Índice General con Páginas

OUR BRAINS ARE DIAZ O — ICPC C++ Team Notebook (Manual Completo de Algoritmos y Estructuras) 1

Bloque 0: Guía de Decisión Rápida, Complejidad y Checklist Anti-WA 1
- 0.1 Selector Rápido: Señal del Problema → Técnica → Condición → Sección 1
- 0.2 Tabla Orientativa de Complejidad según Restricciones (N) 2
- 0.3 Advertencias Críticas de Elección de Técnica 2
- 0.4 Checklist Anti-WA Antes de Enviar 2

Bloque 1: C++ Base, Strings STL, Bitmasks y Cambios de Base 2
- 1.1 Template Base ICPC, Pragmas, Formateo `cout` / `printf` y Comparador `EPS` 2
- 1.2 Generador Aleatorio (`mt19937_64`), `shuffle` y `rd(l, r)` 3
- 1.3 `std::string` : Métodos Clave (`find` , `rfind` , `substr` , `erase` , `insert` , `replace` , `stringstream` , `stoll`) 3
- 1.4 Manipulación de Bits (`__builtin` , LSO, Submáscaras) y `std::bitset` 3
- 1.5 Cambios de Base (Base $A \leftrightarrow B$, Binario, Base Negativa) y Números Gigantes en String 3

Bloque 2: Contenedores STL, PBDS y sus Métodos Más Importantes 4
- 2.1 `std::vector` y `<algorithm>` : Métodos, `sort` , Búsqueda Binaria (`lower_bound` / `upper_bound`) y Compresión de Coordenadas 4
- 2.2 Contenedores Lineales: `std::deque` , `std::queue` y `std::stack` 4
- 2.3 Colas de Prioridad / Heaps (`std::priority_queue`): Max-Heap, Min-Heap, Custom y **Deletable Heap** 4
- 2.4 Árboles Balanceados (`std::set` , `std::multiset` , `std::map`): Métodos, Búsqueda Binaria en `set` y Borrado Seguro en `multiset` 5
- 2.5 Tablas Hash (`unordered_map` , `unordered_set`) con `custom_hash` Anti-Hack (SplitMix64) y Pares Iguales 5
- 2.6 Extensiones GNU: `ordered_set` / `ordered_multiset` (PBDS) y `__gnu_cxx::rope` 5

Bloque 3: Búsquedas, Two Pointers, Prefix Sum y Difference Array 5
- 3.1 Binary Search (Discreta Max/Min y Continua `double`) 5
- 3.2 Ternary Search (Discreta y Continua para Funciones Unimodales) 6
- 3.3 Two Pointers, Monotonic Stack (Next Greater/Smaller) y Monotonic Deque (Sliding Window Minimum) 6
- 3.4 Prefix Sum 1D (Arreglos y Cadenas) y Prefix Sum 2D (Submatrices en $\mathcal{O}(1)$) 6
- 3.5 Difference Array 1D, Doble Difference Array (Greg and Array) y Difference Array 2D 7
- 3.6 Algoritmos Greedy: Interval Scheduling, Greedy con Heap (Regret) y Argumento de Intercambio 7
- 3.7 Barrido de Eventos / Intervalos 1D (Sweep-Line) 8

Bloque 4: Estructuras de Datos Avanzadas (Consultas en Rango y Árboles) 8
- 4.1 Disjoint Set Union (DSU Clásico) y DSU con Rollback + Conectividad Dinámica Offline 8
- 4.2 Sparse Table (RMQ e Idempotentes en $\mathcal{O}(1)$) 9
- 4.3 Fenwick Tree / BIT 1D (con Binary Lifting `lower_bound`) y Fenwick Tree 2D 9
- 4.4 Segment Tree Clásico (Recursivo) y Segment Tree Iterativo 9
- 4.5 Segment Tree con Lazy Propagation, Segment Tree 2D y Persistent Segment Tree 10
- 4.6 SQRT Decomposition y Algoritmo de Mo (con Optimización de **Bloques**) . 11
- 4.7 Treap (BST Aleatorizado) e Implicit Treap (Reversión y Suma en Rango) ... 11
- 4.8 Consultas en Árboles: Heavy-Light Decomposition (HLD) y DSU on Tree (Sack) 12
- 4.9 Link-Cut Tree (LCT — Bosques Dinámicos Online) 12

Bloque 5: Algoritmos de Strings 13
- 5.1 Rotación Cíclica Mínima / Minimum Expression (Duval/Booth $\mathcal{O}(N)$) 13
- 5.2 Double Rolling Hash Polinomial y LCP con Búsqueda Binaria 13
- 5.3 KMP (Función Prefijo π, Pattern Matching, Periodo) y Autómata KMP 13
- 5.4 Z-Algorithm y Algoritmo de Manacher (Palíndromos Pares e Impares) 13
- 5.5 Trie (Strings y Trie Binario para Max XOR) y Aho-Corasick (Múltiples Patrones) 14
- 5.6 Suffix Array + LCP, Suffix Automaton (SAM) y Palindromic Tree (Eertree) . 14

Bloque 6: Grafos, Árboles, Conectividad y Flujos 15
- 6.1 Recorridos BFS: Grid BFS (Flood Fill), Multi-Source BFS (Aristas Invertidas) y 0-1 BFS 15
- 6.2 Patrones DFS: Grafo Bipartito, Ciclos, Euler Tour (`tin` / `tout`), DFS en Complemento y Conteo de Triángulos 16
- 6.3 Caminos Más Cortos: Dijkstra, Bellman-Ford / SPFA (Ciclo Negativo) y Floyd-Warshall 16
- 6.4 Árboles: Diámetro (Doble BFS), MST (Kruskal y Prim), LCA Binary Lifting y Centroid Decomposition 17
- 6.5 Conectividad, DAGs y 2-SAT: Topological Sort, Puentes/Articulación, SCC (Tarjan) y 2-SAT 18
- 6.6 Flujos y Matching: Dinic, Min-Cost Max-Flow (MCMF), Kuhn + König, Algoritmo Húngaro y Blossom 19
- 6.7 Árbol Generador Mínimo Dirigido (Directed MST / Arborescence) 20
- 6.8 Componentes Biconexas y Block-Cut Tree 20
- 6.9 Camino y Ciclo Euleriano (Hierholzer) 21
- 6.10 Flujo Máximo en Redes Densas: Push-Relabel (Highest-Label + Gap) 21
- 6.11 Grafos Funcionales (Successors): Ciclos, Colas y Binary Lifting 21

Bloque 7: Matemáticas, Teoría de Números, Álgebra y Juegos 22
- 7.1 Cribas (Eratóstenes, Divisores, Lineal SPF, Segmentada) y Primalidad $\mathcal{O}(\sqrt{N})$ 22
- 7.2 Primalidad y Factorización Grande: Miller-Rabin (64-bit) y Pollard's Rho . 22
- 7.3 GCD, Diferencia de Cuadrados, Euclides Extendido, Ecuaciones Diofánticas, Ecuación Modular y CRT 23
- 7.4 Aritmética Modular (`mint`), Combinatoria (nCr), Teorema de Lucas y Fórmula de Legendre ($n!$) 23
- 7.5 Funciones Multiplicativas: Totient de Euler (ϕ) y Función de Möbius (μ) . . 24
- 7.6 Álgebra Lineal y Polinomios: Exponenciación de Matrices, Eliminación Gaussiana (`double` , Modular, XOR Basis), FFT, FWT, Logaritmo Discreto y Fracciones 24
- 7.7 XOR en Rango $\mathcal{O}(1)$, Función `mex` , Grundy y Juego de Nim / Misère Nim .26
- 7.8 Programación Lineal: Algoritmo Simplex de Dos Fases 26
- 7.9 Berlekamp-Massey y Recurrencias Lineales Modulares 27
- 7.10 Interpolación de Lagrange en $\mathcal{O}(D)$ y $\mathcal{O}(D^2)$ 27
- 7.11 Conteo Avanzado y Esperanza: Stars and Bars, Inclusión-Exclusión, Contribuciones y Linealidad 27

Bloque 8: Programación Dinámica (DP Clásica, Subconjuntos, Árboles y Optimizaciones) 28
- 8.1 DP Clásica: Knapsack 0/1 y Unbounded, LCS, Edit Distance, LIS $\mathcal{O}(N \log N)$, Digit DP y SOS DP 28
- 8.2 DP de Subconjuntos con Máscaras: Asignación `dp[mask]` y Caminos / TSP `dp[mask][last]` 29
- 8.3 DP en Árboles y Técnica de Rerooting (Cambio de Raíz en $\mathcal{O}(N)$) 29
- 8.4 Optimizaciones de DP: Convex Hull Trick (Estático y Dinámico), Divide & Conquer DP y Knuth's Optimization 29

Bloque 9: Geometría Computacional 2D 30
- 9.1 Geometría 2D: `Pt` , `PtLL` (Entero), Segmentos, Polígonos (Área, Centroide, Punto en Polígono $\mathcal{O}(N)$ y Convexo $\mathcal{O}(\log N)$), `Convex Hull` , Rotating Callipers y Closest Pair 30
- 9.2 Geometría Analítica Básica y Círculos: Proyección, Reflexión, Distancia Punto-Recta y `Circle` 31
- 9.3 Intersección de Semiplanos (Halfplane Intersection) 32
- 9.4 Suma de Minkowski de Polígonos Convexos 32

Bloque 10: Búsqueda en Mitades (MITM), Parsing y Clique Máximo 32
- 10.1 Meet in the Middle (MITM): Subset Sum Exacto, Conteo con Multiplicidades y Máximo $\leq S$ 32
- 10.2 Evaluador de Expresiones Aritméticas (Shunting-Yard Parser) 33
- 10.3 Clique Máximo (Bron-Kerbosch Branch & Bound) 33

Bloque 11: Compendio Teórico, Teoremas, Fórmulas e Identidades 33
- 11.1 Condiciones de Optimización de Programación Dinámica (DP Speedups) 33
- 11.2 Combinatoria (Expresada en Recurrencias, Sumatorias $\sum$ y Productorias $\prod$) 33
- 11.3 Teoría de Números y Aritmética Modular 34
- 11.4 Teoría de Grafos, Árboles y Redes 34
- 11.5 Geometría y Distancias 34
- 11.6 Teoría de Juegos y XOR 34
- 11.7 Reglas Lógicas de 2-SAT 34
- 11.8 Tabla de Constantes, Primos y Módulos Frecuentes 35

Bloque 12: Python 3 para ICPC — Trucos, BigInt, Bases y Librería Estándar 35
- 12.1 Template Base, Fast I/O, Límites de Recursión y Digits en BigInt 35
- 12.2 Conversión de Bases Arbitrarias (Base $A \leftrightarrow B$) y Formateo 35
- 12.3 Aritmética Gigante (BigInt), Potencia Modular y Raíz Entera 35
- 12.4 Fracciones Exactas (`Fraction`) y Decimales de Alta Precisión (`Decimal`) 35
- 12.5 Manejo de Fechas y Calendarios (`datetime`) 35
- 12.6 DP con Memoización Automática (`@cache` / `cache_clear`) 36
- 12.7 Extracción de Números en Texto Sucio con Expresiones Regulares (`re`) . 36
- 12.8 Frecuencias y Álgebra de Multiconjuntos (`Counter`) 36
- 12.9 Agrupación y Grafos Instantáneos (`defaultdict`) 36
- 12.10 Rotación y Transposición de Matrices en 1 Línea (`zip(*mat)`) 36
- 12.11 Álgebra de Conjuntos Nativos (`set`) 36
- 12.12 Bits en Números Gigantes (`bit_count` , `bit_length`) 36
- 12.13 Búsqueda Binaria con Proyección (`bisect` con `key`) y Ordenamiento Multi-Criterio 36
- 12.14 Combinatoria y Generadores Nativos (`comb` , `perm` , `itertools`) 36
- 12.15 Evaluación de Expresiones Aritméticas (`eval`) 37

OUR BRAINS ARE DIAZ O — ICPC C++ Team Notebook (Manual Completo de Algoritmos y Estructuras)

Bloque 0: Guía de Decisión Rápida, Complejidad y Checklist Anti-WA

0.1 Selector Rápido: Señal del Problema → Técnica → Condición → Sección

Señal en el Enunciado	Técnica Recomendada	Condición Obligatoria	Sección
«Maximizar el mínimo» / «Minimizar el máximo»	Binary Search sobre la respuesta	Predicado `check(X)` monótono (VVVFFF o `FFFVVV`)	§3.1
Óptimo de función continua o discreta en U / ∩	Ternary Search	Función **estrictamente unimodal** (sin mesetas planas)	§3.2
Subarreglo más largo/corto con suma $\leq S$	Two Pointers (ventana deslizante)	Elementos **no negativos** ($a_i \geq 0$) para monotonía	§3.3
Subarreglos con suma $= K$ o divisible por M (con < 0)	Prefix Sum + `safe_umap`	Arreglo estático; normalizar residuo `(s%M+M)%M`	§3.4
Subarreglo contiguo de suma máxima	Algoritmo de Kadane	Al menos 1 elemento; maneja todos negativos	§3.4
Mayor rectángulo / primer menor o mayor a izq/der	Monotonic Stack + Contribuciones	Desigualdad estricta de un lado y $\leq$ del otro en duplicados	§3.3, §7.11
Mínimo/máximo en todas las ventanas de tamaño K	Monotonic Deque	Ventana de longitud monótona (extremos avanzan)	§3.3
Sumar V a rangos $[L, R]$ y consultar solo al final	Difference Array 1D / 2D	Todas las consultas ocurren **después** de los updates	§3.5
Máximo número de intervalos disjuntos	Greedy por fin de intervalo	Ordenar por extremo derecho r_i ascendente	§3.6
Tareas con duración t_i y plazo d_i (máx. cantidad)	Greedy con Max-Heap (*Regret*)	Ordenar por d_i; si $\sum t > d_i$, descartar mayor t	§3.6
Intersección máxima o unión de intervalos	Sweep-Line 1D de eventos	Definir desempate en x según intervalo cerrado/abierto	§3.7
Conectividad agregando aristas / MST	DSU Clásico / Kruskal	Solo inserciones (sin borrado de aristas)	§4.1, §6.4
Conectividad agregando y **borrando** aristas	`DynCon` (`RollbackDSU` offline) o `LinkCut`	`DynCon` : todas las operaciones conocidas offline	§4.1, §4.9
RMQ / GCD en rango sin actualizaciones	Sparse Table	Arreglo estático y operación idempotente ($\mathcal{O}(1)$)	§4.2
Suma en rango + update puntual (1D / 2D)	Fenwick Tree (BIT)	Operación invertible (`+` , `^`); 1-indexed	§4.3
Operación asociativa en rango + update puntual/rango	Segment Tree / Lazy SegTree	Monoid asociativo; distributividad del lazy tag	§4.4, §4.5
k-ésimo menor en subarreglo $[L, R]$ estático	`PersistentFreqST`	Compresión de coordenadas previa ($1 \leq L \leq R \leq N$)	§4.5
Consultas complejas en rango sin asociatividad fácil	Algoritmo de Mo	**Offline**, sin updates (o pocos), agregar/quitar $\mathcal{O}(1)$	§4.6
Cortar, pegar, revertir subarreglos dinámicamente	Implicit Treap / `rope`	Índices 0-based; `push` antes de dividir/unir	§4.7, §2.6
Consultas / updates en caminos o subárboles de árbol	LCA / Euler Tour / HLD / Sack	Ver §0.3 para elegir la herramienta mínima suficiente	§4.8, §6.4
Buscar patrón P en texto T o periodo mínimo	KMP / Z-Algorithm	Exacto y determinista en $\mathcal{O}(\lvert T\rvert + \lvert P\rvert)$	§5.3, §5.4
Palíndromo más largo centrado en cada posición	Manacher	Transforma pares e impares en $\mathcal{O}(\lvert S\rvert)$	§5.4
Múltiples patrones / DP prohibiendo subcadenas	Aho-Corasick	Construir `link` y `next` por BFS antes de consultar	§5.5

Señal en el Enunciado	Técnica Recomendada	Condición Obligatoria	Sección
Max/Min XOR con un conjunto de enteros	`BitTrie` / `XorBasis`	`BitTrie` : elemento individual; `XorBasis` : subconjunto	§5.5, §7.6
Subcadenas distintas / LCS contigua / ocurrencias	Suffix Array + LCP o SAM	SA: $\mathcal{O}(N \log N)$; SAM: $\mathcal{O}(N \log N)$ con sort	§5.6
Camino mínimo sin pesos / pesos $\{0,1\}$ / ≥ 0	BFS / 0-1 BFS / Dijkstra	Dijkstra exige **pesos no negativos** ($w \geq 0$)	§6.1, §6.3
Camino mínimo con pesos negativos o detectar ciclo < 0	Bellman-Ford / Floyd-Warshall	$N \leq 2500$ (BF) o $N \leq 400$ (Floyd)	§6.3
Decisiones binarias con cláusulas $(a \vee b)$, $(a \Rightarrow b)$	2-SAT con Tarjan SCC	A lo sumo 2 literales por cláusula	§6.5
Flujo máximo / Corte mínimo / Matching bipartito	Dinic / `BipartiteMVC` (Kuhn+König)	Capacidades ≥ 0; en bipartito Dinic es $\mathcal{O}(E\sqrt{V})$	§6.6
Flujo de costo mínimo / Asignación $N \times M$	MCMF (SPFA) / Húngaro	Sin ciclos de costo negativo; Húngaro para matriz densa	§6.6
Matching máximo en grafo general (con ciclos impares)	Algoritmo de Blossom	$V \leq 500$, $\mathcal{O}(V^3)$	§6.6
Grafo donde cada nodo tiene exactamente 1 salida	Grafo Funcional (Ciclos + Lifting)	`LOG = 62` si $K \leq 10^{18}$	§6.11
Primalidad y factorización hasta 10^{18}	Miller-Rabin + Pollard's Rho	Usar multiplicación `__uint128_t` (`u128`)	§7.2
Sistema de congruencias $x \equiv r_i \pmod{m_i}$	CRT General + `extgcd`	Ensanchar a `i128` antes de multiplicar por m_1	§7.3
Recurrencia lineal a_n para $N \leq 10^{18}$	Exponenciación de Matriz / BM	Matriz conocida $\mathcal{O}(K^3 \log N)$ o inferida con BM	§7.6, §7.9
Convolución de polinomios exacta o módulo 998244353	FFT (`double`) / NTT (`998244353`)	NTT exige primo $c \cdot 2^k + 1$ y raíz primitiva $g = 3$	§7.6
Subconjuntos / Asignación / TSP con $N \leq 20$	Subset DP (`dp[mask]` , `dp[mask][u]`)	$\mathcal{O}(N2^N)$ asignación vs $\mathcal{O}(N^2 2^N)$ TSP	§8.2
Respuesta para cada nodo si fuera la raíz del árbol	Rerooting DP (2 DFS)	Combinar `down[v]` de hijos con contribución exterior `up[u]`	§8.3
DP cuadrática con rectas o costos convexos	CHT / D&C DP / Knuth	Verificar monotonía de pendientes/óptimos (§8.4)	§8.4
Geometría exacta sin errores de redondeo	Kernel Entero `PtLL` (`i128`)	Coordenadas enteras $\leq 10^9$ (productos en `i128`)	§9.1
$N \leq 40$ demasiado grande para 2^N pero $2^{N/2}$ cabe	Meet in the Middle (MITM)	Partir en $N/2$, ordenar una mitad y usar `equal_range`	§10.1

0.2 Tabla Orientativa de Complejidad según Restricciones (N)

Nota sobre límites reales: La tabla asume un límite estándar de **1–2 segundos en C++20** ($\approx 10^8$ operaciones simples/seg). El límite real depende de: (1) **Tiempo límite** del problema, (2) **Lenguaje** (en Python dividir el presupuesto por $10\times$–$30\times$ salvo operaciones vectorizadas/C nativas), (3) **Memoria límite** (evitar tablas `ll` de $> 3 \cdot 10^7$ celdas para 256 MB) y (4) **Constantes ocultas** (`set` / `map` y grafos densos tienen constante mucho mayor que vectores contiguos o Fenwick).

Cota de N	Complejidad Objetivo	Técnicas Típicas
$N \leq 10$	$\mathcal{O}(N!)$, $\mathcal{O}(N \cdot N!)$	`next_permutation` , backtracking exhaustivo
$N \leq 20$	$\mathcal{O}(2^N)$, $\mathcal{O}(N \cdot 2^N)$, $\mathcal{O}(N^2 \cdot 2^N)$	Bitmask DP (`dp[mask]` , TSP `dp[mask][last]`), SOS DP ($N \leq 20$), submáscaras $\mathcal{O}(3^N)$ ($N \leq 15$)
$N \leq 40$	$\mathcal{O}(2^{N/2} \log(2^{N/2}))$	Meet in the Middle (§10.1), Clique Máximo con poda (§10.3)
$N \leq 400$	$\mathcal{O}(N^3)$	Floyd-Warshall, Gauss, Húngaro, Blossom, DP de intervalos / Knuth $\mathcal{O}(N^2)$
$N \leq 3000$	$\mathcal{O}(N^2)$, $\mathcal{O}(N^2/64)$	DP 2D clásica (LCS, Edit Distance, Knapsack), `bitset` $\mathcal{O}(N^2/64)$, árboles cuadráticos
$N \leq 10^5$	$\mathcal{O}(N\sqrt{N})$, $\mathcal{O}(N \log^2 N)$	Algoritmo de Mo, SQRT Decomposition, HLD + SegTree, D&C DP, Centroid Decomp
$N \leq 2 \cdot 10^5 - 5 \cdot 10^5$	$\mathcal{O}(N \log N)$	`sort` , Binary Search, Dijkstra, Fenwick, SegTree, LCA, FFT/NTT, Suffix Array, Convex Hull
$N \leq 10^6 - 5 \cdot 10^6$	$\mathcal{O}(N)$	Two Pointers, Monotonic Stack/Deque, KMP, Z, Manacher, Criba Lineal, Prefix/Diff Array
$N \leq 10^{18}$	$\mathcal{O}(\log N)$, $\mathcal{O}(\sqrt{N})$, $\mathcal{O}(N^{1/4})$	Binary Lifting, Exponenciación de Matrices, Digit DP, Miller-Rabin + Pollard's Rho

0.3 Advertencias Críticas de Elección de Técnica

1. **Sliding Window / Two Pointers de suma exige monotonía**:
   - **Nunca** usar ventana deslizante estándar (`while (sum > K) sum -= a[l++]`) para suma de subarreglo si hay **números negativos**, porque avanzar `l` puede aumentar `sum` y avanzar `r` puede disminuirla.
   - *Alternativa con negativos*: para contar suma exacta K usar `Prefix Sum + safe_umap` (§3.4); para suma máxima usar Kadane (§3.4); para subarreglo más corto/largo con suma $\geq K$ o $\leq K$ con negativos usar `Prefix Sum + Monotonic Deque / Binary Search` .
2. **Mediana transformada a $+1/-1$ (precisión de paridad y desigualdad)**:
   - Al probar si existe un subarreglo con mediana $\geq X$, se transforma cada elemento en $b_i = +1$ si $a_i \geq X$ y $b_i = -1$ si $a_i < X$.
   - **No usar `sum == 0` como regla general**:
     - Para la **mediana inferior** (el elemento $\lfloor (L+1)/2 \rfloor$-ésimo en orden 1-based, donde en longitud par $L = 2k$ es el k-ésimo): la mediana es $\geq X$ si y solo si la suma del subarreglo cumple $\mathbf{sum > 0}$ ($\text{sum} \geq 1$).
     - Para la **mediana superior** (el elemento $\lfloor L/2 \rfloor + 1$-ésimo, donde en $L = 2k$ es el $(k+1)$-ésimo): la mediana es $\geq X$ si y solo si $\mathbf{sum \geq 0}$ (en longitud impar equivale a $\text{sum} \geq 1$).
   - Distinguir también si el problema pide mediana $\geq X$ (búsqueda binaria sobre X con $+1/-1$) o mediana **exactamente igual** a un valor X sin elementos repetidos (donde $a_i > X \to +1$, $a_i < X \to -1$, $a_i = X \to 0$ y se busca subarreglo que contenga a X con suma 0 o $+1$).
3. **Consultas en Árboles — Cuándo bastan LCA/Prefijos y cuándo sí se necesita HLD**:
   - **Árbol estático + operación invertible en camino** (suma, XOR, conteo): basta guardar `pref[u]` desde la raíz y usar `LCA` (§6.4): $\text{camino}(u,v) = \text{pref}[u] + \text{pref}[v] - 2 \cdot \text{pref}[\text{lca}(u,v)] + \text{val}[\text{lca}(u,v)]$ en $\mathcal{O}(\log N)$ (sin HLD).
   - **Updates en nodos + suma en subárbol** (o update de suma en camino + consulta puntual en nodo): basta **Euler Tour (`tin` / `tout` , §6.2) + Fenwick Tree (§4.3)** en $\mathcal{O}(\log N)$.
   - **Updates en subárbol + consultas en subárbol**: basta **Euler Tour (`tin` / `tout`) + Lazy SegTree (§4.5)** porque todo subárbol es el rango contiguo `[tin[u], tout[u]]` .
   - **Cuándo SÍ usar HLD (§4.8)**: cuando hay **updates dinámicos** y consultas en **camino** de una operación **no invertible** (`max` , `min` , `gcd` , matriz), o cuando hay simultáneamente **updates en camino y consultas en camino/subárbol**.

0.4 Checklist Anti-WA Antes de Enviar

- ☐ **Overflow entero**: ¿Usaste `ll` en sumas de prefijos, distancias de Dijkstra, costos de DP y conteos de pares `1LL * n * (n - 1) / 2` ? ¿Usaste `i128` antes de multiplicar en CRT, geometría entera (`PtLL`) o `mul_mod64` (`u128`)?
- ☐ **Indexación (0-based vs 1-based)**: Fenwick, DSU, LCA, HLD y grafos suelen ser `1..N` ; Segment Tree, Sparse Table, Strings y Geometría suelen ser `0..N-1` . ¿Ajustaste `--u, --v` al leer?
- ☐ **Casos múltiples (`T` testcases) y reinicio**: ¿Limpiaste `adj[1..n]` , `vis[1..n]` y contadores entre casos? **Prohibido** hacer `memset(arr, 0, sizeof(arr))` sobre un arreglo global de $2 \cdot 10^5$ dentro de cada caso cuando $\sum N \leq 2 \cdot 10^5$ (causa TLE $\mathcal{O}(T \cdot N_{\max})$); usar `fill(arr, arr + n + 1, 0)` .
- ☐ **Constante `INF`** : ¿Es suficientemente grande (`1e18` para `ll` , no `1e9`) pero protegida contra overflow al hacer `dist[u] + w` (`if (dist[u] < INF)`)?
- ☐ **Negativos en módulo e inversos**: En C++ `(-a) % MOD` es negativo; usar `(x % MOD + MOD) % MOD` . Antes de usar `inv_fermat(a, p)` , verificar que p sea primo y $a \not\equiv 0 \pmod{p}$; para módulo compuesto usar `inv_extgcd` solo si $\gcd(a, m) = 1$.
- ☐ **Recursión y pila**: En grafos/árboles con $N = 2 \cdot 10^5$, no pasar vectores por valor en el DFS. En Python, configurar `sys.setrecursionlimit(300000)` o reescribir el DFS iterativo.
- ☐ **Precisión en punto flotante**: Usar `ld` (`long double`), imprimir con `fixed << setprecision(10)` , comparar con `eps_eq` / `eps_lt` , y **nunca** pasar un comparador aproximado `a + EPS < b` a `std::set` / `std::map` .
- ☐ **Estructura vacía y bordes**: Verificar $N = 1$, $K = 0$, grafo desconectado, llamar a `ms.find(x)` antes de `ms.erase(it)` en `multiset` , y comprobar `!s.empty()` antes de `*s.begin()` , `*s.rbegin()` o `pq.top()` .

Bloque 1: C++ Base, Strings STL, Bitmasks y Cambios de Base

1.1 Template Base ICPC, Pragmas, Formateo `cout` / `printf` y Comparador `EPS`

- **Cuándo usar:** Esqueleto base para todo programa C++ en competencia: incluye fast I/O, macros de iteración (`forn` , `for1` , `fore` , `ford` , `fored`), comparación segura con `EPS` (`eps_lt` , `eps_eq`) y cuantización para contenedores.
- **Cuándo no usar / alternativa:** No descomentar `#pragma GCC target("avx2...")` si el juez corre en una arquitectura distinta sin soporte AVX2 o si el problema es I/O-bound.
- **Contrato:** Define alias `ll` , `ull` , `ld` , `pii` , `pll` , `vi` , `vll` , macros `forn` / `for1` / `fore` / `ford` / `fored` y constantes `INF_INT` , `INF` , `EPS` , `PI` . `eps_eq(a, b)` comprueba `|a - b| <= EPS` .
- **Complejidad:** Fast I/O y comparadores en $\mathcal{O}(1)$.

```cpp
// Descomentar pragmas solo si se requiere optimizacion SIMD
// #pragma GCC optimize("O3,unroll-loops")
// #pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")

#include <bits/stdc++.h>
#include <cassert>
using namespace std;

using ll = long long;
using ull = unsigned long long;
using ld = long double;
using pii = pair<int, int>;
using pll = pair<ll, ll>;
using vi = vector<int>;
using vll = vector<ll>;

#define forn(i,n) for(int i=0; i< (int)n; ++i)
#define for1(i,n) for(int i=1; i<= (int)n; ++i)
#define fore(i,l,r) for(int i=(int)l; i<= (int)r; ++i)
#define ford(i,n) for(int i=(int)(n) - 1; i>= 0; --i)
#define fored(i,l,r) for(int i=(int)r; i>= (int)l; --i)
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (int)(x).size()
#define pb push_back
#define fi first
#define se second

const int INF_INT = 1e9 + 7;
const ll INF = 1e18;
const ld EPS = 1e-9;
const ld PI = acos(-1.0L);

// Comparacion aproximada en punto flotante (NO usar a + EPS
// < b en set/map,
// porque la equivalencia |a - b| <= EPS no es transitiva y
// rompe std::set).
inline int eps_sgn(ld x) { return (x > EPS) - (x < -EPS); }
inline bool eps_eq(ld a, ld b) { return eps_sgn(a - b) == 0;
  }
inline bool eps_lt(ld a, ld b) { return eps_sgn(a - b) < 0;
  }
// Para usar doubles como clave en set/map con rejilla fija,
// cuantizar a ll:
inline ll quantize_eps(ld x, ld step = EPS) {
  return llround(x / step); }

void io_formatting_tricks() {
  // 1. Imprimir punto flotante con exactamente 6 decimales:
  cout << fixed << setprecision(6) << 3.14159265 << "\n";
  // 2. Rellenar con ceros a la izquierda (ej. hora 05:09):
  int h = 5, m = 9;
  cout << setfill('0') << setw(2) << h << ":"
       << setw(2) << m << "\n";
  // 3. Imprimir en base hexadecimal, octal y volver a
  // decimal:
```

```
  cout << hex << 255 << " " << oct << 255 << dec << "\n";
}

void solve() {
  // Logica por caso de prueba
}

int main() {
  ios_base::sync_with_stdio(false);
  cin.tie(nullptr);
  int t = 1;
  // cin >> t;
  while (t--) solve();
  return 0;
}
```

- **Trampas y adaptaciones:** (1) Jamás mezclar `scanf` / `printf` con `cin` / `cout` tras `sync_with_stdio(false)`; (2) no usar `eps_lt` como comparador directo de `std::set` porque la cercanía `|a-b| <= EPS` no es transitiva (usar `quantize_eps`).

1.2 Generador Aleatorio (`mt19937_64`), `shuffle` y `rd(l, r)`

- **Cuándo usar:** Generar enteros o reales uniformes en 64 bits (`rd(l, r)`), barajar arreglos antes de `sort`/QuickSelect o alimentar algoritmos aleatorizados (Pollard's Rho, Treap).
- **Cuándo no usar / alternativa:** Nunca usar `rand()` en Windows/MinGW porque `RAND_MAX = 32767` deja sin cubrir índices > 32767.
- **Contrato:** Rango cerrado `[l, r]` con `l <= r`. Para pruebas deterministas reproducibles, fijar semilla `rng.seed(1337)`.
- **Complejidad:** $\mathcal{O}(1)$ por llamada a `rd(l, r)`; $\mathcal{O}(N)$ para `shuffle`.

```
mt19937_64 rng(
  chrono::steady_clock::now().time_since_epoch().count()
);

// Entero aleatorio uniforme en el rango cerrado [l, r]
ll rd(ll l, ll r) {
  return uniform_int_distribution<ll>(l, r)(rng);
}

// Real aleatorio uniforme en [l, r]
double rd_real(double l, double r) {
  return uniform_real_distribution<double>(l, r)(rng);
}

// Barajar un vector aleatoriamente:
// shuffle(all(v), rng);
```

- **Trampas y adaptaciones:** (1) Declarar `rng` una sola vez como global (no recrearlo dentro de una función en cada llamada); (2) asegurar `l <= r` antes de llamar a `uniform_int_distribution`.

1.3 `std::string`: Métodos Clave (`find`, `rfind`, `substr`, `erase`, `insert`, `replace`, `stringstream`, `stoll`)

- **Cuándo usar:** Separar líneas por un delimitador fijo (CSV, horas `HH:MM:SS`) o por espacios arbitrarios, buscar subcadenas (`find`), extraer rangos (`substr`) y convertir entre strings y enteros (`stoll`).
- **Cuándo no usar / alternativa:** Si necesitas buscar un patrón millones de veces o sobre textos de 10^6 caracteres con garantías lineales en el peor caso, no usar `s.find` repetido ($\mathcal{O}(N \cdot M)$); usar KMP (§5.3) o Z-Algorithm (§5.4).
- **Contrato:** `split_delim(s, delim)` y `split_spaces(s)` retornan `vector<string>`. En `s.substr(pos, len)`, el segundo parámetro es la **longitud**, no el índice final.
- **Complejidad:** `split_delim` y `split_spaces` en $\mathcal{O}(|S|)$; `find`, `substr`, `erase`, `insert` en $\mathcal{O}(|S|)$.

```
// 1. Separar string por un caracter delimitador (ej. ',',
// ':')
vector<string> split_delim(const string& s, char delim) {
  vector<string> tokens;
  stringstream ss(s);
  string item;
  while (getline(ss, item, delim)) tokens.push_back(item);
  return tokens;
}

// 2. Separar string por espacios (ignora espacios
// multiples)
vector<string> split_spaces(const string& s) {
  vector<string> tokens;
  stringstream ss(s);
  string w;
  while (ss >> w) tokens.push_back(w);
  return tokens;
}

void string_methods_demo() {
  string s = "icpc_regional_icpc_2026";

  // --- BUSQUEDA (find / rfind) ---
  // Retorna indice 0-based o string::npos si NO existe
  size_t p1 = s.find("icpc");       // 0 (primera aparicion)
  // 14 (busca desde idx 1)
  size_t p2 = s.find("icpc", 1);
  size_t p3 = s.rfind("icpc");      // 14 (ultima aparicion)
  if (s.find("xyz") == string::npos) { /* No esta */ }

  // Hallar TODAS las ocurrencias de un patron 'pat':
  string pat = "icpc";
  for (size_t pos = s.find(pat); pos != string::npos;
       pos = s.find(pat, pos + 1)) {
    // 'pos' es el inicio 0-based de cada coincidencia
  }

  // --- SUBCADENAS (substr) ---
  // ¡CUIDADO! substr(pos_inicio, LONGITUD)
  // Para extraer el rango [L, R] inclusivo: s.substr(L,
  // R-L+1)
  string sub1 = s.substr(5, 8);     // "regional" (8 chars)
  // "2026" (hasta el final)
  string sub2 = s.substr(19);

  // --- EDICION (erase / insert / replace) ---
  s.erase(4, 9);           // Borra 9 chars desde idx 4
  s.insert(4, "_world");  // Inserta en idx 4
  // Reemplaza 4 chars desde 0 por "acm"
  s.replace(0, 4, "acm");

  // --- CONVERSIONES ---
  // string -> long long
  ll val = stoll("1234567890123");
  ll from_bin = stoll("101101", 0, 2); // base 2 -> ll (45)
  // base 16 -> ll (255)
  ll from_hex = stoll("FF", 0, 16);
  double d = stod("3.14159");          // string -> double
  string num_s = to_string(val);       // numero -> string

  // --- MAYUSCULAS / MINUSCULAS ---
  transform(all(s), s.begin(), ::tolower);
  transform(all(s), s.begin(), ::toupper);
}
```

- **Trampas y adaptaciones:** (1) Al alternar `cin >> x` con `getline`, usar `getline(cin >> ws, s)` si deseas ignorar saltos de línea previos, pero si los espacios iniciales de la línea son significativos consumir solo `cin.ignore()`; (2) comparar el retorno de `find` siempre con `string::npos`.

1.4 Manipulación de Bits (`__builtin`, LSO, Submáscaras) y `std::bitset`

- **Cuándo usar:** Operaciones sobre máscaras de bits ($N \leq 60$), conteo de bits encendidos (`__builtin_popcountll`), bit menos/más significativo, iteración de submáscaras en $\mathcal{O}(3^N)$ y conjuntos booleanos acelerados por hardware (`bitset<MAXN>`).
- **Cuándo no usar / alternativa:** Si el tamaño del `bitset` no se conoce en tiempo de compilación y varía mucho por caso, o si solo hay pocos bits encendidos dispersos en $[0, 10^9]$ (usar `vector<int>` ordenado o `safe_uset`).
- **Contrato:** `__builtin_ctzll(x)` y `__builtin_clzll(x)` son **comportamiento indefinido (UB) si** `x == 0`; verificar siempre `x > 0`. Para desplazamientos ≥ 31 bits usar siempre `1LL << k` o `1ULL << k`.
- **Complejidad:** Operaciones `__builtin_*` en $\mathcal{O}(1)$; recorrer todas las submáscaras de una máscara m toma $\mathcal{O}(2^{\text{popcount}(m)})$; operaciones de `bitset<N>` toman $\mathcal{O}(N/64)$.

```
void bitmask_tricks(ll x) {
  // 1. Funciones de hardware GCC (sufijo 'll' para 64 bits)
  // Cantidad de bits en 1
  int ones = __builtin_popcountll(x);
  // Ceros a la derecha (x>0)
  int ctz  = __builtin_ctzll(x);
  // Ceros a la izq (x>0)
  int clz  = __builtin_clzll(x);
  // floor(log2(x)) exacto
  int msb  = 63 - __builtin_clzll(x);

  // 2. Operaciones sobre el k-esimo bit (0-indexed)
  int k = 4;
  x |= (1LL << k);          // Prender bit k
  x &= ~(1LL << k);         // Apagar bit k
  x ^= (1LL << k);          // Invertir (flip) bit k
  bool on = (x >> k) & 1LL; // Consultar si bit k es 1

  // 3. Trucos del bit menos significativo (LSO / LSB)
  ll lso = x & -x;          // Aisla el 1 mas bajo
  ll clear_lso = x & (x-1); // Apaga el 1 mas bajo
  ll set_low0  = x | (x+1); // Prende el 0 mas bajo
  bool is_pow2 = (x > 0) && ((x & (x - 1)) == 0);

  // 4. Iterar sobre todas las submascaras no vacias de 'm'
  int m = 0b10110;
  // Orden decreciente:
  for (int sub = m; sub > 0; sub = (sub - 1) & m) {
    // procesar 'sub'
  }
  // Orden creciente:
  for (int sub = 0; (sub = (sub - m) & m); ) {
    // procesar 'sub'
  }
}

void bitset_methods_demo() {
  bitset<1005> bs;
  bs.set();        // Pone todos los bits en 1
  bs.reset();      // Pone todos los bits en 0
  bs.set(5, 1);    // Prende el bit 5 (bs[5] = 1)
  bs.reset(5);     // Apaga el bit 5 (bs[5] = 0)
  bs.flip(2);      // Invierte el bit 2
  int c = bs.count();   // Numero de bits en 1: O(N/64)
  bool a = bs.any();    // true si al menos un bit es 1
  bool n = bs.none();   // true si todos son 0

  // Recorrer SOLO los indices con bit en 1 en O(#unos +
  // N/64):
  for (int i = bs._Find_first(); i < (int)bs.size();
       i = bs._Find_next(i)) {
    // 'i' es la posicion del bit encendido
  }
}
```

- **Trampas y adaptaciones:** (1) Usar siempre los sufijos `ll` (`__builtin_popcountll`, `__builtin_clzll`, `__builtin_ctzll`) para tipos `long long`; sin `ll` truncan a 32 bits; (2) `1 << 35` hace overflow en `int`, escribir `1LL << 35`.

1.5 Cambios de Base (Base $A \leftrightarrow B$, Binario, Base Negativa) y Números Gigantes en String

- **Cuándo usar:** Convertir números entre bases $B \in [2, 36]$ (`decimal_to_base`, `base_to_decimal`), representar en base negativa como -2 (`to_negative_base`), o dividir/calcular el módulo de un entero de 10^5 dígitos leído como `string` (`divide_large_number`, `mod_large_number`).
- **Cuándo no usar / alternativa:** Si se requiere multiplicación repetida de enteros gigantes sin límite de tiempo estricto y Python está permitido, usar BigInt nativo de Python (§12.2–§12.3).
- **Contrato:** `to_binary_string(num, n)` pone el LSB en `s[0]`; `decimal_to_base` pone el MSB en `s[0]`. `divide_large_number(s, n)` y `mod_large_number(s, m)` asumen `s` decimal no negativo y divisor `n, m > 0`.
- **Complejidad:** Cambio de base en $\mathcal{O}(\log_B(\text{num}))$; división y módulo de string en $\mathcal{O}(|S|)$.

```
// 1. Numero a string binario de longitud fija n (s[0] =
// LSB)
string to_binary_string(ll num, int n) {
  string s = "";
  for (int i = 0; i < n; i++) {
    s += (num % 2 == 0 ? '0' : '1');
    num /= 2;
  }
  return s; // s[0] es el bit 0 (menos significativo)
}
```

```cpp
// 2. Conversor general entre bases [2, 36] (MSB primero)
int char_to_val(char c) {
  if (isdigit(c)) return c - '0';
  return toupper(c) - 'A' + 10;
}
char val_to_char(int v) {
  return (v < 10) ? ('0' + v) : ('A' + v - 10);
}

string decimal_to_base(ll num, int base) {
  if (num == 0) return "0";
  bool neg = (num < 0);
  if (neg) num = -num;
  string res = "";
  while (num > 0) {
    res += val_to_char(num % base);
    num /= base;
  }
  if (neg) res += '-';
  reverse(all(res));
  return res;
}

ll base_to_decimal(const string& s, int base) {
  if (s.empty()) return 0;
  ll res = 0;
  int st = (s[0] == '-' || s[0] == '+') ? 1 : 0;
  for (int i = st; i < sz(s); ++i)
    res = res * base + char_to_val(s[i]);
  return (s[0] == '-') ? -res : res;
}

// 3. Cambio a base negativa (ej. neg_base = -2)
string to_negative_base(ll n, int neg_base) {
  if (n == 0) return "0";
  string res = "";
  while (n != 0) {
    ll rem = n % neg_base;
    n /= neg_base;
    if (rem < 0) { rem += (-neg_base); n += 1; }
    res += (char)('0' + rem);
  }
  reverse(all(res));
  return res;
}

// 4. Division entera y modulo de numero gigante en string
// Ejemplo: divide_large_number("10010", 14) -> "715"
string divide_large_number(const string& s, ll n) {
  string res = "";
  ll rem = 0;
  for (char c : s) {
    rem = rem * 10 + (c - '0');
    if (!res.empty() || rem / n > 0)
      res += to_string(rem / n);
    rem %= n;
  }
  return res.empty() ? "0" : res;
}

ll mod_large_number(const string& s, ll m) {
  ll rem = 0;
  for (char c : s) rem = (rem * 10 + (c - '0')) % m;
  return rem;
}
```

- **Trampas y adaptaciones:** (1) `to_binary_string` devuelve el bit menos significativo en el índice 0, mientras que `decimal_to_base(x, 2)` devuelve el más significativo en el índice 0; (2) en C++, `rem % neg_base` puede ser negativo, por lo que `to_negative_base` suma `-neg_base` e incrementa el cociente.

Bloque 2: Contenedores STL, PBDS y sus Métodos Más Importantes

2.1 `std::vector` y `<algorithm>`: Métodos, `sort`, Búsqueda Binaria (`lower_bound` / `upper_bound`) y Compresión de Coordenadas

- **Cuándo usar:** Mapear valores arbitrarios en $[-10^{18}, 10^{18}]$ a rangos densos $[0, K-1]$ (`CoordComp`) para usarlos como índices de Fenwick/Segment Tree, y realizar búsquedas binarias en arreglos ordenados.
- **Cuándo no usar / alternativa:** Si el conjunto de llaves cambia dinámicamente con inserciones de valores nuevos no conocidos al inicio y necesitas orden, usar `std::set`, `ordered_set` (§2.6) o Segment Tree dinámico.
- **Contrato:** `CoordComp::init(a)` ordena y elimina duplicados; `get_id(x)` retorna el índice 0-based en $[0, K-1]$ del primer valor $\geq x$; `get_val(id)` recupera el valor original.
- **Complejidad:** `init` en $\mathcal{O}(N \log N)$ tiempo y $\mathcal{O}(N)$ memoria; `get_id` en $\mathcal{O}(\log N)$.

```cpp
// Compresion de Coordenadas: mapea valores arbitrarios a
// [0, K-1]
struct CoordComp {
  vll vals;
  void init(const vll& a) {
    vals = a;
    sort(all(vals));
    vals.erase(unique(all(vals)), vals.end());
  }
  int get_id(ll x) const { // Indice 0-based en O(log N)
    return lower_bound(all(vals), x) - vals.begin();
  }
  ll get_val(int id) const { return vals[id]; }
  int size() const { return sz(vals); }
};

void vector_and_bs_cheatsheet() {
  vi v = {1, 2, 2, 5, 5, 8, 9}; // DEBE ESTAR ORDENADO

  // 1. Primer elemento >= x (lower_bound)
  // Apunta al 1er 5 (idx 3)
  auto it_ge = lower_bound(all(v), 5);
  int idx_ge = it_ge - v.begin();

  // 2. Primer elemento > x (upper_bound)
  // Apunta al 8 (idx 5)
  auto it_gt = upper_bound(all(v), 5);
  int idx_gt = it_gt - v.begin();

  // 3. Ultimo elemento <= x (predecesor inclusivo)
  auto it_le = upper_bound(all(v), 5);
  if (it_le != v.begin()) {
    int val_le = *prev(it_le); // 5 (idx 4)
  }

  // 4. Ultimo elemento < x (predecesor estricto)
  auto it_lt = lower_bound(all(v), 5);
  if (it_lt != v.begin()) {
    int val_lt = *prev(it_lt); // 2 (idx 2)
  }

  // 5. Contar cuantos elementos estan en el rango [L, R]
  int L = 2, R = 8;
  int cnt_in_range = upper_bound(all(v), R)
                   - lower_bound(all(v), L); // 5 elementos

  // 6. Utilidades clave de <algorithm> y <numeric>
  ll suma = accumulate(all(v), 0LL); // ¡Usar 0LL, no 0!
  int mx = *max_element(all(v));
  int mn = *min_element(all(v));

  // 7. Generar todas las permutaciones en O(N! * N)
  vi p = {3, 1, 2};
  sort(all(p)); // OBLIGATORIO ordenar antes del do-while
  do {
    // procesar permutacion 'p'
  } while (next_permutation(all(p)));
}
```

- **Trampas y adaptaciones:** (1) En `accumulate(all(v), 0LL)`, pasar siempre `0LL` para evitar overflow en `int`; (2) si el Fenwick Tree es 1-indexed, sumar `+ 1` al resultado de `cc.get_id(x)`.

2.2 Contenedores Lineales: `std::deque`, `std::queue` y `std::stack`

- **Cuándo usar:** `deque` para inserción/borrado en ambos extremos y acceso aleatorio `dq[i]` en $\mathcal{O}(1)$ (0-1 BFS, ventana deslizante); `queue` para BFS FIFO; `stack` para LIFO y pilas monótonas.
- **Cuándo no usar / alternativa:** No llamar nunca a `front()`, `back()`, `top()` o `pop()` sobre un contenedor vacío (causa comportamiento indefinido / Segmentation Fault).
- **Contrato:** Operaciones sobre extremos en `deque`, `queue` y `stack`. Verificar siempre `!q.empty()` antes de consultar o extraer.
- **Complejidad:** $\mathcal{O}(1)$ tiempo por operación; $\mathcal{O}(N)$ memoria.

```cpp
void linear_containers_demo() {
  // 1. std::deque (Doble cola + acceso aleatorio O(1))
  deque<int> dq;
  dq.push_back(20);     // [20]
  dq.push_front(10);    // [10, 20]
  int f = dq.front();   // 10
  int b = dq.back();    // 20
  int x = dq[1];        // 20 (acceso por indice O(1))
  dq.pop_front();       // Quita el 10
  dq.pop_back();        // Quita el 20

  // 2. std::queue (FIFO - Primero en entrar, primero en
  // salir)
  queue<int> q;
  q.push(1); q.push(2);
  int primero = q.front(); // 1
  q.pop();                 // Elimina el frente (1)

  // 3. std::stack (LIFO - Ultimo en entrar, primero en
  // salir)
  stack<int> st;
  st.push(10); st.push(20);
  int tope = st.top();     // 20
  st.pop();                // Elimina el tope (20)
}
```

- **Trampas y adaptaciones:** (1) `std::queue` y `std::stack` no tienen método `.clear()`; para vaciarlos rápido reasignar `q = queue<int>()` o declararlos locales al caso de prueba; (2) `deque` tiene mayor constante de memoria que `vector`.

2.3 Colas de Prioridad / Heaps (`std::priority_queue`): Max-Heap, Min-Heap, Custom y Deletable Heap

- **Cuándo usar:** Extraer el mínimo o máximo dinámico en $\mathcal{O}(\log N)$, y usar `DeletableMinHeap` cuando además de `push` / `top` necesites borrar valores arbitrarios presentes sin pagar la constante de `std::multiset`.
- **Cuándo no usar / alternativa:** En `DeletableMinHeap`, **nunca** llamar `erase(x)` con un valor `x` que no esté actualmente presente en `pq` (corrompería la cola `del` al cancelar inserciones futuras).
- **Contrato:** `DeletableMinHeap<T>`: `push(x)`, `erase(x)` (precondición: `x` está presente), `top()` y `pop()` (precondición: `!empty()`). En `struct State`, `operator<` invierte el signo (`cost > o.cost`) porque `std::priority_queue` es un Max-Heap por defecto.
- **Complejidad:** `push`, `pop`, `erase` en $\mathcal{O}(\log N)$ amortizado; `top` en $\mathcal{O}(1)$ amortizado; $\mathcal{O}(N)$ memoria.

```cpp
// 1. Comparador en struct para Min-Heap (invertir el signo
// '<')
struct State {
  int u;
  ll cost;
  bool operator<(const State& o) const {
    return cost > o.cost; // El menor 'cost' queda en top()
  }
};

// 2. Deletable Min-Heap (Borrado arbitrario en O(log N))
template <typename T>
struct DeletableMinHeap {
  priority_queue<T, vector<T>, greater<T>> pq, del;

  void push(T x) { pq.push(x); }
  void erase(T x) { del.push(x); } // Borrado perezoso

  void clean() {
    while (!pq.empty() && !del.empty() && pq.top(
      ) == del.top()) {
      pq.pop();
      del.pop();
    }
  }
  T top() { clean(); return pq.top(); }
  void pop() { clean(); pq.pop(); }
  bool empty() { clean(); return pq.empty(); }
  int size() const { return sz(pq) - sz(del); }
};

void priority_queue_demo() {
  // Max-Heap (por defecto: saca el MAYOR primero)
  priority_queue<ll> max_pq;
  max_pq.push(10); max_pq.push(50);
  ll mayor = max_pq.top(); // 50
  max_pq.pop();

  // Min-Heap (saca el MENOR primero)
  priority_queue<ll, vector<ll>, greater<ll>> min_pq;
  min_pq.push(10); min_pq.push(50);
  ll menor = min_pq.top(); // 10
```

```
  min_pq.pop();
}
```

- **Trampas y adaptaciones:** (1) Recordar que `priority_queue<T>` por defecto saca el **mayor** elemento primero; para Min-Heap usar `priority_queue<T, vector<T>, greater<T>>`; (2) verificar `!dh.empty()` antes de `dh.top()`.

2.4 Árboles Balanceados (`std::set`, `std::multiset`, `std::map`): Métodos, Búsqueda Binaria en `set` y Borrado Seguro en `multiset`

- **Cuándo usar:** Mantener un conjunto o multiconjunto ordenado dinámico con consultas de sucesor (`lower_bound`, `upper_bound`) y predecesor (`prev(it)`) en $\mathcal{O}(\log N)$.
- **Cuándo no usar / alternativa:** Si no necesitas orden ni consultas de rango/vecinos, usar `safe_umap` / `safe_uset` (§2.5) en $\mathcal{O}(1)$ promedio.
- **Contrato:** Usar siempre el método miembro `s.lower_bound(x)` ($\mathcal{O}(\log N)$) y jamás `std::lower_bound(all(s), x)` ($\mathcal{O}(N)$). En `multiset`, `ms.erase(ms.find(x))` borra **una sola** copia (verificando `it != ms.end()`).
- **Complejidad:** `insert`, `erase(it)`, `find`, `lower_bound`, `upper_bound` en $\mathcal{O}(\log N)$.

```
void set_multiset_map_cheatsheet() {
  set<int> s = {10, 20, 30, 40};
  s.insert(25);
  s.erase(20);                    // Borra el 20 en O(log N)
  // Minimo actual (10) en O(1)
  int mn = *s.begin();
  // Maximo actual (40) en O(1)
  int mx = *s.rbegin();

  // --- BUSQUEDA BINARIA EN SET / MULTISET ---
  // ¡PROHIBIDO usar std::lower_bound(all(s), x) porque es
  // O(N)!
  // Usar SIEMPRE el metodo miembro s.lower_bound(x) ->
  // O(log N):

  int x = 25;
  // 1) Primer elemento >= x:
  auto it_ge = s.lower_bound(x);
  if (it_ge != s.end()) { int val_ge = *it_ge; }

  // 2) Primer elemento > x:
  auto it_gt = s.upper_bound(x);
  if (it_gt != s.end()) { int val_gt = *it_gt; }

  // 3) Mayor elemento < x (predecesor estricto):
  auto it_lt = s.lower_bound(x);
  if (it_lt != s.begin()) { int val_lt = *prev(it_lt); }

  // 4) Mayor elemento <= x (predecesor inclusivo):
  auto it_le = s.upper_bound(x);
  if (it_le != s.begin()) { int val_le = *prev(it_le); }

  // --- TRAMPAS CLASICAS DE MULTISET ---
  multiset<int> ms = {5, 5, 5, 10};
  // Trampa 1: ms.erase(5) borra TODAS las apariciones del
  // 5.
  // Para borrar UNA SOLA copia del 5 en O(log N):
  auto it = ms.find(5);
  if (it != ms.end()) ms.erase(it); // Queda {5, 5, 10}

  // Trampa 2: ms.count(x) es O(k + log N) donde k =
  // repeticiones.
  // Para saber solo si existe en O(log N), usar:
  bool existe = (ms.find(5) != ms.end());

  // --- CUIDADO CON STD::MAP ---
  map<int, int> mp;
  // Hacer if (mp[key] == 0) INSERTA 'key' en el mapa y
  // gasta RAM/tiempo.
  // Para consultar sin insertar:
  if (mp.find(100) != mp.end()) { int val = mp[100]; }
}
```

- **Trampas y adaptaciones:** (1) `ms.count(x)` en un `multiset` tarda $\mathcal{O}(k + \log N)$ donde k es la multiplicidad (usar `ms.find(x) != ms.end()` para existencia); (2) consultar `mp[k]` en `std::map` **inserta** la llave `k` con valor 0 si no existía (usar `mp.find(k)`).

2.5 Tablas Hash (`unordered_map`, `unordered_set`) con `custom_hash` Anti-Hack (SplitMix64) y Pares Iguales

- **Cuándo usar:** Conteo de frecuencias, memorización dispersa o búsqueda de llaves (incluyendo `pair<T1, T2>`) en $\mathcal{O}(1)$ promedio protegiendo contra colisiones maliciosas (*anti-hash tests*).
- **Cuándo no usar / alternativa:** Si necesitas recorrer las llaves en orden creciente o consultar predecesor/sucesor, usar `std::map` / `std::set` (§2.4); si las llaves son enteros en $[0, N]$, usar un `vector` directo.
- **Contrato:** `safe_umap<K, V>` y `safe_uset<K>` aceptan tipos enteros y `pair<T1, T2>`. `count_equal_pairs(a)` cuenta pares $(i < j)$ con $a[i] = a[j]$.
- **Complejidad:** $\mathcal{O}(1)$ tiempo esperado por operación; $\mathcal{O}(N)$ para `count_equal_pairs`.

```
struct custom_hash {
  static uint64_t splitmix64(uint64_t x) {
    x += 0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
    return x ^ (x >> 31);
  }
  size_t operator()(uint64_t x) const {
    static const uint64_t FIXED_RANDOM =
      chrono::steady_clock::now().time_since_epoch().count(
        );
    return splitmix64(x + FIXED_RANDOM);
  }
  // Permite usar pair<T1, T2> como llave en safe_umap
  template <typename T1, typename T2>
  size_t operator()(const pair<T1, T2>& p) const {
    uint64_t h1 = (*this)(p.first);
    uint64_t h2 = (*this)(p.second);
    return h1 ^ (h2 + 0x9e3779b97f4a7c15 + (h1<<6) + (
      h1>>2));
  }
};

template <typename K, typename V>
using safe_umap = unordered_map<K, V, custom_hash>;

template <typename K>
using safe_uset = unordered_set<K, custom_hash>;

// Aplicacion: Contar pares (i < j) con a[i] == a[j] en O(N)
// Ejemplo: a = [1, 2, 1, 1, 2] -> 3 pares de 1 + 1 par de 2
// = 4
ll count_equal_pairs(const vi& a) {
  safe_umap<int, int> freq;
  freq.reserve(sz(a) * 2);
  freq.max_load_factor(0.25f);
  for (int x : a) freq[x]++;
  ll pairs = 0;
  for (auto& [val, cnt] : freq)
    pairs += 1LL * cnt * (cnt - 1) / 2;
  return pairs;
}
```

- **Trampas y adaptaciones:** (1) Llamar a `mp.reserve(2 * N)` y `mp.max_load_factor(0.25f)` reduce los *rehashes* y acelera $2\times$ la tabla; (2) para `string` usar el hash nativo o Rolling Hash (§5.2).

2.6 Extensiones GNU: `ordered_set` / `ordered_multiset` (PBDS) y `__gnu_cxx::rope`

- **Cuándo usar:** `ordered_set` / `ordered_multiset` para responder en $\mathcal{O}(\log N)$ el k-ésimo elemento (`find_by_order(k)`, 0-based) y cuántos elementos son $< x$ (`order_of_key(x)`); `rope` para cortar y pegar subarreglos en $\mathcal{O}(\log N)$.
- **Cuándo no usar / alternativa:** **Prohibido** usar `less_equal<T>` en PBDS para simular un multiset: rompe la asimetría del comparador y hace que `find`, `lower_bound` y `erase` fallen silenciosamente. Usar siempre `ordered_multiset<T>` con `pair<T, int>` e `id` único.
- **Contrato:** `find_by_order(k)` requiere $0 \le k < \text{size}()$. En `ordered_multiset<T>`, insertar `{x, uid++}`; `order_of_key({x, -1})` cuenta elementos $< x$ y `order_of_key({x, INF_INT})` cuenta elementos $\le x$.
- **Complejidad:** $\mathcal{O}(\log N)$ tiempo por operación; $\mathcal{O}(N)$ memoria.

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <ext/rope>
using namespace __gnu_pbds;
using namespace __gnu_cxx;

// 1. Ordered Set (elementos unicos)
template <typename T>
using ordered_set = tree<
  T, null_type, less<T>, rb_tree_tag,
  tree_order_statistics_node_update>;

// 2. Ordered Multiset seguro con pair<T, int> y less<>
// (NUNCA usar less_equal,
// porque viola asimetria y corrompe find, lower_bound y
// erase).
template <typename T>
using ordered_multiset = tree<
  pair<T, int>, null_type, less<pair<T, int>>, rb_tree_tag,
  tree_order_statistics_node_update>;

void pbds_and_rope_demo() {
  ordered_set<int> os;
  os.insert(10); os.insert(20); os.insert(50);

  // k-esimo elemento (0-indexed; precondicion: 0 <= k <
  // sz(os)):
  int elem_idx_1 = *os.find_by_order(1); // 20
  // Cantidad de elementos estrictamente < 25:
  // 2 (el 10 y el 20)
  int menores_25 = os.order_of_key(25);

  // Multiset con duplicados, conteos por rango y borrado de
  // UNA sola copia:
  ordered_multiset<int> oms;
  int uid = 0;
  oms.insert({5, uid++}); oms.insert({5, uid++});
    oms.insert({10, uid++});
  // 0 (< 5)
  int menores_5 = oms.order_of_key({5, -1});
  // 2 (<= 5)
  int men_ig_5  = oms.order_of_key({5, INF_INT});
  auto it = oms.lower_bound({5, -1});
  // Borra 1 copia de 5
  if (it != oms.end() && it->first == 5) oms.erase(it);
  (void)elem_idx_1; (void)menores_25; (void)menores_5;
    (void)men_ig_5;

  // 3. Uso de rope<int> o crope (rope<char>)
  // Cortar el rango 0-based [1, r] y moverlo al inicio:
  rope<int> rp;
  // [1,2,3,4,5]
  for (int i = 1; i <= 5; ++i) rp.push_back(i);
  int l = 1, r = 3, len = r - l + 1;
  rope<int> sub = rp.substr(l, len); // Extrae [2, 3, 4]
  rp.erase(l, len);                  // Queda [1, 5]
  // Queda [2, 3, 4, 1, 5]
  rp.insert(rp.mutable_begin(), sub);
}
```

- **Trampas y adaptaciones:** (1) Verificar siempre `k < (int)oms.size()` antes de desreferenciar `find_by_order(k)`; (2) `rope` tiene constante alta; si el límite de tiempo es ajustado o hay reversiones de rango, usar `ImplicitTreap` (§4.7).

Bloque 3: Búsquedas, Two Pointers, Prefix Sum y Difference Array

3.1 Binary Search (Discreta Max/Min y Continua `double`)

3.1.1 Binary Search Discreta sobre la Respuesta (`bs_max` y `bs_min`)

- **Cuándo usar:** Problemas de «maximizar el mínimo» (`bs_max`, predicado `V, V, ..., F, F`) o «minimizar el máximo» (`bs_min`, predicado `F, F, ..., V, V`) sobre un dominio entero `[low, high]`.
- **Cuándo no usar / alternativa:** Si el predicado `check(mid)` **no es monótono** en `[low, high]`, la búsqueda binaria descartará la mitad equivocada.
- **Contrato:** `bs_max(low, high, check)` retorna el mayor entero en `[low, high]` con `check(mid) == true`, o `low - 1` si ninguno cumple. `bs_min(low, high, check)` retorna el menor entero con `check(mid) == true`, o `high + 1` si ninguno cumple.
- **Complejidad:** $\mathcal{O}(\log(\text{high} - \text{low} + 1) \cdot T_{\text{check}})$ tiempo, $\mathcal{O}(1)$ memoria.

```
// 1. Maximizar respuesta discreta: [True, True, ..., False,
// False]
template <typename F>
ll bs_max(ll low, ll high, F check) {
  ll ans = low - 1; // Centinela si ninguno cumple
  while (low <= high) {
    ll mid = low + (high - low) / 2;
    if (check(mid)) {
```

```
      ans = mid;      // mid sirve, probar mayores
      low = mid + 1;
    } else {
      high = mid - 1;
    }
  }
  return ans;
}

// 2. Minimizar respuesta discreta: [False, False, ...,
// True, True]
template <typename F>
ll bs_min(ll low, ll high, F check) {
  ll ans = high + 1;
  while (low <= high) {
    ll mid = low + (high - low) / 2;
    if (check(mid)) {
      ans = mid;      // mid sirve, probar menores
      high = mid - 1;
    } else {
      low = mid + 1;
    }
  }
  return ans;
}
```

- **Trampas y adaptaciones:** (1) Nunca evaluar `L = 0` si `check(L)` divide entre `L` (`low >= 1`); (2) elegir `high` suficientemente grande pero sin causar overflow en `mid * algo` dentro de `check(mid)` .

3.1.2 Binary Search Continua en Punto Flotante (`bs_double`)

- **Cuándo usar:** Hallar un umbral real $x \in [l, r]$ cuando `check(x)` es monótono (tiempo mínimo de encuentro, promedio máximo $0/1$ fraccionario).
- **Cuándo no usar / alternativa:** No usar `while (r - l > EPS)` como condición de parada: si $l, r \approx 10^9$ y `EPS = 1e-9` , la distancia entre `double` consecutivos supera `EPS` y produce un **bucle infinito (TLE)**.
- **Contrato:** `bs_double(l, r, check, it = 90)` ejecuta `it` iteraciones fijas reduciendo el intervalo en 2^{-90}. Asume `check(mid) == true` en la mitad izquierda (`[True, False]`); si es `[False, True]` , intercambiar `l = mid` y `r = mid` .
- **Complejidad:** $\mathcal{O}(\text{it} \cdot T_{\text{check}})$ con `it = 90` .

```
// 3. Binary Search continua en double (iteraciones fijas)
template <typename F>
double bs_double(double l, double r, F check, int it = 90) {
  for (int i = 0; i < it; ++i) {
    double mid = l + (r - l) / 2.0;
    // Si maximiza (cambiar a r=mid si minimiza)
    if (check(mid)) l = mid;
    else r = mid;
  }
  return l;
}
```

- **Trampas y adaptaciones:** (1) Recordar que `double` tiene 53 bits de mantisa (≈ 15 dígitos significativos): la precisión real es relativa a la escala ($|x| \cdot 2^{-53}$); usar `long double` (`ld`) si el rango supera 10^9.

3.2 Ternary Search (Discreta y Continua para Funciones Unimodales)

3.2.1 Ternary Search Continua (`ternary_continuous`)

- **Cuándo usar:** Minimizar (o maximizar) una función continua **estrictamente unimodal** $f(x)$ en $[l, r]$ (p. ej., suma de distancias convexas).
- **Cuándo no usar / alternativa:** Si $f(x)$ tiene tramos planos constantes (mesetas) o múltiples mínimos locales, la búsqueda ternaria puede converger a un punto subóptimo.
- **Contrato:** `ternary_continuous(l, r, f, it = 100)` retorna el **valor mínimo** $f(x^*)$ en $[l, r]$. Para maximizar, cambiar `<` por `>` o minimizar `-f(x)` .
- **Complejidad:** $\mathcal{O}(\text{it} \cdot T_f)$ con `it = 100` (reduce el intervalo por $(2/3)^{100} \approx 2.4 \cdot 10^{-18}$).

```
// 1. Ternary Search Continua (halla el MINIMO en [l, r])
// Para hallar el MAXIMO: cambiar f(m1) < f(m2) por f(m1) >
// f(m2)
template <typename F>
double ternary_continuous(double l, double r, F f,
  int it = 100) {
  for (int i = 0; i < it; ++i) {
    double m1 = l + (r - l) / 3.0;
    double m2 = r - (r - l) / 3.0;
    if (f(m1) < f(m2)) r = m2;
    else l = m1;
  }
  // Retorna el valor minimo (o 'l' para la posicion)
  return f(l);
}
```

- **Trampas y adaptaciones:** (1) Toda suma de funciones convexas (como distancias euclidianas a puntos o rectas) es convexa y por tanto unimodal; (2) si necesitas la abscisa x^* en vez de $f(x^*)$, retornar `l` al final.

3.2.2 Ternary Search Discreta en Enteros (`ternary_discrete`)

- **Cuándo usar:** Minimizar una función entera estrictamente unimodal $f(x)$ para $x \in [\text{low}, \text{high}] \cap \mathbb{Z}$.
- **Cuándo no usar / alternativa:** Si existen valores consecutivos con $f(x) = f(x+1)$ fuera del óptimo global, la ternaria falla; si $f(x+1) - f(x)$ es monótona (función convexa), también puedes hacer búsqueda binaria sobre la derivada discreta $\Delta f(x) = f(x+1) - f(x) \geq 0$.
- **Contrato:** `ternary_discrete(low, high, f)` retorna $\min_{x \in [\text{low}, \text{high}]} f(x)$, evaluando exhaustivamente cuando el rango se reduce a ≤ 3 elementos.
- **Complejidad:** $\mathcal{O}(\log_{3/2}(\text{high} - \text{low} + 1) \cdot T_f)$.

```
// 2. Ternary Search Discreta en enteros [low, high]
// (MINIMO)
template <typename F>
ll ternary_discrete(ll low, ll high, F f) {
  while (high - low >= 3) {
    ll m1 = low + (high - low) / 3;
    ll m2 = high - (high - low) / 3;
    if (f(m1) < f(m2)) high = m2;
    else low = m1;
  }
  ll best = f(low);
  for (ll x = low + 1; x <= high; ++x)
    best = min(best, f(x));
  return best;
}
```

- **Trampas y adaptaciones:** (1) Nunca reducir directamente a un solo punto sin el bucle `while (high - low >= 3)` + barrido final de 3 enteros, o ciclará por división entera.

3.3 Two Pointers, Monotonic Stack (Next Greater/Smaller) y Monotonic Deque (Sliding Window Minimum)

3.3.1 Two Pointers / Ventana Deslizante (`max_subarray_sum_bound`)

- **Cuándo usar:** Hallar el subarreglo más largo (o más corto, o contar cuántos subarreglos) que cumple una condición **monótona** al expandir `r` y contraer `l` (p. ej., suma $\leq S$ cuando $a[i] \geq 0$).
- **Cuándo no usar / alternativa: Jamás** usar esta ventana deslizante para suma de subarreglos si existen números **negativos** ($a[i] < 0$), porque quitar `a[l]` aumentaría la suma y rompería la monotonía (ver §0.3 y §3.4.2).
- **Contrato:** Precondición obligatoria: $a[i] \geq 0$ para todo i y `max_sum >= 0` . Retorna la mayor longitud de un subarreglo contiguo con suma $\leq \text{max_sum}$ (`0` si ninguno cumple).
- **Complejidad:** $\mathcal{O}(N)$ tiempo (`l` y `r` avanzan a lo sumo N veces) y $\mathcal{O}(1)$ memoria.

```
// 1. Two Pointers: Subarreglo mas largo con suma <= max_sum
// (a[i] >= 0)
int max_subarray_sum_bound(const vll& a, ll max_sum) {
  int n = sz(a), max_len = 0, l = 0;
  ll cur_sum = 0;
  for (int r = 0; r < n; ++r) {
    cur_sum += a[r];
    while (cur_sum > max_sum && l <= r) {
      cur_sum -= a[l++];
    }
    max_len = max(max_len, r - l + 1);
  }
  return max_len;
}
```

- **Trampas y adaptaciones:** (1) Para contar **cuántos** subarreglos tienen suma $\leq \text{max_sum}$ (con $a[i] \geq 0$), en cada `r` sumar `(r - l + 1)` a un acumulador `ll` después del `while` .

3.3.2 Monotonic Stack: Primer Mayor/Menor a la Derecha o Izquierda (`next_greater_right`)

- **Cuándo usar:** Encontrar para cada índice i el primer índice $j > i$ (o $j < i$) con $a[j] > a[i]$ (o $<, \geq, \leq$). Base de mayor rectángulo en histograma y técnica de contribuciones (§7.11).
- **Cuándo no usar / alternativa:** Si los elementos cambian dinámicamente y se pide el primer índice $\geq X$ en un rango $[L, R]$, usar **Segment Tree walk** en $\mathcal{O}(\log N)$.
- **Contrato:** Índices `0..N-1` . `next_greater_right(a)` retorna `nxt[0..N-1]` donde `nxt[i]` es el menor $j > i$ con `a[j] > a[i]` , o `-1` si no existe.
- **Complejidad:** $\mathcal{O}(N)$ tiempo y $\mathcal{O}(N)$ memoria.

```
// 2. Monotonic Stack: Indice del primer elemento > a[i] a
// la derecha
// Retorna -1 si no existe. (Para < a[i], cambiar '<' por
// '>')
vi next_greater_right(const vll& a) {
  int n = sz(a);
  vi nxt(n, -1);
  stack<int> st;
  for (int i = 0; i < n; ++i) {
    while (!st.empty() && a[st.top()] < a[i]) {
      nxt[st.top()] = i;
      st.pop();
    }
    st.push(i);
  }
  return nxt;
}
```

- **Trampas y adaptaciones:** (1) Para hallar el primer elemento **menor** a la derecha, cambiar `a[st.top()] < a[i]` por `a[st.top()] > a[i]` ; (2) para el primer mayor a la **izquierda**, recorrer de derecha a izquierda o consultar el tope de la pila antes de hacer `push(i)` .

3.3.3 Monotonic Deque: Mínimo/Máximo en Ventana Deslizante (`sliding_window_min`)

- **Cuándo usar:** Calcular el mínimo (o máximo) de todas las subventanas contiguas de longitud fija K en $\mathcal{O}(N)$, o acelerar DP con transiciones en rango $[i - K, i - 1]$.
- **Cuándo no usar / alternativa:** Si la longitud de la ventana no es monótona (los extremos `L` y `R` retroceden arbitrariamente), usar **Sparse Table** (§4.2) en $\mathcal{O}(1)$ por consulta.
- **Contrato:** Precondición: $1 \leq K \leq N$. Retorna un vector de tamaño $N - K + 1$ donde la posición i contiene $\min(a[i..i + K - 1])$.
- **Complejidad:** $\mathcal{O}(N)$ tiempo y $\mathcal{O}(N)$ memoria.

```
// 3. Monotonic Deque: Minimo de cada ventana de longitud K
// Retorna vector con N - K + 1 respuestas. (Para Maximo:
// cambiar >= por <=)
vll sliding_window_min(const vll& a, int k) {
  int n = sz(a);
  // Guarda indices en orden creciente de valor
  deque<int> dq;
  vll ans;
  for (int i = 0; i < n; ++i) {
    if (!dq.empty() && dq.front() <= i - k) dq.pop_front();
    while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back(
      );
    dq.push_back(i);
    if (i >= k - 1) ans.push_back(a[dq.front()]);
  }
  return ans;
}
```

- **Trampas y adaptaciones:** (1) La `deque` guarda **índices** (no valores) para poder detectar cuándo `dq.front() <= i - k` sale de la ventana; (2) para ventana de **máximo**, cambiar `a[dq.back()] >= a[i]` por `<=` .

3.4 Prefix Sum 1D (Arreglos y Cadenas) y Prefix Sum 2D (Submatrices en $\mathcal{O}(1)$)

3.4.1 Prefix Sum 1D (`PrefixSum`)

- **Cuándo usar:** Responder Q consultas de suma en un rango $[l, r]$ sobre un arreglo **estático** en $\mathcal{O}(1)$ por consulta.
- **Cuándo no usar / alternativa:** Si hay actualizaciones del arreglo intercaladas con las consultas de suma, usar **Fenwick Tree** (§4.3.1) en $\mathcal{O}(\log N)$.
- **Contrato:** El constructor recibe `a` (0-indexed), pero las consultas `query(l, r)` son **1-indexed inclusivas** ($1 \leq l \leq r \leq N$). Si $l > r$, retorna `0` .

- **Complejidad:** Construcción $\mathcal{O}(N)$, consulta $\mathcal{O}(1)$, memoria $\mathcal{O}(N)$.

```cpp
// 1. Prefix Sum 1D (1-indexed en las consultas [1, r])
struct PrefixSum {
  vll pref;
  PrefixSum(const vll& a) : pref(sz(a) + 1, 0) {
    for (int i = 0; i < sz(a); ++i)
      pref[i + 1] = pref[i] + a[i];
  }
  // Constructor para contar caracteres adyacentes iguales
  // (CF 313B)
  PrefixSum(const string& s) : pref(sz(s) + 1, 0) {
    for (int i = 0; i + 1 < sz(s); ++i)
      pref[i + 1] = pref[i] + (s[i] == s[i + 1]);
    if (!s.empty()) pref[sz(s)] = pref[sz(s) - 1];
  }
  ll query(int l, int r) const { // 1-based [l, r]
    return (l > r) ? 0 : pref[r] - pref[l - 1];
  }
};
```

- **Trampas y adaptaciones:** (1) Los prefijos deben guardarse siempre en `ll` (`vll pref`) aunque el arreglo original quepa en `int`; (2) si las consultas del enunciado son 0-based `[l, r]`, llamar `ps.query(l + 1, r + 1)`.

3.4.2 Prefix Sum 2D en Submatrices (`PrefixSum2D`)

- **Cuándo usar:** Consultar la suma de cualquier submatriz rectangular $[r_1..r_2] \times [c_1..c_2]$ en una grilla estática en $\mathcal{O}(1)$.
- **Cuándo no usar / alternativa:** Si la matriz recibe actualizaciones puntuales entre consultas, usar **Fenwick Tree 2D** (§4.3.2).
- **Contrato:** El constructor recibe `mat` 0-indexed de dimensión $N \times M$; `query(r1, c1, r2, c2)` usa coordenadas **1-indexed inclusivas** ($1 \le r_1 \le r_2 \le N$, $1 \le c_1 \le c_2 \le M$).
- **Complejidad:** Construcción $\mathcal{O}(N \cdot M)$, consulta $\mathcal{O}(1)$, memoria $\mathcal{O}(N \cdot M)$.

```cpp
// 2. Prefix Sum 2D (Submatrices 1-based [r1..r2][c1..c2])
struct PrefixSum2D {
  vector<vll> p;
  PrefixSum2D(const vector<vll>& mat) {
    int n = sz(mat), m = n ? sz(mat[0]) : 0;
    p.assign(n + 1, vll(m + 1, 0));
    for (int i = 1; i <= n; ++i)
      for (int j = 1; j <= m; ++j)
        p[i][j] = mat[i - 1][j - 1] + p[i - 1][j]
                + p[i][j - 1] - p[i - 1][j - 1];
  }
  ll query(int r1, int c1, int r2, int c2) const {
    if (r1 > r2 || c1 > c2) return 0;
    return p[r2][c2] - p[r1 - 1][c2]
         - p[r2][c1 - 1] + p[r1 - 1][c1 - 1];
  }
};
```

- **Trampas y adaptaciones:** (1) Al aplicar inclusión-exclusión 2D, recordar sumar de vuelta `+ p[r1 - 1][c1 - 1]` que fue restado dos veces.

3.4.3 Patrones de Prefijos (Suma K, Divisibles por M, Prefix XOR) y Kadane

- **Cuándo usar:** Contar subarreglos con suma exacta K (incluso con **negativos**), suma divisible por M, XOR igual a K, o hallar la máxima suma de un subarreglo contiguo no vacío (`kadane_max_subarray`).
- **Cuándo no usar / alternativa:** Para subarreglo de suma $\le K$ con negativos, `count_subarrays_sum_k` solo cuenta igualdades exactas; usar árbol de orden (`ordered_multiset`, §2.6) o Fenwick con compresión sobre los prefijos.
- **Contrato:** `kadane_max_subarray(a)` requiere `!a.empty()` y retorna el mayor elemento si todos son negativos. `count_subarrays_div_m(a, m)` requiere $m > 0$ y normaliza negativos con `((pref + x) % m + m) % m`. Dependencias: `safe_umap` (§2.5).
- **Complejidad:** $\mathcal{O}(N)$ tiempo y $\mathcal{O}(N)$ memoria ($\mathcal{O}(1)$ memoria en Kadane).

```cpp
// 3. Patrones de Prefijos: Subarreglos con Suma = K,
// Divisibles por M,
// Prefix XOR = K y Subarreglo de Suma Maxima (Kadane) en
// O(N).
ll count_subarrays_sum_k(const vll& a, ll k) {
  safe_umap<ll, int> freq;
  freq[0] = 1;
  ll pref = 0, ans = 0;
  for (ll x : a) {
    pref += x;
    auto it = freq.find(pref - k);
    if (it != freq.end()) ans += it->se;
    freq[pref]++;
  }
  return ans;
}

ll count_subarrays_div_m(const vll& a, ll m) {
  vi freq(m, 0);
  freq[0] = 1;
  ll pref = 0, ans = 0;
  for (ll x : a) {
    pref = ((pref + x) % m + m) % m;
    ans += freq[pref]++;
  }
  return ans;
}

ll count_subarrays_xor_k(const vi& a, int k) {
  safe_umap<int, int> freq;
  freq[0] = 1;
  int px = 0;
  ll ans = 0;
  for (int x : a) {
    px ^= x;
    auto it = freq.find(px ^ k);
    if (it != freq.end()) ans += it->se;
    freq[px]++;
  }
  return ans;
}

// Kadane: mayor suma de un subarreglo contiguo NO vacio
// (admite todo < 0)
ll kadane_max_subarray(const vll& a) {
  assert(!a.empty());
  ll cur = a[0], best = a[0];
  for (int i = 1; i < sz(a); ++i) {
    cur = max(a[i], cur + a[i]);
    best = max(best, cur);
  }
  return best;
}
```

- **Trampas y adaptaciones:** (1) Inicializar siempre `freq[0] = 1` antes de recorrer el arreglo para contar los subarreglos que empiezan en el índice 0; (2) no inicializar `best = 0` en Kadane si el subarreglo no puede ser vacío y todos los números son negativos.

3.5 Difference Array 1D, Doble Difference Array (Greg and Array) y Difference Array 2D

3.5.1 Difference Array 1D y Doble Difference Array (`DiffArray1D`, `apply_double_diff`)

- **Cuándo usar:** Aplicar Q sumas en rango (`sumar val a [l, r]`) en $\mathcal{O}(1)$ cada una cuando **todas las consultas ocurren al final**, o aplicar rangos de operaciones sobre un arreglo (`apply_double_diff`, estilo *Greg and Array*).
- **Cuándo no usar / alternativa:** Si hay consultas de suma intercaladas con actualizaciones de rango, usar **Lazy Segment Tree** (§4.5.1) o dos Fenwick Trees.
- **Contrato:** Indexación **1-based** (`1..n`). `add_range(l, r, val)` suma `val` a `[l..r]`; `build()` devuelve `res[1..n]`.
- **Complejidad:** $\mathcal{O}(1)$ por actualización; $\mathcal{O}(N)$ para `build()`.

```cpp
// 1. Difference Array 1D (1-indexed [1..n])
struct DiffArray1D {
  int n;
  vll diff;
  DiffArray1D(int n) : n(n), diff(n + 2, 0) {}
  void add_range(int l, int r, ll val) { // 1-based [l, r]
    diff[l] += val;
    diff[r + 1] -= val;
  }
  vll build() const {
    vll res(n + 1, 0);
    for (int i = 1; i <= n; ++i) res[i] = res[i - 1]
      + diff[i];
    return res; // res[1..n]
  }
};

// 2. Doble Difference Array (Operaciones de rango sobre
// operaciones)
// Se tienen M operaciones (sumar d_i en [l_i, r_i]) y K
// consultas
// que aplican todas las operaciones desde la x_j hasta la
// y_j (CF 295A).
struct Op { int l, r; ll d; };
vll apply_double_diff(int n, int m, const vll& base_arr,
                      const vector<Op>& ops,
                      const vector<pii>& queries) {
  DiffArray1D op_diff(m);
  for (auto [x, y] : queries) op_diff.add_range(x, y, 1);
  vll op_cnt = op_diff.build();

  DiffArray1D arr_diff(n);
  for (int i = 1; i <= m; ++i)
    if (op_cnt[i] > 0)
      arr_diff.add_range(ops[i].l, ops[i].r,
        ops[i].d * op_cnt[i]);

  vll delta = arr_diff.build(), ans(n + 1, 0);
  for (int i = 1; i <= n; ++i) ans[i] = base_arr[i]
    + delta[i];
  return ans;
}
```

- **Trampas y adaptaciones:** (1) El vector interno `diff` debe tener tamaño `n + 2` porque `add_range` accede a `diff[r + 1]` cuando `r == n`.

3.5.2 Difference Array 2D en Submatrices (`DiffArray2D`)

- **Cuándo usar:** Sumar `val` a Q rectángulos $[r_1..r_2] \times [c_1..c_2]$ en $\mathcal{O}(1)$ cada uno y reconstruir la matriz final una sola vez al terminar.
- **Cuándo no usar / alternativa:** Si se intercalan actualizaciones rectangulares y consultas de submatriz, no basta un Difference Array estático.
- **Contrato:** Coordenadas **1-based** ($1 \le r_1 \le r_2 \le N$, $1 \le c_1 \le c_2 \le M$). `build()` retorna matriz `(n + 1) x (m + 1)`.
- **Complejidad:** $\mathcal{O}(1)$ por `add_rect`; $\mathcal{O}(N \cdot M)$ en `build()`.

```cpp
// 3. Difference Array 2D (Sumar 'val' al rectangulo
// [r1..r2][c1..c2])
struct DiffArray2D {
  int n, m;
  vector<vll> d;
  DiffArray2D(int n, int m) : n(n), m(m), d(n + 2, vll(
    m + 2, 0)) {}
  void add_rect(int r1, int c1, int r2, int c2, ll val) {
    d[r1][c1] += val;
    d[r1][c2 + 1] -= val;
    d[r2 + 1][c1] -= val;
    d[r2 + 1][c2 + 1] += val;
  }
  vector<vll> build() const {
    vector<vll> res(n + 1, vll(m + 1, 0));
    for (int i = 1; i <= n; ++i)
      for (int j = 1; j <= m; ++j)
        res[i][j] = d[i][j] + res[i - 1][j]
                  + res[i][j - 1] - res[i - 1][j - 1];
    return res;
  }
};
```

- **Trampas y adaptaciones:** (1) Recordar los signos de las 4 esquinas: `+val` en `(r1, c1)` y `(r2+1, c2+1)`, y `-val` en `(r1, c2+1)` y `(r2+1, c1)`.

3.6 Algoritmos Greedy: Interval Scheduling, Greedy con Heap (Regret) y Argumento de Intercambio

3.6.1 Interval Scheduling (Selección Máxima de Intervalos Disjuntos)

- **Cuándo usar:** Seleccionar la máxima cantidad de intervalos mutuamente disjuntos ordenando por menor extremo derecho r_i.
- **Cuándo no usar / alternativa:** Si cada intervalo tiene un **peso/ganancia** arbitraria y quieres maximizar la suma de pesos, el greedy por extremo derecho falla; usar **DP con búsqueda binaria / Segment Tree** en $\mathcal{O}(N \log N)$.
- **Contrato:** `interval_scheduling(ivs, strict_disjoint)` recibe pares `{l, r}`. Si `strict_disjoint = true`, intervalos que comparten borde ($[1, 2]$ y $[2, 3]$) chocan (`l > last_r`); si `false`, pueden tocarse en el borde (`l >= last_r`).
- **Complejidad:** $\mathcal{O}(N \log N)$ tiempo y $\mathcal{O}(N)$ memoria.

```cpp
// 1. Interval Scheduling: maximo numero de intervalos
// disjuntos
int interval_scheduling(vector<pll> ivs,
                        bool strict_disjoint = false) {
  sort(all(ivs), [](const pll& a, const pll& b) {
    return a.se != b.se ? a.se < b.se : a.fi < b.fi;
  });
  int cnt = 0;
  ll last_r = -INF;
  for (auto [l, r] : ivs) {
    if (strict_disjoint ? (l > last_r) : (l >= last_r)) {
      cnt++;
      last_r = r;
    }
```

```
  }
  return cnt;
}
```

- **Trampas y adaptaciones:** (1) Nunca ordenar por extremo izquierdo l_i ni por menor longitud $r_i - l_i$: ambos contraejemplos clásicos fallan.

3.6.2 Greedy con arrepentimiento (Moore-Hodgson con Max-Heap)

- **Cuándo usar:** Maximizar la cantidad de tareas completadas a tiempo cuando cada tarea tiene duración $t_i \geq 0$ y fecha límite d_i.
- **Cuándo no usar / alternativa:** Si cada tarea tiene una ganancia distinta w_i además de t_i y d_i, requiere DP tipo Knapsack sobre el tiempo ordenado por d_i.
- **Contrato:** Ordena por `deadline` d_i ascendente y mantiene las duraciones elegidas en un max-heap; si el tiempo acumulado supera d_i, descarta la tarea aceptada más larga (`pq.top()`).
- **Complejidad:** $\mathcal{O}(N \log N)$ tiempo y $\mathcal{O}(N)$ memoria.

```
// 2. Regret Greedy con Max-Heap (Moore-Hodgson):
//    tasks[i] = {duracion t_i, deadline d_i}.
int schedule_deadlines_heap(vector<pll> tasks) {
  sort(all(tasks), [](const pll& a, const pll& b) {
    return a.se < b.se;
  });
  priority_queue<ll> pq;
  ll cur_time = 0;
  for (auto [t, d] : tasks) {
    cur_time += t;
    pq.push(t);
    if (cur_time > d) {
      cur_time -= pq.top();
      pq.pop();
    }
  }
  return sz(pq);
}
```

- **Trampas y adaptaciones:** (1) El acumulador `cur_time` debe ser `ll` para no desbordar `int` cuando $t_i \leq 10^9$.

3.6.3 Argumento de Intercambio de Smith (`exchange_argument_order`)

- **Cuándo usar:** Ordenar tareas con duración t_i y penalización por unidad de tiempo w_i para minimizar $\sum w_i C_i$, donde C_i es el tiempo de finalización acumulado.
- **Cuándo no usar / alternativa:** Si hay precedencias (DAG de dependencias generales), el orden plano por t_i/w_i no respeta topología (en árbol requiere fusión con DSU + heap).
- **Contrato:** Compara dos tareas a, b con producto cruzado en 128 bits `(i128)a.t * b.w < (i128)b.t * a.w` sin usar punto flotante. Retorna `{costo_minimo, orden_indices}` .
- **Complejidad:** $\mathcal{O}(N \log N)$ tiempo y $\mathcal{O}(N)$ memoria.

```
// 3. Argumento de intercambio: minimizar suma(w_i * C_i)
//    Ordena por menor razon t_i / w_i sin punto flotante.
struct JobEx { int id; ll t, w; };
vector<JobEx> exchange_argument_order(vector<JobEx> jobs) {
  sort(all(jobs), [](const JobEx& a, const JobEx& b) {
    __int128 lhs = (__int128)a.t * b.w;
    __int128 rhs = (__int128)b.t * a.w;
    return lhs != rhs ? lhs < rhs : a.id < b.id;
  });
  return jobs;
}
```

- **Trampas y adaptaciones:** (1) Tratar $w_i = 0, t_i > 0$ enviándolos al final; (2) el comparador debe usar `<` estricto (nunca `<=`) para no producir UB en `std::sort` .

3.7 Barrido de Eventos / Intervalos 1D (Sweep-Line)

3.7.1 Máximo Solapamiento de Intervalos por Sweep-Line (`max_interval_overlap`)

- **Cuándo usar:** Hallar la máxima cantidad de intervalos simultáneamente activos y el primer punto donde se alcanza dicho máximo.
- **Cuándo no usar / alternativa:** Si necesitas actualizar y consultar solapamientos de rangos dinámicamente con inserciones y borrados intercalados sobre coordenadas fijas, usar **Lazy Segment Tree** (§4.5.1).
- **Contrato:** Si `closed_ends = true` (intervalos cerrados $[l, r]$), la entrada `+1` procesa **antes** que la salida `-1` en la misma coordenada; si `false` (semi-abiertos $[l, r)$), la salida `-1` procesa **antes** que la entrada `+1` . Retorna `{max_solapamiento, coordenada}` .
- **Complejidad:** $\mathcal{O}(N \log N)$ tiempo y $\mathcal{O}(N)$ memoria.

```
// 1. Maxima cantidad de intervalos solapados en un punto
//    closed_ends = true para [l, r]; false para [l, r).
int max_interval_overlap(const vector<pll>& ivs,
                         bool closed_ends = true) {
  vector<pair<ll, int>> ev;
  ev.reserve(2 * sz(ivs));
  for (auto [l, r] : ivs) {
    ev.pb({l, +1});
    ev.pb({r, -1});
  }
  sort(all(ev), [&](const auto& a, const auto& b) {
    if (a.fi != b.fi) return a.fi < b.fi;
    // En [l, r] entra (+1) antes de salir (-1);
    // en [l, r) sale (-1) antes de entrar (+1).
    return closed_ends ? (a.se > b.se) : (a.se < b.se);
  });
  int cur = 0, best = 0;
  for (auto [x, d] : ev) {
    cur += d;
    best = max(best, cur);
  }
  return best;
}
```

- **Trampas y adaptaciones:** (1) El orden de desempate en `a.fi == b.fi` entre eventos de apertura y cierre es la causa #1 de WA en sweep-line.

3.7.2 Unión de Intervalos Cerrados y Longitud Cubierta (`merge_intervals`)

- **Cuándo usar:** Colapsar una lista de intervalos cerrados $[l_i, r_i]$ posiblemente solapados en una lista ordenada de intervalos disjuntos y medir la longitud continua cubierta.
- **Cuándo no usar / alternativa:** Si buscas intersección de todos los intervalos en vez de su unión, basta `[max(l_i), min(r_i)]` en $\mathcal{O}(N)$.
- **Contrato:** Fusiona intervalos cuando `ivs[i].fi <= res.back().se` . Retorna `{intervalos_unidos, longitud_continua_total}` donde cada intervalo aporta `r - l` .
- **Complejidad:** $\mathcal{O}(N \log N)$ tiempo y $\mathcal{O}(N)$ memoria.

```
// 2. Union de intervalos cerrados [l, r] y longitud
// cubierta
pair<vector<pll>, ll> merge_intervals(vector<pll> ivs) {
  if (ivs.empty()) return {{}, 0};
  sort(all(ivs));
  vector<pll> res = {ivs[0]};
  for (int i = 1; i < sz(ivs); ++i) {
    if (ivs[i].fi <= res.back().se)
      res.back().se = max(res.back().se, ivs[i].se);
    else
      res.pb(ivs[i]);
  }
  ll total_len = 0;
  for (auto [l, r] : res) total_len += (r - l);
  return {res, total_len};
}
```

- **Trampas y adaptaciones:** (1) Si se cuenta la cantidad de **puntos enteros** cubiertos por intervalos cerrados $[l, r]$, cada intervalo aporta `(r - l + 1)` y dos intervalos contiguos `[1, 2]` y `[3, 4]` pueden fusionarse con `ivs[i].fi <= res.back().se + 1` .

Bloque 4: Estructuras de Datos Avanzadas (Consultas en Rango y Árboles)

4.1 Disjoint Set Union (DSU Clásico) y DSU con Rollback + Conectividad Dinámica Offline

4.1.1 Disjoint Set Union Clásico (`DSU`)

- **Cuándo usar:** Mantener componentes conexas al agregar aristas, consultar si dos nodos pertenecen al mismo componente (`same(u, v)`), conocer el tamaño del componente (`get_size(u)`) o el total de componentes (`num_comp`).
- **Cuándo no usar / alternativa:** Si el problema también **elimina** aristas, el DSU clásico con compresión de caminos no puede deshacer uniones; usar `RollbackDSU` + `DynCon` (§4.1.2) o `LinkCut` (§4.9).
- **Contrato:** Nodos `1..N` (también admite `0` porque reserva `n + 1`). `unite(u, v)` retorna `true` si unió dos componentes distintos y `false` si ya estaban unidos.
- **Complejidad:** Construcción $\mathcal{O}(N)$; `find` , `unite` , `same` , `get_size` en $\mathcal{O}(\alpha(N))$ amortizado; $\mathcal{O}(N)$ memoria.

```
// 1. DSU Clasico (Path Compression + Union by Size)
struct DSU {
  vi parent, sz;
  int num_comp;
  DSU(int n) : parent(n + 1), num_comp(n) {
    sz.assign(n + 1, 1);
    iota(all(parent), 0);
  }
  int find(int i) {
    return (parent[i] == i) ? i : (parent[i] = find(
      parent[i]));
  }
  bool unite(int i, int j) {
    int ri = find(i), rj = find(j);
    if (ri == rj) return false;
    if (sz[ri] < sz[rj]) swap(ri, rj);
    parent[rj] = ri;
    sz[ri] += sz[rj];
    num_comp--;
    return true;
  }
  bool same(int i, int j) { return find(i) == find(j); }
  int get_size(int i) { return sz[find(i)]; }
};
```

- **Trampas y adaptaciones:** (1) Acceder siempre a las propiedades del componente a través de su representante `dsu.find(u)` (nunca `sz[u]` directo); (2) `num_comp` se inicializa en `n` (si usas nodos `1..n` , ignora el índice `0`).

4.1.2 DSU con Rollback y Conectividad Dinámica Offline (`RollbackDSU` , `DynCon`)

- **Cuándo usar:** Responder el número de componentes conexas (o propiedades de DSU) tras una secuencia de **inserciones y eliminaciones de aristas** conocidas de antemano (offline) dividiendo en el árbol de intervalos de tiempo.
- **Cuándo no usar / alternativa:** Si las operaciones son estrictamente **online** (forzadas por codificación de la respuesta anterior), usar **Link-Cut Tree** (§4.9) cuando el grafo se mantiene acíclico.
- **Contrato:** Nodos `1..N` . En `DynCon(n)` , `add_edge(u, v)` agrega una arista activa y `rem_edge(u, v)` elimina una arista **actualmente activa**; `add_query()` registra una consulta y `solve()` retorna el vector de respuestas en orden cronológico.
- **Complejidad:** `RollbackDSU` : `unite` / `find` en $\mathcal{O}(\log N)$ (sin compresión de caminos), `rollback` en $\mathcal{O}(1)$ por unión revertida. `DynCon::solve()` : $\mathcal{O}(Q \log Q \log N)$ tiempo y $\mathcal{O}(N + Q)$ memoria.

```
// 2. DSU con Rollback + Dynamic Connectivity Offline
struct RollbackDSU {
  vi p, r, hist;
  int comp;
  RollbackDSU(int n) : p(n + 1), r(n + 1, 1), comp(n) {
    iota(all(p), 0);
  }
  int find(int i) const {
    return (p[i] == i) ? i : find(p[i]); }
  void unite(int i, int j) {
    i = find(i); j = find(j);
    if (i == j) return;
    if (r[i] > r[j]) swap(i, j);
    p[i] = j; r[j] += r[i];
    hist.pb(i); comp--;
  }
  int snapshot() const { return sz(hist); }
  void rollback(int snap) {
    while (sz(hist) > snap) {
      int i = hist.back(); hist.pop_back();
      r[p[i]] -= r[i]; p[i] = i; comp++;
    }
  }
};

struct DynCon {
  enum { ADD, DEL, QUERY };
  struct Ev { int type, u, v; };
  vector<Ev> q;
  RollbackDSU dsu;
  vi mt, ans;
  map<pii, int> active;

  DynCon(int n) : dsu(n) {}
  void add_edge(int u, int v) {
    if (u > v) swap(u, v);
    active[{u, v}] = sz(q);
    q.pb({ADD, u, v}); mt.pb(-1);
  }
  void rem_edge(int u, int v) {
    if (u > v) swap(u, v);
    int pr = active[{u, v}];
```

```
    mt[pr] = sz(q);
    q.pb({DEL, u, v}); mt.pb(pr);
  }
  void add_query() { q.pb({QUERY, -1, -1}); mt.pb(-1); }

  void go(int l, int r) {
    if (l + 1 == r) {
      if (q[l].type == QUERY) ans.pb(dsu.comp);
      return;
    }
    int snap = dsu.snapshot(), mid = (l + r) / 2;
    for (int i = mid; i < r; ++i)
      if (mt[i] != -1 && mt[i] < l) dsu.unite(q[i].u,
        q[i].v);
    go(l, mid); dsu.rollback(snap);
    for (int i = l; i < mid; ++i)
      if (mt[i] >= r) dsu.unite(q[i].u, q[i].v);
    go(mid, r); dsu.rollback(snap);
  }
  vi solve() {
    if (q.empty()) return {};
    for (int i = 0; i < sz(q); ++i)
      if (q[i].type == ADD && mt[i] == -1) mt[i] = sz(q);
    go(0, sz(q));
    return ans;
  }
};
```

- **Trampas y adaptaciones:** (1) No agregar compresión de caminos a `RollbackDSU::find`, porque destruiría el historial de `rollback`; (2) si pueden insertarse aristas paralelas idénticas, guardar un `vector<int>` de tiempos de inicio en `active[{u, v}]` en vez de un solo `int`.

4.2 Sparse Table (RMQ e Idempotentes en $\mathcal{O}(1)$)

- **Cuándo usar:** Responder muchas consultas ($Q \leq 10^6$) de **Mínimo, Máximo, GCD, AND u OR** en un rango $[L, R]$ sobre un arreglo **100% estático** en $\mathcal{O}(1)$ puro.
- **Cuándo no usar / alternativa:** **No funciona** en $\mathcal{O}(1)$ para operaciones **no idempotentes** como la **suma** o **XOR** porque los dos bloques de longitud 2^j se solapan (usar `PrefixSum` §3.4.1). Si hay modificaciones en el arreglo, usar `IterSegTree` (§4.4.1).
- **Contrato:** Arreglo y consultas **0-indexed** ($0 \leq L \leq R < N$, precondición: `N >= 1`). Cambiar `op(a, b)` a `max(a, b)` o `std::gcd(a, b)` según el problema.
- **Complejidad:** Construcción $\mathcal{O}(N \log N)$ tiempo y memoria; consulta `query(L, R)` en $\mathcal{O}(1)$.

```
struct SparseTable {
  int n, K;
  vector<vll> st;
  vi lg;

  ll op(ll a, ll b) const {
    // Cambiar a max(a, b) o std::gcd(a, b)
    return min(a, b);
  }

  SparseTable(const vll& a) {
    n = sz(a);
    K = __lg(max(1, n)) + 1;
    st.assign(K, vll(n));
    lg.assign(n + 1, 0);
    for (int i = 2; i <= n; ++i) lg[i] = lg[i / 2] + 1;
    st[0] = a;
    for (int j = 1; j < K; ++j)
      for (int i = 0; i + (1 << j) <= n; ++i)
        st[j][i] = op(st[j - 1][i], st[j - 1][i + (1 << (
          j - 1))]);
  }

  // Consulta en rango 0-indexed [L, R] inclusivo en O(1)
  ll query(int L, int R) const {
    int j = lg[R - L + 1];
    return op(st[j][L], st[j][R - (1 << j) + 1]);
  }
};
```

- **Trampas y adaptaciones:** (1) Verificar siempre `L <= R` antes de llamar a `query(L, R)` (de lo contrario `R - L + 1 <= 0` indexa fuera de `lg`).

4.3 Fenwick Tree / BIT 1D (con Binary Lifting `lower_bound`) y Fenwick Tree 2D

4.3.1 Fenwick Tree / BIT 1D con Binary Lifting (`Fenwick`)

- **Cuándo usar:** Actualizaciones puntuales (`add(idx, delta)`) y sumas de rango (`query(1, r)`) dinámicas con constante mínima, o hallar en $\mathcal{O}(\log N)$ el primer índice cuyo prefijo suma $\geq K$ (`lower_bound(k)`).
- **Cuándo no usar / alternativa:** No sirve para mínimo/máximo en rango arbitrario $[L, R]$ con actualizaciones arbitrarias (usar `IterSegTree` §4.4.1). `lower_bound(k)` exige que todos los valores almacenados sean **no negativos**.
- **Contrato:** Indexación estrictamente **1-based** (`1 <= idx <= n`). `lower_bound(k)` retorna el menor índice $p \in [1, n]$ con $\text{query}(p) \geq k$, o `n + 1` si la suma total es $< k$.
- **Complejidad:** Construcción $\mathcal{O}(N)$; `add`, `query`, `lower_bound` en $\mathcal{O}(\log N)$ tiempo; $\mathcal{O}(N)$ memoria.

```
// 1. Fenwick Tree 1D (1-indexed: 1 <= idx <= n)
struct Fenwick {
  int n;
  vll bit;
  Fenwick(int n) : n(n), bit(n + 1, 0) {}

  void add(int idx, ll delta) {
    for (; idx <= n; idx += idx & -idx) bit[idx] += delta;
  }
  ll query(int idx) const { // Suma en [1..idx]
    ll sum = 0;
    for (; idx > 0; idx -= idx & -idx) sum += bit[idx];
    return sum;
  }
  ll query(int l, int r) const { // Suma en [l..r]
    return (l > r) ? 0 : query(r) - query(l - 1);
  }
  // Menor indice idx tal que query(1..idx) >= target
  // (valores >= 0)
  int lower_bound(ll target) const {
    int idx = 0;
    ll cur = 0;
    for (int pw = 1 << __lg(max(1, n)); pw > 0; pw >>= 1) {
      if (idx + pw <= n && cur + bit[idx + pw] < target) {
        idx += pw;
        cur += bit[idx];
      }
    }
    return idx + 1;
  }
};
```

- **Trampas y adaptaciones:** (1) **Jamás** llamar `ft.add(0, delta)`: como `0 & -0 == 0`, produce un **bucle infinito**; si tus datos son 0-based, suma `+ 1` al índice.

4.3.2 Fenwick Tree 2D (`Fenwick2D`)

- **Cuándo usar:** Actualizaciones puntuales `add(r, c, delta)` y consultas de suma en submatrices `[r1..r2] x [c1..c2]` dinámicas cuando $N, M \leq 2500$.
- **Cuándo no usar / alternativa:** Si $N, M = 10^5$ la matriz $N \times M$ da MLE; usar compresión 2D offline o CDQ Divide & Conquer.
- **Contrato:** Coordenadas **1-based** (`1 <= r <= n, 1 <= c <= m`).
- **Complejidad:** $\mathcal{O}(\log N \log M)$ por `add` y `query`; $\mathcal{O}(N \cdot M)$ memoria.

```
// 2. Fenwick Tree 2D (1-indexed: 1 <= r <= n, 1 <= c <= m)
struct Fenwick2D {
  int n, m;
  vector<vll> bit;
  Fenwick2D(int n, int m) : n(n), m(m), bit(n + 1, vll(
    m + 1, 0)) {}

  void add(int r, int c, ll delta) {
    for (int i = r; i <= n; i += i & -i)
      for (int j = c; j <= m; j += j & -j)
        bit[i][j] += delta;
  }
  ll query(int r, int c) const { // Suma en [1..r][1..c]
    ll sum = 0;
    for (int i = r; i > 0; i -= i & -i)
      for (int j = c; j > 0; j -= j & -j)
        sum += bit[i][j];
    return sum;
  }
  ll query(int r1, int c1, int r2, int c2) const {
    return query(r2, c2) - query(r1 - 1, c2)
         - query(r2, c1 - 1) + query(r1 - 1, c1 - 1);
  }
};
```

- **Trampas y adaptaciones:** (1) Verificar que $r, c \geq 1$ antes de llamar a `add(r, c, delta)` para evitar bucle infinito en 0.

4.4 Segment Tree Clásico (Recursivo) y Segment Tree Iterativo

4.4.1 Segment Tree Iterativo (`IterSegTree`)

- **Cuándo usar:** Actualizaciones puntuales (`a[p] = val`) y consultas en rango `[l, r]` de cualquier operación asociativa (suma, min, max, gcd) con mitad de memoria (`2*N`) y $3\times$ menor constante que el recursivo.
- **Cuándo no usar / alternativa:** Si la operación no es conmutativa (p. ej. multiplicación de matrices o concatenación de hashes), en el bucle `query` cuidar el orden `res_left = op(res_left, tree[l++])` y `res_right = op(tree[--r], res_right)` (o usar `SegTree` recursivo §4.4.2).
- **Contrato:** Indexación **0-based** (`0 <= p < n`, consulta **inclusiva** `query(l, r)` en `[l, r]`). Ajustar `NEUT` (`0` para suma/gcd, `INF` para min, `-INF` para max) y `op(a, b)`.
- **Complejidad:** Construcción $\mathcal{O}(N)$; `update` y `query` en $\mathcal{O}(\log N)$ tiempo; $\mathcal{O}(N)$ memoria.

```
// 1. Segment Tree Iterativo (0-indexed, ocupa solo 2*N de
// memoria)
struct IterSegTree {
  int n;
  // INF para min, -INF para max, 0 para suma
  const ll NEUTRAL = INF;
  vll t;
  ll op(ll a, ll b) const { return min(a, b); }

  IterSegTree(const vll& a) : n(sz(a)), t(2 * sz(a),
    NEUTRAL) {
    for (int i = 0; i < n; ++i) t[n + i] = a[i];
    for (int i = n - 1; i >= 1; --i)
      t[i] = op(t[i << 1], t[i << 1 | 1]);
  }
  // Asigna a[p] = val (0-indexed)
  void update(int p, ll val) {
    for (t[p += n] = val; p > 1; p >>= 1)
      t[p >> 1] = op(t[p], t[p ^ 1]);
  }
  // Rango CERRADO 0-indexed [l, r]
  ll query(int l, int r) const {
    ll vl = NEUTRAL, vr = NEUTRAL;
    for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
      if (l & 1) vl = op(vl, t[l++]);
      if (r & 1) vr = op(t[--r], vr);
    }
    return op(vl, vr);
  }
};
```

- **Trampas y adaptaciones:** (1) Aunque internamente el bucle convierte a semi-abierto `[l, r + 1)`, la interfaz pública `query(l, r)` recibe un rango **cerrado** `[l, r]` 0-based.

4.4.2 Segment Tree Recursivo Clásico (`SegTree`)

- **Cuándo usar:** Consultas y actualizaciones puntuales cuando la operación es **no conmutativa** (matrices, subarreglo de suma máxima en rango) o cuando necesitas hacer búsqueda binaria dentro del árbol (*Segment Tree walk*).
- **Cuándo no usar / alternativa:** Si también hay actualizaciones en rango `[l, r]`, usar `LazySegTree` (§4.5.1).
- **Contrato:** Indexación **0-based** (`0 <= l <= r < n`).Reserva `4 * n` nodos.
- **Complejidad:** Construcción $\mathcal{O}(N)$; `update` y `query` en $\mathcal{O}(\log N)$; memoria $\mathcal{O}(4N)$.

```
// 2. Segment Tree Recursivo Clasico (0-indexed)
struct SegTree {
  int n;
  const ll NEUTRAL = 0;
  vll t;
  ll merge(ll a, ll b) const { return a + b; }

  SegTree(const vll& a) : n(sz(a)), t(4 * sz(a), NEUTRAL) {
    if (n > 0) build(1, 0, n - 1, a);
  }
  void build(int v, int l, int r, const vll& a) {
    if (l == r) { t[v] = a[l]; return; }
    int m = (l + r) / 2;
    build(v * 2, l, m, a);
    build(v * 2 + 1, m + 1, r, a);
    t[v] = merge(t[v * 2], t[v * 2 + 1]);
  }
```

```
  void update(int v, int l, int r, int pos, ll val) {
    if (l == r) { t[v] = val; return; }
    int m = (l + r) / 2;
    if (pos <= m) update(v * 2, l, m, pos, val);
    else update(v * 2 + 1, m + 1, r, pos, val);
    t[v] = merge(t[v * 2], t[v * 2 + 1]);
  }
  ll query(int v, int l, int r, int ql, int qr) const {
    if (qr < l || r < ql) return NEUTRAL;
    if (ql <= l && r <= qr) return t[v];
    int m = (l + r) / 2;
    return merge(query(v * 2, l, m, ql, qr),
                 query(v * 2 + 1, m + 1, r, ql, qr));
  }
  void update(int pos, ll val) {
    update(1, 0, n - 1, pos, val); }
  ll query(int ql, int qr) const {
    return query(1, 0, n - 1, ql, qr); }
};
```

- **Trampas y adaptaciones:** (1) Nunca dimensionar el vector del árbol recursivo como `2 * n` (cuando `n` no es potencia de 2 puede acceder hasta casi `4 * n`); usar siempre `4 * n`.

4.5 Segment Tree con Lazy Propagation, Segment Tree 2D y Persistent Segment Tree

4.5.1 Segment Tree con Lazy Propagation (`LazySegTree`)

- **Cuándo usar:** Actualizaciones en rango (`add_range(l, r, val)`) y consultas en rango (`query_sum(l, r)` , `query_min(l, r)`) intercaladas dinámicamente.
- **Cuándo no usar / alternativa:** Si todas las actualizaciones de rango ocurren antes de cualquier consulta, basta un `DiffArray1D` + `PrefixSum` (§3.5.1) en $\mathcal{O}(N)$.
- **Contrato:** Indexación **0-based** (`0 <= l <= r < n`). `apply(node, l, r, v)` suma `v * (r - l + 1)` a `sum` y `v` a `mn` y `lazy` .
- **Complejidad:** Construcción $\mathcal{O}(N)$; `add_range` , `query_sum` , `query_min` en $\mathcal{O}(\log N)$; memoria $\mathcal{O}(4N)$.

```
// 1. Segment Tree con Lazy Propagation (Suma en rango +
// Query Suma y Min)
struct LazySegTree {
  int n;
  vll sum_t, min_t, lazy;

  LazySegTree(const vll& a)
    : n(sz(a)), sum_t(4 * n, 0), min_t(4 * n, 0), lazy(
      4 * n, 0) {
    if (n > 0) build(1, 0, n - 1, a);
  }
  void apply(int v, int l, int r, ll val) {
    sum_t[v] += val * (r - l + 1);
    min_t[v] += val;
    lazy[v] += val;
  }
  void push(int v, int l, int r) {
    if (lazy[v] != 0) {
      int m = (l + r) / 2;
      apply(v * 2, l, m, lazy[v]);
      apply(v * 2 + 1, m + 1, r, lazy[v]);
      lazy[v] = 0;
    }
  }
  void pull(int v) {
    sum_t[v] = sum_t[v * 2] + sum_t[v * 2 + 1];
    min_t[v] = min(min_t[v * 2], min_t[v * 2 + 1]);
  }
  void build(int v, int l, int r, const vll& a) {
    if (l == r) { sum_t[v] = min_t[v] = a[l]; return; }
    int m = (l + r) / 2;
    build(v * 2, l, m, a);
    build(v * 2 + 1, m + 1, r, a);
    pull(v);
  }
  void add_range(int v, int l, int r, int ql, int qr,
    ll val) {
    if (qr < l || r < ql) return;
    if (ql <= l && r <= qr) { apply(v, l, r, val); return; }
    push(v, l, r);
    int m = (l + r) / 2;
    add_range(v * 2, l, m, ql, qr, val);
    add_range(v * 2 + 1, m + 1, r, ql, qr, val);
    pull(v);
  }
  ll query_sum(int v, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return 0;
    if (ql <= l && r <= qr) return sum_t[v];
    push(v, l, r);
    int m = (l + r) / 2;
    return query_sum(v * 2, l, m, ql, qr)
         + query_sum(v * 2 + 1, m + 1, r, ql, qr);
  }
  ll query_min(int v, int l, int r, int ql, int qr) {
    if (qr < l || r < ql) return INF;
    if (ql <= l && r <= qr) return min_t[v];
    push(v, l, r);
    int m = (l + r) / 2;
    return min(query_min(v * 2, l, m, ql, qr),
               query_min(v * 2 + 1, m + 1, r, ql, qr));
  }
  void add_range(int ql, int qr, ll val) {
    add_range(1, 0, n - 1, ql, qr, val);
  }
  ll query_sum(int ql, int qr) {
    return query_sum(1, 0, n - 1, ql, qr); }
  ll query_min(int ql, int qr) {
    return query_min(1, 0, n - 1, ql, qr); }
};
```

- **Trampas y adaptaciones:** (1) Para soportar asignación en rango (`a[l..r] = v`) además de suma, mantener un booleano `has_set` y al aplicar un `set` limpiar el `lazy` de suma previo.

4.5.2 Segment Tree 2D Iterativo (`SegTree2D`)

- **Cuándo usar:** Actualizaciones puntuales `update(x, y, val)` (reemplazo) y consultas rectangulares `[x0..x1] x [y0..y1]` de operaciones como suma, `min` , `max` o `gcd` donde Fenwick 2D no aplica por no ser invertible.
- **Cuándo no usar / alternativa:** Si la operación es suma (`a[x][y] += delta`), `Fenwick2D` (§4.3.2) usa $4\times$ menos memoria y código.
- **Contrato:** Coordenadas **0-based** (`0 <= x0 <= x1 < n` , `0 <= y0 <= y1 < m`).
- **Complejidad:** Construcción $\mathcal{O}(N \cdot M)$; `update` y `query` en $\mathcal{O}(\log N \log M)$; memoria $\mathcal{O}(4NM)$.

```
// 2. Segment Tree 2D Iterativo (0-indexed:
// [x0..x1][y0..y1])
struct SegTree2D {
  int n, m;
  const ll NEUTRAL = 0;
  vector<vll> st;
  ll op(ll a, ll b) const { return a + b; }

  SegTree2D(const vector<vll>& a)
    : n(sz(a)), m(n ? sz(a[0]) : 0), st(2 * n, vll(2 * m,
      0)) {
    for (int i = 0; i < n; ++i)
      for (int j = 0; j < m; ++j) st[i + n][j + m]
        = a[i][j];
    for (int i = 0; i < n; ++i)
      for (int j = m - 1; j > 0; --j)
        st[i + n][j] = op(st[i + n][j << 1],
          st[i + n][j << 1 | 1]);
    for (int i = n - 1; i > 0; --i)
      for (int j = 0; j < 2 * m; ++j)
        st[i][j] = op(st[i << 1][j], st[i << 1 | 1][j]);
  }
  void update(int x, int y, ll v) {
    st[x + n][y + m] = v;
    for (int j = y + m; j > 1; j >>= 1)
      st[x + n][j >> 1] = op(st[x + n][j],
        st[x + n][j ^ 1]);
    for (int i = x + n; i > 1; i >>= 1)
      for (int j = y + m; j > 0; j >>= 1)
        st[i >> 1][j] = op(st[i][j], st[i ^ 1][j]);
  }
  ll query(int x0, int x1, int y0, int y1) const {
    ll res = NEUTRAL;
    for (int i0 = x0 + n,
      i1 = x1 + n + 1; i0 < i1; i0 >>= 1, i1 >>= 1) {
      int rows[2], q = 0;
      if (i0 & 1) rows[q++] = i0++;
      if (i1 & 1) rows[q++] = --i1;
      for (int k = 0; k < q; ++k)
        for (int j0 = y0 + m,
          j1 = y1 + m + 1; j0 < j1; j0 >>= 1, j1 >>= 1) {
          if (j0 & 1) res = op(res, st[rows[k]][j0++]);
          if (j1 & 1) res = op(res, st[rows[k]][--j1]);
        }
    }
    return res;
  }
};
```

- **Trampas y adaptaciones:** (1) `update(x, y, val)` **reemplaza** el valor de la celda `(x, y)` por `val` ; si quieres sumar `delta` , hacer `t[x + n][y + m] += val` .

4.5.3 Persistent Segment Tree de Sumas en Versiones Históricas (`PersistentST`)

- **Cuándo usar:** Clonar el arreglo en el tiempo al hacer actualizaciones puntuales (`create_version`) y consultar la suma en `[ql, qr]` sobre cualquier versión histórica en $\mathcal{O}(\log N)$.
- **Cuándo no usar / alternativa:** Si necesitas hallar el k**-ésimo menor elemento en un subarreglo** $[L, R]$, esta estructura de sumas por posición no basta: usar `PersistentFreqST` (§4.5.4).
- **Contrato:** Índices **0-based** (`0 <= pos < n`). `roots[0]` es la versión inicial; `create_version(ver, pos, val)` crea una nueva raíz en `roots.back()` y retorna su ID de versión.
- **Complejidad:** Construcción $\mathcal{O}(N)$; cada actualización crea $\mathcal{O}(\log N)$ nodos nuevos en tiempo y memoria.

```
// 3. Persistent Segment Tree (Suma en versiones historicas)
struct PersistentST {
  struct Node { ll val; int l, r; };
  int n;
  vector<Node> t;
  vi roots;

  PersistentST(const vll& a) : n(sz(a)) {
    t.reserve(n * 22);
    roots.pb(build(0, n - 1, a));
  }
  int build(int tl, int tr, const vll& a) {
    int v = sz(t); t.pb({0, 0, 0});
    if (tl == tr) { t[v].val = a[tl]; return v; }
    int tm = (tl + tr) / 2;
    int left = build(tl, tm, a);
    int right = build(tm + 1, tr, a);
    t[v] = {t[left].val + t[right].val, left, right};
    return v;
  }
  int update(int prv, int tl, int tr, int pos, ll new_val) {
    int v = sz(t); t.pb(t[prv]);
    if (tl == tr) { t[v].val = new_val; return v; }
    int tm = (tl + tr) / 2;
    if (pos <= tm) {
      int nl = update(t[prv].l, tl, tm, pos, new_val);
      t[v].l = nl;
    } else {
      int nr = update(t[prv].r, tm + 1, tr, pos, new_val);
      t[v].r = nr;
    }
    t[v].val = t[t[v].l].val + t[t[v].r].val;
    return v;
  }
  ll query(int v, int tl, int tr, int ql, int qr) const {
    if (qr < tl || tr < ql) return 0;
    if (ql <= tl && tr <= qr) return t[v].val;
    int tm = (tl + tr) / 2;
    return query(t[v].l, tl, tm, ql, qr)
         + query(t[v].r, tm + 1, tr, ql, qr);
  }
  // Crea una nueva version a partir de 'ver' y retorna su
  // ID
  int create_version(int ver, int pos, ll val) {
    int new_root = update(roots[ver], 0, n - 1, pos, val);
    roots.pb(new_root);
    return sz(roots) - 1;
  }
  ll query_version(int ver, int ql, int qr) const {
    return query(roots[ver], 0, n - 1, ql, qr);
  }
};
```

- **Trampas y adaptaciones:** (1) Con Q actualizaciones se crean $\approx Q(\lceil \log_2 N \rceil + 2)$ nodos; `reserve` evita realocaciones.

4.5.4 Persistent Segment Tree de Frecuencias: k-ésimo en Subarreglo (`PersistentFreqST`)

- **Cuándo usar:** Responder online cuál es el k**-ésimo menor valor** en cualquier subarreglo estático `a[l..r]` (0-indexed, $k \geq 1$) en $\mathcal{O}(\log N)$.
- **Cuándo no usar / alternativa:** Si el arreglo también sufre modificaciones puntuales entre consultas de k-ésimo, un árbol persistente de prefijos estático no basta (requiere Fenwick de Segment Trees dinámicos en $\mathcal{O}(\log^2 N)$).
- **Contrato:** Arreglo `a` **0-indexed** (`0 <= l <= r < n`) y orden `1 <= k <= r - l + 1` . Comprime coordenadas internamente y devuelve el valor real `vals[idx]` .
- **Complejidad:** Construcción $\mathcal{O}(N \log N)$ tiempo y memoria; `query_kth(l, r, k)` en $\mathcal{O}(\log N)$.

```
// 4. Persistente de Frecuencias: k-esimo menor en
// subarreglo [l..r] (0-indexed, k >= 1)
```

```cpp
struct PersistentFreqST {
  struct Node { int cnt, l, r; };
  int m; // Tamano del alfabeto de valores comprimidos
  vll vals;
  vector<Node> t;
  vi roots;

  PersistentFreqST(const vll& a) : vals(a) {
    sort(all(vals));
    vals.erase(unique(all(vals)), vals.end());
    m = max(1, sz(vals));
    t.reserve((sz(a) + 1) * 20);
    roots.pb(build(0, m - 1));
    for (ll x : a) {
      int pos = lower_bound(all(vals), x) - vals.begin();
      roots.pb(insert(roots.back(), 0, m - 1, pos));
    }
  }
  int build(int tl, int tr) {
    int v = sz(t); t.pb({0, 0, 0});
    if (tl == tr) return v;
    int tm = (tl + tr) / 2;
    int left = build(tl, tm), right = build(tm + 1, tr);
    t[v] = {0, left, right};
    return v;
  }
  int insert(int prv, int tl, int tr, int pos) {
    int v = sz(t); t.pb(t[prv]);
    t[v].cnt++;
    if (tl == tr) return v;
    int tm = (tl + tr) / 2;
    if (pos <= tm) { int nl = insert(t[prv].l, tl, tm, pos);
      t[v].l = nl; }
    else { int nr = insert(t[prv].r, tm + 1, tr, pos);
      t[v].r = nr; }
    return v;
  }
  int kth_idx(int vl, int vr, int tl, int tr, int k) const {
    if (tl == tr) return tl;
    int tm = (tl + tr) / 2;
    int left_cnt = t[t[vr].l].cnt - t[t[vl].l].cnt;
    if (k <= left_cnt) return kth_idx(t[vl].l, t[vr].l, tl,
      tm, k);
    return kth_idx(t[vl].r, t[vr].r, tm + 1, tr,
      k - left_cnt);
  }
  // Retorna el k-esimo menor valor (1-indexed: 1 <= k <= r
  // - l + 1) en a[l..r]
  ll query_kth(int l, int r, int k) const {
    return vals[kth_idx(roots[l], roots[r + 1], 0, m - 1,
      k)];
  }
};
```

- **Trampas y adaptaciones:** (1) Verificar siempre `1 <= k && k <= r - l + 1` antes de llamar a `query_kth(l, r, k)`.

4.6 SQRT Decomposition y Algoritmo de Mo (con Optimización de Bloques)

4.6.1 SQRT Decomposition en Bloques (`SqrtDecomp`)

- **Cuándo usar:** Actualizar elementos y responder consultas en rango cuando la estructura por bloques de tamaño $B \approx \sqrt{N}$ es mucho más simple de adaptar que un Segment Tree (p. ej., mantener bloques ordenados para contar $< X$).
- **Cuándo no usar / alternativa:** Para suma/mínimo estándar, `Fenwick` (§4.3.1) o `IterSegTree` (§4.4.1) son $\mathcal{O}(\log N)$ y más veloces.
- **Contrato:** Indexación **0-based** (`0 <= l <= r < n`). `B = sqrt(n) + 1`.
- **Complejidad:** Construcción $\mathcal{O}(N)$; `update` en $\mathcal{O}(1)$; `query` en $\mathcal{O}(\sqrt{N})$.

```cpp
// 1. SQRT Decomposition en Bloques (0-indexed)
struct SqrtDecomp {
  int n, B;
  vll a, b;
  SqrtDecomp(const vll& arr)
    : n(sz(arr)), B((int)sqrt(n) + 1), a(arr), b(B, 0) {
    for (int i = 0; i < n; ++i) b[i / B] += a[i];
  }
  void update(int pos, ll val) { // O(1)
    b[pos / B] += val - a[pos];
    a[pos] = val;
  }
  ll query(int l, int r) const { // O(sqrt(N))
    ll sum = 0;
    int bl = l / B, br = r / B;
    if (bl == br) {
      for (int i = l; i <= r; ++i) sum += a[i];
    } else {
      for (int i = l, end = (
        bl + 1) * B - 1; i <= end; ++i) sum += a[i];
      for (int i = bl + 1; i < br; ++i) sum += b[i];
      for (int i = br * B; i <= r; ++i) sum += a[i];
    }
    return sum;
  }
};
```

- **Trampas y adaptaciones:** (1) En lugar de iterar `i` y comprobar `i % B == 0`, manejar por separado `b_l == b_r` y luego los bordes más los bloques completos `b_l + 1 .. b_r - 1` es $3\times$ más rápido.

4.6.2 Algoritmo de Mo Offline (`MoQuery` , `solve_mo`)

- **Cuándo usar:** Responder Q consultas **offline** sobre rangos `[l, r]` cuando no hay forma sencilla de fusionar dos mitades en un Segment Tree, pero agregar o quitar 1 elemento en los extremos del rango toma $\mathcal{O}(1)$ (frecuencias, elementos distintos, pares iguales).
- **Cuándo no usar / alternativa:** Si las consultas son online o si agregar/quitar un elemento cuesta $\mathcal{O}(\log N)$ con $N, Q = 2 \cdot 10^5$ ($\mathcal{O}(N\sqrt{Q}\log N)$ suele dar TLE).
- **Contrato:** Arreglo `a` y consultas `queries` son **0-indexed** (`0 <= l <= r < n`). Los valores de `a[i]` deben estar en `[0, MAX_VAL - 1]` (comprimir coordenadas con `CoordComp` §2.1 si superan 10^6).
- **Complejidad:** $\mathcal{O}((N+Q)\sqrt{N})$ tiempo y $\mathcal{O}(N + Q + V_{\max})$ memoria.

```cpp
// 2. Algoritmo de Mo (Offline con ordenamiento serpenteante
// par/impar)
// ~ N / sqrt(Q) para N, Q <= 2e5
const int BLOCK_SIZE = 450;
struct MoQuery {
  int l, r, idx;
  bool operator<(const MoQuery& o) const {
    int b1 = l / BLOCK_SIZE, b2 = o.l / BLOCK_SIZE;
    if (b1 != b2) return b1 < b2;
    // Optimiza el puntero R
    return (b1 & 1) ? (r < o.r) : (r > o.r);
  }
};

// Ejemplo: Contar cantidad de elementos distintos en cada
// rango [l, r]
vll solve_mo(const vi& a, vector<MoQuery>& queries) {
  int max_val = *max_element(all(a));
  vi freq(max_val + 1, 0);
  int distinct_cnt = 0;

  auto add = [&](int idx) {
    if (++freq[a[idx]] == 1) distinct_cnt++;
  };
  auto del = [&](int idx) {
    if (--freq[a[idx]] == 0) distinct_cnt--;
  };

  sort(all(queries));
  vll ans(sz(queries));
  int cur_l = 0, cur_r = -1;

  // ¡IMPORTANTE! Primero expandir (cur_l--, cur_r++), luego
  // contraer
  for (const auto& q : queries) {
    while (cur_l > q.l) add(--cur_l);
    while (cur_r < q.r) add(++cur_r);
    while (cur_l < q.l) del(cur_l++);
    while (cur_r > q.r) del(cur_r--);
    ans[q.idx] = distinct_cnt;
  }
  return ans;
}
```

- **Trampas y adaptaciones:** (1) **Orden de los 4** `while` : siempre expandir (`--cur_l` , `++cur_r`) **antes** de contraer (`cur_l++` , `cur_r--`); de lo contrario `cur_l` puede cruzar a `cur_r + 2` y causar conteos negativos.

4.7 Treap (BST Aleatorizado) e Implicit Treap (Reversión y Suma en Rango)

- **Cuándo usar:** Mantener un arreglo dinámico que soporte **revertir** un subarreglo `[l, r]` , **sumar** una constante a `[l, r]` , **cortar y mover** bloques a otra posición y consultar la **suma** de `[l, r]` en $\mathcal{O}(\log N)$.
- **Cuándo no usar / alternativa:** Si no hay reversiones ni cortes/inserciones de posición (solo sumas y consultas en rango sobre índices fijos), usar `LazySegTree` (§4.5.1), que tiene constante mucho menor.
- **Contrato:** Posiciones **0-based** (`0 <= l <= r < cnt(root)`). Llamar siempre a `push(t)` al inicio de `split` y `merge` , y a `pull(t)` antes de retornar. Dependencias: `rng` (§1.2).
- **Complejidad:** $\mathcal{O}(\log N)$ tiempo esperado por operación; $\mathcal{O}(N)$ memoria.

```cpp
struct ImplicitTreap {
  struct Node {
    ll val, sum, lazy_add;
    int pr, cnt;
    bool rev;
    Node *l = nullptr, *r = nullptr;
    Node(ll v)
      : val(v), sum(v), lazy_add(0), pr( (int)rng() ), cnt(
        1), rev(false) {}
  };
  using pnode = Node*;
  pnode root = nullptr;

  int cnt(pnode t) const { return t ? t->cnt : 0; }
  ll sum(pnode t) const { return t ? t->sum : 0; }

  void apply_add(pnode t, ll add_v) {
    if (!t) return;
    t->val += add_v;
    t->sum += add_v * t->cnt;
    t->lazy_add += add_v;
  }
  void apply_rev(pnode t) {
    if (!t) return;
    t->rev ^= 1;
    swap(t->l, t->r);
  }
  void push(pnode t) {
    if (!t) return;
    if (t->rev) {
      apply_rev(t->l);
      apply_rev(t->r);
      t->rev = false;
    }
    if (t->lazy_add != 0) {
      apply_add(t->l, t->lazy_add);
      apply_add(t->r, t->lazy_add);
      t->lazy_add = 0;
    }
  }
  void pull(pnode t) {
    if (!t) return;
    t->cnt = 1 + cnt(t->l) + cnt(t->r);
    t->sum = t->val + sum(t->l) + sum(t->r);
  }
  // Divide 't' en 'l' (primeros k elementos) y 'r' (el
  // resto)
  void split(pnode t, pnode& l, pnode& r, int k) {
    if (!t) { l = r = nullptr; return; }
    push(t);
    if (cnt(t->l) >= k) {
      split(t->l, l, t->l, k);
      r = t;
    } else {
      split(t->r, t->r, r, k - cnt(t->l) - 1);
      l = t;
    }
    pull(t);
  }
  void merge(pnode& t, pnode l, pnode r) {
    push(l); push(r);
    if (!l || !r) t = l ? l : r;
    else if (l->pr > r->pr) { merge(l->r, l->r, r); t = l; }
    else { merge(r->l, l, r->l); t = r; }
    pull(t);
  }
  void push_back(ll val) { merge(root, root, new Node(val));
    }

  // Invierte el subarreglo 0-indexed [ql, qr]
  void reverse_range(int ql, int qr) {
    pnode t1, t2, t3;
    split(root, t1, t2, ql);
    split(t2, t2, t3, qr - ql + 1);
    apply_rev(t2);
    merge(root, t1, t2);
    merge(root, root, t3);
  }
  // Suma 'val' al subarreglo 0-indexed [ql, qr]
  void add_range(int ql, int qr, ll val) {
    pnode t1, t2, t3;
    split(root, t1, t2, ql);
```

```
    split(t2, t2, t3, qr - ql + 1);
    apply_add(t2, val);
    merge(root, t1, t2);
    merge(root, root, t3);
  }
  // Consulta suma en 0-indexed [ql, qr]
  ll query_sum(int ql, int qr) {
    pnode t1, t2, t3;
    split(root, t1, t2, ql);
    split(t2, t2, t3, qr - ql + 1);
    ll res = sum(t2);
    merge(root, t1, t2);
    merge(root, root, t3);
    return res;
  }
};
```

- **Trampas y adaptaciones:** (1) En `split(t, l, r, k)`, el parámetro `k` es la **cantidad de elementos** que quedan en el árbol izquierdo `l` (es decir, índices `0..k-1`), no un valor de llave.

4.8 Consultas en Árboles: Heavy-Light Decomposition (HLD) y DSU on Tree (Sack)

4.8.1 Heavy-Light Decomposition (`HLD`)

- **Cuándo usar:** Resolver consultas en **caminos** (`query_path(u, v)`) y en **subárboles** (`query_subtree(u)`) con actualizaciones dinámicas de nodos cuando la operación no es invertible (como `max`, `min`, `gcd`) o cuando se combinan caminos y subárboles.
- **Cuándo no usar / alternativa:** Si el árbol es estático y la operación es suma/XOR, basta `LCA` (§6.4.3) con sumas a la raíz; si solo hay updates en nodos y sumas en subárboles, basta Euler Tour + Fenwick (§0.3).
- **Contrato:** Nodos **1-based** (`1..n`), valores iniciales `vals[1..n]`. Esta implementación incluye `update_node(u, val)` puntual; para actualizar todo un camino `u..v`, reemplazar el `IterSegTree` interno por `LazySegTree` (§4.5.1) e invocar `st.add_range(pos[head[u]], pos[u], val)` en el bucle de subida. Dependencias: `IterSegTree` (§4.4.1).
- **Complejidad:** Construcción $\mathcal{O}(N \log N)$ (por las N llamadas iniciales a `st.update`); `update_node` y `query_subtree` en $\mathcal{O}(\log N)$; `query_path` en $\mathcal{O}(\log^2 N)$.

```
// 1. Heavy-Light Decomposition (HLD) + Segment Tree
// Iterativo
struct HLD {
  int n, timer = 0;
  vector<vi> adj;
  vi par, dep, heavy, head, pos, sz_sub;
  IterSegTree st;

  HLD(int n, int root, const vector<vi>& g, const vll& vals)
    : n(n), adj(g), par(n + 1), dep(n + 1, 0), heavy(n + 1,
      -1),
      head(n + 1), pos(n + 1), sz_sub(n + 1, 1), st(vll(n,
        0)) {
    par[root] = root;
    dfs_sz(root);
    dfs_hld(root, root);
    for (int u = 1; u <= n; ++u) st.update(pos[u], vals[u]);
  }
  void dfs_sz(int u) {
    int max_c = 0;
    for (int v : adj[u]) {
      if (v == par[u]) continue;
      par[v] = u;
      dep[v] = dep[u] + 1;
      dfs_sz(v);
      sz_sub[u] += sz_sub[v];
      if (sz_sub[v] > max_c) {
        max_c = sz_sub[v];
        heavy[u] = v;
      }
    }
  }
  void dfs_hld(int u, int h) {
    head[u] = h;
    pos[u] = timer++;
    if (heavy[u] != -1) dfs_hld(heavy[u], h);
    for (int v : adj[u])
      if (v != par[u] && v != heavy[u])
        dfs_hld(v, v);
  }
  void update_node(int u, ll val) { st.update(pos[u], val);
```

```
    }

  // Consulta en el camino entre u y v (minimo en este
  // ejemplo)
  ll query_path(int u, int v) const {
    ll res = st.NEUTRAL;
    for (; head[u] != head[v]; v = par[head[v]]) {
      if (dep[head[u]] > dep[head[v]]) swap(u, v);
      res = st.op(res, st.query(pos[head[v]], pos[v]));
    }
    if (dep[u] > dep[v]) swap(u, v);
    res = st.op(res, st.query(pos[u], pos[v]));
    // NOTA: Si los pesos estan en las ARISTAS (guardados en
    // el hijo),
    // cambiar la linea anterior por:
    // if (u != v) res = st.op(res, st.query(pos[u] + 1,
    // pos[v]));
    return res;
  }
  // Consulta en todo el subarbol de u en O(log N):
  ll query_subtree(int u) const {
    return st.query(pos[u], pos[u] + sz_sub[u] - 1);
  }
};
```

- **Trampas y adaptaciones:** (1) Si los valores están en las **aristas** en lugar de los nodos, asignar el peso de cada arista al nodo hijo (el más profundo) y en la última etapa de `query_path` consultar `st.query(pos[u] + 1, pos[v])` para excluir el LCA `u`.

4.8.2 DSU on Tree / Small-to-Large en Subárboles (`pre_sack` , `dfs_sack`)

- **Cuándo usar:** Responder consultas **offline** sobre todos los subárboles (p. ej., color más frecuente en el subárbol de u o cantidad de valores distintos) en $\mathcal{O}(N \log N)$ sin estructuras pesadas.
- **Cuándo no usar / alternativa:** Si hay actualizaciones de colores intercaladas con consultas online, Sack no aplica.
- **Contrato:** Nodos **1-based** (`1..n`). Llenar `adj_s[1..n]` y `col_s[1..n]`, llamar `pre_sack(1, 0)` y luego `dfs_sack(1, 0, false)`. Respuestas en `ans_sack[1..n]`.
- **Complejidad:** $\mathcal{O}(N \log N)$ tiempo total y $\mathcal{O}(N)$ memoria.

```
// 2. DSU on Tree (Sack) - Consultas de frecuencia en
// subarboles en O(N log N)
const int MAX_SACK = 200005;
vi adj_s[MAX_SACK];
int col_s[MAX_SACK], cnt_s[MAX_SACK], big_s[MAX_SACK];
int tin_s[MAX_SACK], tout_s[MAX_SACK], ver_s[MAX_SACK],
  t_sack = 0;

int pre_sack(int u, int p) {
  int sz = 1, mx = 0;
  tin_s[u] = ++t_sack;
  ver_s[t_sack] = u;
  big_s[u] = -1;
  for (int v : adj_s[u]) {
    if (v == p) continue;
    int c_sz = pre_sack(v, u);
    sz += c_sz;
    if (c_sz > mx) { mx = c_sz; big_s[u] = v; }
  }
  tout_s[u] = t_sack;
  return sz;
}
void add_node_sack(int u, int delta) {
  // Actualizar estructura de frecuencias
  cnt_s[col_s[u]] += delta;
}
void dfs_sack(int u, int p, bool keep) {
  for (int v : adj_s[u])
    if (v != p && v != big_s[u]) dfs_sack(v, u, false);
  if (big_s[u] != -1) dfs_sack(big_s[u], u, true);

  for (int v : adj_s[u])
    if (v != p && v != big_s[u])
      for (int t = tin_s[v]; t <= tout_s[v]; ++t)
        add_node_sack(ver_s[t], +1);
  add_node_sack(u, +1);

  // AQUI: cnt_s[c] tiene la cantidad exacta de nodos de
  // color 'c'
  // en el subarbol de 'u'. Responder todas las queries de
  // 'u'.

  if (!keep)
    for (int t = tin_s[u]; t <= tout_s[u]; ++t)
      add_node_sack(ver_s[t], -1);
}
```

- **Trampas y adaptaciones:** (1) Entre casos de prueba múltiples, limpiar `adj_s[1..n]`, `big_c[1..n] = 0`, `timer_s = 0` y asegurarse de que la raíz se llame con `keep = false` si quieres dejar `cnt_col` limpio en 0 al terminar.

4.9 Link-Cut Tree (LCT — Bosques Dinámicos Online)

- **Cuándo usar:** Mantener un bosque de árboles **online** bajo inserción de aristas (`link(u, v)`), borrado de aristas (`cut(u, v)`), prueba de conectividad (`connected(u, v)`), actualizaciones puntuales y sumas en el camino entre u y v (`query_path(u, v)`).
- **Cuándo no usar / alternativa:** Si el árbol nunca cambia su topología de aristas, usar `HLD` (§4.8.1); si solo se pide conectividad y todas las operaciones se conocen de antemano, usar `DynCon` (§4.1.2). Link-Cut Tree solo mantiene **bosques** (sin ciclos).
- **Contrato:** Nodos **1-based** (`1..n`). `link(u, v)` requiere `!connected(u, v)`; `cut(u, v)` requiere que la arista `(u, v)` exista actualmente.
- **Complejidad:** $\mathcal{O}(\log N)$ amortizado por operación; $\mathcal{O}(N)$ memoria.

```
// Splay Tree Node. 'pp' apunta al padre del arbol
// representado
struct LCTNode {
  LCTNode *p = 0, *pp = 0, *c[2] = {0, 0};
  bool flip = 0;
  void fix() {
    for (int i = 0; i < 2; ++i) if (c[i]) c[i]->p = this;
  }
  int up() { return p ? (p->c[1] == this) : -1; }
  void push() {
    if (!flip) return;
    flip = 0; swap(c[0], c[1]);
    for (int i = 0; i < 2; ++i) if (c[i]) c[i]->flip ^= 1;
  }
  void rot(int i, int b) {
    int h = i ^ b;
    LCTNode *x = c[i], *y = (b == 2 ? x : x->c[h]), *z = (
      b ? y : x);
    if ((y->p = p)) p->c[up()] = y;
    c[i] = z->c[i ^ 1];
    if (b < 2) x->c[h] = y->c[h ^ 1], z->c[h ^ 1] = (
      b ? x : this);
    y->c[i ^ 1] = (b ? this : x);
    fix(); x->fix(); y->fix();
    if (p) p->fix();
    swap(pp, y->pp);
  }
  void splay() {
    for (push(); p;) {
      if (p->p) p->p->push();
      p->push(); push();
      int c1 = up(), c2 = p->up();
      if (c2 == -1) p->rot(c1, 2);
      else p->p->rot(c2, c1 != c2);
    }
  }
  LCTNode* first() {
    push();
    return c[0] ? c[0]->first() : (splay(), this);
  }
};

struct LinkCut {
  vector<LCTNode> node;
  LinkCut(int n) : node(n + 1) {}

  LCTNode* access(LCTNode* u) {
    u->splay();
    while (LCTNode* pp = u->pp) {
      pp->splay(); u->pp = 0;
      if (pp->c[1]) pp->c[1]->p = 0, pp->c[1]->pp = pp;
      pp->c[1] = u; pp->fix(); u = pp;
    }
    return u;
  }
  void make_root(int u) {
    access(&node[u]); node[u].splay();
    if (node[u].c[0]) {
      node[u].c[0]->p = 0;
      node[u].c[0]->flip ^= 1;
      node[u].c[0]->pp = &node[u];
      node[u].c[0] = 0; node[u].fix();
    }
  }
  bool connected(int u, int v) {
    return access(&node[u])->first() == access(
```

```
      &node[v])->first();
  }
  void link(int u, int v) {
    if (connected(u, v)) return;
    make_root(u); node[u].pp = &node[v];
  }
  void cut(int u, int v) {
    make_root(v); node[u].splay();
    if (node[u].pp == &node[v]) node[u].pp = 0;
    else {
      node[u].c[0] = node[v].p = 0;
      node[u].fix();
    }
  }
};
```

- **Trampas y adaptaciones:** (1) Verificar siempre `!lct.connected(u, v)` antes de `lct.link(u, v)` porque crear un ciclo rompe la invariante de los Splay Trees auxiliares.

Bloque 5: Algoritmos de Strings

5.1 Rotación Cíclica Mínima / Minimum Expression (Duval/Booth $\mathcal{O}(N)$)

- **Cuándo usar:** Encontrar el índice de inicio de la rotación cíclica lexicográficamente menor de una cadena S en $\mathcal{O}(|S|)$ (canonización de collares o polígonos).
- **Cuándo no usar / alternativa:** Si necesitas ordenar **todas** las rotaciones cíclicas de S, usar `SuffixArray` sobre $S+S$ (§5.6.1).
- **Contrato:** `min_cyclic_rotation(s)` retorna el índice **0-based** $i \in [0, |S|-1]$ donde empieza la menor rotación. La cadena rotada es `s.substr(i) + s.substr(0, i)`.
- **Complejidad:** $\mathcal{O}(|S|)$ tiempo y $\mathcal{O}(|S|)$ memoria.

```
// Retorna el indice 0-based donde empieza la menor rotacion
// ciclica
// Ejemplo: s = "bbaaca" -> retorna 2 (rotacion "aacabb")
int min_cyclic_rotation(string s) {
  int n = sz(s);
  s += s;
  int i = 0, j = 1, k = 0;
  while (i < n && j < n && k < n) {
    if (s[i + k] == s[j + k]) {
      k++;
    } else {
      if (s[i + k] > s[j + k]) i += k + 1;
      else j += k + 1;
      if (i == j) j++;
      k = 0;
    }
  }
  return min(i, j);
}
```

- **Trampas y adaptaciones:** (1) Si todas las rotaciones mínimas empatan, el algoritmo de Duval retorna la primera aparición del bloque mínimo.

5.2 Double Rolling Hash Polinomial y LCP con Búsqueda Binaria

- **Cuándo usar:** Comparar subcadenas `s[l1..r1]` y `s[l2..r2]` por igualdad en $\mathcal{O}(1)$ y hallar su Mayor Prefijo Común (`lcp`) u orden lexicográfico (`cmp_substr`) en $\mathcal{O}(\log N)$.
- **Cuándo no usar / alternativa:** Si solo necesitas comparar prefijos de la misma cadena con el prefijo global `s[0..n-1]`, usar **Z-Algorithm** (§5.4.1) que es determinista y $\mathcal{O}(N)$.
- **Contrato:** Índices **0-based** (`0 <= l <= r < n`). `get(l, r)` retorna el par de hashes `(h1, h2)` de `s[l..r]`. El doble módulo reduce drásticamente la tasa de colisión accidental, pero con base fija un adversario que conozca `BASE` y `M1, M2` puede construir colisiones (aleatorizar `BASE` con `rd(257, 1e9)` si hay *open hacking*).
- **Complejidad:** Construcción $\mathcal{O}(N)$; `get(l, r)` en $\mathcal{O}(1)$; `lcp` y `cmp_substr` en $\mathcal{O}(\log N)$.

```
struct PolyHash {
  static const ll M1 = 1000000007LL;
  static const ll M2 = 1000000009LL;
  static const ll BASE = 911382323LL;
  int n;
  vll h1, h2, p1, p2;
  string s;

  PolyHash(const string& str)
    : n(sz(str)), h1(n + 1, 0), h2(n + 1, 0),
      p1(n + 1, 1), p2(n + 1, 1), s(str) {
    for (int i = 0; i < n; ++i) {
      p1[i + 1] = (p1[i] * BASE) % M1;
      p2[i + 1] = (p2[i] * BASE) % M2;
      h1[i + 1] = (h1[i] * BASE + s[i]) % M1;
      h2[i + 1] = (h2[i] * BASE + s[i]) % M2;
    }
  }
  // Hash de s[l..r] (0-indexed, inclusivo) en O(1)
  pll get(int l, int r) const {
    ll r1 = (h1[r + 1] - h1[l] * p1[r - l + 1]) % M1;
    if (r1 < 0) r1 += M1;
    ll r2 = (h2[r + 1] - h2[l] * p2[r - l + 1]) % M2;
    if (r2 < 0) r2 += M2;
    return {r1, r2};
  }
  // Longitud del mayor prefijo comun entre s[i..n-1] y
  // s[j..n-1] en O(log N)
  int lcp(int i, int j) const {
    int low = 1, high = n - max(i, j), ans = 0;
    while (low <= high) {
      int mid = (low + high) / 2;
      if (get(i, i + mid - 1) == get(j, j + mid - 1)) {
        ans = mid; low = mid + 1;
      } else high = mid - 1;
    }
    return ans;
  }
  // Retorna true si s[l1..r1] < s[l2..r2]
  // lexicograficamente en O(log N)
  bool cmp_substr(int l1, int r1, int l2, int r2) const {
    int len1 = r1 - l1 + 1, len2 = r2 - l2 + 1;
    int common = min({lcp(l1, l2), len1, len2});
    if (common == len1 || common
      == len2) return len1 < len2;
    return s[l1 + common] < s[l2 + common];
  }
};
```

- **Trampas y adaptaciones:** (1) Nunca mapear `'a' -> 0` (si `'a' == 0`, entonces `"a"`, `"aa"` y `"aaa"` tendrían todas hash `0`).

5.3 KMP (Función Prefijo π, Pattern Matching, Periodo) y Autómata KMP

5.3.1 KMP: Función Prefijo π y Búsqueda de Patrón (`prefix_function`, `kmp_search`)

- **Cuándo usar:** Buscar todas las ocurrencias exactas de un patrón `pat` dentro de `text` en $\mathcal{O}(|T|+|P|)$ determinista, o hallar el periodo mínimo de una cadena ($k = n - \pi[n-1]$).
- **Cuándo no usar / alternativa:** Si hay **múltiples patrones** simultáneos ($P_1, \ldots, P_k$) que buscar sobre el mismo texto, usar **Aho-Corasick** (§5.5.3).
- **Contrato:** Índices **0-based**. `pi[i]` es la longitud del mayor prefijo propio de `s[0..i]` que también es sufijo de `s[0..i]`. `kmp_search(text, pat)` retorna los índices de inicio 0-based en `text` (vacío si no ocurre).
- **Complejidad:** $\mathcal{O}(|T|+|P|)$ tiempo y $\mathcal{O}(|P|)$ memoria.

```
// 1. Funcion Prefijo pi[0..n-1] en O(N)
vi prefix_function(const string& s) {
  int n = sz(s);
  vi pi(n, 0);
  for (int i = 1; i < n; ++i) {
    int j = pi[i - 1];
    while (j > 0 && s[i] != s[j]) j = pi[j - 1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
  }
  return pi;
}

// 2. Buscar todos los indices 0-based donde 'pat' ocurre en
// 'text'
vi kmp_search(const string& text, const string& pat) {
  vi pi = prefix_function(pat), matches;
  int m = sz(pat), j = 0;
  for (int i = 0; i < sz(text); ++i) {
    while (j > 0 && text[i] != pat[j]) j = pi[j - 1];
    if (text[i] == pat[j]) j++;
    if (j == m) {
      matches.pb(i - m + 1);
      j = pi[j - 1];
    }
  }
  return matches;
}
```

- **Trampas y adaptaciones:** (1) Si las cadenas pueden contener espacios, leer con `getline` en vez de `cin >>`; (2) el periodo $k = n - \pi[n-1]$ es un periodo exacto completo si y solo si $n \bmod k == 0$.

5.3.2 Autómata KMP (`build_kmp_automaton`)

- **Cuándo usar:** Construir el DFA completo del patrón `p` (`aut[estado][letra]`) para hacer **DP o exponenciación de matrices** contando cuántas cadenas de longitud L contienen (o no contienen) a `p` como subcadena.
- **Cuándo no usar / alternativa:** Si solo buscas `p` una vez en un texto fijo, `kmp_search` (§5.3.1) ahorra el factor 26 de tiempo y memoria.
- **Contrato:** Requiere alfabeto `'a'..'z'` minúsculas. `aut` tiene dimensión `(|p| + 1) x 26`; el estado `|p|` representa haber completado el patrón `p`. Dependencias: `prefix_function` (§5.3.1).
- **Complejidad:** $\mathcal{O}(26 \cdot |P|)$ tiempo y memoria.

```
// 3. Automata KMP: aut[i][c] = nuevo estado si llevamos 'i'
// chars de 'p'
//    coincidentes y agregamos la letra ('a' + c).
vector<vi> build_kmp_automaton(const string& p) {
  int m = sz(p);
  vi pi = prefix_function(p);
  vector<vi> aut(m + 1, vi(26, 0));
  for (int i = 0; i <= m; ++i) {
    for (int c = 0; c < 26; ++c) {
      if (i == m || (i > 0 && 'a' + c != p[i]))
        aut[i][c] = aut[pi[i - 1]][c];
      else
        aut[i][c] = i + ('a' + c == p[i]);
    }
  }
  return aut;
}
```

- **Trampas y adaptaciones:** (1) Si el alfabeto incluye mayúsculas o dígitos, ampliar la segunda dimensión de `26` al tamaño del alfabeto y ajustar `c - 'a'`.

5.4 Z-Algorithm y Algoritmo de Manacher (Palíndromos Pares e Impares)

5.4.1 Z-Algorithm (`z_function`)

- **Cuándo usar:** Calcular en $\mathcal{O}(N)$ para todo $i \in [1, N-1]$ la longitud del Mayor Prefijo Común (`z[i]`) entre `s` y el sufijo `s[i..N-1]`.
- **Cuándo no usar / alternativa:** Si necesitas el LCP entre **cualquier** par de sufijos `s[i..]` y `s[j..]`, usar `SuffixArray` + Kasai (§5.6.1) o `PolyHash` (§5.2).
- **Contrato:** Índices **0-based**. Por convención `z[0] = 0` (algunos problemas definen `z[0] = n`; asignarlo manualmente si se requiere).
- **Complejidad:** $\mathcal{O}(|S|)$ tiempo y $\mathcal{O}(|S|)$ memoria.

```
// 1. Z-Algorithm: z[i] = LCP(s[0..n-1], s[i..n-1])
vi z_function(const string& s) {
  int n = sz(s), l = 0, r = 0;
  vi z(n, 0);
  for (int i = 1; i < n; ++i) {
    if (i < r) z[i] = min(r - i, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] > r) { l = i; r = i + z[i]; }
  }
  return z;
}
```

- **Trampas y adaptaciones:** (1) Para buscar `pat` en `text` con Z-Algorithm, construir `pat + '#' + text` con un carácter separador `'#'` que no aparezca en ninguna de las dos cadenas.

5.4.2 Algoritmo de Manacher para Palíndromos Pares e Impares (`Manacher`)

- **Cuándo usar:** Encontrar en $\mathcal{O}(|S|)$ el radio del mayor palíndromo centrado en cada carácter y entre cada par de caracteres adyacentes, y consultar si cualquier subcadena `s[l..r]` es palíndromo en $\mathcal{O}(1)$ (`is_pal(l, r)`).
- **Cuándo no usar / alternativa:** Si necesitas contar cuántos palíndromos **distintos** existen o sus frecuencias como estados de un autómata, usar `PalindromicTree` / Eertree (§5.6.3).

- **Contrato:** Índices **0-based** (`0 <= l <= r < |s|`). `is_pal(l, r)` responde en $\mathcal{O}(1)$; `max_pal_len()` retorna la longitud del mayor palíndromo en `s` .
- **Complejidad:** Construcción $\mathcal{O}(|S|)$ tiempo y memoria; `is_pal(l, r)` en $\mathcal{O}(1)$.

```cpp
// 2. Manacher (Unificado con '#' para pares e impares)
// Transforma "abba" -> "#a#b#b#a#".
// En el vector retornado 'p', p[i] - 1 es la longitud real
// del mayor
// palindromo centrado en t[i].
// Ademas, devuelve si s[l..r] (0-indexed) es palindromo en
// O(1)!
struct Manacher {
  vi p;
  Manacher(const string& s) {
    string t = "#";
    for (char c : s) { t += c; t += "#"; }
    int n = sz(t), l = 0, r = -1;
    p.assign(n, 0);
    for (int i = 0; i < n; ++i) {
      int k = (i > r) ? 1 : min(p[l + r - i], r - i + 1);
      while (
        0 <= i - k && i + k < n && t[i - k] == t[i
          + k]) k++;
      p[i] = k--;
      if (i + k > r) { l = i - k; r = i + k; }
    }
  }
  // Consulta en O(1) si s[l..r] (0-indexed) es palindromo:
  bool is_pal(int l, int r) const {
    int center = l + r + 1;
    int len = r - l + 1;
    return (p[center] - 1) >= len;
  }
  int max_pal_len() const {
    return *max_element(all(p)) - 1;
  }
};
```

- **Trampas y adaptaciones:** (1) La cadena transformada interna `t` tiene tamaño `2 * |s| + 3` con centinelas `^` y `$`; usar siempre el método `is_pal(l, r)` para evitar errores de mapeo de índices.

5.5 Trie (Strings y Trie Binario para Max XOR) y Aho-Corasick (Múltiples Patrones)

5.5.1 Trie de Strings: Prefijos y Palabras Exactas (`StringTrie`)

- **Cuándo usar:** Insertar un diccionario de cadenas y consultar cuántas palabras comparten un prefijo dado (`count_prefix(s)`) o cuántas veces se insertó exactamente `s` (`count_exact(s)`).
- **Cuándo no usar / alternativa:** Si necesitas buscar todas las palabras del diccionario como subcadenas dentro de un texto largo, usar **Aho-Corasick** (§5.5.3) que agrega enlaces de fallo al Trie.
- **Contrato:** Alfabeto `'a'..'z'` minúsculas. `count_prefix(s)` y `count_exact(s)` retornan `0` si el camino no existe.
- **Complejidad:** `insert` , `count_prefix` , `count_exact` en $\mathcal{O}(|S|)$ tiempo; memoria $\mathcal{O}(26 \cdot \sum |S|)$.

```cpp
// 0. Trie de Strings (Conteo de prefijos y palabras exactas
// en O(|S|))
struct StringTrie {
  struct Node { int next[26], pref = 0, end_w = 0; Node() {
    fill(begin(next), end(next), -1); } };
  vector<Node> t = {Node()};
  void insert(const string& s) {
    int u = 0; t[u].pref++;
    for (char c : s) {
      int ch = c - 'a';
      if (t[u].next[ch] == -1) { t[u].next[ch] = sz(t);
        t.emplace_back(); }
      u = t[u].next[ch]; t[u].pref++;
    }
    t[u].end_w++;
  }
  int count_prefix(const string& p) const {
    int u = 0;
    for (char c : p) {
      int ch = c - 'a';
      if (t[u].next[ch] == -1) return 0;
      u = t[u].next[ch];
    }
    return t[u].pref;
  }
};
```

- **Trampas y adaptaciones:** (1) Si la memoria es ajustada y el alfabeto es grande, reemplazar `int nxt[26]` por `map<char, int>` (paga $\mathcal{O}(\log \Sigma)$ pero ahorra memoria).

5.5.2 Trie Binario para Max/Min XOR Dinámico (`BitTrie`)

- **Cuándo usar:** Mantener un multiconjunto dinámico de enteros bajo inserción (`update(x, +1)`), borrado (`update(x, -1)`) y consultar en $\mathcal{O}(B)$ qué elemento y del conjunto maximiza `x ^ y` (`max_xor(x)`) o minimiza `x ^ y` (`min_xor(x)`).
- **Cuándo no usar / alternativa:** Si el problema pide maximizar el XOR de un **subconjunto de elementos** (no de un único elemento y), usar **Base de XOR / Gauss Binario** (`XorBasis` , §7.6.2).
- **Contrato:** Precondición obligatoria: el Trie **no debe estar vacío** al llamar `max_xor(x)` o `min_xor(x)` , y solo se debe borrar (`delta = -1`) un valor que haya sido insertado previamente. `B = 30` cubre $0 \le x < 2^{30}$ (cambiar `B = 61` para `ll`).
- **Complejidad:** $\mathcal{O}(B)$ tiempo por `update` , `max_xor` , `min_xor` ; memoria $\mathcal{O}(N \cdot B)$.

```cpp
// 1. Trie Binario (Max/Min XOR con insercion y borrado;
// precondicion: no vacio)
struct BitTrie {
  static const int B = 60; // 30 para int, 60 para long long
  struct Node { int ch[2] = {-1, -1}, cnt = 0; };
  vector<Node> t = {Node()};

  // delta = +1 insertar, -1 borrar (x debe existir)
  void update(ll x, int delta) {
    int u = 0;
    for (int b = B; b >= 0; --b) {
      int bit = (x >> b) & 1LL;
      if (t[u].ch[bit] == -1) {
        t[u].ch[bit] = sz(t);
        t.emplace_back();
      }
      u = t[u].ch[bit];
      t[u].cnt += delta;
    }
  }
  // Retorna max(x ^ y) para 'y' activo en el Trie
  ll max_xor(ll x) const {
    int u = 0;
    ll ans = 0;
    for (int b = B; b >= 0; --b) {
      int bit = (x >> b) & 1LL, want = bit ^ 1;
      if (t[u].ch[want] != -1 && t[t[u].ch[want]].cnt > 0) {
        ans |= (1LL << b);
        u = t[u].ch[want];
      } else {
        u = t[u].ch[bit];
      }
    }
    return ans;
  }
  // Retorna min(x ^ y) para 'y' activo en el Trie
  ll min_xor(ll x) const {
    int u = 0;
    ll ans = 0;
    for (int b = B; b >= 0; --b) {
      int bit = (x >> b) & 1LL;
      if (t[u].ch[bit] != -1 && t[t[u].ch[bit]].cnt > 0) {
        u = t[u].ch[bit];
      } else {
        ans |= (1LL << b);
        u = t[u].ch[bit ^ 1];
      }
    }
    return ans;
  }
};
```

- **Trampas y adaptaciones:** (1) Para contar cuántos elementos y cumplen $(x \oplus y) < K$, recorrer de `B` a `0` : si el bit de K es `1` , sumar `cnt` de la rama donde `(x ^ y)` tiene bit `0` y bajar por la rama donde tiene bit `1` .

5.5.3 Autómata de Aho-Corasick para Múltiples Patrones (`AhoCorasick`)

- **Cuándo usar:** Buscar simultáneamente un diccionario de patrones $P_1, ..., P_k$ dentro de un texto T en tiempo lineal, o hacer DP sobre el autómata para contar cadenas que evitan todas las palabras prohibidas.
- **Cuándo no usar / alternativa:** Para un solo patrón, KMP (§5.3.1) no requiere la matriz `26 * sum(|P|)` .
- **Contrato:** Alfabeto `'a'..'z'` . Llamar a `insert(s, id)` para cada patrón y **obligatoriamente** llamar a `build()` una sola vez antes de procesar textos o usar las transiciones `t[u].next[c]` .
- **Complejidad:** `insert` en $\mathcal{O}(\sum |P_i|)$; `build` en $\mathcal{O}(26 \sum |P_i|)$; `count_matches(text)` en $\mathcal{O}(|T|)$.

```cpp
// 2. Aho-Corasick (Multi-Pattern Matching + Automaton en
// O(sum|P| * 26))
struct AhoCorasick {
  struct Node {
    int next[26], link = 0, exit_link = 0, cnt = 0;
    Node() { fill(begin(next), end(next), 0); }
  };
  vector<Node> t = {Node()};

  void insert(const string& s) {
    int u = 0;
    for (char c : s) {
      int ch = c - 'a';
      if (!t[u].next[ch]) {
        t[u].next[ch] = sz(t);
        t.emplace_back();
      }
      u = t[u].next[ch];
    }
    // Cantidad de patrones que terminan exactamente en u
    t[u].cnt++;
  }
  void build() {
    queue<int> q;
    for (int c = 0; c < 26; ++c)
      if (t[0].next[c]) q.push(t[0].next[c]);
    while (!q.empty()) {
      int u = q.front(); q.pop();
      int f = t[u].link;
      // Acumula ocurrencias de sufijos
      t[u].cnt += t[f].cnt;
      t[u].exit_link = (t[f].cnt > 0) ? f : t[f].exit_link;
      for (int c = 0; c < 26; ++c) {
        int v = t[u].next[c];
        if (v) {
          t[v].link = t[f].next[c];
          q.push(v);
        } else {
          // Transicion directa de automata
          t[u].next[c] = t[f].next[c];
        }
      }
    }
  }
  // Cuenta el total de apariciones de todos los patrones
  // dentro de 'text'
  ll count_matches(const string& text) const {
    int u = 0;
    ll total = 0;
    for (char c : text) {
      u = t[u].next[c - 'a'];
      total += t[u].cnt;
    }
    return total;
  }
};
```

- **Trampas y adaptaciones:** (1) Si necesitas la frecuencia individual de **cada** patrón P_i (en vez de la suma total), acumular visitas en los nodos durante el recorrido de `text` y propagar en orden inverso de BFS hacia `t[u].link` .

5.6 Suffix Array + LCP, Suffix Automaton (SAM) y Palindromic Tree (Eertree)

5.6.1 Suffix Array y Arreglo LCP de Kasai (`SuffixArray`)

- **Cuándo usar:** Ordenar lexicográficamente todos los sufijos de S, calcular el LCP entre sufijos adyacentes (`lcp[i]`) y contar subcadenas distintas ($\frac{N(N+1)}{2} - \sum \text{lcp}[i]$) o buscar patrones con búsqueda binaria.
- **Cuándo no usar / alternativa:** Si necesitas consultar el número de ocurrencias de subcadenas por longitud o el estado mínimo que las acepta, `SuffixAutomaton` (§5.6.2) es más directo.
- **Contrato:** `sa` tiene tamaño `N + 1` porque incluye el sufijo vacío centinela en `sa[0] = N` ; los sufijos reales están en `sa[1..N]` . `lcp[i]` es el LCP entre `sa[i]` y `sa[i - 1]` .
- **Complejidad:** Construcción de `sa` en $\mathcal{O}(N \log N)$ y `lcp` (Kasai) en $\mathcal{O}(N)$; memoria $\mathcal{O}(N)$.

```cpp
// 1. Suffix Array + LCP (Kasai) en O(N log N)
struct SuffixArray {
  vi sa, lcp, rank;
  SuffixArray(const string& s, int lim = 256) {
    int n = sz(s) + 1, k = 0, a, b;
    vi x(all(s) + 1), y(n), ws(max(n, lim));
```

```
    sa = lcp = rank = y;
    iota(all(sa), 0);
    for (int j = 0, p = 0; p < n; j = max(1, j * 2),
      lim = p) {
      p = j;
      for (int i = 0; i < n; ++i)
        y[i] = (sa[i] - j >= 0 ? 0 : n) + sa[i] - j;
      fill(all(ws), 0);
      for (int i = 0; i < n; ++i) ws[x[i]]++;
      for (int i = 1; i < lim; ++i) ws[i] += ws[i - 1];
      for (int i = n; i--; ) sa[--ws[x[y[i]]]] = y[i];
      swap(x, y); p = 1; x[sa[0]] = 0;
      for (int i = 1; i < n; ++i) {
        a = sa[i - 1]; b = sa[i];
        x[b] = (
          y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1
            : p++;
      }
    }
    for (int i = 1; i < n; ++i) rank[sa[i]] = i;
    for (int i = 0, j; i < n - 1; lcp[rank[i++]] = k)
      for (k && k--,
        j = sa[rank[i] - 1]; s[i + k] == s[j + k]; k++);
  }
  // Nota: sa[0] es el sufijo vacio '$'. Los sufijos reales
  // estan en sa[1..n].
  // lcp[i] = LCP(sa[i - 1], sa[i]) para i in [1..n].
};
```

- **Trampas y adaptaciones:** (1) Para el LCP entre dos sufijos cualesquiera `i` y `j` con `rk[i] < rk[j]`, construir una `SparseTable` (§4.2) sobre `lcp` y consultar el mínimo en `[rk[i] + 1, rk[j]]`.

5.6.2 Suffix Automaton (`SuffixAutomaton`)

- **Cuándo usar:** Representar de forma comprimida todas las subcadenas de S en a lo sumo $2|S| - 1$ estados: contar subcadenas distintas, hallar la **subcadena contigua común más larga** (`longest_common_substring`) entre S y T, o contar cuántas veces aparece cada subcadena (`propagate_occurrences`).
- **Cuándo no usar / alternativa:** No confundir `longest_common_substring` (subcadena **contigua** en $\mathcal{O}(|S| + |T|)$) con `lcs_string` (§8.1.2, subsecuencia no necesariamente contigua en $\mathcal{O}(|S| \cdot |T|)$).
- **Contrato:** Estado raíz = `0`. Cada estado `u` representa todas las subcadenas de longitud en `[st[st[u].link].len + 1, st[u].len]`. `propagate_occurrences()` ordena por longitud para acumular `cnt` hacia `link`.
- **Complejidad:** Construcción $\mathcal{O}(|S| \log \Sigma)$ (con `map`); `propagate_occurrences` en $\mathcal{O}(|S| \log |S|)$ por el `sort`; `longest_common_substring` en $\mathcal{O}(|T| \log \Sigma)$.

```
// 2. Suffix Automaton (SAM) en O(N)
struct SuffixAutomaton {
  struct State {
    int len = 0, link = -1;
    int next[26];
    // Frecuencia de aparicion del estado (endpos size)
    ll cnt = 0;
    State() { fill(begin(next), end(next), -1); }
  };
  vector<State> st;
  int last = 0;

  SuffixAutomaton(const string& s) {
    st.reserve(2 * sz(s) + 1);
    st.emplace_back();
    for (char c : s) extend(c - 'a');
    // Propagar frecuencias (cnt) de estados largos a sus
    // suffix links:
    vi ord(sz(st) - 1);
    iota(all(ord), 1);
    sort(all(ord), [&](int a, int b) {
      return st[a].len > st[b].len; });
    for (int u : ord) st[st[u].link].cnt += st[u].cnt;
  }
  void extend(int c) {
    int cur = sz(st);
    st.emplace_back();
    st[cur].len = st[last].len + 1;
    st[cur].cnt = 1;
    int p = last;
    while (p != -1 && st[p].next[c] == -1) {
      st[p].next[c] = cur;
      p = st[p].link;
    }
    if (p == -1) {
      st[cur].link = 0;
    } else {
      int q = st[p].next[c];
      if (st[p].len + 1 == st[q].len) {
        st[cur].link = q;
      } else {
        int clone = sz(st);
        st.pb(st[q]);
        st[clone].len = st[p].len + 1;
        // Los clones no suman nueva ocurrencia base
        st[clone].cnt = 0;
        while (p != -1 && st[p].next[c] == q) {
          st[p].next[c] = clone;
          p = st[p].link;
        }
        st[q].link = st[cur].link = clone;
      }
    }
    last = cur;
  }
  // Total de subcadenas distintas de S en O(|st|):
  ll count_distinct_substrings() const {
    ll ans = 0;
    for (int i = 1; i < sz(st); ++i)
      ans += st[i].len - st[st[i].link].len;
    return ans;
  }
  // Subcadena comun mas larga (LCS) entre S y otro string T
  // en O(|T|):
  string longest_common_substring(const string& t) const {
    int v = 0, l = 0, best_len = 0, best_pos = 0;
    for (int i = 0; i < sz(t); ++i) {
      int c = t[i] - 'a';
      while (v != 0 && st[v].next[c] == -1) {
        v = st[v].link;
        l = st[v].len;
      }
      if (st[v].next[c] != -1) { v = st[v].next[c]; l++; }
      if (l > best_len) { best_len = l; best_pos = i; }
    }
    return t.substr(best_pos - best_len + 1, best_len);
  }
};
```

- **Trampas y adaptaciones:** (1) Si el alfabeto es fijo `'a'..'z'` y el tiempo es muy ajustado, cambiar `map<char, int> next` por `int next[26]` inicializado en `-1` para quitar el factor $\log \Sigma$.

5.6.3 Palindromic Tree / Eertree (`PalindromicTree`)

- **Cuándo usar:** Construir en $\mathcal{O}(|S|)$ el árbol de todas las subcadenas palindrómicas **distintas** de S (a lo sumo $|S|$ palíndromos distintos), manteniendo sus longitudes, enlaces sufijos (`link`) y frecuencias.
- **Cuándo no usar / alternativa:** Si solo necesitas saber si `s[1..r]` es palíndromo o el palíndromo más largo global, `Manacher` (§5.4.2) es más corto.
- **Contrato:** Alfabeto `'a'..'z'`. El nodo `1` es la raíz imaginaria de longitud `-1` y el nodo `2` es la raíz vacía de longitud `0`; los palíndromos reales son los nodos `3..sz(t)-1` (por lo que hay `sz(t) - 3` palíndromos distintos).
- **Complejidad:** $\mathcal{O}(|S|)$ tiempo y $\mathcal{O}(26 \cdot |S|)$ memoria.

```
// 3. Palindromic Tree (Eertree) en O(N)
struct PalindromicTree {
  struct Node {
    int next[26], len, link;
    ll cnt = 0;
    Node(int l = 0, int lk = 0) : len(l), link(lk) {
      fill(begin(next), end(next), 0);
    }
  };
  vector<Node> st;
  string s;
  int last = 1;

  PalindromicTree(const string& str) : s(str) {
    // Nodo 0: raiz de longitud impar (-1)
    st.pb(Node(-1, 0));
    st.pb(Node(0, 0));  // Nodo 1: raiz de longitud par (0)
    for (int i = 0; i < sz(s); ++i) add(i);
    for (int i = sz(st) - 1; i >= 2; --i)
      st[st[i].link].cnt += st[i].cnt;
  }
  int get_link(int v, int i) const {
    while (
      i - st[v].len - 1 < 0 || s[i - st[v].len - 1] != s[i])
      v = st[v].link;
    return v;
  }
  void add(int i) {
    int c = s[i] - 'a';
    int cur = get_link(last, i);
    if (!st[cur].next[c]) {
      int nw = sz(st);
      int lk = st[get_link(st[cur].link, i)].next[c];
      st.pb(Node(st[cur].len + 2, max(1, lk)));
      st[cur].next[c] = nw;
    }
    last = st[cur].next[c];
    st[last].cnt++;
  }
  // Cantidad de palindromos DISTINTOS en S: sz(st) - 2
};
```

- **Trampas y adaptaciones:** (1) Para obtener el número total de ocurrencias de cada palíndromo en S, recorrer los nodos desde `sz(pt.t) - 1` hacia `3` haciendo `t[t[i].link].cnt += t[i].cnt`.

Bloque 6: Grafos, Árboles, Conectividad y Flujos

6.1 Recorridos BFS: Grid BFS (Flood Fill), Multi-Source BFS (Aristas Invertidas) y 0-1 BFS

6.1.1 Grid BFS / Flood Fill con Identificador de Componentes (`flood_fill_bfs`)

- **Cuándo usar:** Etiquetar componentes conexas en una grilla $R \times C$ con movimiento en 4 direcciones (`dx4, dy4`) y precalcular una métrica por componente (`ans_comp[id]`) para responder consultas en $\mathcal{O}(1)$.
- **Cuándo no usar / alternativa:** Si las celdas cambian dinámicamente (se abren paredes online), usar **DSU** (§4.1.1) aplanando `(r, c) -> r * m + c`.
- **Contrato:** Grilla 0-indexed de tamaño `n_g x m_g` en `grid_g`. Inicializar `comp_g` en `-1` antes de recorrer las celdas `..`.
- **Complejidad:** $\mathcal{O}(R \cdot C)$ tiempo y memoria.

```
// 1. Grid BFS / Flood Fill con Identificador de Componentes
const int MAX_R = 1005, MAX_C = 1005;
int n_g, m_g, comp_g[MAX_R][MAX_C], ans_comp[MAX_R * MAX_C];
char grid_g[MAX_R][MAX_C];
int dx4[] = {-1, 1, 0, 0}, dy4[] = {0, 0, -1, 1};

void flood_fill_bfs(int sr, int sc, int id) {
  queue<pii> q;
  q.push({sr, sc});
  comp_g[sr][sc] = id;
  int metric = 0;
  while (!q.empty()) {
    auto [r, c] = q.front(); q.pop();
    for (int d = 0; d < 4; ++d) {
      int nr = r + dx4[d], nc = c + dy4[d];
      if (nr < 0 || nr >= n_g || nc < 0 || nc
        >= m_g) continue;
      if (grid_g[nr][nc] == '*') {
        metric++; // Ejemplo: contar paredes '*' adyacentes
      } else if (
        grid_g[nr][nc] == '.' && comp_g[nr][nc] == -1) {
        comp_g[nr][nc] = id;
        q.push({nr, nc});
      }
    }
  }
  ans_comp[id] = metric;
}
```

- **Trampas y adaptaciones:** (1) Marcar `comp_g[nr][nc] = id` **al encolar** (no al desencolar) para no insertar la misma celda múltiples veces y provocar TLE/MLE.

6.1.2 Multi-Source BFS y Aristas Invertidas (`multi_source_bfs`)

- **Cuándo usar:** Hallar la distancia mínima desde cada nodo hacia el nodo más cercano de un conjunto de fuentes/salidas S en un grafo sin pesos.
- **Cuándo no usar / alternativa:** Si las aristas tienen pesos positivos arbitrarios, inicializar un **Dijkstra multi-fuente** (§6.3.1) metiendo todas las fuentes con distancia `0` al min-heap.
- **Contrato:** Nodos 1-indexed (`1..n`). Si el grafo es dirigido y quieres ir *hacia* las salidas S, cargar las aristas al revés (`v -> u`) en `adj_rev`. `dist_ms[u] = -1` si es inalcanzable.
- **Complejidad:** $\mathcal{O}(V + E)$ tiempo y memoria.

```
// 2. Multi-Source BFS (con aristas invertidas adj_rev si es
// dirigido)
vi adj_rev[200005];
int dist_ms[200005];

void multi_source_bfs(const vi& sources, int n) {
  fill(dist_ms + 1, dist_ms + n + 1, -1);
  queue<int> q;
  for (int s : sources) { dist_ms[s] = 0; q.push(s); }
  while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : adj_rev[u]) {
```

```
      if (dist_ms[v] == -1) {
        dist_ms[v] = dist_ms[u] + 1;
        q.push(v);
      }
    }
  }
}
```

- **Trampas y adaptaciones:** (1) No correr un BFS separado desde cada fuente ($\mathcal{O}(K(V+E))$ da TLE); meter las K fuentes juntas en la cola inicial.

6.1.3 0-1 BFS con `std::deque` y Reconstrucción (`bfs_01` , `get_path_01`)

- **Cuándo usar:** Camino más corto desde `s` cuando todos los pesos de arista son exclusivamente 0 o 1 (ej. laberintos donde romper pared o cambiar dirección cuesta 1 y avanzar cuesta 0).
- **Cuándo no usar / alternativa:** Si hay pesos mayores que 1 (como $\{0,1,2\}$), el `deque` pierde monotonía; usar **Dijkstra** (§6.3.1).
- **Contrato:** Nodos 1-indexed (`1..n`). Arista de peso `0` se inserta con `push_front` , peso `1` con `push_back` .
- **Complejidad:** $\mathcal{O}(V+E)$ tiempo y memoria.

```
// 3. 0-1 BFS en Grafo General o Grilla con Reconstruccion
// de Camino
vector<pii> adj01[200005]; // {vecino, peso_0_o_1}
int dist01[200005], par01[200005];

void bfs_01(int s, int n) {
  fill(dist01 + 1, dist01 + n + 1, INF_INT);
  fill(par01 + 1, par01 + n + 1, -1);
  deque<int> dq;
  dist01[s] = 0;
  dq.push_front(s);
  while (!dq.empty()) {
    int u = dq.front(); dq.pop_front();
    for (auto [v, w] : adj01[u]) {
      if (dist01[u] + w < dist01[v]) {
        dist01[v] = dist01[u] + w;
        par01[v] = u;
        if (w == 0) dq.push_front(v);
        else dq.push_back(v);
      }
    }
  }
}
vi get_path_01(int target) {
  if (dist01[target] == INF_INT) return {};
  vi path;
  for (int cur = target; cur != -1; cur
    = par01[cur]) path.pb(cur);
  reverse(all(path));
  return path;
}
```

- **Trampas y adaptaciones:** (1) Un mismo nodo puede entrar al `deque` primero con costo $+1$ y luego mejorarse con costo $+0$; verificar `dist01[u] + w < dist01[v]` antes de relajar.

6.2 Patrones DFS: Grafo Bipartito, Ciclos, Euler Tour (`tin` / `tout`), DFS en Complemento y Conteo de Triángulos

6.2.1 Grafo Bipartito y 2-Coloreo por DFS (`dfs_bipartite`)

- **Cuándo usar:** Determinar si un grafo no dirigido carece de ciclos de longitud impar y particionar sus nodos en dos conjuntos independientes (colores `1` y `2`).
- **Cuándo no usar / alternativa:** Si las aristas se agregan online y quieres mantener bipartición con deshacer, usar **DSU con paridad / Rollback** (§4.1.2).
- **Contrato:** Nodos 1-indexed (`1..n`) en `adj_dfs` . Inicializar `col_bip[1..n] = 0` y lanzar `dfs_bipartite(i, 1)` desde cada componente no visitada.
- **Complejidad:** $\mathcal{O}(V+E)$ tiempo y memoria.

```
// 1. Grafo Bipartito (2-Coloreo con colores 1 y 2)
vi adj_dfs[200005];
int col_bip[200005]; // Inicializar en 0
bool dfs_bipartite(int u, int c) {
  col_bip[u] = c;
  for (int v : adj_dfs[u]) {
    if (col_bip[v] == 0) {
      if (!dfs_bipartite(v, 3 - c)) return false;
    } else if (col_bip[v] == col_bip[u]) return false;
  }
  return true;
}
```

- **Trampas y adaptaciones:** (1) Iterar `for (int i = 1; i <= n; ++i)` porque el grafo puede ser desconexo.

6.2.2 Detección de Ciclo en Grafo Dirigido con 3 Estados (`dfs_dir_cycle`)

- **Cuándo usar:** Detectar si un grafo dirigido contiene algún ciclo dirigido usando coloreo de 3 estados (`0` no visitado, `1` en la pila recursiva actual, `2` cerrado).
- **Cuándo no usar / alternativa:** Si además necesitas el orden topológico o comprimir los ciclos en un DAG, usar `topological_sort` (§6.5.1) o `TarjanSCC` (§6.5.3).
- **Contrato:** Nodos 1-indexed (`1..n`) en `adj_dfs` . Un ciclo dirigido existe si y solo si se encuentra una arista hacia un nodo con `state_dir[v] == 1` . Dependencias: `adj_dfs` (§6.2.1).
- **Complejidad:** $\mathcal{O}(V+E)$ tiempo y memoria.

```
// 2. Ciclo en Grafo Dirigido (0 = blanco, 1 = gris/activo,
// 2 = negro)
int state_dir[200005];
bool dfs_dir_cycle(int u) {
  state_dir[u] = 1;
  for (int v : adj_dfs[u]) {
    // Back-edge encontrada
    if (state_dir[v] == 1) return true;
    if (state_dir[v] == 0 && dfs_dir_cycle(v)) return true;
  }
  state_dir[u] = 2;
  return false;
}
```

- **Trampas y adaptaciones:** (1) En grafos dirigidos, ver un nodo ya visitado con `state_dir[v] == 2` (*cross-edge* o *forward-edge*) **no** es un ciclo; solo `state_dir[v] == 1` (*back-edge*) indica ciclo.

6.2.3 Euler Tour en Árboles y Ancestralidad en $\mathcal{O}(1)$ (`dfs_euler` , `is_ancestor`)

- **Cuándo usar:** Aplanar un árbol en un arreglo contiguo donde el subárbol de cada nodo `u` ocupa exactamente el intervalo `[tin_e[u], tout_e[u]]` , permitiendo consultar en $\mathcal{O}(1)$ si `u` es ancestro de `v` o actualizar/consultar subárboles con Fenwick/Segment Tree.
- **Cuándo no usar / alternativa:** Si las consultas son sobre **caminos** entre `u` y `v` con actualizaciones de peso, el rango `[tin, tout]` no representa caminos; usar **HLD** (§4.8).
- **Contrato:** Árbol 1-indexed en `adj_dfs` . Reiniciar `timer_e = 0` antes de llamar `dfs_euler(root, 0)` . Dependencias: `adj_dfs` (§6.2.1).
- **Complejidad:** $\mathcal{O}(N)$ preprocesamiento y $\mathcal{O}(1)$ por consulta `is_ancestor` .

```
// 3. Euler Tour en Arboles (tin, tout y ancestralidad en
// O(1))
int tin_e[200005], tout_e[200005], timer_e = 0;
void dfs_euler(int u, int p) {
  tin_e[u] = ++timer_e;
  for (int v : adj_dfs[u])
    if (v != p) dfs_euler(v, u);
  tout_e[u] = timer_e;
}
bool is_ancestor(int u, int v) { // ¿u es ancestro de v?
  return tin_e[u] <= tin_e[v] && tout_e[v] <= tout_e[u];
}
```

- **Trampas y adaptaciones:** (1) Con este Euler Tour de N tiempos (`++timer_e` solo al entrar), el tamaño del subárbol de `u` es `tout_e[u] - tin_e[u] + 1` .

6.2.4 Componentes Conexas en el Grafo Complemento (`complement_components`)

- **Cuándo usar:** Hallar las componentes conexas de un grafo donde existen todas las $\binom{N}{2}$ aristas **excepto** M aristas prohibidas dadas en la entrada ($N, M \le 2 \cdot 10^5$).
- **Cuándo no usar / alternativa:** Si $N \le 2000$, construir la matriz de adyacencia explícita y hacer BFS normal en $\mathcal{O}(N^2)$ es más simple.
- **Contrato:** Nodos 1-indexed (`1..n`). `forbidden_adj[u]` contiene los vecinos prohibidos de `u` . Mantiene los nodos aún no visitados en un `std::set<int>` y salta sobre los prohibidos.
- **Complejidad:** $\mathcal{O}((N+M)\log N)$ tiempo y $\mathcal{O}(N+M)$ memoria.

```
// 4. Componentes Conexas en el Grafo Complemento en O((N +
// M) log N)
vector<vi> complement_components(int n,
  vector<vi>& forbidden_adj) {
  for (int i = 1; i <= n; ++i) sort(all(forbidden_adj[i]));
  set<int> unvis;
  for (int i = 1; i <= n; ++i) unvis.insert(i);
  vector<vi> comps;

  for (int i = 1; i <= n; ++i) {
    if (!unvis.count(i)) continue;
    vi comp;
    queue<int> q;
    q.push(i);
    unvis.erase(i);
    while (!q.empty()) {
      int u = q.front(); q.pop();
      comp.pb(u);
      for (auto it = unvis.begin(); it != unvis.end(); ) {
        int v = *it;
        if (!binary_search(all(forbidden_adj[u]), v)) {
          q.push(v);
          it = unvis.erase(it); // Borra en O(1) amortizado!
        } else ++it;
      }
    }
    comps.pb(comp);
  }
  return comps;
}
```

- **Trampas y adaptaciones:** (1) El algoritmo es $\mathcal{O}((N+M)\log N)$ porque cada vez que `binary_search` da `true` se paga una de las $2M$ aristas prohibidas, y cuando da `false` el nodo se borra definitivamente del `set` .

6.2.5 Conteo de Triángulos (K_3) por Orientación de Grado (`count_triangles`)

- **Cuándo usar:** Contar exactamente cuántos ciclos de longitud 3 ($u-v-w-u$) existen en un grafo simple no dirigido con $N, M \le 2 \cdot 10^5$.
- **Cuándo no usar / alternativa:** Si el grafo tiene multiaristas o lazos, limpiar primero `edges` con `sort` + `unique` y descartar `u == v` .
- **Contrato:** Nodos 1-indexed (`1..n`). Orienta cada arista desde el nodo de menor grado hacia el de mayor grado (desempatando por índice), garantizando grado de salida $\le \sqrt{2M}$ en el DAG resultante.
- **Complejidad:** $\mathcal{O}(M\sqrt{M})$ tiempo y $\mathcal{O}(N+M)$ memoria.

```
// 5. Contar Triangulos (3-ciclos) en un grafo en O(M *
// sqrt(M))
ll count_triangles(int n, const vector<pii>& edges) {
  vi deg(n + 1, 0), mark(n + 1, 0);
  for (auto [u, v] : edges) { deg[u]++; deg[v]++; }
  vector<vi> dag(n + 1);
  for (auto [u, v] : edges) {
    if (tie(deg[u], u) > tie(deg[v], v)) swap(u, v);
    dag[u].pb(v); // Orienta de menor grado a mayor grado
  }
  ll triangles = 0;
  for (int u = 1; u <= n; ++u) {
    for (int v : dag[u]) mark[v] = u;
    for (int v : dag[u])
      for (int w : dag[v])
        if (mark[w] == u) triangles++;
  }
  return triangles;
}
```

- **Trampas y adaptaciones:** (1) La misma orientación por grado sirve para enumerar ciclos de longitud 4 (C_4) en $\mathcal{O}(M\sqrt{M})$.

6.3 Caminos Más Cortos: Dijkstra, Bellman-Ford / SPFA (Ciclo Negativo) y Floyd-Warshall

6.3.1 Dijkstra con Min-Heap y Reconstrucción (`dijkstra` , `get_path_dij`)

- **Cuándo usar:** Caminos más cortos desde una fuente `s` en grafos dirigidos o no dirigidos con pesos **no negativos** ($w_e \ge 0$).
- **Cuándo no usar / alternativa:** Si existe alguna arista con peso negativo ($w_e < 0$), Dijkstra falla; usar **Bellman-Ford** (§6.3.2).
- **Contrato:** Nodos 1-indexed (`1..n`) en `adj_dij` . Inicializa `dist_dij` con `INF` (10^{18}). `get_path_dij(t)` devuelve `{}` si `t` es inalcanzable.
- **Complejidad:** $\mathcal{O}((V+E)\log V)$ tiempo y $\mathcal{O}(V+E)$ memoria.

```
// 1. Dijkstra con Reconstruccion de Camino (1-indexed)
const int MAX_V_DIJ = 200005;
vector<pair<int, ll>> adj_dij[MAX_V_DIJ]; // {vecino, peso}
ll dist_dij[MAX_V_DIJ];
int par_dij[MAX_V_DIJ];
```

```cpp
void dijkstra(int s, int n) {
  fill(dist_dij + 1, dist_dij + n + 1, INF);
  fill(par_dij + 1, par_dij + n + 1, -1);
  priority_queue<pair<ll, int>, vector<pair<ll, int>>,
    greater<>> pq;
  dist_dij[s] = 0;
  pq.push({0, s});
  while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist_dij[u]) continue;
    for (auto [v, w] : adj_dij[u]) {
      if (dist_dij[u] + w < dist_dij[v]) {
        dist_dij[v] = dist_dij[u] + w;
        par_dij[v] = u;
        pq.push({dist_dij[v], v});
      }
    }
  }
}
vi get_path_dij(int target) {
  if (dist_dij[target] == INF) return {};
  vi path;
  for (int cur = target; cur != -1; cur
    = par_dij[cur]) path.pb(cur);
  reverse(all(path));
  return path;
}
```

- **Trampas y adaptaciones:** (1) La guarda `if (d > dist_dij[u]) continue;` es obligatoria para mantener complejidad $\mathcal{O}(E \log V)$ sin `decrease-key`.

6.3.2 Bellman-Ford con Reconstrucción de Ciclo Negativo (`bellman_ford_cycle`)

- **Cuándo usar:** Caminos más cortos con pesos negativos y detección/reconstrucción explícita de un ciclo negativo alcanzable desde `s`.
- **Cuándo no usar / alternativa:** Si todos los pesos son ≥ 0, usar `dijkstra` (§6.3.1).
- **Contrato:** Nodos 1-indexed (`1..n`). Satura `d[e.v] = max(-INF, d[e.u] + e.w)` para evitar overflow a `LLONG_MIN`. Si retorna `true`, `cycle` contiene los vértices del ciclo negativo en orden dirigido.
- **Complejidad:** $\mathcal{O}(V \cdot E)$ tiempo y $\mathcal{O}(V + E)$ memoria.

```cpp
// 2. Bellman-Ford con Reconstruccion de Ciclo Negativo
// Alcanzable
// (Si quieres detectar CUALQUIER ciclo negativo en el grafo
// aunque no sea
// alcanzable desde 's', inicializa todos los d[1..n] = 0 en
// vez de INF).
struct BFEdge { int u, v; ll w; };
bool bellman_ford_cycle(int s, int n,
  const vector<BFEdge>& edges,
                          vll& d, vi& neg_cycle) {
  d.assign(n + 1, INF);
  vi p(n + 1, -1);
  d[s] = 0;
  int x = -1;
  for (int i = 1; i <= n; ++i) {
    x = -1;
    for (const auto& e : edges) {
      if (d[e.u] >= INF) continue;
      ll nd = max(-INF, d[e.u] + e.w);
      if (nd < d[e.v]) {
        d[e.v] = nd;
        p[e.v] = e.u;
        x = e.v;
      }
    }
  }
  if (x == -1) return false; // No hay ciclo negativo
  // Entrar con certeza al ciclo
  for (int i = 0; i < n; ++i) x = p[x];
  neg_cycle.clear();
  for (int cur = x; ; cur = p[cur]) {
    neg_cycle.pb(cur);
    if (cur == x && sz(neg_cycle) > 1) break;
  }
  reverse(all(neg_cycle));
  return true;
}
```

- **Trampas y adaptaciones:** (1) Retroceder `n` veces con `x = p[x]` antes de reconstruir el ciclo es indispensable porque el último nodo relajado en la iteración N puede estar en una cola que sale del ciclo negativo y no dentro del ciclo mismo; (2) para detectar cualquier ciclo negativo global sin importar `s`, inicializar `d[1..n] = 0`.

6.3.3 Floyd-Warshall Todos contra Todos (`floyd_warshall`)

- **Cuándo usar:** Distancias mínimas entre **todos los pares** de vértices (i, j) en grafos pequeños ($V \leq 400$–500), o clausura transitiva.
- **Cuándo no usar / alternativa:** Si $V > 500$ y los pesos son no negativos, correr `dijkstra` desde cada nodo cuesta $\mathcal{O}(V(V+E)\log V)$.
- **Contrato:** Matriz `d` 1-indexed de tamaño `(n + 1) x (n + 1)`, inicializada con `d[i][i] = 0` y `d[i][j] = INF` si no hay arista. Al finalizar, `d[i][i] < 0` indica que `i` pertenece a un ciclo negativo.
- **Complejidad:** $\mathcal{O}(V^3)$ tiempo y $\mathcal{O}(V^2)$ memoria.

```cpp
// 3. Floyd-Warshall (All-Pairs Shortest Path en O(V^3))
void floyd_warshall(int n, vector<vll>& d) {
  // Precondicion: d[i][i] = 0, d[u][v] = min_peso (o INF si
  // no hay arista)
  for (int k = 1; k <= n; ++k)
    for (int i = 1; i <= n; ++i)
      if (d[i][k] < INF)
        for (int j = 1; j <= n; ++j)
          if (d[k][j] < INF)
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
}
```

- **Trampas y adaptaciones:** (1) El nodo intermedio `k` **siempre** debe ser el bucle más externo (`for k ... for i ... for j`); (2) al leer la entrada, hacer `d[u][v] = min(d[u][v], w)` por si vienen aristas paralelas.

6.4 Árboles: Diámetro (Doble BFS), MST (Kruskal y Prim), LCA Binary Lifting y Centroid Decomposition

6.4.1 Diámetro de un Árbol por Doble BFS (`bfs_farthest` , `get_diameter_path`)

- **Cuándo usar:** Hallar el camino simple más largo (y su centro) en un árbol sin pesos (o con pesos no negativos).
- **Cuándo no usar / alternativa:** Si el árbol tiene aristas de **peso negativo**, el truco de los 2 BFS falla; usar **DP en árboles** (§8.3) guardando las dos mayores ramas hacia abajo desde cada nodo.
- **Contrato:** Árbol 1-indexed (`1..n`) en `adj_tr`. `get_diameter_path(n)` devuelve todos los nodos en la ruta del diámetro; su longitud en aristas es `sz(path) - 1`.
- **Complejidad:** $\mathcal{O}(N)$ tiempo y memoria.

```cpp
// 1. Diametro de un Arbol (Doble BFS con reconstruccion de
// ruta)
vi adj_tr[200005];
int par_tr[200005];

pii bfs_farthest(int start, int n) {
  vi d(n + 1, -1);
  queue<int> q;
  q.push(start); d[start] = 0; par_tr[start] = -1;
  int far = start;
  while (!q.empty()) {
    int u = q.front(); q.pop();
    if (d[u] > d[far]) far = u;
    for (int v : adj_tr[u]) {
      if (d[v] == -1) {
        d[v] = d[u] + 1;
        par_tr[v] = u;
        q.push(v);
      }
    }
  }
  return {far, d[far]};
}
vi get_diameter_path(int n) {
  pii p1 = bfs_farthest(1, n);
  pii p2 = bfs_farthest(p1.fi, n);
  vi path;
  for (int cur = p2.fi; cur != -1; cur
    = par_tr[cur]) path.pb(cur);
  reverse(all(path));
  // Longitud del diametro en aristas = sz(path) - 1
  return path;
}
```

- **Trampas y adaptaciones:** (1) El centro del árbol (nodo que minimiza la máxima distancia al resto) es `path[sz(path) / 2]`.

6.4.2 Árbol Generador Mínimo de Kruskal (`kruskal_mst`)

- **Cuándo usar:** Hallar el subgrafo conexo acíclico de menor peso total (MST) en un grafo no dirigido pesado.
- **Cuándo no usar / alternativa:** Si el grafo es dirigido y buscas una arborescencia desde una raíz fija, Kruskal no aplica; usar `directed_mst` (§6.7).
- **Contrato:** Nodos 1-indexed (`1..n`). Retorna el costo total del MST y llena `mst` con las $N-1$ aristas elegidas, o retorna `-1` si el grafo es desconexo. Dependencias: `DSU` (§4.1.1).
- **Complejidad:** $\mathcal{O}(E \log E)$ tiempo y $\mathcal{O}(V + E)$ memoria.

```cpp
// 2. Kruskal MST (requiere DSU de la seccion 4.1)
struct KrEdge {
  int u, v; ll w;
  bool operator<(const KrEdge& o) const { return w < o.w; }
};
ll kruskal_mst(int n, vector<KrEdge>& edges,
  vector<KrEdge>& mst) {
  sort(all(edges));
  DSU dsu(n);
  ll cost = 0;
  for (const auto& e : edges)
    if (dsu.unite(e.u, e.v)) { cost += e.w; mst.pb(e); }
  return (dsu.num_comp == 1) ? cost : -1;
}
```

- **Trampas y adaptaciones:** (1) Para Maximum Spanning Tree, invertir el comparador (`w > o.w`); (2) si el problema pide un *Minimum Spanning Forest*, ignorar el chequeo `dsu.num_comp == 1` y usar `cost` directamente.

6.4.3 LCA, K-ésimo Ancestro y Distancia por Binary Lifting (`LCA`)

- **Cuándo usar:** Responder consultas de Ancestro Común Más Bajo (`lca(u, v)`), k-ésimo ancestro (`kth_ancestor(u, k)`) y distancia en aristas (`dist(u, v)`) en un árbol estático.
- **Cuándo no usar / alternativa:** Si los pesos o valores de los nodos cambian dinámicamente y necesitas consultar agregados en caminos, usar **HLD** (§4.8).
- **Contrato:** Árbol 1-indexed (`1..n`). El padre de la raíz `root` se define como `root` mismo, por lo que pedir un ancestro más arriba de la raíz se satura en `root`.
- **Complejidad:** Construcción $\mathcal{O}(N \log N)$ tiempo y memoria; cada consulta en $\mathcal{O}(\log N)$.

```cpp
// 3. LCA (Lowest Common Ancestor) + K-th Ancestor +
// Distancia en Arbol
struct LCA {
  int n, LOG;
  vi dep;
  vector<vi> up;

  LCA(int n, int root, const vector<vi>& adj) : n(n) {
    LOG = __lg(max(1, n)) + 1;
    dep.assign(n + 1, 0);
    up.assign(n + 1, vi(LOG, root));
    dfs(root, root, adj);
  }
  void dfs(int u, int p, const vector<vi>& adj) {
    up[u][0] = p;
    for (int j = 1; j < LOG; ++j)
      up[u][j] = up[up[u][j - 1]][j - 1];
    for (int v : adj[u]) {
      if (v == p) continue;
      dep[v] = dep[u] + 1;
      dfs(v, u, adj);
    }
  }
  int kth_ancestor(int u, int k) const {
    for (int j = 0; j < LOG; ++j)
      if ((k >> j) & 1) u = up[u][j];
    return u;
  }
  int lca(int u, int v) const {
    if (dep[u] < dep[v]) swap(u, v);
    u = kth_ancestor(u, dep[u] - dep[v]);
    if (u == v) return u;
    for (int j = LOG - 1; j >= 0; --j)
      if (up[u][j] != up[v][j]) { u = up[u][j];
        v = up[v][j]; }
    return up[u][0];
  }
  int dist(int u, int v) const {
    return dep[u] + dep[v] - 2 * dep[lca(u, v)];
  }
};
```

- **Trampas y adaptaciones:** (1) Para encontrar el nodo en el camino de `u` a `v` a distancia `d` desde `u` ($0 \leq d \leq \text{dist}(u,v)$): sea $w = \text{lca}(u,v)$; si $d \leq \text{dep}[u] - \text{dep}[w]$ es `kth_ancestor(u, d)`, si no es `kth_ancestor(v, dist(u, v) - d)`.

6.4.4 Descomposición por Centroides (`CentroidDecomp`)

- **Cuándo usar:** Contar caminos en un árbol que cumplen una propiedad global (longitud exacta K, XOR igual a X, etc.) o mantener el nodo marcado más cercano en $\mathcal{O}(\log N)$ por actualización/consulta.
- **Cuándo no usar / alternativa:** Si las consultas son simplemente sobre un camino entre `u` y `v` dados en la entrada, usar `LCA` (§6.4.3) o `HLD` (§4.8).
- **Contrato:** Árbol 1-indexed (`1..n`). Construye el árbol de centroides de altura $\leq \lceil \log_2 N \rceil$, donde `par_c[c]` es el padre de `c` en el árbol de centroides (`0` en la raíz).
- **Complejidad:** $\mathcal{O}(N \log N)$ tiempo y $\mathcal{O}(N)$ memoria.

```cpp
// 4. Centroid Decomposition (Arbol de Centroides en O(N log
// N))
struct CentroidDecomp {
  int n;
  vector<vi> adj;
  vi sz_c, par_c;
  vector<bool> rem;

  CentroidDecomp(int n, const vector<vi>& g)
    : n(n), adj(g), sz_c(n + 1, 0), par_c(n + 1, 0), rem(
      n + 1, false) {
    build(1, 0);
  }
  int dfs_sz(int u, int p) {
    sz_c[u] = 1;
    for (int v : adj[u])
      if (v != p && !rem[v]) sz_c[u] += dfs_sz(v, u);
    return sz_c[u];
  }
  int get_centroid(int u, int p, int tot) const {
    for (int v : adj[u])
      if (v != p && !rem[v] && sz_c[v] * 2 > tot)
        return get_centroid(v, u, tot);
    return u;
  }
  void build(int u, int p) {
    int tot = dfs_sz(u, 0);
    int c = get_centroid(u, 0, tot);
    // Padre en el arbol de centroides (0 si es la raiz)
    par_c[c] = p;
    rem[c] = true;
    for (int v : adj[c])
      if (!rem[v]) build(v, c);
  }
};
```

- **Trampas y adaptaciones:** (1) En el árbol de centroides, cualquier camino original $u \leftrightarrow v$ pasa por su ancestro común más bajo en el árbol de centroides, que está a lo sumo a $\mathcal{O}(\log N)$ saltos por `par_c` .

6.5 Conectividad, DAGs y 2-SAT: Topological Sort, Puentes/Articulación, SCC (Tarjan) y 2-SAT

6.5.1 Orden Topológico de Kahn por BFS (`topological_sort`)

- **Cuándo usar:** Obtener una permutación lineal de los nodos de un DAG donde toda arista $u \to v$ cumple que u aparece antes que v (ideal para DP sobre DAGs o verificar si existe un ciclo dirigido).
- **Cuándo no usar / alternativa:** Si el grafo tiene ciclos y necesitas procesarlo por bloques, primero comprimirlo en un DAG con `TarjanSCC` (§6.5.3).
- **Contrato:** Nodos 1-indexed (`1..n`). Si `sz(order) < n` , el grafo dirigido **tiene al menos un ciclo** y no admite orden topológico.
- **Complejidad:** $\mathcal{O}(V + E)$ tiempo y memoria.

```cpp
// 1. Orden Topologico (Kahn BFS — detecta ciclos si
// sz(order) < n)
vi topological_sort(int n, const vector<vi>& adj) {
  vi in_deg(n + 1, 0), order;
  for (int u = 1; u <= n; ++u)
    for (int v : adj[u]) in_deg[v]++;
  // Usar Min-Heap si piden orden lexicografico menor
  queue<int> q;
  for (int i = 1; i <= n; ++i)
    if (in_deg[i] == 0) q.push(i);
  while (!q.empty()) {
    int u = q.front(); q.pop();
    order.pb(u);
    for (int v : adj[u])
      if (--in_deg[v] == 0) q.push(v);
  }
  return order;
}
```

- **Trampas y adaptaciones:** (1) Si el problema pide el **orden topológico lexicográficamente menor**, cambiar `queue<int> q` por `priority_queue<int, vi, greater<int>> q` ($\mathcal{O}((V+E)\log V)$).

6.5.2 Puentes y Puntos de Articulación de Tarjan (`BridgesAndCuts`)

- **Cuándo usar:** Encontrar todas las aristas cuya eliminación desconecta el grafo (**puentes**, `low[v] > tin[u]`) y los vértices cuya eliminación desconecta el grafo (**puntos de articulación**, `low[v] >= tin[u]`).
- **Cuándo no usar / alternativa:** Si necesitas el árbol comprimido de componentes biconexas por vértices, usar `BlockCutTree` (§6.8).
- **Contrato:** Nodos 1-indexed (`1..n`). Maneja correctamente **aristas paralelas** ignorando únicamente la primera ocurrencia de la arista hacia el padre (`skipped_parent`), e ignora auto-bucles (`v == u`).
- **Complejidad:** $\mathcal{O}(V + E)$ tiempo y memoria.

```cpp
// 2. Puentes y Puntos de Articulacion en Grafo No Dirigido
// (Tarjan)
struct BridgesAndCuts {
  int n, timer = 0;
  vi tin, low;
  vector<bool> is_cut;
  vector<pii> bridges;

  BridgesAndCuts(int n, const vector<vi>& adj)
    : n(n), tin(n + 1, 0), low(n + 1, 0), is_cut(n + 1,
      false) {
    for (int i = 1; i <= n; ++i)
      if (!tin[i]) dfs(i, -1, adj);
  }
  void dfs(int u, int p, const vector<vi>& adj) {
    tin[u] = low[u] = ++timer;
    int children = 0;
    bool skipped_parent = false;
    for (int v : adj[u]) {
      if (v == u) continue;
      if (v == p && !skipped_parent) {
        skipped_parent = true; continue; }
      if (tin[v]) {
        low[u] = min(low[u], tin[v]);
      } else {
        children++;
        dfs(v, u, adj);
        low[u] = min(low[u], low[v]);
        if (low[v] > tin[u]) bridges.pb({u, v});
        if (p != -1 && low[v] >= tin[u]) is_cut[u] = true;
      }
    }
    if (p == -1 && children > 1) is_cut[u] = true;
  }
};
```

- **Trampas y adaptaciones:** (1) La raíz del árbol DFS (`p == -1`) es punto de articulación si y solo si tiene `children > 1` en el árbol DFS (no en la lista de adyacencia).

6.5.3 Componentes Fuertemente Conexas de Tarjan (`TarjanSCC`)

- **Cuándo usar:** Particionar un grafo dirigido en sus Componentes Fuertemente Conexas maximales (`comp_id[u]`) en una sola pasada DFS para condensarlo en un DAG.
- **Cuándo no usar / alternativa:** Si el grafo es no dirigido, las componentes se hallan con BFS/DFS o `DSU` (§4.1.1).
- **Contrato:** Nodos 1-indexed (`1..n`). `scc_cnt` es la cantidad de SCCs (`1..scc_cnt`). Tarjan emite las SCCs en **orden topológico inverso** (los pozos/sinks reciben menor `comp_id` y las fuentes mayor `comp_id`).
- **Complejidad:** $\mathcal{O}(V + E)$ tiempo y memoria.

```cpp
// 3. Tarjan SCC (Componentes Fuertemente Conexas en Grafo
// Dirigido)
// Nota: comp_id[u] queda en orden topologico INVERSO
// (scc_cnt..1)
struct TarjanSCC {
  int n, timer = 0, scc_cnt = 0;
  vi tin, low, comp_id, st;
  vector<bool> in_st;

  TarjanSCC(int n, const vector<vi>& adj)
    : n(n), tin(n + 1, 0), low(n + 1, 0),
      comp_id(n + 1, 0), in_st(n + 1, false) {
    for (int i = 1; i <= n; ++i)
      if (!tin[i]) dfs(i, adj);
  }
  void dfs(int u, const vector<vi>& adj) {
    tin[u] = low[u] = ++timer;
    st.pb(u); in_st[u] = true;
    for (int v : adj[u]) {
      if (!tin[v]) {
        dfs(v, adj);
        low[u] = min(low[u], low[v]);
      } else if (in_st[v]) {
        low[u] = min(low[u], tin[v]);
      }
    }
    if (low[u] == tin[u]) {
      scc_cnt++;
      while (true) {
        int v = st.back(); st.pop_back();
        in_st[v] = false;
        comp_id[v] = scc_cnt;
        if (u == v) break;
      }
    }
  }
};
```

- **Trampas y adaptaciones:** (1) Al actualizar `low[u]` con un vecino ya visitado `v` , verificar obligatoriamente `if (in_st[v])` (si `v` ya salió de la pila en otra SCC cerrada, ignorarlo).

6.5.4 Solucionador 2-SAT en Tiempo Lineal (`TwoSAT`)

- **Cuándo usar:** Asignar valores booleanos a N variables ($x_0, ..., x_{N-1}$) sujetas a cláusulas de disyunción de 2 literales ($a \vee b$), implicación ($a \Rightarrow b$), igualdad ($a = b$) o desigualdad ($a \neq b$).
- **Cuándo no usar / alternativa:** Si las cláusulas tienen 3 o más literales (3-SAT), el problema es NP-completo.
- **Contrato:** Variables **0-indexed** (`0..n-1`). `solve()` retorna `false` si alguna variable e inverso caen en la misma SCC; si retorna `true` , la asignación válida queda en `ans[0..n-1]` . Dependencias: `TarjanSCC` (§6.5.3).
- **Complejidad:** $\mathcal{O}(N + M)$ tiempo y memoria.

```cpp
// 4. 2-SAT Solver en O(N + M) (Variables 0-indexed: 0..n-1)
struct TwoSAT {
  int n;
  vector<vi> adj;
  vector<bool> ans;

  TwoSAT(int n) : n(n), adj(2 * n + 1), ans(n, false) {}
  // Nodo para variable 'x' (0..n-1):
  // Si val == true -> nodo (x + 1); si val == false -> nodo
  // (x + 1 + n)
  int var_node(int x, bool val) const {
    return val ? (x + 1) : (x + 1 + n); }
  int neg(int u) const {
    return (u <= n) ? (u + n) : (u - n); }

  void add_imp(int u, int v) {
    adj[u].pb(v);
    adj[neg(v)].pb(neg(u));
  }
  // (x == val_x) OR (y == val_y)
  void add_or(int x, bool vx, int y, bool vy) {
    int u = var_node(x, vx), v = var_node(y, vy);
    add_imp(neg(u), v);
  }
  void set_val(int x, bool vx) { add_or(x, vx, x, vx); }
  void make_eq(int x, int y) {
    add_or(x, true, y, false);
    add_or(x, false, y, true);
  }
  void make_diff(int x, int y) {
    add_or(x, true, y, true);
    add_or(x, false, y, false);
  }
  bool solve() {
    TarjanSCC scc(2 * n, adj);
    for (int i = 0; i < n; ++i) {
      int u = i + 1, nu = i + 1 + n;
      if (scc.comp_id[u] == scc.comp_id[nu]) return false;
      // En Tarjan, menor comp_id significa mas abajo en el
      // DAG (sink)
      ans[i] = (scc.comp_id[u] < scc.comp_id[nu]);
    }
    return true;
  }
};
```

- **Trampas y adaptaciones:** (1) Para prohibir que x e y sean ambas verdaderas ($\neg(x \wedge y)$), llamar `add_or(x, false, y, false)` .

6.6 Flujos y Matching: Dinic, Min-Cost Max-Flow (MCMF), Kuhn + König, Algoritmo Húngaro y Blossom

6.6.1 Flujo Máximo y Corte Mínimo de Dinic (`Dinic`)

- **Cuándo usar:** Calcular el flujo máximo $s \to t$ y reconstruir el corte mínimo $s - t$ (`left_of_min_cut`), emparejamiento bipartito en $\mathcal{O}(E\sqrt{V})$, o problemas de selección de proyectos (*Project Selection*).
- **Cuándo no usar / alternativa:** Si cada arista tiene además un costo por unidad de flujo y buscas costo mínimo, usar `MCMF` (§6.6.2).
- **Contrato:** Nodos 1-indexed (`1..n`). Para aristas no dirigidas de capacidad `cap` , cambiar el `0` de la arista reversa en `add_edge` por `cap` . Llamar `left_of_min_cut(u)` después de `max_flow()` .
- **Complejidad:** $\mathcal{O}(V^2E)$ en redes generales; $\mathcal{O}(E\sqrt{V})$ en redes unitarias / matching bipartito.

```
// 1. Dinic (Max Flow + Min Cut + Bipartite Matching en O(E
// sqrt(V)))
struct Dinic {
  struct Edge { int to, rev; ll cap, flow = 0; };
  int n, s, t;
  vector<vector<Edge>> g;
  vi lvl, ptr;

  Dinic(int n, int s, int t) : n(n), s(s), t(t), g(n + 1),
    lvl(n + 1), ptr(n + 1) {}
  void add_edge(int u, int v, ll cap) {
    g[u].pb({v, sz(g[v]), cap, 0});
    // Cambiar 0 por cap si es no dirigido
    g[v].pb({u, sz(g[u]) - 1, 0, 0});
  }
  bool bfs() {
    fill(all(lvl), -1);
    queue<int> q;
    lvl[s] = 0; q.push(s);
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (const auto& e : g[u])
        if (e.cap - e.flow > 0 && lvl[e.to] == -1) {
          lvl[e.to] = lvl[u] + 1;
          q.push(e.to);
        }
    }
    return lvl[t] != -1;
  }
  ll dfs(int u, ll pushed) {
    if (pushed == 0 || u == t) return pushed;
    for (int& cid = ptr[u]; cid < sz(g[u]); ++cid) {
      auto& e = g[u][cid];
      if (lvl[u] + 1 != lvl[e.to] || e.cap - e.flow
        == 0) continue;
      ll tr = dfs(e.to, min(pushed, e.cap - e.flow));
      if (tr == 0) continue;
      e.flow += tr;
      g[e.to][e.rev].flow -= tr;
      return tr;
    }
    return 0;
  }
  ll max_flow() {
    ll flow = 0;
    while (bfs()) {
      fill(all(ptr), 0);
      while (ll pushed = dfs(s, INF)) flow += pushed;
    }
    return flow;
  }
  // Retorna true si 'u' queda del lado de la fuente S en el
  // Min-Cut
  // (Llamar DESPUES de max_flow()):
  bool left_of_min_cut(int u) const { return lvl[u] != -1; }
};
```

- **Trampas y adaptaciones:** (1) El puntero `int& cid = ptr[u]` (*current arc optimization*) es indispensable: sin referencia `&` , Dinic degrada a $\mathcal{O}(VE^2)$ y da TLE.

6.6.2 Flujo Máximo de Costo Mínimo (`MCMF`)

- **Cuándo usar:** Enviar flujo desde `s` hasta `t` (o a lo sumo `max_f` unidades) minimizando el costo total $\sum f_e \cdot \text{cost}_e$ en una red sin ciclos de costo negativo.
- **Cuándo no usar / alternativa:** Si solo necesitas asignación 1-a-1 entre N filas y M columnas en grafo bipartito denso, el `hungarian` (§6.6.4) es más rápido y corto.
- **Contrato:** Nodos 1-indexed (`1..n`). La arista reversa se crea automáticamente con capacidad `0` y costo `-cost` . `solve(s, t, max_f)` retorna `{flujo_enviado, costo_minimo}` .
- **Complejidad:** $\mathcal{O}(F \cdot kE)$ práctico con SPFA (donde F es el flujo total).

```
// 2. Min-Cost Max-Flow (MCMF con SPFA; requiere red sin
// ciclos negativos)
struct MCMF {
  struct Edge { int to, rev; ll cap, cost, flow = 0; };
  int n;
  vector<vector<Edge>> g;
  vll dist, cap_pushed;
  vi par_u, par_e;
  vector<bool> in_q;

  MCMF(int n)
    : n(n), g(n + 1), dist(n + 1), cap_pushed(n + 1),
      par_u(n + 1), par_e(n + 1), in_q(n + 1) {}

  void add_edge(int u, int v, ll cap, ll cost) {
    g[u].pb({v, sz(g[v]), cap, cost, 0});
    g[v].pb({u, sz(g[u]) - 1, 0, -cost, 0});
  }
  bool spfa(int s, int t) {
    fill(all(dist), INF);
    fill(all(in_q), false);
    queue<int> q;
    dist[s] = 0; cap_pushed[s] = INF;
    q.push(s); in_q[s] = true;
    while (!q.empty()) {
      int u = q.front(); q.pop();
      in_q[u] = false;
      for (int i = 0; i < sz(g[u]); ++i) {
        auto& e = g[u][i];
        if (e.cap - e.flow > 0 && dist[u]
          + e.cost < dist[e.to]) {
          dist[e.to] = dist[u] + e.cost;
          par_u[e.to] = u; par_e[e.to] = i;
          cap_pushed[e.to] = min(cap_pushed[u],
            e.cap - e.flow);
          if (!in_q[e.to]) { q.push(e.to);
            in_q[e.to] = true; }
        }
      }
    }
    return dist[t] != INF;
  }
  // Retorna {flujo_total, costo_minimo} enviando a lo sumo
  // max_f
  pll solve(int s, int t, ll max_f = INF) {
    ll flow = 0, cost = 0;
    while (flow < max_f && spfa(s, t)) {
      ll f = min(max_f - flow, cap_pushed[t]);
      flow += f;
      cost += f * dist[t];
      for (int cur = t; cur != s; cur = par_u[cur]) {
        int p = par_u[cur], idx = par_e[cur];
        g[p][idx].flow += f;
        g[cur][g[p][idx].rev].flow -= f;
      }
    }
    return {flow, cost};
  }
};
```

- **Trampas y adaptaciones:** (1) Para **Max-Cost Max-Flow** en un DAG, insertar los costos negados (`-cost`) y negar el costo retornado.

6.6.3 Matching Bipartito de Kuhn y Reconstrucción de König (`BipartiteMVC`)

- **Cuándo usar:** Emparejamiento máximo en grafos bipartitos pequeños/medianos ($N_1 \cdot E \le 10^7$) y reconstrucción explícita de los vértices del **Minimum Vertex Cover** (o Maximum Independent Set) mediante el Teorema de König.
- **Cuándo no usar / alternativa:** Si $N_1, N_2 \approx 5 \cdot 10^4$, Kuhn $\mathcal{O}(N_1 E)$ puede dar TLE; usar `Dinic` (§6.6.1) en $\mathcal{O}(E\sqrt{V})$.
- **Contrato:** Lado izquierdo `1..n1` , lado derecho `1..n2` . `min_vertex_cover()` retorna `{cov_l, cov_r}` donde $|\text{cov}_l| + |\text{cov}_r| = \text{max_matching}()$.
- **Complejidad:** $\mathcal{O}(N_1 \cdot E)$ para el matching; $\mathcal{O}(N_1 + N_2 + E)$ para la reconstrucción de König.

```
// 2b. Kuhn (Matching Bipartito en O(N1 * E)) +
// Reconstruccion de Konig (Min Vertex Cover)
// Lado izquierdo: 1..n1, Lado derecho: 1..n2.
struct BipartiteMVC {
  int n1, n2;
  vector<vi> adj;
  vi mt_l, mt_r, vis_l, vis_r;
  int timer = 0;

  BipartiteMVC(int n1, int n2)
    : n1(n1), n2(n2), adj(n1 + 1), mt_l(n1 + 1, 0),
      mt_r(n2 + 1, 0), vis_l(n1 + 1, 0), vis_r(n2 + 1, 0) {}
  // u in [1..n1], v in [1..n2]
  void add_edge(int u, int v) { adj[u].pb(v); }

  bool dfs(int u) {
    if (vis_l[u] == timer) return false;
    vis_l[u] = timer;
    for (int v : adj[u]) {
      if (!mt_r[v] || dfs(mt_r[v])) {
        mt_l[u] = v; mt_r[v] = u;
        return true;
      }
    }
    return false;
  }
  int max_matching() {
    int matched = 0;
    for (int u = 1; u <= n1; ++u)
      if (!mt_l[u]) { ++timer; if (dfs(u)) matched++; }
    return matched;
  }
  void dfs_konig(int u) {
    vis_l[u] = 1;
    for (int v : adj[u]) {
      if (!vis_r[v] && mt_l[u] != v) {
        vis_r[v] = 1;
        if (mt_r[v] && !vis_l[mt_r[v]]) dfs_konig(mt_r[v]);
      }
    }
  }
  // Retorna {nodos_izquierda_en_MVC, nodos_derecha_en_MVC}
  // (Teorema de Konig)
  pair<vi, vi> min_vertex_cover() {
    max_matching();
    fill(all(vis_l), 0); fill(all(vis_r), 0);
    for (int u = 1; u <= n1; ++u)
      if (!mt_l[u]) dfs_konig(u);
    vi cov_l, cov_r;
    for (int u = 1; u <= n1; ++u) if (!vis_l[u]) cov_l.pb(
      u);
    for (int v = 1; v <= n2; ++v) if (vis_r[v]) cov_r.pb(v);
    return {cov_l, cov_r};
  }
};
```

- **Trampas y adaptaciones:** (1) El **Maximum Independent Set** bipartito es exactamente el complemento del Minimum Vertex Cover: los nodos con `vis_l[u] == 1` a la izquierda y `vis_r[v] == 0` a la derecha.

6.6.4 Algoritmo Húngaro de Asignación de Costo Mínimo (`hungarian`)

- **Cuándo usar:** Asignar cada una de las N filas a una columna distinta entre M columnas ($N \le M \le 500$) minimizando la suma de costos $\sum_{i=1}^{N} a[i][\text{match}[i]]$.
- **Cuándo no usar / alternativa:** Si el grafo bipartito es muy disperso con capacidades > 1, usar `MCMF` (§6.6.2).
- **Contrato:** Matriz `a` **1-indexed** de dimensión `(n + 1) x (m + 1)` con `n <= m` .admite costos negativos o cero. Retorna `{costo_minimo, match_row}` .
- **Complejidad:** $\mathcal{O}(N^2M)$ tiempo y $\mathcal{O}(NM)$ memoria.

```
// 3. Algoritmo Hungaro (Asignacion N x M de Costo Minimo en
// O(N^2 * M))
// Matriz 'a' 1-indexed de tamano (n+1) x (m+1) con n <= m.
// Para MAXIMIZAR costo: negar todos los valores de la
// matriz antes de llamar.
pair<ll, vi> hungarian(int n, int m, const vector<vll>& a) {
  vll u(n + 1, 0), v(m + 1, 0);
  vi p(m + 1, 0), way(m + 1, 0);
  for (int i = 1; i <= n; ++i) {
    p[0] = i;
    int j0 = 0;
    vll minv(m + 1, INF);
    vector<bool> used(m + 1, false);
    do {
      used[j0] = true;
      int i0 = p[j0], j1 = 0;
      ll delta = INF;
      for (int j = 1; j <= m; ++j) {
        if (!used[j]) {
          ll cur = a[i0][j] - u[i0] - v[j];
          if (cur < minv[j]) { minv[j] = cur; way[j] = j0; }
          if (minv[j] < delta) { delta = minv[j]; j1 = j; }
        }
      }
      for (int j = 0; j <= m; ++j) {
        if (used[j]) { u[p[j]] += delta; v[j] -= delta; }
        else minv[j] -= delta;
      }
      j0 = j1;
    } while (p[j0] != 0);
    do {
      int j1 = way[j0];
```

```
      p[j0] = p[j1];
      j0 = j1;
    } while (j0);
  }
  vi match_row(n + 1, 0);
  for (int j = 1; j <= m; ++j) if (
    p[j] > 0) match_row[p[j]] = j;
  // {costo_minimo, columna_asignada_a_cada_fila}
  return {-v[0], match_row};
}
```

- **Trampas y adaptaciones:** (1) Si $N > M$, transponer la matriz antes de llamar; (2) para **maximizar** la ganancia total, negar todas las entradas `a[i][j] = -a[i][j]` y tomar `-cost`.

6.6.5 Matching Máximo en Grafo General: Algoritmo Blossom de Edmonds (`Blossom`)

- **Cuándo usar:** Hallar el máximo número de parejas disjuntas en un grafo no dirigido **general (no bipartito, con posibles ciclos impares)** para $V \leq 500$.
- **Cuándo no usar / alternativa:** Si el grafo es bipartito, usar `Dinic` (§6.6.1) o `BipartiteMVC` (§6.6.3); en grafos no bipartitos el teorema de König NO se cumple.
- **Contrato:** Nodos 1-indexed (`1..n`). `solve()` retorna el número de parejas emparejadas y deja en `match[u]` la pareja del nodo `u` (`0` si quedó libre).
- **Complejidad:** $\mathcal{O}(V^3)$ tiempo y $\mathcal{O}(V+E)$ memoria.

```
// 4. Blossom (Max Matching en Grafo General No Bipartito en
// O(V^3))
struct Blossom {
  int n, timer = 0;
  vector<vi> adj;
  vi match, p, base, vis, in_q, in_b;
  queue<int> q;

  Blossom(int n)
    : n(n), adj(n + 1), match(n + 1, 0), p(n + 1, 0),
      base(n + 1, 0), vis(n + 1, 0), in_q(n + 1, 0), in_b(
        n + 1, 0) {}

  void add_edge(int u, int v) { adj[u].pb(v); adj[v].pb(u);
    }

  int lca(int u, int v) {
    ++timer;
    while (true) {
      if (u) {
        u = base[u];
        if (vis[u] == timer) return u;
        vis[u] = timer;
        u = p[match[u]];
      }
      swap(u, v);
    }
  }
  void mark_blossom(int u, int v, int b) {
    while (base[u] != b) {
      in_b[base[u]] = in_b[base[match[u]]] = 1;
      p[u] = v;
      v = match[u];
      u = p[v];
    }
  }
  int bfs(int s) {
    fill(all(p), 0); fill(all(in_q), 0);
    for (int i = 1; i <= n; ++i) base[i] = i;
    while (!q.empty()) q.pop();
    q.push(s); in_q[s] = 1;
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (int v : adj[u]) {
        if (base[u] == base[v] || match[u] == v) continue;
        if (v == s || (match[v] && p[match[v]])) {
          int b = lca(u, v);
          fill(all(in_b), 0);
          mark_blossom(u, v, b);
          mark_blossom(v, u, b);
          for (int i = 1; i <= n; ++i)
            if (in_b[base[i]]) {
              base[i] = b;
              if (!in_q[i]) { q.push(i); in_q[i] = 1; }
            }
        } else if (!p[v]) {
          p[v] = u;
          if (!match[v]) return v;
          v = match[v];
          q.push(v); in_q[v] = 1;
        }
      }
    }
    return 0;
  }
  int solve() {
    int ans = 0;
    for (int i = 1; i <= n; ++i) {
      if (!match[i]) {
        int cur = bfs(i);
        if (cur) {
          ans++;
          while (cur) {
            int nxt = match[p[cur]];
            match[cur] = p[cur];
            match[p[cur]] = cur;
            cur = nxt;
          }
        }
      }
    }
    // Cantidad de parejas; match[u] indica pareja de u
    return ans;
  }
};
```

- **Trampas y adaptaciones:** (1) No agregar dos veces la arista en la entrada porque `add_edge(u, v)` ya inserta ambas direcciones.

6.7 Árbol Generador Mínimo Dirigido (Directed MST / Arborescence)

- **Cuándo usar:** Hallar la arborescencia dirigida de peso mínimo enraizada en `root` que alcance todos los demás nodos en un grafo **dirigido** con pesos.
- **Cuándo no usar / alternativa:** Si el grafo es no dirigido, usar `kruskal_mst` (§6.4.2).
- **Contrato:** Nodos **0-indexed** (`0..n-1`). Retorna `{costo_total, parent_edge}`, o `{-1, {}}` si algún nodo es inalcanzable desde `root`. Usa *Skew Heaps* con propagación perezosa y `RollbackDSU` (§4.1.2). Dependencias: `RollbackDSU` (§4.1.2).
- **Complejidad:** $\mathcal{O}(E \log V)$ tiempo y $\mathcal{O}(V+E)$ memoria.

```
// Edmonds' Chu-Liu-Edmonds en O(E log V) con Skew Heaps
struct DirEdge { int u, v; ll w; };
struct SkewNode {
  DirEdge key;
  SkewNode *l = 0, *r = 0;
  ll delta = 0;
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  }
  DirEdge top() { prop(); return key; }
};
SkewNode* skew_merge(SkewNode *a, SkewNode *b) {
  if (!a || !b) return a ? a : b;
  a->prop(); b->prop();
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = skew_merge(b, a->r)));
  return a;
}
void skew_pop(SkewNode*& a) {
  a->prop(); a = skew_merge(a->l, a->r);
}

// Retorna {costo_total, parent_edge} (-1 si no es conexo)
pair<ll, vi> directed_mst(int n, int root,
  vector<DirEdge>& edges) {
  struct ArbUF {
    vi p, r;
    vector<pair<int, int>> hist;
    ArbUF(int n) : p(n), r(n, 0) { iota(all(p), 0); }
    int find(int i) { return p[i] == i ? i : find(p[i]); }
    bool unite(int i, int j) {
      i = find(i); j = find(j);
      if (i == j) return false;
      if (r[i] < r[j]) swap(i, j);
      hist.push_back({j, r[i] == r[j]});
      p[j] = i;
      if (r[i] == r[j]) r[i]++;
      return true;
    }
    int time() { return sz(hist); }
    void rollback(int t) {
      while (sz(hist) > t) {
        auto [j, inc] = hist.back(); hist.pop_back();
        if (inc) r[p[j]]--;
        p[j] = j;
      }
    }
  } uf(n);

  vector<SkewNode*> heap(n, 0);
  for (auto& e : edges) heap[e.v] = skew_merge(heap[e.v],
    new SkewNode{e});
  ll res = 0;
  vi seen(n, -1), path(n), par(n, -1);
  seen[root] = root;
  vector<DirEdge> Q(n), in(n, {-1, -1, 0});
  deque<tuple<int, int, vector<DirEdge>>> cycs;

  for (int s = 0; s < n; ++s) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
      if (!heap[u]) return {-1, {}}; // Inalcanzable
      DirEdge e = heap[u]->top();
      heap[u]->delta -= e.w; skew_pop(heap[u]);
      Q[qi] = e; path[qi++] = u; seen[u] = s;
      res += e.w; u = uf.find(e.u);
      if (seen[u] == s) { // Ciclo detectado: contraer
        SkewNode* cyc = 0;
        int end = qi, t = uf.time();
        do cyc = skew_merge(cyc, heap[w = path[--qi]]);
        while (uf.unite(u, w));
        u = uf.find(u); heap[u] = cyc; seen[u] = -1;
        cycs.push_front({u, t, {&Q[qi], &Q[end]}});
      }
    }
    for (int i = 0; i < qi; ++i) in[uf.find(Q[i].v)] = Q[i];
  }
  for (auto& [u, t, comp] : cycs) { // Reconstruccion
    uf.rollback(t);
    DirEdge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.v)] = e;
    in[uf.find(inEdge.v)] = inEdge;
  }
  for (int i = 0; i < n; ++i) par[i] = in[i].u;
  return {res, par};
}
```

- **Trampas y adaptaciones:** (1) A diferencia del resto del bloque de grafos, esta implementación de Edmonds es **0-indexed** (`0..n-1`); restar `1` a `u, v, root` si la entrada viene 1-indexed.

6.8 Componentes Biconexas y Block-Cut Tree

- **Cuándo usar:** Comprimir un grafo no dirigido general en el árbol bipartito entre nodos originales (`1..N`) y bloques biconexos (`N+1..N+sz(blocks)`), por ejemplo para saber si todo camino simple entre `u` y `v` debe pasar por un punto de articulación `c`.
- **Cuándo no usar / alternativa:** Si solo necesitas las componentes 2-arista-conexas (sin aristas puente), basta colapsar componentes separadas por `BridgesAndCuts::bridges` (§6.5.2).
- **Contrato:** Nodos 1-indexed (`1..n`). Tras `build()`, `bct_adj` tiene `bct_nodes = n + sz(blocks)` vértices: una arista $(u, N+b)$ en `bct_adj` indica que el vértice original `u` pertenece al bloque `b`. Maneja correctamente nodos aislados y aristas paralelas.
- **Complejidad:** $\mathcal{O}(V+E)$ tiempo y memoria.

```
struct BlockCutTree {
  struct Edge { int to, id; };
  int n, m = 0;
  vector<vector<Edge>> adj;
  vector<vi> bct_adj, blocks;
  vi dfn, low, is_cut, bct_id;
  vector<pair<pii, int>> st;
  int timer = 0, bct_nodes = 0;
  vector<vector<pii>> biconn_edges;

  BlockCutTree(int n) : n(n), adj(n + 1), dfn(n + 1, 0),
    low(n + 1, 0), is_cut(n + 1, 0), bct_id(n + 1, 0) {}

  void add_edge(int u, int v) {
    if (u == v) return;
    ++m;
    adj[u].push_back({v, m});
    adj[v].push_back({u, m});
  }
  void dfs(int u, int p_edge = 0) {
    dfn[u] = low[u] = ++timer;
    if (adj[u].empty()) {
      blocks.push_back({u});
      return;
```

```
    }
    int children = 0;
    for (auto [v, id] : adj[u]) {
      if (id == p_edge) continue;
      if (dfn[v]) {
        low[u] = min(low[u], dfn[v]);
        if (dfn[v] < dfn[u]) st.push_back({{u, v}, id});
      } else {
        children++;
        st.push_back({{u, v}, id});
        dfs(v, id);
        low[u] = min(low[u], low[v]);
        if (low[v] >= dfn[u]) {
          if (p_edge != 0 || children > 1) is_cut[u] = 1;
          biconn_edges.push_back({});
          vi comp_v;
          while (true) {
            auto [edge, eid] = st.back(); st.pop_back();
            biconn_edges.back().push_back(edge);
            comp_v.push_back(edge.fi);
            comp_v.push_back(edge.se);
            if (eid == id) break;
          }
          sort(all(comp_v));
          comp_v.erase(unique(all(comp_v)), comp_v.end());
          blocks.push_back(comp_v);
        }
      }
    }
  }
  void build() {
    for (int i = 1; i <= n; ++i) if (!dfn[i]) dfs(i);
    int cuts = 0;
    for (int i = 1; i <= n; ++i) if (
      is_cut[i]) bct_id[i] = ++cuts;
    bct_nodes = cuts;
    bct_adj.assign(cuts + sz(blocks) + 1, {});

    for (const auto& verts : blocks) {
      int block_node = ++bct_nodes;
      for (int u : verts) {
        if (is_cut[u]) {
          bct_adj[bct_id[u]].push_back(block_node);
          bct_adj[block_node].push_back(bct_id[u]);
        } else bct_id[u] = block_node;
      }
    }
  }
};
```

- **Trampas y adaptaciones:** (1) Un vértice `c` es obligatorio en todo camino simple entre `u` y `v` ($c \neq u, v$) si y solo si `c` es punto de articulación (`is_cut[c] == 1`) y `c` está en el camino del árbol `bct_adj` entre `u` y `v`.

6.9 Camino y Ciclo Euleriano (Hierholzer)

- **Cuándo usar:** Encontrar un recorrido que utilice **exactamente una vez cada arista** de un grafo dirigido (con posibles multiaristas y auto-bucles), como en reconstrucción de cadenas por k-mers (grafos de De Bruijn).
- **Cuándo no usar / alternativa:** Si buscas visitar cada **vértice** exactamente una vez (Camino Hamiltoniano), el problema es NP-completo y requiere `tsp_subset_dp` (§8.2.2) para $N \leq 20$.
- **Contrato:** Nodos 1-indexed (`1..n`). `find_tour()` verifica grados de entrada/salida, reinicia su estado interno y retorna los $M+1$ vértices del recorrido en orden, o `{}` si el grafo no es euleriano (o tiene aristas en componentes desconectadas).
- **Complejidad:** $\mathcal{O}(V+E)$ tiempo y memoria.

```
// Hierholzer Canónico para Grafo Dirigido con multiaristas
// (1-indexed)
// Nota: find_tour() reinicia cur_edge y used_edge, por lo
// que admite llamadas repetidas.
struct EulerianDirected {
  int n, m = 0;
  vector<vector<pii>> adj; // {v, edge_id}
  vi in_deg, out_deg, cur_edge;
  vector<bool> used_edge;

  EulerianDirected(int n) : n(n), adj(n + 1), in_deg(n + 1,
    0),
    out_deg(n + 1, 0), cur_edge(n + 1, 0) {}

  void add_edge(int u, int v) {
    adj[u].push_back({v, ++m});
    out_deg[u]++; in_deg[v]++;
  }
  vi find_tour() {
    if (m == 0) return {1};
    fill(all(cur_edge), 0);
    used_edge.assign(m + 1, false);
    int start = 1, one_out = 0, one_in = 0;
    for (int i = 1; i <= n; ++i) {
      if (out_deg[i] - in_deg[i] == 1) { one_out++;
        start = i; }
      else if (in_deg[i] - out_deg[i] == 1) one_in++;
      // Imposible
      else if (in_deg[i] != out_deg[i]) return {};
    }
    if (!(one_out == 0 && one_in == 0) && !(
      one_out == 1 && one_in == 1))
      return {};

    for (int i = 1; i <= n; ++i)
      if (out_deg[i] > 0 && one_out == 0) { start = i;
        break; }

    vi path, st = {start};
    while (!st.empty()) {
      int u = st.back();
      while (cur_edge[u] < sz(
        adj[u]) && used_edge[adj[u][cur_edge[u]].second])
        cur_edge[u]++;
      if (cur_edge[u] < sz(adj[u])) {
        auto [v, id] = adj[u][cur_edge[u]++];
        used_edge[id] = true;
        st.push_back(v);
      } else {
        path.push_back(u);
        st.pop_back();
      }
    }
    reverse(all(path));
    if (sz(path) != m + 1) return {}; // Grafo desconectado
    return path;
  }
};
```

- **Trampas y adaptaciones:** (1) Para grafo **no dirigido**, existe camino euleriano si a lo sumo 2 vértices tienen grado impar y todas las aristas pertenecen a una sola componente conexa; marcar `used_edge[id] = true` para ambas direcciones de la arista `id`.

6.10 Flujo Máximo en Redes Densas: Push-Relabel (Highest-Label + Gap)

- **Cuándo usar:** Flujo máximo $s \to t$ en redes muy densas ($V \leq 5000, E \leq 10^6$) con capacidades grandes donde Dinic sufre demasiadas fases de aumento.
- **Cuándo no usar / alternativa:** En redes unitarias o matching bipartito, `Dinic` (§6.6.1) es más corto y corre en $\mathcal{O}(E\sqrt{V})$.
- **Contrato:** Nodos 1-indexed (`1..n`). Implementa preflujo con selección *Highest-Label* (`B[b_max]`) y heurística *Gap*.
- **Complejidad:** $\mathcal{O}(V^2\sqrt{E})$ peor caso teórico (muy rápido en la práctica).

```
// Push-Relabel (Highest-Label Preflow-Push O(V^2 sqrt(E)))
struct PushRelabel {
  struct Edge { int to, rev; ll cap, flow; };
  int n;
  vector<vector<Edge>> adj;
  vector<ll> excess;
  vi height, count;
  vector<bool> active;
  vector<vi> B;
  int b_max = 0;

  PushRelabel(int n) : n(n), adj(n + 1), excess(n + 1, 0),
    height(n + 1, 0), count(2 * n + 2, 0), active(n + 1,
      false), B(2 * n + 2) {}

  void add_edge(int from, int to, ll cap) {
    adj[from].push_back({to, sz(adj[to]), cap, 0});
    adj[to].push_back({from, sz(adj[from]) - 1, 0, 0});
  }
  void enqueue(int v) {
    if (!active[v] && excess[v] > 0 && height[v] < n) {
      active[v] = true;
      B[height[v]].push_back(v);
      b_max = max(b_max, height[v]);
    }
  }
  void push(int u, Edge& e) {
    ll push_amt = min(excess[u], e.cap - e.flow);
    if (push_amt == 0 || height[u] <= height[e.to]) return;
    e.flow += push_amt;
    adj[e.to][e.rev].flow -= push_amt;
    excess[u] -= push_amt;
    excess[e.to] += push_amt;
    enqueue(e.to);
  }
  void gap(int k) {
    for (int v = 1; v <= n; ++v) {
      if (height[v] > k && height[v] < n) {
        count[height[v]]--;
        height[v] = n + 1;
        count[height[v]]++;
        enqueue(v);
      }
    }
  }
  void relabel(int v) {
    int old_h = height[v];
    count[old_h]--;
    int d = 2 * n;
    for (auto& e : adj[v])
      if (e.cap - e.flow > 0) d = min(d, height[e.to]);
    height[v] = d + 1;
    count[height[v]]++;
    enqueue(v);
    if (old_h < n && count[old_h] == 0) gap(old_h);
  }
  ll max_flow(int s, int t) {
    if (s == t) return 0;
    fill(all(count), 0);
    count[0] = n - 1; count[n] = 1;
    height[s] = n; active[s] = active[t] = true;
    for (auto& e : adj[s]) {
      excess[s] += e.cap;
      push(s, e);
    }
    while (b_max >= 0) {
      if (B[b_max].empty()) { b_max--; continue; }
      int v = B[b_max].back(); B[b_max].pop_back();
      active[v] = false;
      for (auto& e : adj[v]) {
        if (excess[v] == 0) break;
        push(v, e);
      }
      if (excess[v] > 0) relabel(v);
    }
    return excess[t];
  }
};
```

- **Trampas y adaptaciones:** (1) Para capacidades no dirigidas, agregar dos llamadas `add_edge(u, v, cap)` y `add_edge(v, u, cap)`.

6.11 Grafos Funcionales (Successors): Ciclos, Colas y Binary Lifting

- **Cuándo usar:** Grafos dirigidos donde **cada nodo tiene grado de salida exactamente 1** ($u \to \text{to}[u]$): hallar el nodo tras $K \leq 10^{18}$ pasos (`jump`), el mínimo número de pasos de `u` a `v` (`min_steps`), o la longitud e índice del ciclo de cada componente.
- **Cuándo no usar / alternativa:** Si los nodos tienen múltiples aristas de salida, usar `TarjanSCC` (§6.5.3) o `LCA` en árboles (§6.4.3).
- **Contrato:** Nodos 1-indexed (`1..n`); `to[u] \in [1, n]`. `LOG = 62` soporta $K \leq 2^{61} > 2 \cdot 10^{18}$. `min_steps(u, v)` retorna `-1` si `v` es inalcanzable desde `u`.
- **Complejidad:** Construcción $\mathcal{O}(N \log K_{\max})$ tiempo y memoria; `jump` y `min_steps` en $\mathcal{O}(\log K_{\max})$.

```
struct FunctionalGraph {
  static const int LOG = 62; // Soporta K <= 2e18
  int n, cyc_cnt = 0;
  vi to, in_deg, dist_cyc, root_cyc, cyc_id, cyc_pos,
    cyc_len;
  vector<vi> up;

  FunctionalGraph(int n, const vi& nxt)
    : n(n), to(nxt), in_deg(n + 1, 0), dist_cyc(n + 1, 0),
      root_cyc(n + 1, 0), cyc_id(n + 1, 0), cyc_pos(n + 1,
        0),
      cyc_len(n + 1, 0), up(LOG, vi(n + 1, 0)) {
    for (int i = 1; i <= n; ++i) {
      up[0][i] = to[i];
      in_deg[to[i]]++;
    }
    for (int j = 1; j < LOG; ++j)
      for (int i = 1; i <= n; ++i)
        up[j][i] = up[j - 1][up[j - 1][i]];

    queue<int> q;
```

```
    vector<bool> on_cyc(n + 1, true);
    for (int i = 1; i <= n; ++i)
      if (in_deg[i] == 0) { q.push(i); on_cyc[i] = false; }
    while (!q.empty()) {
      int u = q.front(); q.pop();
      int v = to[u];
      if (--in_deg[v] == 0) {
        on_cyc[v] = false;
        q.push(v);
      }
    }
    for (int i = 1; i <= n; ++i) {
      if (on_cyc[i] && !cyc_id[i]) {
        ++cyc_cnt;
        int cur = i, pos = 0;
        while (!cyc_id[cur]) {
          cyc_id[cur] = cyc_cnt;
          cyc_pos[cur] = pos++;
          root_cyc[cur] = cur;
          cur = to[cur];
        }
        cyc_len[cyc_cnt] = pos;
      }
    }
    for (int i = 1; i <= n; ++i)
      if (!on_cyc[i]) resolve_tree(i);
  }
  void resolve_tree(int u) {
    if (cyc_id[u]) return;
    int v = to[u];
    if (!cyc_id[v]) resolve_tree(v);
    dist_cyc[u] = dist_cyc[v] + 1;
    root_cyc[u] = root_cyc[v];
    cyc_id[u] = cyc_id[v];
  }
  int jump(int u, ll k) const {
    for (int j = 0; j < LOG; ++j)
      if ((k >> j) & 1LL) u = up[j][u];
    return u;
  }
  // Minima cantidad de pasos para ir de u a v (-1 si
  // imposible)
  ll min_steps(int u, int v) const {
    if (cyc_id[u] != cyc_id[v]) return -1;
    if (dist_cyc[u] < dist_cyc[v]) return -1;
    ll d = dist_cyc[u] - dist_cyc[v];
    int u_anc = jump(u, d);
    if (dist_cyc[v] > 0) return (u_anc == v) ? d : -1;
    int len = cyc_len[cyc_id[v]];
    int diff = (cyc_pos[v] - cyc_pos[u_anc] + len) % len;
    return d + diff;
  }
};
```

- **Trampas y adaptaciones:** (1) Un nodo en la cola (`dist_cyc > 0`) nunca puede ser alcanzado desde un nodo que ya entró al ciclo (`dist_cyc == 0`).

Bloque 7: Matemáticas, Teoría de Números, Álgebra y Juegos

7.1 Cribas (Eratóstenes, Divisores, Lineal SPF, Segmentada) y Primalidad $\mathcal{O}(\sqrt{N})$

7.1.1 Criba de Eratóstenes (`sieve`)

- **Cuándo usar:** Precalcular primalidad `is_prime[0..MAX_P-1]` en $\mathcal{O}(N \log \log N)$ para consultas en $\mathcal{O}(1)$ hasta $N \approx 10^7$.
- **Cuándo no usar / alternativa:** Si además necesitas factorizar muchos números rápido en $\mathcal{O}(\log X)$, usar `linear_sieve_spf` (§7.1.3).
- **Contrato:** Marca `is_prime[0] = is_prime[1] = false` y desde `i = 2` tacha múltiplos empezando en `i * i` .
- **Complejidad:** $\mathcal{O}(N \log \log N)$ tiempo y $\mathcal{O}(N)$ memoria.

```
// 1. Criba de Eratostenes en O(MAX_P log log MAX_P)
const int MAX_P = 1000005;
bool is_prime[MAX_P];
void sieve() {
  is_prime[0] = is_prime[1] = false;
  fill(is_prime + 2, is_prime + MAX_P, true);
  for (ll i = 2; i * i < MAX_P; ++i)
    if (is_prime[i])
      for (ll j = i * i; j < MAX_P; j += i) is_prime[j]
        = false;
}
```

- **Trampas y adaptaciones:** (1) El bucle interno `for (ll j = i * i; ...)` debe usar `ll` si `MAX_P` supera 46340, pues `i * i` desbordaría `int` .

7.1.2 Criba Armónica de Divisores (`harmonic_sieve`)

- **Cuándo usar:** Precalcular la cantidad de divisores `num_div[x]` ($\tau(x)$) y la suma de divisores `sum_div[x]` ($\sigma(x)$) para todos los enteros $x \in [1, N]$.
- **Cuándo no usar / alternativa:** Si solo necesitas los divisores de un único $N \leq 10^{12}$, iterar hasta $\sqrt{N}$ en $\mathcal{O}(\sqrt{N})$.
- **Contrato:** Recorre cada divisor `i` y suma a todos sus múltiplos `j = i, 2i, 3i, ... <= n` . Dependencias: `MAX_P` (§7.1.1).
- **Complejidad:** $\mathcal{O}(N \log N)$ tiempo y $\mathcal{O}(N)$ memoria.

```
// 2. Criba Armonica de Divisores en O(N log N)
int num_div[MAX_P];
ll sum_div[MAX_P];
void harmonic_sieve(int n) {
  for (int i = 1; i <= n; ++i)
    for (int j = i; j <= n; j += i) {
      num_div[j]++;
      sum_div[j] += i;
    }
}
```

- **Trampas y adaptaciones:** (1) La serie armónica $\sum_{i=1}^{N} \lfloor N/i \rfloor = \mathcal{O}(N \log N)$ es mucho más rápida que factorizar cada número por separado.

7.1.3 Criba Lineal SPF y Factorización en $\mathcal{O}(\log X)$ (`linear_sieve_spf` , `get_prime_factors`)

- **Cuándo usar:** Factorizar hasta $Q \leq 10^6$ números con $X \leq 10^7$ en $\mathcal{O}(\log X)$ por consulta dividiendo repetidamente por el menor factor primo `spf[x]` .
- **Cuándo no usar / alternativa:** Si $X \leq 10^{18}$ no cabe en memoria para precalcular `spf` , usar `factorize_64` con Pollard's Rho (§7.2.2).
- **Contrato:** Cada compuesto m es tachado **exactamente una vez** por su menor factor primo `spf[m]` . `get_prime_factors(x)` retorna los factores primos en orden no decreciente con multiplicidad.
- **Complejidad:** Precomputación $\mathcal{O}(N)$; factorización de X en $\mathcal{O}(\log X)$.

```
// 3. Criba Lineal O(N) con Menor Factor Primo (SPF) y
// Factorizacion O(log X)
const int MAX_SPF = 10000005;
int spf[MAX_SPF];
vi primes_list;
void linear_sieve_spf(int n = MAX_SPF - 1) {
  for (int i = 2; i <= n; ++i) {
    if (spf[i] == 0) { spf[i] = i; primes_list.pb(i); }
    for (int p : primes_list) {
      if (p > spf[i] || 1LL * i * p > n) break;
      spf[i * p] = p;
    }
  }
}
// Retorna factores primos con multiplicidad
vi get_prime_factors(int x) {
  vi f;
  while (x > 1) { f.pb(spf[x]); x /= spf[x]; }
  return f;
}
```

- **Trampas y adaptaciones:** (1) No olvidar llamar `linear_sieve_spf(max_val)` una vez al inicio de `main()` antes de invocar `get_prime_factors(x)` .

7.1.4 Criba Segmentada en Rango $[L, R]$ (`segmented_sieve`)

- **Cuándo usar:** Encontrar todos los primos en un intervalo desplazado $[L, R]$ cuando $R \leq 10^{12}$ es demasiado grande para un arreglo global pero el ancho $R - L + 1 \leq 10^7$ es pequeño.
- **Cuándo no usar / alternativa:** Si solo necesitas verificar unos pocos números aislados en $[L, R]$, usar `miller_rabin` (§7.2.1).
- **Contrato:** Retorna `vector<bool> is_p(R - L + 1)` donde `is_p[x - L]` indica si `x` es primo. Maneja explícitamente los bordes `L = 0` y `L = 1` .
- **Complejidad:** $\mathcal{O}(\sqrt{R} \log \log \sqrt{R} + (R - L + 1) \log \log R)$ tiempo y $\mathcal{O}(\sqrt{R} + R - L + 1)$ memoria.

```
// 4. Criba Segmentada en [L, R] para R <= 10^12 y (R - L +
// 1) <= 10^7
// Retorna vector<bool> donde res[x - L] == true si y solo
// si x es primo.
vector<bool> segmented_sieve(ll L, ll R) {
  ll lim = sqrt(R);
  vector<bool> mark(lim + 1, false);
  vll pr;
  for (ll i = 2; i <= lim; ++i) {
    if (!mark[i]) {
      pr.pb(i);
      for (ll j = i * i; j <= lim; j += i) mark[j] = true;
    }
  }
  vector<bool> res(R - L + 1, true);
  for (ll p : pr) {
    ll start = max(p * p, (L + p - 1) / p * p);
    for (ll j = start; j <= R; j += p) res[j - L] = false;
  }
  for (ll x = max(L, 0LL); x <= min(R,
    1LL); ++x) res[x - L] = false;
  return res;
}
```

- **Trampas y adaptaciones:** (1) El primer múltiplo de `p` a tachar en $[L, R]$ debe ser `max(p * p, ((L + p - 1) / p) * p)` para no tachar a `p` mismo cuando $L \leq p$.

7.1.5 Test de Primalidad Individual en $\mathcal{O}(\sqrt{N})$ (`is_prime_single`)

- **Cuándo usar:** Verificar primalidad de un único entero $N \leq 10^{12}$ sin necesidad de tablas ni de `__uint128_t` .
- **Cuándo no usar / alternativa:** Si N llega hasta 10^{18} o hay miles de consultas grandes, $\mathcal{O}(\sqrt{N})$ da TLE; usar `miller_rabin` (§7.2.1).
- **Contrato:** Salta múltiplos de 2 y 3 e inspecciona divisores de la forma $6k \pm 1$ hasta $\sqrt{N}$.
- **Complejidad:** $\mathcal{O}(\sqrt{N})$ tiempo y $\mathcal{O}(1)$ memoria.

```
// 5. Primalidad Individual en O(sqrt(N)) para N <= 10^12
bool is_prime_single(ll n) {
  if (n <= 1) return false;
  if (n <= 3) return true;
  if (n % 2 == 0 || n % 3 == 0) return false;
  for (ll i = 5; i * i <= n; i += 6)
    if (n % i == 0 || n % (i + 2) == 0) return false;
  return true;
}
```

- **Trampas y adaptaciones:** (1) La condición `i * i <= n` con `ll i` es segura hasta $N \leq 3 \cdot 10^{18}$.

7.2 Primalidad y Factorización Grande: Miller-Rabin (64-bit) y Pollard's Rho

7.2.1 Test de Primalidad Determinista Miller-Rabin 64-bit (`miller_rabin`)

- **Cuándo usar:** Determinar con 100% de exactitud si un entero de 64 bits ($N < 2^{64} \approx 1.84 \cdot 10^{19}$) es primo.
- **Cuándo no usar / alternativa:** Para $N \leq 10^6$ con millones de consultas, precalcular `sieve()` (§7.1.1) es $\mathcal{O}(1)$ por consulta.
- **Contrato:** Usa `u128` (`__uint128_t`) en `mul_mod64` para evitar overflow con signo cuando $N \geq 2^{63}$, y evalúa las 7 bases deterministas de Jim Sinclair.
- **Complejidad:** $\mathcal{O}(7 \log N)$ tiempo y $\mathcal{O}(1)$ memoria.

```
ull mul_mod64(ull a, ull b, ull m) {
  return (ull)((__uint128_t)a * b % m);
}
ull pow_mod64(ull b, ull e, ull m) {
  ull r = 1;
  for (b %= m; e > 0; e >>= 1) {
    if (e & 1) r = mul_mod64(r, b, m);
    b = mul_mod64(b, b, m);
  }
  return r;
}

// 1. Miller-Rabin Determinista para todo entero de 64 bits
// (< 2^64)
bool miller_rabin(ull n) {
  if (n < 2) return false;
  for (ull p : {
    2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
    if (n % p == 0) return n == p;
  }
  ull d = n - 1, s = 0;
  while ((d & 1) == 0) { d >>= 1; s++; }
  for (ull a : {2ULL, 325ULL, 9375ULL, 28178ULL,
                450775ULL, 9780504ULL, 1795265022ULL}) {
    if (a % n == 0) continue;
    ull x = pow_mod64(a, d, n);
    if (x == 1 || x == n - 1) continue;
    bool composite = true;
    for (ull r = 1; r < s; ++r) {
      x = mul_mod64(x, x, n);
      if (x == n - 1) { composite = false; break; }
    }
    if (composite) return false;
  }
```

```
  return true;
}
```

- **Trampas y adaptaciones:** (1) Nunca usar `__int128` con signo para módulos $\geq 2^{63}$; usar siempre `u128` (`__uint128_t`).

7.2.2 Factorización de 64-bit con Pollard's Rho (`pollard_rho`, `factorize_64`)

- **Cuándo usar:** Obtener todos los factores primos de un entero $N \leq 10^{18}$ en aproximadamente $\mathcal{O}(N^{1/4})$ operaciones.
- **Cuándo no usar / alternativa:** Si todos los números a factorizar son $\leq 10^7$, `linear_sieve_spf` (§7.1.3) es mucho más rápido.
- **Contrato:** `factorize_64(n, fac)` llena `fac` con todos los factores primos de `n` (con multiplicidad). Si `n <= 1`, no agrega factores. Dependencias: `miller_rabin`, `mul_mod64` (§7.2.1), `rd` (§1.2.1).
- **Complejidad:** $\mathcal{O}(N^{1/4} \log N)$ esperado en tiempo; $\mathcal{O}(\log N)$ memoria.

```
// 2. Pollard's Rho (Factorizacion completa en O(N^(1/4)))
ull pollard_f(ull x, ull c, ull m) {
  return (mul_mod64(x, x, m) + c) % m;
}
ull pollard_rho(ull n) {
  if ((n & 1) == 0) return 2;
  ull x = 2, y = 2, d = 1, c = rd(1, n - 1);
  while (d == 1) {
    x = pollard_f(x, c, n);
    y = pollard_f(pollard_f(y, c, n), c, n);
    d = std::gcd(x > y ? x - y : y - x, n);
  }
  return (d == n) ? pollard_rho(n) : d;
}
void factorize_64(ull n, map<ull, int>& factors) {
  if (n == 1) return;
  if (miller_rabin(n)) { factors[n]++; return; }
  ull div = pollard_rho(n);
  factorize_64(div, factors);
  factorize_64(n / div, factors);
}
```

- **Trampas y adaptaciones:** (1) `factorize_64` no ordena automáticamente el vector `fac`; llamar `sort(all(fac))` después si necesitas los factores en orden creciente.

7.3 GCD, Diferencia de Cuadrados, Euclides Extendido, Ecuaciones Diofánticas, Ecuación Modular y CRT

7.3.1 GCD de Arreglo tras Sumar X (`array_diff_gcd`)

- **Cuándo usar:** Calcular $\gcd(a_0 + X, a_1 + X, ..., a_{n-1} + X)$ para múltiples consultas de X en $\mathcal{O}(\log V)$ por consulta tras un preprocesamiento $\mathcal{O}(N \log V)$.
- **Cuándo no usar / alternativa:** Si el arreglo cambia con actualizaciones en rango además de consultar GCD en subarreglos, combinar diferencias adyacentes con un `SegTree` (§4.4) + `Fenwick` (§4.3.1).
- **Contrato:** Por propiedad de Euclides, $\gcd(a_0 + X, ..., a_{n-1} + X) = \gcd(a_0 + X, |a_1 - a_0|, ..., |a_{n-1} - a_0|)$.
- **Complejidad:** $\mathcal{O}(N + \log V)$ tiempo y $\mathcal{O}(1)$ memoria.

```
// 1. GCD de todo el arreglo A despues de sumar X a cada
// elemento
ll array_diff_gcd(const vll& a, ll x) {
  int n = sz(a);
  if (n == 1) return abs(a[0] + x);
  ll diff_g = 0;
  for (int i = 1; i < n; ++i) diff_g = std::gcd(diff_g, abs(
    a[i] - a[i - 1]));
  return std::gcd(abs(a[0] + x), diff_g);
}
```

- **Trampas y adaptaciones:** (1) Usar `abs(a[i] - a[0])` y `abs(a[0] + x)` para garantizar que el GCD retornado sea siempre no negativo.

7.3.2 Representación como Diferencia de Cuadrados $N = a^2 - b^2$ (`solve_diff_of_squares`)

- **Cuándo usar:** Encontrar todos los pares de enteros no negativos (a, b) tales que $a^2 - b^2 = (a - b)(a + b) = N$.
- **Cuándo no usar / alternativa:** Si $N \equiv 2 \pmod 4$, no existe ninguna solución entera porque $(a - b)$ y $(a + b)$ tienen la misma paridad.
- **Contrato:** Factoriza $N = d_1 \cdot d_2$ con $d_1 \leq d_2$ y $d_1 \equiv d_2 \pmod 2$, dando $a = (d_1 + d_2)/2$ y $b = (d_2 - d_1)/2$.
- **Complejidad:** $\mathcal{O}(\sqrt{N})$ tiempo y $\mathcal{O}(\tau(N))$ memoria.

```
// 2. Expresar N = a^2 - b^2 (retorna pares {a, b} >= 0)
vector<pll> solve_diff_of_squares(ll n) {
  vector<pll> sols;
  for (ll d1 = 1; d1 * d1 <= n; ++d1) {
    if (n % d1 == 0) {
      ll d2 = n / d1;
      if ((d1 + d2) % 2 == 0) sols.pb({
        (d1 + d2) / 2, (d2 - d1) / 2});
    }
  }
  return sols;
}
```

- **Trampas y adaptaciones:** (1) Si el problema exige enteros **estrictamente positivos** ($b > 0$), descartar los pares donde $d_1 = d_2$ (N cuadrado perfecto).

7.3.3 Algoritmo de Euclides Extendido (`extgcd`)

- **Cuándo usar:** Hallar $g = \gcd(a, b)$ junto con coeficientes de Bézout (x, y) tales que $ax + by = g$, o calcular inversos modulares para módulos compuestos.
- **Cuándo no usar / alternativa:** Si solo necesitas $\gcd(a, b)$, usar `std::gcd(a, b)` de `<numeric>`.
- **Contrato:** Para $a, b \geq 0$ no ambos nulos, produce $|x| \leq b/g$ y $|y| \leq a/g$.
- **Complejidad:** $\mathcal{O}(\log(\min(a, b)))$ tiempo y $\mathcal{O}(1)$ memoria.

```
// 3. Euclides Extendido: halla g = gcd(a, b) y (x, y) tal
// que a*x + b*y = g
ll extgcd(ll a, ll b, ll& x, ll& y) {
  if (b == 0) { x = 1; y = 0; return a; }
  ll x1, y1;
  ll g = extgcd(b, a % b, x1, y1);
  x = y1;
  y = x1 - y1 * (a / b);
  return g;
}
```

- **Trampas y adaptaciones:** (1) Si `a` o `b` pueden ser negativos, normalizar signos afuera (como hace `find_diophantine` en §7.3.4).

7.3.4 Ecuación Diofántica Lineal $ax + by = c$ (`find_diophantine`)

- **Cuándo usar:** Encontrar una solución entera particular (x_0, y_0) y el paso de la familia general de soluciones de $ax + by = c$.
- **Cuándo no usar / alternativa:** Si $a = 0$ y $b = 0$, la función retorna `false` (tratar el caso $0x + 0y = 0$ por separado si puede ocurrir).
- **Contrato:** Existe solución entera si y solo si $g = \gcd(|a|, |b|)$ divide a c. La familia general es $x = x_0 + k \cdot (b/g)$, $y = y_0 - k \cdot (a/g)$ para $k \in \mathbb{Z}$. Usa `i128` al escalar por c/g para evitar overflow. Dependencias: `extgcd` (§7.3.3).
- **Complejidad:** $\mathcal{O}(\log(\min(|a|, |b|)))$ tiempo y $\mathcal{O}(1)$ memoria.

```
// 4. Ecuacion Diofantica Lineal: a*x + b*y = c
// Retorna true si existe solucion; deja una solucion
// particular en (x0, y0).
// Familia general de soluciones: x = x0 + k*(b/g), y = y0 -
// k*(a/g).
bool find_diophantine(ll a, ll b, ll c, ll& x0, ll& y0,
  ll& g) {
  g = extgcd(abs(a), abs(b), x0, y0);
  if (c % g != 0) return false;
  x0 *= c / g;
  y0 *= c / g;
  if (a < 0) x0 = -x0;
  if (b < 0) y0 = -y0;
  return true;
}
```

- **Trampas y adaptaciones:** (1) Para hallar la solución con el menor $x \geq 0$ (cuando $b > 0$),Sea `step = b / g`; hacer `x = (x % step + step) % step` y recalcular `y = (c - a * x) / b` en `i128`.

7.3.5 Teorema Chino del Resto General (`crt_general`)

- **Cuándo usar:** Resolver un sistema de congruencias simultáneas $x \equiv r_i \pmod{m_i}$ cuando los módulos m_i **no son necesariamente coprimos** entre sí.
- **Cuándo no usar / alternativa:** Si el MCM acumulado `m1 * (m2 / g)` supera $9 \cdot 10^{18}$, el resultado no cabe en `ll`.
- **Contrato:** Retorna `{x, lcm}` con $0 \leq x < \text{lcm}$, o `{-1, -1}` si dos congruencias son incompatibles ($r_1 \not\equiv r_2 \pmod{\gcd(m_1, m_2)}$). Usa `i128` en el producto intermedio para evitar overflow de `ll`. Dependencias: `extgcd` (§7.3.3).
- **Complejidad:** $\mathcal{O}(K \log(\text{LCM}))$ tiempo y $\mathcal{O}(1)$ memoria.

```
// 5. Teorema Chino del Resto General (CRT para modulos NO
// necesariamente coprimos)
// Recibe vectores 'r' y 'm' (x == r[i] mod m[i]).
// Retorna {x_minimo_no_negativo, lcm_total}, o {-1, -1} si
// es incompatible.
pll crt_general(const vll& r, const vll& m) {
  ll r0 = 0, m0 = 1;
  for (int i = 0; i < sz(r); ++i) {
    ll r1 = ((r[i] % m[i]) + m[i]) % m[i], m1 = m[i];
    ll x, y;
    ll g = extgcd(m0, m1, x, y);
    if ((r1 - r0) % g != 0) return {-1, -1}; // Sin solucion
    ll mod_step = m1 / g;
    ll diff = ((r1 - r0) / g) % mod_step;
    if (diff < 0) diff += mod_step;
    ll inv_part = (x % mod_step + mod_step) % mod_step;
    ll step = (ll)((__int128)diff * inv_part % mod_step);
    __int128 lcm128 = (__int128)m0 * mod_step;
    ll lcm = (ll)lcm128;
    r0 = (ll)((r0 + (__int128)step * m0) % lcm128);
    if (r0 < 0) r0 += lcm;
    m0 = lcm;
  }
  return {r0, m0};
}
```

- **Trampas y adaptaciones:** (1) Toda ecuación lineal modular $ax \equiv b \pmod m$ puede reducirse a $x \equiv r' \pmod{m/g}$ con `extgcd` antes de meterla en `crt_general`.

7.4 Aritmética Modular (`mint`), Combinatoria (nCr), Teorema de Lucas y Fórmula de Legendre ($n!$)

7.4.1 Aritmética Modular Segura e Inverso Modular (`norm_mod`, `binpow`, `inv_fermat`, `inv_extgcd`)

- **Cuándo usar:** Operaciones básicas $(+, -, \times, a^e, a^{-1}) \pmod m$ inmunes a operandos negativos y a overflow cuando $m \leq 3 \cdot 10^{18}$.
- **Cuándo no usar / alternativa:** No usar `inv_fermat(a, m)` si el módulo `m` **no es primo**; para `m` compuesto usar siempre `inv_extgcd(a, m)`.
- **Contrato:** `norm_mod` lleva cualquier `ll` (incluso negativo) a $[0, m-1]$. `inv_extgcd(a, m)` retorna `-1` si $m \leq 1$ o $\gcd(a, m) \neq 1$. Dependencias: `extgcd` (§7.3.3).
- **Complejidad:** $\mathcal{O}(1)$ para suma/resta/producto; $\mathcal{O}(\log E)$ para `binpow` e inversos.

```
constexpr ll MOD = 1000000007LL;

inline ll norm_mod(ll x, ll m = MOD) {
  x %= m;
  return (x < 0) ? (x + m) : x;
}
inline ll add_mod(ll a, ll b, ll m = MOD) {
  ll res = norm_mod(a, m) + norm_mod(b, m);
  return (res >= m) ? (res - m) : res;
}
inline ll sub_mod(ll a, ll b, ll m = MOD) {
  ll res = norm_mod(a, m) - norm_mod(b, m);
  return (res < 0) ? (res + m) : res;
}
inline ll mul_mod(ll a, ll b, ll m = MOD) {
  return (ll)((__int128)norm_mod(a, m) *
    norm_mod(b, m) % m);
}
ll binpow(ll base, ll exp, ll m = MOD) {
  if (m == 1) return 0;
  base = norm_mod(base, m);
  ll res = 1;
  while (exp > 0) {
    if (exp & 1) res = mul_mod(res, base, m);
    base = mul_mod(base, base, m);
    exp >>= 1;
  }
  return res;
}
// Inverso cuando m es PRIMO (Fermat):
inline ll inv_fermat(ll a, ll m = MOD) {
  return binpow(a, m - 2, m); }

// Inverso cuando m es COMPUESTO (requiere m > 1 y gcd(a, m)
// == 1):
ll inv_extgcd(ll a, ll m) {
  if (m <= 1) return -1;
  ll x, y;
  ll g = extgcd(norm_mod(a, m), m, x, y);
  return (g == 1) ? norm_mod(x, m) : -1;
}
```

- **Trampas y adaptaciones:** (1) En C++, `-4 % 5 == -4`; `add_mod` y `sub_mod` normalizan ambos operandos antes de operar para que nunca retornen residuos negativos.

7.4.2 Combinatoria en $\mathcal{O}(1)$, Producto Recurrente, Lucas y Legendre (`init_fact` , `nCr` , `nCr_iter` , `lucas_nCr` , `legendre_prime_in_fact`)

- **Cuándo usar:** Calcular $\binom{n}{r} \pmod p$ en $\mathcal{O}(1)$ con factoriales recurrentes (`nCr`), en $\mathcal{O}(r)$ exacto mediante productoria $\binom{n}{r} = \prod_{i=1}^{r} \frac{n-r+i}{i}$ (`nCr_iter`), con dígitos en base p $\binom{n}{r} \equiv \prod_{i\geq 0} \binom{n_i}{r_i} \pmod p$ (`lucas_nCr`), o sumar $v_p(n!) = \sum_{k\geq 1} \lfloor n/p^k \rfloor$ (`legendre_prime_in_fact`).
- **Cuándo no usar / alternativa:** Si el módulo M no es primo y $n \leq 2000$, usar la recurrencia aditiva de Pascal $C[n][r] = (C[n-1][r-1] + C[n-1][r]) \bmod M$ sin divisiones.
- **Contrato:** Llamar `init_fact(max_n, mod)` una vez antes de consultar `nCr` o `nPr` . Para `lucas_nCr(n, r, p)` con p primo, llamar antes `init_fact(p - 1, p)` . Dependencias: `inv_fermat` (§7.4.1).
- **Complejidad:** Precomputación $\mathcal{O}(N)$; `nCr` en $\mathcal{O}(1)$; `nCr_iter` en $\mathcal{O}(r)$; `lucas_nCr` y `legendre_prime_in_fact` en $\mathcal{O}(\log_p N)$.

```
// Combinatoria O(N) precalculo, O(1) consulta:
// Recurrencias: fact[i] = fact[i-1] * i % m
//   inv_num[i] = (m - (m/i) * inv_num[m%i] % m) % m
//   inv_fact[i] = inv_fact[i-1] * inv_num[i] % m
const int MAX_F = 1000005;
ll fact[MAX_F], inv_fact[MAX_F], inv_num[MAX_F];
void init_fact(int n = MAX_F - 1, ll m = MOD) {
  fact[0] = inv_fact[0] = 1;
  inv_num[0] = 0; inv_num[1] = 1;
  for (int i = 1; i <= n; ++i) fact[i] = (
    fact[i - 1] * i) % m;
  inv_fact[n] = inv_fermat(fact[n], m);
  for (int i = n - 1; i >= 1; --i)
    inv_fact[i] = (inv_fact[i + 1] * (i + 1)) % m;
  for (int i = 2; i <= n; ++i)
    inv_num[i] = (m - (m / i) * inv_num[m % i] % m) % m;
}
inline ll nCr(int n, int r, ll m = MOD) {
  if (r < 0 || r > n || n < 0) return 0;
  return fact[n] * inv_fact[r] % m * inv_fact[n - r] % m;
}
inline ll nPr(int n, int r, ll m = MOD) {
  if (r < 0 || r > n || n < 0) return 0;
  return fact[n] * inv_fact[n - r] % m;
}
// Producto recurrente O(r) sin tablas:
// C(n, r) = prod_{i=1..r} (n - r + i) / i
ll nCr_iter(ll n, ll r) {
  if (r < 0 || r > n) return 0;
  r = min(r, n - r);
  ll res = 1;
  for (ll i = 1; i <= r; ++i) res = res * (n - r + i) / i;
  return res;
}

// Teorema de Lucas: C(n, r) = prod_{i>=0} C(n_i, r_i) mod p
// (Requiere haber llamado init_fact(p - 1, p))
ll lucas_nCr(ll n, ll r, ll p) {
  if (r < 0 || r > n) return 0;
  if (r == 0) return 1;
  return lucas_nCr(n / p, r / p, p) * nCr(n % p, r % p,
    p) % p;
}

// Formula de Legendre: v_p(n!) = sum_{k>=1} floor(n / p^k)
ll legendre_prime_in_fact(ll n, ll p) {
  ll cnt = 0;
  while (n > 0) { cnt += n / p; n /= p; }
  return cnt;
}
```

- **Trampas y adaptaciones:** (1) Por el Teorema de Lucas en base 2, $\binom{n}{r}$ es **impar** si y solo si $(n \text{ AND } r) == r$ (r es submáscara de bits de n).

7.5 Funciones Multiplicativas: Totient de Euler (ϕ) y Función de Möbius (μ)

7.5.1 Totient de Euler Individual $\phi(N)$ en $\mathcal{O}(\sqrt{N})$ (`euler_phi`)

- **Cuándo usar:** Contar cuántos enteros en $[1, N]$ son coprimos con N para un único $N \leq 10^{12}$, o aplicar el Teorema de Euler $a^{\phi(m)} \equiv 1 \pmod m$ cuando $\gcd(a, m) = 1$.
- **Cuándo no usar / alternativa:** Si necesitas $\phi(x)$ para todos los $x \in [1, N]$, usar la criba `init_phi_and_mobius` (§7.5.2).
- **Contrato:** Aplica $\phi(n) = n \prod_{p|n} (1 - 1/p)$ usando `ans -= ans / d` para no desbordar.
- **Complejidad:** $\mathcal{O}(\sqrt{N})$ tiempo y $\mathcal{O}(1)$ memoria.

```
// 1. Totient de Euler individual en O(sqrt(N))
ll euler_phi(ll n) {
  ll ans = n;
  for (ll d = 2; d * d <= n; ++d) {
    if (n % d == 0) {
      while (n % d == 0) n /= d;
      ans -= ans / d;
    }
  }
  if (n > 1) ans -= ans / n;
  return ans;
}
```

- **Trampas y adaptaciones:** (1) Para reducir exponentes gigantes cuando $\gcd(a, m) > 1$ (Exponenciación Generalizada de Euler) y $e \geq \phi(m)$: $a^e \equiv a^{(e \bmod \phi(m)) + \phi(m)} \pmod m$.

7.5.2 Criba de Euler $\phi[1..N]$ y Función de Möbius $\mu[1..N]$ (`init_phi_and_mobius`)

- **Cuándo usar:** Problemas de inversión de Möbius y conteo de pares/tuplas con $\gcd(x, y) = k$ usando $\sum_{d|g} \mu(d) = [g = 1]$ y $\sum_{d|g} \phi(d) = g$.
- **Cuándo no usar / alternativa:** Si solo necesitas $\phi(N)$ para un único número grande, usar `euler_phi(N)` (§7.5.1).
- **Contrato:** Llena `phi_arr[1..n]` y `mu_arr[1..n]` ($n <$ MAX_P). Dependencias: `MAX_P` (§7.1.1).
- **Complejidad:** $\mathcal{O}(N \log N)$ tiempo y $\mathcal{O}(N)$ memoria.

```
// 2. Precalculo de phi[1..n] en O(N log log N) y mu[1..n]
// en O(N log N)
int phi_arr[MAX_P], mu_arr[MAX_P];
void init_phi_and_mobius(int n) {
  for (int i = 1; i <= n; ++i) { phi_arr[i] = i;
    mu_arr[i] = 0; }
  for (int i = 2; i <= n; ++i) {
    if (phi_arr[i] == i) { // i es primo
      for (int j = i; j
        <= n; j += i) phi_arr[j] -= phi_arr[j] / i;
    }
  }
  mu_arr[1] = 1;
  for (int i = 1; i <= n; ++i)
    for (int j = 2 * i; j <= n; j += i)
      mu_arr[j] -= mu_arr[i];
}
```

- **Trampas y adaptaciones:** (1) La cantidad de pares ordenados $(x, y) \in [1, N] \times [1, M]$ con $\gcd(x, y) = 1$ es $\sum_{d=1}^{\min(N,M)} \mu(d) \lfloor N/d \rfloor \lfloor M/d \rfloor$.

7.6 Álgebra Lineal y Polinomios: Exponenciación de Matrices, Eliminación Gaussiana (`double` , Modular, XOR Basis), FFT, FWT, Logaritmo Discreto y Fracciones

7.6.1 Exponenciación de Matrices y Fibonacci Rápido (`Matrix` , `mat_pow` , `fib_fast`)

- **Cuándo usar:** Acelerar recurrencias lineales de dimensión fija $D \leq 100$ o contar caminos de longitud $K \leq 10^{18}$ en un grafo pequeño en $\mathcal{O}(D^3 \log K)$.
- **Cuándo no usar / alternativa:** Si la recurrencia tiene orden $D \approx 2000$ o no la conoces de antemano, usar `BerlekampMassey` (§7.9) en $\mathcal{O}(D^2 \log K)$.
- **Contrato:** Matriz 0-indexed de dimensión `d x d` módulo `MOD` . `fib_fast(n)` retorna `{F_n, F_{n+1}}` mod `MOD` por duplicación rápida sin crear matriz. Dependencias: `MOD` (§7.4.1).
- **Complejidad:** $\mathcal{O}(D^3 \log E)$ para `mat_pow` ; $\mathcal{O}(\log N)$ para `fib_fast` .

```
// 1. Exponenciacion de Matrices en O(D^3 log E)
struct Matrix {
  int n;
  vector<vll> m;
  Matrix(int n, ll diag = 0) : n(n), m(n, vll(n, 0)) {
    for (int i = 0; i < n; ++i) m[i][i] = diag;
  }
  Matrix operator*(const Matrix& o) const {
    Matrix res(n, 0);
    for (int i = 0; i < n; ++i)
      for (int k = 0; k < n; ++k) {
        if (!m[i][k]) continue;
        for (int j = 0; j < n; ++j)
          res.m[i][j] = (
            res.m[i][j] + m[i][k] * o.m[k][j]) % MOD;
      }
    return res;
  }
};
Matrix mat_pow(Matrix b, ll e) {
  Matrix res(b.n, 1); // Identidad
  for (; e > 0; e >>= 1) {
    if (e & 1) res = res * b;
    b = b * b;
  }
  return res;
}

// Fibonacci en O(log N) usando duplicacion rapida: retorna
// {F_n, F_{n+1}}
pll fib_fast(ll n, ll m = MOD) {
  if (n == 0) return {0, 1};
  auto [a, b] = fib_fast(n >> 1, m);
  ll c = a * (2 * b - a + m) % m;
  ll d = (a * a % m + b * b % m) % m;
  return (n & 1) ? pll{d, (c + d) % m} : pll{c, d};
}
```

- **Trampas y adaptaciones:** (1) En el triple bucle de `operator*` , poner `k` como segundo bucle (`for i ... for k ... if (a[i][k]) for j`) mejora drásticamente la localidad de caché.

7.6.2 Base Lineal de XOR / Eliminación Gaussiana en $\mathbb{F}_2$ (`XorBasis`)

- **Cuándo usar:** Mantener un conjunto de enteros de hasta 62 bits bajo la operación XOR: consultar el máximo XOR obtenible (`max_xor`), si un valor `x` puede formarse, o cuántos subconjuntos tienen XOR igual a `x` ($2^{N-\text{sz}}$).
- **Cuándo no usar / alternativa:** Si buscas el máximo XOR entre `x` y **un único** elemento del conjunto (no un subconjunto arbitrario), usar `BitTrie` (§5.5.2).
- **Contrato:** Soporta enteros de hasta 62 bits (`B = 62`). El número de valores distintos generables es 2^{sz}.
- **Complejidad:** $\mathcal{O}(B)$ por inserción y consulta ($B = 62$).

```
// 2. Base de XOR (Gauss Binario en O(B) por insercion)
struct XorBasis {
  static const int B = 60;
  ll basis[B + 1] = {0};
  int sz_b = 0;

  bool insert(ll x) {
    for (int i = B; i >= 0; --i) {
      if (!((x >> i) & 1LL)) continue;
      if (!basis[i]) { basis[i] = x; sz_b++; return true; }
      x ^= basis[i];
    }
    return false; // x ya era linealmente dependiente
  }
  // Maximo valor posible de (initial_val ^ subconjunto)
  ll max_xor(ll ans = 0) const {
    for (int i = B; i >= 0; --i)
      ans = max(ans, ans ^ basis[i]);
    return ans;
  }
  bool can_form(ll x) const {
    for (int i = B; i >= 0; --i)
      if ((x >> i) & 1LL) x ^= basis[i];
    return x == 0;
  }
};
```

- **Trampas y adaptaciones:** (1) El menor XOR no nulo obtenible es simplemente el menor `b[i] > 0` de la base reducida.

7.6.3 Eliminación Gaussiana en Punto Flotante (`gauss_double`)

- **Cuándo usar:** Resolver sistemas de N ecuaciones lineales con M incógnitas en reales (ld), como sistemas de esperanzas o cadenas de Markov con ciclos.
- **Cuándo no usar / alternativa:** Si el sistema es módulo un primo P, usar `gauss_mod` (§7.6.4).
- **Contrato:** Recibe la matriz aumentada `a` de tamaño $N \times (M + 1)$ (donde la columna `M` es el término independiente b_i). Usa pivoteo parcial por mayor valor absoluto. Retorna `0` (sin solución), `1` (única en `ans`), o `2` (infinitas soluciones, dejando una particular en `ans`). Dependencias: `EPS` (§1.1).
- **Complejidad:** $\mathcal{O}(N \cdot M \cdot \min(N, M))$ tiempo y $\mathcal{O}(NM)$ memoria.

```
// 3. Eliminacion Gaussiana en punto flotante O(N^3)
// Matriz aumentada 'a' de tamano N x (M + 1).
// Retorna: 0 (sin solucion), 1 (solucion unica en 'ans'), 2
// (infinitas).
int gauss_double(vector<vector<double>> a,
  vector<double>& ans) {
  int n = sz(a), m = sz(a[0]) - 1;
  vi where(m, -1);
```

```
  for (int col = 0, row = 0; col < m && row < n; ++col) {
    int sel = row;
    for (int i = row; i < n; ++i)
      if (abs(a[i][col]) > abs(a[sel][col])) sel = i;
    if (abs(a[sel][col]) < 1e-9) continue;
    swap(a[sel], a[row]);
    where[col] = row;
    for (int i = 0; i < n; ++i)
      if (i != row) {
        double c = a[i][col] / a[row][col];
        for (int j = col; j
          <= m; ++j) a[i][j] -= a[row][j] * c;
      }
    row++;
  }
  ans.assign(m, 0);
  for (int i = 0; i < m; ++i)
    if (where[i] != -1) ans[i]
      = a[where[i]][m] / a[where[i]][i];
  for (int i = 0; i < n; ++i) {
    double sum = 0;
    for (int j = 0; j < m; ++j) sum += ans[j] * a[i][j];
    if (abs(sum - a[i][m]) > 1e-9) return 0;
  }
  for (int i = 0; i < m; ++i) if (where[i] == -1) return 2;
  return 1;
}
```

- **Trampas y adaptaciones:** (1) El pivoteo parcial (`abs(a[i][col]) > abs(a[sel][col])`) es obligatorio; elegir el primer no nulo causa catástrofe numérica.

7.6.4 Eliminación Gaussiana Modular (`gauss_mod`)

- **Cuándo usar:** Resolver sistemas lineales $Ax \equiv b \pmod p$ sobre un cuerpo primo p (ej. sistemas de ecuaciones modulares o conteo de soluciones libres p^{libres}).
- **Cuándo no usar / alternativa:** Si el sistema es sobre $\mathbb{F}_2$ ($p = 2$) con $M \le 2000$, usar `std::bitset` reduce la complejidad a $\mathcal{O}(NM^2/64)$.
- **Contrato:** Recibe matriz aumentada `a` de tamaño $N \times (M+1)$ y primo `mod`. Retorna `0` (incompatible), `1` (única), o `2` (infinitas, con una solución particular en `ans`). Dependencias: `norm_mod`, `sub_mod`, `mul_mod`, `inv_fermat` (§7.4.1).
- **Complejidad:** $\mathcal{O}(N \cdot M \cdot \min(N, M))$ tiempo y $\mathcal{O}(NM)$ memoria.

```
// 3b. Eliminacion Gaussiana Modular sobre primo MOD en O(N
// * M * min(N, M))
// Matriz aumentada 'a' de tamano N x (M + 1). Retorna 0
// (sin sol), 1 (unica), 2 (infinitas).
int gauss_mod(vector<vll> a, vll& ans, ll mod = MOD) {
  int n = sz(a), m = sz(a[0]) - 1;
  vi where(m, -1);
  for (int col = 0, row = 0; col < m && row < n; ++col) {
    int sel = -1;
    for (int i = row; i < n; ++i)
      if (norm_mod(a[i][col], mod) != 0) { sel = i; break; }
    if (sel == -1) continue;
    swap(a[sel], a[row]);
    where[col] = row;
    ll inv_piv = inv_fermat(a[row][col], mod);
    for (int j = col; j <= m; ++j)
      a[row][j] = norm_mod(a[row][j], mod) * inv_piv % mod;
    for (int i = 0; i < n; ++i)
      if (i != row && norm_mod(a[i][col], mod) != 0) {
        ll factor = norm_mod(a[i][col], mod);
        for (int j = col; j <= m; ++j)
          a[i][j] = sub_mod(a[i][j], factor * a[row][j],
            mod);
      }
    row++;
  }
  ans.assign(m, 0);
  for (int i = 0; i < m; ++i)
    if (where[i] != -1) ans[i] = norm_mod(a[where[i]][m],
      mod);
  for (int i = 0; i < n; ++i) {
    ll sum = 0;
    for (int j = 0; j < m; ++j)
      sum = (sum + ans[j] * norm_mod(a[i][j], mod)) % mod;
    if (sum != norm_mod(a[i][m], mod)) return 0;
  }
  for (int i = 0; i < m; ++i) if (where[i] == -1) return 2;
  return 1;
}
```

- **Trampas y adaptaciones:** (1) El número de variables libres es $M - \text{rango}$, por lo que cuando retorna `2` existen exactamente $\text{mod}^{M-\text{rango}}$ soluciones distintas.

7.6.5 Transformada Rápida de Fourier para Polinomios (`fft` , `multiply_poly`)

- **Cuándo usar:** Multiplicar dos polinomios con coeficientes enteros (convolución $c_k = \sum_{i+j=k} a_i b_j$) en $\mathcal{O}(N \log N)$ cuando los coeficientes resultantes no exceden $\approx 10^{14}$.
- **Cuándo no usar / alternativa:** Si la convolución se pide módulo `998244353` o los coeficientes son más grandes y no toleran error de punto flotante, usar `multiply_ntt` (§7.6.6).
- **Contrato:** Redondea cada coeficiente final con `llround(fa[i].real())`. Dependencias: `PI` (§1.1).
- **Complejidad:** $\mathcal{O}(N \log N)$ tiempo y $\mathcal{O}(N)$ memoria.

```
// 4. Fast Fourier Transform (FFT - Multiplicacion de
// Polinomios en O(N log N))
using cd = complex<double>;
void fft(vector<cd>& a, bool inv) {
  int n = sz(a);
  for (int i = 1, j = 0; i < n; ++i) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) swap(a[i], a[j]);
  }
  for (int len = 2; len <= n; len <<= 1) {
    double ang = 2 * acos(-1.0) / len * (inv ? -1 : 1);
    cd wlen(cos(ang), sin(ang));
    for (int i = 0; i < n; i += len) {
      cd w(1);
      for (int j = 0; j < len / 2; ++j) {
        cd u = a[i + j], v = a[i + j + len / 2] * w;
        a[i + j] = u + v;
        a[i + j + len / 2] = u - v;
        w *= wlen;
      }
    }
  }
  if (inv) for (cd& x : a) x /= n;
}
vll multiply_poly(const vll& a, const vll& b) {
  vector<cd> fa(all(a)), fb(all(b));
  int n = 1;
  while (n < sz(a) + sz(b)) n <<= 1;
  fa.resize(n); fb.resize(n);
  fft(fa, false); fft(fb, false);
  for (int i = 0; i < n; ++i) fa[i] *= fb[i];
  fft(fa, true);
  vll res(sz(a) + sz(b) - 1);
  for (int i = 0; i < sz(res); ++i) res[i] = llround(
    fa[i].real());
  return res;
}
```

- **Trampas y adaptaciones:** (1) Para hacer *string matching con comodines* o contar sumas $a_i + b_j = S$, invertir el segundo arreglo (`reverse(all(b))`) para convertir diferencias $i - j$ en sumas $i + (M - 1 - j)$.

7.6.6 Transformada Numérica Modular Exacta (`ntt` , `multiply_ntt`)

- **Cuándo usar:** Multiplicar polinomios exactamente módulo $998244353 = 119 \cdot 2^{23} + 1$ (raíz primitiva $g = 3$) sin punto flotante ni error de redondeo.
- **Cuándo no usar / alternativa:** Si el módulo del problema no es de la forma $c \cdot 2^k + 1$ (ej. $10^9 + 7$) y los coeficientes finales caben en 10^{14}, usar `multiply_poly` (§7.6.5) y tomar módulo al final.
- **Contrato:** Requiere longitud `n` potencia de 2 con $n \le 2^{23}$ (`multiply_ntt` ajusta la potencia de 2 automáticamente).
- **Complejidad:** $\mathcal{O}(N \log N)$ tiempo y $\mathcal{O}(N)$ memoria.

```
// 4b. Number Theoretic Transform (NTT exacto mod 998244353,
// raiz g = 3)
// Requisitos: modulo primo de la forma p = c * 2^k + 1
// (998244353 = 119 * 2^23 + 1),
// longitud n potencia de 2 con n <= 2^23. A diferencia de
// FFT (double), NTT no sufre
// error de redondeo pero calcula los coeficientes modulo
// NTT_MOD.
const ll NTT_MOD = 998244353;
const ll NTT_G = 3;

ll ntt_pow(ll b, ll e) {
  ll r = 1;
  for (b %= NTT_MOD; e > 0; e >>= 1, b = b * b % NTT_MOD)
    if (e & 1) r = r * b % NTT_MOD;
  return r;
}
void ntt(vll& a, bool inv) {
  int n = sz(a);
  for (int i = 1, j = 0; i < n; ++i) {
    int bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j ^= bit;
    if (i < j) swap(a[i], a[j]);
  }
  for (int len = 2; len <= n; len <<= 1) {
    ll wlen = ntt_pow(NTT_G, (NTT_MOD - 1) / len);
    if (inv) wlen = ntt_pow(wlen, NTT_MOD - 2);
    for (int i = 0; i < n; i += len) {
      ll w = 1;
      for (int j = 0; j < len / 2; ++j) {
        ll u = a[i + j];
        ll v = a[i + j + len / 2] * w % NTT_MOD;
        a[i + j] = (u + v < NTT_MOD) ? (u + v) : (
          u + v - NTT_MOD);
        a[i + j + len / 2] = (u - v >= 0) ? (u - v) : (
          u - v + NTT_MOD);
        w = w * wlen % NTT_MOD;
      }
    }
  }
  if (inv) {
    ll inv_n = ntt_pow(n, NTT_MOD - 2);
    for (ll& x : a) x = x * inv_n % NTT_MOD;
  }
}
vll multiply_ntt(vll a, vll b) {
  if (a.empty() || b.empty()) return {};
  int need = sz(a) + sz(b) - 1, n = 1;
  while (n < need) n <<= 1;
  for (ll& x : a) x = (x % NTT_MOD + NTT_MOD) % NTT_MOD;
  for (ll& x : b) x = (x % NTT_MOD + NTT_MOD) % NTT_MOD;
  a.resize(n, 0); b.resize(n, 0);
  ntt(a, false); ntt(b, false);
  for (int i = 0; i < n; ++i) a[i] = a[i] * b[i] % NTT_MOD;
  ntt(a, true);
  a.resize(need);
  return a;
}
```

- **Trampas y adaptaciones:** (1) Otros primos NTT frecuentes: $p = 7340033 = 7 \cdot 2^{20} + 1$ ($g = 3$) y $p = 469762049 = 7 \cdot 2^{26} + 1$ ($g = 3$).

7.6.7 Transformada Rápida de Walsh-Hadamard para Convolución XOR (`fwt_xor`)

- **Cuándo usar:** Calcular el arreglo $C[k] = \sum_{i \oplus j = k} A[i]B[j]$ en $\mathcal{O}(N \log N)$ (donde $N = 2^B$ es potencia de 2).
- **Cuándo no usar / alternativa:** Para convolución de suma de índices ($i + j = k$), usar `multiply_poly` o `multiply_ntt` (§7.6.5–§7.6.6).
- **Contrato:** El tamaño de `a` **debe ser potencia de 2**. Para convolucionar: `fwt_xor(a, false)` , `fwt_xor(b, false)` , multiplicar punto a punto `a[i] *= b[i]` , y llamar `fwt_xor(a, true)` .
- **Complejidad:** $\mathcal{O}(N \log N)$ tiempo y $\mathcal{O}(1)$ memoria extra.

```
// 5. Fast Walsh-Hadamard Transform (FWT para Convolucion
// XOR en O(N log N))
// n debe ser potencia de 2.
void fwt_xor(vll& a, bool inv) {
  int n = sz(a);
  for (int len = 1; 2 * len <= n; len <<= 1) {
    for (int i = 0; i < n; i += 2 * len) {
      for (int j = 0; j < len; ++j) {
        ll u = a[i + j], v = a[i + len + j];
        a[i + j] = u + v;
        a[i + len + j] = u - v;
        if (inv) { a[i + j] /= 2; a[i + len + j] /= 2; }
      }
    }
  }
}
```

- **Trampas y adaptaciones:** (1) Si se trabaja módulo `MOD` , en la inversa (`inv == true`) multiplicar por el inverso modular de 2 en vez de dividir por `2` .

7.6.8 Logaritmo Discreto Baby-Step Giant-Step (`discrete_log`)

- **Cuándo usar:** Hallar el menor exponente entero $x \ge 0$ tal que $a^x \equiv b \pmod m$ (incluso cuando $\gcd(a, m) > 1$).
- **Cuándo no usar / alternativa:** Si la incógnita está en la base ($x^k \equiv b \pmod p$), reducir primero a logaritmo discreto tomando una raíz primitiva g del primo p.
- **Contrato:** Extrae factores comunes $\gcd(a, m)$ antes de aplicar Baby-Step Giant-Step. Retorna el menor $x \ge 0$, o `-1` si no existe solución.
- **Complejidad:** $\mathcal{O}(\sqrt{m} \log \sqrt{m})$ (o $\mathcal{O}(\sqrt{m})$ con `safe_umap`) tiempo y $\mathcal{O}(\sqrt{m})$ memoria.

```
// 6. Logaritmo Discreto (Baby-Step Giant-Step): menor x >=
// 0 con a^x = b (mod m)
```

```
ll discrete_log(ll a, ll b, ll m) {
  a %= m; b %= m;
  ll k = 1, add = 0, g;
  while ((g = std::gcd(a, m)) > 1) {
    if (b == k) return add;
    if (b % g != 0) return -1;
    b /= g; m /= g; ++add;
    k = (k * a / g) % m;
  }
  ll n = sqrt(m) + 1, an = 1;
  for (int i = 0; i < n; ++i) an = (an * a) % m;
  safe_umap<ll, ll> vals;
  for (ll q = 0, cur = b; q <= n; ++q) {
    vals[cur] = q;
    cur = (cur * a) % m;
  }
  for (ll p = 1, cur = k; p <= n; ++p) {
    cur = (cur * an) % m;
    if (vals.find(cur) != vals.end())
      return n * p - vals[cur] + add;
  }
  return -1;
}
```

- **Trampas y adaptaciones:** (1) Definimos $0^0 \equiv 1 \pmod m$, por lo que si `b % m == 1 % m` la respuesta siempre es `0`.

7.6.9 Estructura de Fracciones Racionales Exactas (`Frac`)

- **Cuándo usar:** Operar y comparar números racionales p/q de forma exacta sin errores de redondeo de punto flotante (pendientes de rectas, probabilidades racionales pequeñas, etc.).
- **Cuándo no usar / alternativa:** Si haces decenas de sumas encadenadas con denominadores distintos, `p` y `q` crecerán exponencialmente y desbordarán `ll`; en ese caso usar Python `Fraction` (§12.4) o aritmética modular.
- **Contrato:** Mantiene siempre `q > 0` y $\gcd(|p|, q) = 1$. El comparador `<` multiplica en `i128` para soportar numeradores y denominadores de hasta 10^{18} sin overflow.
- **Complejidad:** $\mathcal{O}(\log(\min(|p|, q)))$ por operación aritmética; $\mathcal{O}(1)$ por comparación.

```
// 7. Estructura de Fracciones Exactas (evita errores de
// precision de double)
struct Frac {
  ll num, den;
  Frac(ll n = 0, ll d = 1) : num(n), den(d) {
    if (den < 0) { num = -num; den = -den; }
    ll g = std::gcd(abs(num), den);
    if (g) { num /= g; den /= g; }
  }
  Frac operator+(const Frac& o) const { return {
    num*o.den + o.num*den, den*o.den}; }
  Frac operator-(const Frac& o) const { return {
    num*o.den - o.num*den, den*o.den}; }
  Frac operator*(const Frac& o) const { return {
    num*o.num, den*o.den}; }
  Frac operator/(const Frac& o) const { return {
    num*o.den, den*o.num}; }
  bool operator<(const Frac& o) const {
    return (__int128)num * o.den < (__int128)o.num * den; }
  bool operator==(const Frac& o) const {
    return num == o.num && den == o.den; }
};
```

- **Trampas y adaptaciones:** (1) Para representar pendientes dirigidas o no dirigidas que pueden ser verticales ($q = 0$), normalizar con `g = gcd(abs(dx), abs(dy))` sin llamar a `Frac(p, 0)` (que divide por `g` pero asume $q \neq 0$ en sumas).

7.7 XOR en Rango $\mathcal{O}(1)$, Función `mex`, Grundy y Juego de Nim / Misère Nim

7.7.1 XOR Acumulado en Rango $[L, R]$ en $\mathcal{O}(1)$ (`xor_n`, `xor_range`)

- **Cuándo usar:** Calcular $L \oplus (L+1) \oplus \ldots \oplus R$ en tiempo constante para $L, R \leq 10^{18}$.
- **Cuándo no usar / alternativa:** Si los números no son consecutivos sino un arreglo arbitrario, usar un arreglo de prefijos XOR.
- **Contrato:** Basado en el ciclo de longitud 4 de $0 \oplus 1 \oplus \ldots \oplus n$. Para $l > r$ o $n < 0$ retorna `0`.
- **Complejidad:** $\mathcal{O}(1)$ tiempo y memoria.

```
// 1. XOR de 0 a n en O(1)
ll xor_n(ll n) {
  if (n < 0) return 0;
  if (n % 4 == 0) return n;
  if (n % 4 == 1) return 1;
  if (n % 4 == 2) return n + 1;
  return 0;
}
// XOR de todos los enteros en el intervalo cerrado [l, r]
// en O(1)
ll xor_range(ll l, ll r) {
  return (l > r) ? 0 : (xor_n(r) ^ xor_n(l - 1));
}
```

- **Trampas y adaptaciones:** (1) Nunca escribir `n % 4 == 2 ? n + 1` pensando en OR bit a bit sin paréntesis por precedencia de operadores.

7.7.2 Mínimo Excluido No Negativo (`mex`) para Grundy / Sprague-Grundy

- **Cuándo usar:** Calcular en $\mathcal{O}(N)$ el menor entero ≥ 0 que no aparece en el vector `v` (base de los valores Grundy $G(u) = \text{mex}(\{G(v) : u \to v\})$ en juegos imparciales).
- **Cuándo no usar / alternativa:** Si necesitas mantener el MEX de un conjunto que sufre inserciones y borrados online, mantener los ausentes en un `std::set<int>` o usar un Segment Tree de mínimos sobre la última posición vista.
- **Contrato:** Por principio del palomar, el MEX de N números está siempre en $[0, N]$, por lo que ignora valores < 0 o $> N$.
- **Complejidad:** $\mathcal{O}(N)$ tiempo y memoria.

```
// 2. MEX (Minimum Excluded >= 0) en O(N)
int mex(const vi& v) {
  int n = sz(v);
  vector<bool> pres(n + 1, false);
  for (int x : v) if (0 <= x && x <= n) pres[x] = true;
  for (int i = 0; i <= n; ++i) if (!pres[i]) return i;
  return n + 1;
}
```

- **Trampas y adaptaciones:** (1) En la suma disjunta de K juegos imparciales independientes, el valor Grundy total es $G(s_1) \oplus G(s_2) \oplus \ldots \oplus G(s_K)$.

7.7.3 Juego de Nim Normal y Misère Nim (`nim_normal_win`, `nim_misere_win`)

- **Cuándo usar:** Determinar si el primer jugador tiene estrategia ganadora en Nim normal (gana quien retira la última piedra) o en **Misère Nim** (pierde quien se ve forzado a retirar la última piedra).
- **Cuándo no usar / alternativa:** En juegos imparciales generales bajo regla Misère donde un movimiento desde un montón de Grundy > 1 no puede elegir arbitrariamente cualquier Grundy menor, la reducción de Misère Nim no aplica directamente.
- **Contrato:** Ignora correctamente montones vacíos (`p == 0`). En Misère Nim, si todos los montones tienen tamaño ≤ 1, el primer jugador gana si y solo si la cantidad de montones de tamaño `1` es **par** (`ones % 2 == 0`); si existe algún montón > 1, gana si y solo si el XOR total es $\neq 0$.
- **Complejidad:** $\mathcal{O}(N)$ tiempo y $\mathcal{O}(1)$ memoria.

```
// 3. Nim Normal (gana quien toma el ultimo) y Misere Nim
// (pierde quien toma el ultimo)
bool nim_normal_win(const vll& piles) {
  ll x = 0;
  for (ll p : piles) x ^= p;
  return x != 0;
}
bool nim_misere_win(const vll& piles) {
  bool all_le_one = true;
  int ones = 0;
  ll x = 0;
  for (ll p : piles) {
    if (p > 1) all_le_one = false;
    if (p == 1) ones++;
    x ^= p;
  }
  return all_le_one ? (ones % 2 == 0) : (x != 0);
}
```

- **Trampas y adaptaciones:** (1) No usar `*max_element <= 1 && sz(piles) % 2 == 0` porque los montones de tamaño `0` alterarían `sz(piles)` sin ser jugadas reales; contar explícitamente `if (p == 1) ones++`.

7.8 Programación Lineal: Algoritmo Simplex de Dos Fases

- **Cuándo usar:** Maximizar una función objetivo lineal $c^T x$ sujeta a $Ax \leq b, x \geq 0$ con variables continuas (o con matriz totalmente unimodular) cuando $N, M \leq 100$.
- **Cuándo no usar / alternativa:** Si el problema es de flujo o corte en un grafo grande, usar `Dinic` (§6.6.1) o `MCMF` (§6.6.2) en lugar de programación lineal general.
- **Contrato:** Fase 1 halla una solución básica factible inicial incluso cuando algún $b_i < 0$ (retornando `-INF_LP` si el poliedro es vacío/infactible); Fase 2 optimiza $c^T x$ con regla de desempate lexicográfica anti-ciclos de Bland (retornando `+INF_LP` si es no acotado). Soporta valores óptimos negativos cuando el origen no es factible.
- **Complejidad:** Polinomial en promedio práctico; exponencial solo en casos patológicos artificiales.

```
// Simplex de Dos Fases con desempate de Bland (Anti-ciclos)
// Maximiza c^T x sujeto a A x <= b, x >= 0.
struct Simplex {
  const ld EPS_LP = 1e-9, INF_LP = 1e18;
  int n, m; // n restricciones, m variables
  vi B, N;
  vector<ld> b, c;
  vector<vector<ld>> a, D;

  Simplex(int n, int m) : n(n), m(m), b(n), c(m), a(n,
    vector<ld>(m, 0)) {}

  void pivot(int r, int s) {
    ld inv = 1.0L / D[r][s];
    for (int i = 0; i < n + 2; ++i) if (i != r && abs(
      D[i][s]) > EPS_LP) {
      ld inv2 = D[i][s] * inv;
      for (int j = 0; j < m
        + 2; ++j) D[i][j] -= D[r][j] * inv2;
      D[i][s] = D[r][s] * inv2;
    }
    for (int j = 0; j < m + 2; ++j) if (
      j != s) D[r][j] *= inv;
    for (int i = 0; i < n + 2; ++i) if (
      i != r) D[i][s] *= -inv;
    D[r][s] = inv;
    swap(B[r], N[s]);
  }

  bool run_simplex(int phase) {
    int x = n + (phase == 1);
    while (true) {
      int s = -1;
      for (int j = 0; j <= m; ++j) {
        if (phase == 2 && N[j] == -1) continue;
        if (D[x][j] < -EPS_LP) {
          if (s == -1 || make_pair(D[x][j],
            N[j]) < make_pair(D[x][s], N[s]))
            s = j;
        }
      }
      if (s == -1) return true;
      int r = -1;
      for (int i = 0; i < n; ++i) {
        if (D[i][s] <= EPS_LP) continue;
        ld ratio = D[i][m + 1] / D[i][s];
        if (r == -1 ||
            make_pair(ratio, B[i]) < make_pair(
              D[r][m + 1] / D[r][s], B[r]))
          r = i;
      }
      if (r == -1) return false; // No acotado
      pivot(r, s);
    }
  }

  // Retorna {max_Z, x}. Infactible: {-INF_LP, {}}. No
  // acotado: {INF_LP, {}}.
  pair<ld, vector<ld>> solve() {
    B.resize(n); N.resize(m + 1);
    D.assign(n + 2, vector<ld>(m + 2, 0));
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < m; ++j) D[i][j] = a[i][j];
      B[i] = m + i;
      D[i][m] = -1;
      D[i][m + 1] = b[i];
    }
    for (int j = 0; j < m; ++j) { N[j] = j; D[n][j] = -c[j];
    }
    N[m] = -1; D[n + 1][m] = 1;
    int r = 0;
    for (int i = 1; i < n; ++i) if (
```

```
      D[i][m + 1] < D[r][m + 1]) r = i;
    if (D[r][m + 1] < -EPS_LP) {
      pivot(r, m);
      if (!run_simplex(
        1) || D[n + 1][m + 1] < -EPS_LP) return {-INF_LP, {
          }};
      for (int i = 0; i < n; ++i) if (B[i] == -1) {
        int s = 0;
        for (int j = 1; j <= m; ++j)
          if (s == -1 || make_pair(D[i][j],
            N[j]) < make_pair(D[i][s], N[s]))
            s = j;
        pivot(i, s);
      }
    }
    if (!run_simplex(2)) return {INF_LP, {}};
    vector<ld> x(m, 0);
    for (int i = 0; i < n; ++i) if (
      B[i] < m) x[B[i]] = D[i][m + 1];
    return {D[n][m + 1], x};
  }
};
```

- **Trampas y adaptaciones:** (1) Una restricción $\geq$ ($a^T x \geq b$) se convierte multiplicando por -1: $(-a)^T x \leq -b$; una igualdad $a^T x = b$ se representa con ambas desigualdades $\leq$ y $\geq$.

7.9 Berlekamp-Massey y Recurrencias Lineales Modulares

- **Cuándo usar:** Descubrir automáticamente la recurrencia lineal más corta que genera una sucesión modular dada (a partir de sus primeros $2D$ términos) y evaluar su término K-ésimo ($K \leq 10^{18}$) en $\mathcal{O}(D^2 \log K)$.
- **Cuándo no usar / alternativa:** Si la sucesión no proviene de una recurrencia lineal de orden pequeño sino de un polinomio en k de grado D, también funciona (orden $D+1$), pero la interpolación de Lagrange (§7.10.1) la evalúa en $\mathcal{O}(D)$.
- **Contrato:** Índices **0-based** (`get_kth(s, 0)` retorna `s[0]`). Normaliza discrepancias negativas con `norm()` y soporta módulos primos. Dependencias: `MOD` (§7.4.1).
- **Complejidad:** Inferencia de recurrencia $\mathcal{O}(N^2)$; evaluación de `get_kth` en $\mathcal{O}(D^2 \log K)$.

```
// Berlekamp-Massey + Cayley-Hamilton para sucesiones
// modulares
struct BerlekampMassey {
  ll mod;
  BerlekampMassey(ll m = MOD) : mod(m) {}

  ll extgcd(ll a, ll b, ll& x, ll& y) {
    if (!b) { x = 1; y = 0; return a; }
    ll d = extgcd(b, a % b, y, x);
    y -= (a / b) * x;
    return d;
  }
  ll norm(ll a) const {
    a %= mod;
    return (a < 0) ? (a + mod) : a;
  }
  ll inv(ll a) {
    ll x, y; extgcd(norm(a), mod, x, y);
    return norm(x);
  }

  // Requiere mod primo para que toda discrepancia no nula
  // sea invertible
  vll BM(const vll& x) {
    vll ls, cur;
    int lf = 0; ll ld = 0;
    for (int i = 0; i < sz(x); ++i) {
      ll t = 0;
      for (int j = 0; j < sz(cur); ++j)
        t = (t + norm(x[i - j - 1]) * cur[j]) % mod;
      ll d = norm(x[i] - t);
      if (d == 0) continue;
      if (cur.empty()) {
        cur.resize(i + 1, 0);
        lf = i; ld = d;
        continue;
      }
      ll k = d * inv(ld) % mod;
      vll c(i - lf - 1, 0); c.push_back(k);
      for (int j = 0; j < sz(ls); ++j)
        c.push_back(norm(-ls[j]) * k % mod);
      if (sz(c) < sz(cur)) c.resize(sz(cur), 0);
      for (int j = 0; j < sz(cur); ++j)
        c[j] = (c[j] + cur[j]) % mod;
      if (i - lf + sz(ls) >= sz(cur)) {
        ls = cur; lf = i; ld = d;
      }
      cur = c;
    }
    for (ll& val : cur) val = norm(val);
    return cur;
  }

  vll poly_mul(const vll& p, const vll& q, const vll& h,
    int m) {
    vll t(2 * m, 0);
    for (int i = 0; i < m; ++i) if (p[i])
      for (int j = 0; j < m; ++j)
        t[i + j] = (t[i + j] + p[i] * q[j]) % mod;
    for (int i = 2 * m - 1; i >= m; --i) if (t[i])
      for (int j = 0; j < m; ++j)
        t[i - j - 1] = (t[i - j - 1] + t[i] * h[j]) % mod;
    t.resize(m);
    return t;
  }

  ll get_kth(const vll& first_terms, ll k) {
    if (k < sz(first_terms)) return (
      first_terms[k] % mod + mod) % mod;
    vll h = BM(first_terms);
    int m = sz(h);
    if (!m) return 0;
    vll s(m, 0), t(m, 0);
    s[0] = 1;
    if (m != 1) t[1] = 1; else t[0] = h[0];
    for (; k > 0; k >>= 1) {
      if (k & 1) s = poly_mul(s, t, h, m);
      t = poly_mul(t, t, h, m);
    }
    ll ans = 0;
    for (int i = 0; i < m; ++i)
      ans = (ans + s[i] * first_terms[i]) % mod;
    return (ans % mod + mod) % mod;
  }
};
```

- **Trampas y adaptaciones:** (1) Proveer siempre al menos $2D+2$ términos iniciales calculados por fuerza bruta o DP pequeña para que Berlekamp-Massey no infiera una recurrencia espuria más corta.

7.10 Interpolación de Lagrange en $\mathcal{O}(D)$ y $\mathcal{O}(D^2)$

7.10.1 Interpolación de Lagrange en $\mathcal{O}(D)$ para Abscisas Consecutivas $0..D$ (`lagrange_eval_consecutive`)

- **Cuándo usar:** Evaluar en $\mathcal{O}(D)$ un polinomio $P(x) \pmod{\text{MOD}}$ de grado D (por ejemplo $\sum_{i=1}^{n} i^k$, que tiene grado $D = k+1$) conociendo únicamente sus valores en $x = 0, 1, ..., D$.
- **Cuándo no usar / alternativa:** Si las abscisas x_i donde conoces $P(x_i)$ **no** son $0, 1, ..., D$, usar `lagrange_eval_arbitrary` (§7.10.2).
- **Contrato:** `y` tiene tamaño $D+1$ con `y[i] = P(i) mod MOD`. Requiere haber llamado `init_fact(D, MOD)` previamente. Dependencias: `init_fact`, `norm_mod`, `add_mod`, `sub_mod`, `mul_mod` (§7.4).
- **Complejidad:** $\mathcal{O}(D)$ tiempo y memoria por evaluación.

```
// Interpolacion de Lagrange en O(D) para x = 0, 1, ..., D
// 'y' contiene P(0), P(1), ..., P(D). Evalua P(x) mod MOD.
ll lagrange_eval_consecutive(const vll& y, ll x,
  ll mod = MOD) {
  int n = sz(y); // Grado = n - 1
  if (x < n) return (y[x] % mod + mod) % mod;
  vll pref(n), suff(n), fact(n), inv_fact(n);
  pref[0] = x % mod;
  for (int i = 1; i < n; ++i) pref[i] = pref[i - 1] * ((
    x - i) % mod) % mod;
  suff[n - 1] = (x - (n - 1)) % mod;
  for (int i = n - 2; i >= 0; --i) suff[i] = suff[i + 1] * (
    (x - i) % mod) % mod;

  auto bpow = [&](ll b, ll e) {
    ll r = 1;
    for (; e > 0; e >>= 1, b = b * b % mod) if (
      e & 1) r = r * b % mod;
    return r;
  };
  fact[0] = 1;
  for (int i = 1; i < n; ++i) fact[i] = fact[i
    - 1] * i % mod;
  inv_fact[n - 1] = bpow(fact[n - 1], mod - 2);
  for (int i = n - 2; i >= 0; --i) inv_fact[i] = inv_fact[i
    + 1] * (i + 1) % mod;

  ll ans = 0;
  for (int i = 0; i < n; ++i) {
    ll num = 1;
    if (i > 0) num = num * pref[i - 1] % mod;
    if (i + 1 < n) num = num * suff[i + 1] % mod;
    ll den = inv_fact[i] * inv_fact[n - 1 - i] % mod;
    if ((n - 1 - i) & 1) den = mod - den;
    ans = (ans + y[i] * num % mod * den) % mod;
  }
  return (ans % mod + mod) % mod;
}
```

- **Trampas y adaptaciones:** (1) La suma de potencias $\sum_{i=1}^{n} i^k$ es un polinomio de grado $D = k+1$ (no k), por lo que hay que pasar $k+2$ puntos $P(0), ..., P(k+1)$.

7.10.2 Interpolación de Lagrange en $\mathcal{O}(N^2)$ para Puntos Arbitrarios (`lagrange_eval_arbitrary`)

- **Cuándo usar:** Evaluar $P(x) \pmod{\text{MOD}}$ cuando los N puntos conocidos (x_i, y_i) tienen abscisas arbitrarias distintas módulo `MOD`.
- **Cuándo no usar / alternativa:** Si las abscisas son consecutivas $0..D$, `lagrange_eval_consecutive` (§7.10.1) es $\mathcal{O}(D)$ en lugar de $\mathcal{O}(N^2)$.
- **Contrato:** Requiere que todos los `xs[i]` sean distintos módulo `MOD`. Dependencias: `norm_mod`, `add_mod`, `sub_mod`, `mul_mod`, `inv_fermat` (§7.4.1).
- **Complejidad:** $\mathcal{O}(N^2)$ tiempo y $\mathcal{O}(1)$ memoria extra.

```
// 2. Interpolacion de Lagrange en O(N^2) para puntos
// arbitrarios distintos (xs[i], ys[i])
ll lagrange_eval_arbitrary(const vll& xs, const vll& ys,
  ll x, ll mod = MOD) {
  int n = sz(xs);
  auto norm = [&](ll v) { v %= mod;
    return v < 0 ? v + mod : v; };
  auto bpow = [&](ll b, ll e) {
    ll r = 1;
    for (b = norm(b); e > 0; e >>= 1, b = b * b % mod)
      if (e & 1) r = r * b % mod;
    return r;
  };
  ll ans = 0;
  for (int i = 0; i < n; ++i) {
    ll num = 1, den = 1;
    for (int j = 0; j < n; ++j) {
      if (i == j) continue;
      num = num * norm(x - xs[j]) % mod;
      den = den * norm(xs[i] - xs[j]) % mod;
    }
    ll term = norm(ys[i]) * num % mod * bpow(den,
      mod - 2) % mod;
    ans = (ans + term) % mod;
  }
  return ans;
}
```

- **Trampas y adaptaciones:** (1) Si necesitas recuperar explícitamente el vector de coeficientes $c_0 + c_1 x + ... + c_{N-1} x^{N-1}$, precalcular el polinomio producto $\prod (x - x_i)$ en $\mathcal{O}(N^2)$ y dividir sintéticamente por $(x - x_i)$ para cada término.

7.11 Conteo Avanzado y Esperanza: Stars and Bars, Inclusión-Exclusión, Contribuciones y Linealidad

7.11.1 Stars and Bars: Composiciones No Negativas y Positivas (`stars_and_bars_nonneg`, `stars_and_bars_pos`)

- **Cuándo usar:** Contar cuántas tuplas de k enteros suman exactamente n: con $x_i \geq 0$ es $\binom{n+k-1}{k-1}$, y con $x_i \geq 1$ es $\binom{n-1}{k-1}$.
- **Cuándo no usar / alternativa:** Si además cada variable tiene una cota superior $x_i \leq c$, combinar Stars and Bars con **Inclusión-Exclusión** sobre el subconjunto de variables que violan $x_i \geq c+1$.
- **Contrato:** Requiere haber llamado `init_fact(n + k, MOD)` previamente. Dependencias: `init_fact`, `nCr` (§7.4.2).
- **Complejidad:** $\mathcal{O}(1)$ por consulta tras `init_fact`.

```
// 1. Stars and Bars: x_1 + ... + x_k = n (requiere
// init_fact() de §7.4)
ll stars_and_bars_nonneg(int n, int k) { // x_i >= 0
  if (n < 0 || k <= 0) return (n == 0 && k == 0) ? 1 : 0;
  return nCr(n + k - 1, k - 1);
}
ll stars_and_bars_pos(int n, int k) { // x_i >= 1
  if (n < k || k <= 0) return 0;
  return nCr(n - 1, k - 1);
}
```

- **Trampas y adaptaciones:** (1) Si cada variable exige $x_i \geq a_i$, sustituir $x'_i = x_i - a_i \geq 0$ y llamar `stars_and_bars_nonneg(n - sum(a_i), k)`.

7.11.2 Principio de Inclusión-Exclusión para Múltiplos (`inclusion_exclusion_multiples`)

- **Cuándo usar:** Contar cuántos enteros en $[1, N]$ ($N \leq 10^{18}$) son divisibles por **al menos uno** de los números de una lista pequeña D ($|D| \leq 20$).
- **Cuándo no usar / alternativa:** Si $|D| > 25$, iterar las $2^{|D|}$ máscaras da TLE.
- **Contrato:** Usa `i128` al actualizar el MCM (`lcm = lcm / g * divs[i]`) y corta la rama si `lcm > n` para evitar overflow.
- **Complejidad:** $\mathcal{O}(2^K \cdot K \log N)$ tiempo y $\mathcal{O}(1)$ memoria.

```cpp
// 2. Principio de Inclusion-Exclusion en [1, N]:
// Cuenta enteros en [1, N] divisibles por al menos un d in
// divs.
ll inclusion_exclusion_multiples(ll N, const vll& divs) {
  int k = sz(divs);
  ll ans = 0;
  for (int mask = 1; mask < (1 << k); ++mask) {
    ll lcm_val = 1;
    int bits = __builtin_popcount(mask);
    for (int i = 0; i < k; ++i) {
      if ((mask >> i) & 1) {
        ll g = std::gcd(lcm_val, divs[i]);
        __int128 nxt = (__int128)(lcm_val / g) * divs[i];
        if (nxt > N) { lcm_val = N + 1; break; }
        lcm_val = (ll)nxt;
      }
    }
    if (lcm_val <= N) {
      ll cnt = N / lcm_val;
      ans += (bits & 1) ? cnt : -cnt;
    }
  }
  return ans;
}
```

- **Trampas y adaptaciones:** (1) Para contar cuántos enteros en $[1, N]$ **no** son divisibles por ninguno de los divisores, calcular `n - inclusion_exclusion_multiples(n, divs)`.

7.11.3 Técnica de Contribuciones con Pila Monótona (`sum_all_subarray_mins`)

- **Cuándo usar:** Calcular la suma de $\min(a[l..r])$ (o $\max(a[l..r])$) sobre todos los $\binom{N+1}{2}$ subarreglos en $\mathcal{O}(N)$ contando en cuántos subarreglos cada $a[i]$ actúa como mínimo.
- **Cuándo no usar / alternativa:** Si la consulta pide la suma de mínimos solo dentro de un subrango $[L_q, R_q]$ por query, precalcular saltos en la pila monótona o usar Mo / Segment Tree.
- **Contrato:** Para no contar dos veces subarreglos con mínimos empatados, busca el anterior **estrictamente menor** (`<`) a la izquierda y el siguiente **menor o igual** (`<=`) a la derecha.
- **Complejidad:** $\mathcal{O}(N)$ tiempo y memoria.

```cpp
// 3. Tecnica de Contribuciones: Suma de min(a[l..r]) sobre
// todos los
// subarreglos en O(N). Usa '>' a la izq y '>=' a la der
// para empates.
ll sum_all_subarray_mins(const vll& a) {
  int n = sz(a);
  vi L(n, -1), R(n, n);
  stack<int> st;
  for (int i = 0; i < n; ++i) {
    while (!st.empty() && a[st.top()] > a[i]) st.pop();
    L[i] = st.empty() ? -1 : st.top();
    st.push(i);
  }
  while (!st.empty()) st.pop();
  for (int i = n - 1; i >= 0; --i) {
    while (!st.empty() && a[st.top()] >= a[i]) st.pop();
    R[i] = st.empty() ? n : st.top();
    st.push(i);
  }
  ll total = 0;
  for (int i = 0; i < n; ++i) {
    ll ways = 1LL * (i - L[i]) * (R[i] - i);
    total += a[i] * ways;
  }
  return total;
}
```

- **Trampas y adaptaciones:** (1) Usar `<` de ambos lados cuenta de más cuando hay elementos iguales; usar `<=` de ambos lados cuenta de menos. Siempre emparejar `<` a un lado con `<=` al otro.

7.11.4 Linealidad de la Esperanza: Coupon Collector (`expected_coupon_collector`)

- **Cuándo usar:** Calcular el número esperado de extracciones uniformes con reemplazo necesarias para obtener los N tipos distintos de cupones ($E[T] = \sum_{k=0}^{N-1} \frac{N}{N-k} = N H_N$).
- **Cuándo no usar / alternativa:** Si los cupones tienen probabilidades **no uniformes** p_i, usar Inclusión-Exclusión Min-Max: $E[\max T_i] = \sum_{\varnothing \neq S \subseteq [N]} (-1)^{|S|-1} \frac{1}{\sum_{i \in S} p_i}$.
- **Contrato:** Por linealidad de la esperanza, el tiempo para pasar de k cupones distintos a $k+1$ sigue una distribución geométrica de éxito $p = (N-k)/N$, cuyo valor esperado es $1/p = N/(N-k)$.
- **Complejidad:** $\mathcal{O}(N)$ tiempo y $\mathcal{O}(1)$ memoria.

```cpp
// 4. Linealidad de la Esperanza: Coupon Collector E[pasos]
// = N * H_N
ld expected_coupon_collector(int n) {
  ld h = 0;
  for (int i = 1; i <= n; ++i) h += 1.0L / i;
  return n * h;
}
```

- **Trampas y adaptaciones:** (1) Si el problema pide $N H_N \pmod{998244353}$, reemplazar `1.0L / rem` por `inv_num[rem]` (§7.4.2).

Bloque 8: Programación Dinámica (DP Clásica, Subconjuntos, Árboles y Optimizaciones)

8.1 DP Clásica: Knapsack 0/1 y Unbounded, LCS, Edit Distance, LIS $\mathcal{O}(N \log N)$, Digit DP y SOS DP

8.1.1 Mochila 0/1 en Memoria Lineal (`knapsack_01`)

- **Cuándo usar:** Seleccionar un subconjunto de ítems con peso w_i y valor v_i cuya suma de pesos no supere W ($N \cdot W \leq 10^8$) maximizando el valor total.
- **Cuándo no usar / alternativa:** Si la capacidad W es enorme ($W \leq 10^9$) pero la suma total de valores $V = \sum v_i \leq 10^5$ es pequeña, **invertir el estado**: `dp[v] = menor peso para lograr valor v`.
- **Contrato:** Recorre `w` **hacia atrás** (`W` hacia `weight[i]`) para usar cada ítem a lo sumo una vez.
- **Complejidad:** $\mathcal{O}(N \cdot W)$ tiempo y $\mathcal{O}(W)$ memoria.

```cpp
// 1. Knapsack 0/1 en O(N * W) tiempo y O(W) memoria
ll knapsack_01(int W, const vi& weight, const vll& val) {
  vll dp(W + 1, 0);
  for (int i = 0; i < sz(weight); ++i)
    // Hacia adelante si Unbounded
    for (int w = W; w >= weight[i]; --w)
      dp[w] = max(dp[w], dp[w - weight[i]] + val[i]);
  return dp[W];
}
```

- **Trampas y adaptaciones:** (1) Para **Unbounded Knapsack** (cada ítem se puede repetir infinitas veces), cambiar únicamente el bucle interno para que avance hacia adelante (`for (int w = weight[i]; w <= W; ++w)`).

8.1.2 Subsecuencia Común Más Larga con Reconstrucción (`lcs_string`)

- **Cuándo usar:** Hallar y reconstruir la cadena más larga que aparece como subsecuencia (no necesariamente contigua) tanto en `a` como en `b` ($|a|, |b| \leq 5000$).
- **Cuándo no usar / alternativa:** Si buscas la **subcadena contigua** común más larga, usar `SuffixAutomaton::longest_common_substring` (§5.6.2) en $\mathcal{O}(|a| + |b|)$.
- **Contrato:** Construye la tabla `dp` $(n+1) \times (m+1)$ y reconstruye la cadena retrocediendo desde `(n, m)`.
- **Complejidad:** $\mathcal{O}(N \cdot M)$ tiempo y memoria.

```cpp
// 1b. LCS (Longest Common Subsequence con reconstruccion)
// en O(N * M)
string lcs_string(const string& a, const string& b) {
  int n = sz(a), m = sz(b);
  vector<vi> dp(n + 1, vi(m + 1, 0));
  for (int i = 1; i <= n; ++i)
    for (int j = 1; j <= m; ++j)
      dp[i][j] = (a[i - 1] == b[j - 1])
                   ? (dp[i - 1][j - 1] + 1)
                   : max(dp[i - 1][j], dp[i][j - 1]);
  string res;
  for (int i = n, j = m; i > 0 && j > 0; ) {
    if (a[i - 1] == b[j - 1]) { res += a[i - 1]; --i; --j; }
    else if (dp[i - 1][j] >= dp[i][j - 1]) --i;
    else --j;
  }
  reverse(all(res));
  return res;
}
```

- **Trampas y adaptaciones:** (1) Si uno de los arreglos es una permutación sin elementos repetidos, renombrar sus valores por sus índices convierte el LCS en un **LIS** en $\mathcal{O}(N \log N)$ (§8.1.4).

8.1.3 Distancia de Edición de Levenshtein (`edit_distance`)

- **Cuándo usar:** Mínimo número de inserciones, borrados y sustituciones de un carácter para transformar la cadena `a` en la cadena `b`.
- **Cuándo no usar / alternativa:** Si no se permiten sustituciones (solo insertar y borrar), la respuesta es directamente $|a| + |b| - 2 \cdot |\mathrm{LCS}(a, b)|$.
- **Contrato:** Casos base: `dp[i][0] = i` (borrar `i` caracteres) y `dp[0][j] = j` (insertar `j` caracteres).
- **Complejidad:** $\mathcal{O}(N \cdot M)$ tiempo y memoria.

```cpp
// 2. Edit Distance (Levenshtein) en O(N * M)
int edit_distance(const string& a, const string& b) {
  int n = sz(a), m = sz(b);
  vector<vi> dp(n + 1, vi(m + 1, 0));
  for (int i = 0; i <= n; ++i) dp[i][0] = i;
  for (int j = 0; j <= m; ++j) dp[0][j] = j;
  for (int i = 1; i <= n; ++i)
    for (int j = 1; j <= m; ++j) {
      if (a[i - 1] == b[j - 1]) dp[i][j] = dp[i - 1][j - 1];
      else dp[i][j] = 1 + min({dp[i - 1][j], dp[i][j - 1],
                               dp[i - 1][j - 1]});
    }
  return dp[n][m];
}
```

- **Trampas y adaptaciones:** (1) Si solo se pide el número de operaciones y la memoria es ajustada, mantener solo dos filas (`dp_prev` y `dp_cur`) reduce la memoria a $\mathcal{O}(\min(N, M))$.

8.1.4 Subsecuencia Creciente Más Larga en $\mathcal{O}(N \log N)$ (`lis_length`)

- **Cuándo usar:** Calcular la longitud de la mayor subsecuencia estrictamente creciente ($a_{i_1} < a_{i_2} < \ldots < a_{i_k}$) en $\mathcal{O}(N \log N)$.
- **Cuándo no usar / alternativa:** Si la condición de transición entre elementos consecutivos no es un orden total sino un predicado general, usar Segment Tree o CDQ Divide & Conquer.
- **Contrato:** `tails[k]` almacena el menor valor final posible de una subsecuencia creciente de longitud $k+1$. Usa `lower_bound` para estrictamente creciente (`<`).
- **Complejidad:** $\mathcal{O}(N \log N)$ tiempo y $\mathcal{O}(N)$ memoria.

```cpp
// 3. LIS (Longest Increasing Subsequence) en O(N log N)
int lis_length(const vll& a) {
  vll tails;
  for (ll x : a) {
    // upper_bound para <=
    auto it = lower_bound(all(tails), x);
    if (it == tails.end()) tails.pb(x);
    else *it = x;
  }
  return sz(tails);
}
```

- **Trampas y adaptaciones:** (1) Para subsecuencia **no decreciente** ($\leq$), cambiar `lower_bound` por `upper_bound`; (2) el vector `tails` **no** es necesariamente la subsecuencia en sí, pero su tamaño `sz(tails)` siempre es la longitud exacta del LIS.

8.1.5 Digit DP por Prefijos Decimales (`dp_dig` , `solve_digit_dp`)

- **Cuándo usar:** Contar cuántos enteros en un rango enorme $[L, R]$ ($R \leq 10^{18}$ o dado como string de 10^5 dígitos) cumplen una propiedad sobre sus dígitos (suma divisible por K, sin dígitos adyacentes iguales, etc.).
- **Cuándo no usar / alternativa:** Para contar en $[L, R]$, nunca intentar meter simultáneamente cota inferior y superior si no es necesario; calcular simplemente `solve(R) - solve(L - 1)`.
- **Contrato:** Estado `(pos, rem, tight)` donde `tight` indica si el prefijo actual sigue empatado con los dígitos de `R`.
- **Complejidad:** $\mathcal{O}(10 \cdot \text{dígitos}(R) \cdot K)$ tiempo y $\mathcal{O}(\text{dígitos}(R) \cdot K)$ memoria.

```cpp
// 4. Digit DP: Cuenta numeros en [0, R] con suma de digitos
// % K == 0
string dig_s;
int K_mod;
ll memo_dig[20][200][2];
```

```cpp
ll dp_dig(int pos, int rem, bool tight) {
  if (pos == sz(dig_s)) return (rem == 0) ? 1 : 0;
  ll& res = memo_dig[pos][rem][tight];
  if (res != -1) return res;
  res = 0;
  int lim = tight ? (dig_s[pos] - '0') : 9;
  for (int d = 0; d <= lim; ++d)
    res += dp_dig(pos + 1, (rem + d) % K_mod, tight && (
      d == lim));
  return res;
}
ll solve_digit_dp(ll R, int K) {
  if (R < 0) return 0;
  dig_s = to_string(R); K_mod = K;
  memset(memo_dig, -1, sizeof(memo_dig));
  return dp_dig(0, 0, true);
}
```

- **Trampas y adaptaciones:** (1) Si la propiedad depende de los ceros a la izquierda (ej. contar ceros o exigir dígitos distintos), agregar una bandera booleana `started` al estado.

8.1.6 SOS DP / Suma Sobre Submáscaras (`sos_dp`)

- **Cuándo usar:** Calcular para toda máscara `mask` de N bits la suma $F[\text{mask}] = \sum_{\text{sub} \subseteq \text{mask}} A[\text{sub}]$ en $\mathcal{O}(N \cdot 2^N)$ en lugar de iterar submáscaras en $\mathcal{O}(3^N)$.
- **Cuándo no usar / alternativa:** Si la transición a `mask` depende de valores ya calculados de `dp[sub]` que no son una suma fija independiente, usar iteración de submáscaras `for (int sub = mask; sub; sub = (sub - 1) & mask)` en $\mathcal{O}(3^N)$.
- **Contrato:** `A` debe tener tamaño 2^N. El bucle externo **debe** ser el bit `i` (`0..N-1`) para no sumar la misma submáscara dos veces.
- **Complejidad:** $\mathcal{O}(N \cdot 2^N)$ tiempo y $\mathcal{O}(2^N)$ memoria.

```cpp
// 5. SOS DP (Sum Over Subsets) en O(N * 2^N)
vll sos_dp(int N, const vll& A) {
  vll F = A;
  for (int i = 0; i < N; ++i)
    for (int mask = 0; mask < (1 << N); ++mask)
      if (mask & (1 << i))
        F[mask] += F[mask ^ (1 << i)];
  return F;
}
```

- **Trampas y adaptaciones:** (1) Para sumar sobre **supermáscaras** ($\sup \supseteq \text{mask}$), cambiar la condición `if (mask & (1 << i))` por `if (!(mask & (1 << i))) F[mask] += F[mask | (1 << i)];` .

8.2 DP de Subconjuntos con Máscaras: Asignación `dp[mask]` y Caminos / TSP `dp[mask][last]`

8.2.1 DP de Asignación por Máscaras de Bits (`subset_assignment_dp`)

- **Cuándo usar:** Asignar N elementos a N posiciones/turnos ($N \leq 20$) cuando el paso actual está determinado por la cantidad de bits encendidos `step = __builtin_popcount(mask)` , ahorrando un factor N en estado.
- **Cuándo no usar / alternativa:** Si $N > 22$ y se trata de una asignación 1-a-1 de costo aditivo puro sin dependencia de subconjunto, usar el **Algoritmo Húngaro** (§6.6.4) en $\mathcal{O}(N^3)$.
- **Contrato:** Matriz `cost` 0-indexed de tamaño $N \times N$. Retorna el costo mínimo de asignar el paso `step` al elemento `j` para todo `step = 0..N-1` .
- **Complejidad:** $\mathcal{O}(N \cdot 2^N)$ tiempo y $\mathcal{O}(2^N)$ memoria.

```cpp
// 1. DP de Asignacion con dp[mask] en O(N * 2^N) tiempo y
// O(2^N) RAM
// Asigna la persona i = popcount(mask) a la tarea j no
// usada.
ll subset_assignment_dp(const vector<vll>& cost) {
  int n = sz(cost);
  int full = (1 << n);
  vll dp(full, INF);
  dp[0] = 0;
  for (int mask = 0; mask < full - 1; ++mask) {
    if (dp[mask] >= INF) continue;
    int i = __builtin_popcount(mask);
    for (int j = 0; j < n; ++j) {
      if (!(mask & (1 << j))) {
        int nxt = mask | (1 << j);
        dp[nxt] = min(dp[nxt], dp[mask] + cost[i][j]);
      }
    }
  }
  return dp[full - 1];
}
```

- **Trampas y adaptaciones:** (1) Iterar solo los bits apagados o encendidos con `__builtin_ctz` reduce la constante práctica a la mitad.

8.2.2 DP de Caminos / Travelling Salesperson Problem con Reconstrucción (`tsp_subset_dp`)

- **Cuándo usar:** Encontrar el ciclo o camino Hamiltoniano de menor peso que visita todos los N nodos exactamente una vez ($N \leq 20$) cuando la transición depende del último nodo visitado `last` .
- **Cuándo no usar / alternativa:** Si quieres recorrer todas las **aristas** en vez de los vértices, es un Ciclo Euleriano en $\mathcal{O}(V + E)$ (§6.9).
- **Contrato:** Nodos 0-indexed (`0..n-1`), empezando en `0` . Si `close_cycle = true` , exige regresar al nodo `0` al final. Retorna `{costo_minimo, ruta_reconstruida}` .
- **Complejidad:** $\mathcal{O}(N^2 \cdot 2^N)$ tiempo y $\mathcal{O}(N \cdot 2^N)$ memoria.

```cpp
// 2. DP de Caminos / TSP con dp[mask][last] en O(N^2 * 2^N)
//    y reconstruccion del orden de nodos visitados.
pair<ll, vi> tsp_subset_dp(const vector<vll>& dist,
                           int start = 0,
                           bool close_cycle = true) {
  int n = sz(dist), full = (1 << n);
  vector<vll> dp(full, vll(n, INF));
  vector<vi> par(full, vi(n, -1));
  dp[1 << start][start] = 0;

  for (int mask = 1; mask < full; ++mask) {
    for (int u = 0; u < n; ++u) {
      if (!(mask & (
        1 << u)) || dp[mask][u] >= INF) continue;
      for (int v = 0; v < n; ++v) {
        if ((mask & (
          1 << v)) || dist[u][v] >= INF) continue;
        int nxt = mask | (1 << v);
        ll cand = dp[mask][u] + dist[u][v];
        if (cand < dp[nxt][v]) {
          dp[nxt][v] = cand;
          par[nxt][v] = u;
        }
      }
    }
  }
  ll best = INF;
  int last = -1, all_m = full - 1;
  for (int u = 0; u < n; ++u) {
    if (dp[all_m][u] >= INF) continue;
    ll extra = close_cycle ? dist[u][start] : 0;
    if (extra < INF && dp[all_m][u] + extra < best) {
      best = dp[all_m][u] + extra;
      last = u;
    }
  }
  if (last == -1) return {INF, {}};
  vi path;
  for (int m = all_m, cur = last; cur != -1; ) {
    path.pb(cur);
    int p = par[m][cur];
    m ^= (1 << cur);
    cur = p;
  }
  reverse(all(path));
  return {best, path};
}
```

- **Trampas y adaptaciones:** (1) Para camino Hamiltoniano que puede empezar en **cualquier** nodo, inicializar `dp[1 << i][i] = 0` para todo $i \in [0, N-1]$ y pasar `close_cycle = false` .

8.3 DP en Árboles y Técnica de Rerooting (Cambio de Raíz en $\mathcal{O}(N)$)

- **Cuándo usar:** Calcular una función de árbol (suma de distancias, máxima distancia, DP de subárbol) **para todas las raíces posibles** $u \in [1..N]$ en dos pasadas DFS (`dfs_down` y `dfs_up`).
- **Cuándo no usar / alternativa:** Si la respuesta solo se pide para una raíz fija, basta un solo DFS abajo-arriba.
- **Contrato:** Árbol 1-indexed (`1..n`). `dfs_down` calcula `sub_sz[u]` y `down[u]` ; `dfs_up` propaga hacia cada hijo `v` usando `ans[v] = ans[u] + (n - sub_sz[v]) - sub_sz[v]` .
- **Complejidad:** $\mathcal{O}(N)$ tiempo y memoria.

```cpp
// Rerooting DP: Suma de distancias desde cada nodo u a
// todos los demas
// 1) dfs_down: calcula sz[u] y down[u] (contribucion del
// subarbol de u)
// 2) dfs_up: al mover la raiz de u a su hijo v, los sz[v]
// nodos en el
// subarbol de v se acercan 1 paso y los (n - sz[v]) fuera
// se alejan 1.
vll rerooting_sum_distances(int n, const vector<vi>& adj) {
  vi sub_sz(n + 1, 0);
  vll down(n + 1, 0), ans(n + 1, 0);

  auto dfs_down = [&](auto& self, int u, int p) -> void {
    sub_sz[u] = 1;
    for (int v : adj[u]) {
      if (v == p) continue;
      self(self, v, u);
      sub_sz[u] += sub_sz[v];
      down[u] += down[v] + sub_sz[v];
    }
  };
  auto dfs_up = [&](auto& self, int u, int p) -> void {
    for (int v : adj[u]) {
      if (v == p) continue;
      ans[v] = ans[u] + (n - sub_sz[v]) - sub_sz[v];
      self(self, v, u);
    }
  };
  if (n >= 1) {
    dfs_down(dfs_down, 1, 0);
    ans[1] = down[1];
    dfs_up(dfs_up, 1, 0);
  }
  return ans; // ans[1..n]
}
```

- **Trampas y adaptaciones:** (1) Cuando la operación que combina hijos no es invertible (como `max` o producto módulo M), precalcular prefijos y sufijos sobre la lista de hijos de `u` para excluir la rama de `v` en $\mathcal{O}(1)$.

8.4 Optimizaciones de DP: Convex Hull Trick (Estático y Dinámico), Divide & Conquer DP y Knuth's Optimization

8.4.1 Convex Hull Trick Dinámico (`LineContainer`)

- **Cuándo usar:** Optimizar transiciones de DP de la forma $dp[i] = \max_j(k_j x_i + m_j)$ (o $\min_j$) cuando las pendientes k_j y los puntos de consulta x_i llegan en **orden arbitrario**.
- **Cuándo no usar / alternativa:** Si tanto las pendientes como los puntos de consulta son monótonos, `MonotoneCHT` (§8.4.2) corre en $\mathcal{O}(N)$ sin `std::multiset` .
- **Contrato:** `add(k, m)` inserta $y = kx + m$; `query(x)` devuelve el **máximo** en `x` . Para consultar el **mínimo**, insertar `add(-k, -m)` y tomar `-query(x)` .
- **Complejidad:** $\mathcal{O}(\log N)$ amortizado por inserción y consulta; $\mathcal{O}(N)$ memoria.

```cpp
// 1. Convex Hull Trick Dinamico (LineContainer - inserta
// rectas y = k*x + m
// en cualquier orden y consulta el MAXIMO en x en O(log
// N)).
// NOTA: Para consultar el MINIMO, insertar (-k, -m) y
// devolver -query(x).
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  static const ll inf = LLONG_MAX;
  // Division entera hacia abajo (floor)
  ll div(ll a, ll b) {
    return a / b - ((a ^ b) < 0 && a % b);
  }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(ll k, ll m) { // Agrega la recta y = k*x + m
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(
      y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
```

```
  }
  ll query(ll x) const { // Retorna max(k*x + m) en O(log N)
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

- **Trampas y adaptaciones:** (1) No llamar `query(x)` con el contenedor vacío (`assert(!empty())`); insertar primero el caso base `dp[0]`.

8.4.2 Convex Hull Trick Estático / Monótono con Deque (`MonotoneCHT`)

- **Cuándo usar:** Consultar el **mínimo** de rectas $y = mx + b$ en $\mathcal{O}(1)$ amortizado cuando las pendientes m se insertan en orden estrictamente **decreciente** y las consultas x se hacen en orden **creciente**.
- **Cuándo no usar / alternativa:** Si las pendientes o las consultas no son monótonas, usar `LineContainer` (§8.4.1).
- **Contrato:** Compara intersecciones con multiplicación cruzada en `i128` sin división ni punto flotante.
- **Complejidad:** $\mathcal{O}(1)$ amortizado por `add` y `query_monotone`; $\mathcal{O}(N)$ memoria.

```
// 1b. CHT Estático/Monótono con Deque en O(N) (para MINIMO:
// insertar pendientes m decrecientes
//      y consultar x crecientes).
struct MonotoneCHT {
  struct L { ll m, b; ll eval(ll x) const {
    return m * x + b; } };
  deque<L> dq;
  bool bad(const L& l1, const L& l2, const L& l3) const {
    return (__int128)(l3.b - l1.b) * (l1.m - l2.m) <=
      (__int128)(l2.b - l1.b) * (l1.m - l3.m);
  }
  // Precondicion: m estrictamente decreciente
  void add(ll m, ll b) {
    L cur = {m, b};
    while (sz(dq) >= 2 && bad(dq[sz(dq) - 2], dq.back(),
      cur)) dq.pop_back();
    dq.pb(cur);
  }
  // Precondicion: x creciente y !dq.empty()
  ll query_monotone(ll x) {
    while (sz(dq) >= 2 && dq[0].eval(x) >= dq[1].eval(
      x)) dq.pop_front();
    return dq[0].eval(x);
  }
};
```

- **Trampas y adaptaciones:** (1) Si tienes pendientes iguales con distinta ordenada al origen, conservar solo la de menor `b` antes de llamar a `add`.

8.4.3 Optimización Divide & Conquer DP (`compute_dc`, `solve_dc_dp`)

- **Cuándo usar:** Particionar un arreglo `1..N` en exactamente `K` subarreglos contiguos minimizando $\sum C(j,i)$ cuando la función de costo cumple la **desigualdad cuadrangular** $C(a,c) + C(b,d) \leq C(a,d) + C(b,c)$ (lo que implica monotonía del punto óptimo $\text{opt}[k][i] \leq \text{opt}[k][i+1]$).
- **Cuándo no usar / alternativa:** Si el costo no cumple monotonía del óptimo pero es bilineal $f(j)g(i)$, usar CHT (§8.4.1).
- **Contrato:** Índices **1-based** (`1..n`). `cost_fn(j, i)` devuelve el costo del bloque cerrado `[j, i]`. Ignora estados inalcanzables (`dp_prev[j - 1] >= INF`) sin corromper `opt`.
- **Complejidad:** $\mathcal{O}(K \cdot N \log N)$ tiempo y $\mathcal{O}(N)$ memoria.

```
// 2. Divide and Conquer DP Optimization en O(K * N log N)
vll dp_prev, dp_cur;

template <typename CostFn>
void compute_dc(int l, int r, int optl, int optr,
  const CostFn& cost_fn) {
  if (l > r) return;
  int mid = (l + r) / 2;
  pair<ll, int> best = {INF, optl};
  int hi = min(mid, optr);
  for (int j = optl; j <= hi; ++j) {
    if (dp_prev[j - 1] >= INF) continue;
    ll cand = dp_prev[j - 1] + cost_fn(j, mid);
    if (cand < best.fi) best = {cand, j};
  }
  dp_cur[mid] = best.fi;
  int opt = best.se;
  compute_dc(l, mid - 1, optl, (best.fi < INF ? opt : hi),
    cost_fn);
  compute_dc(mid + 1, r, (best.fi < INF ? opt : optl), optr,
    cost_fn);
}

// Driver completo de D&C DP: particionar 1..n en
// exactamente k grupos (1-indexed)
template <typename CostFn>
ll solve_dc_dp(int n, int k, const CostFn& cost_fn) {
  dp_prev.assign(n + 1, INF);
  dp_cur.assign(n + 1, INF);
  dp_prev[0] = 0;
  for (int step = 1; step <= k; ++step) {
    fill(all(dp_cur), INF);
    compute_dc(step, n, step, n, cost_fn);
    dp_prev = dp_cur;
  }
  return dp_prev[n];
}
```

- **Trampas y adaptaciones:** (1) En problemas donde evaluar $C(l,r)$ cuesta $\mathcal{O}(1)$ con dos punteros tipo Mo (ej. pares iguales en `[l, r]`), mantener una ventana `[cur_l, cur_r]` global dentro de `compute_dc` sigue costando $\mathcal{O}(KN \log N)$.

8.4.4 Optimización de Knuth para DP de Intervalos (`solve_knuth`)

- **Cuándo usar:** Reducir de $\mathcal{O}(N^3)$ a $\mathcal{O}(N^2)$ la DP de intervalos $dp[l][r] = \min_{l \leq k < r}(dp[l][k] + dp[k+1][r]) + C[l][r]$ (como *Optimal Binary Search Tree* o partir una barra/cuerda) cuando $C[l][r]$ cumple desigualdad cuadrangular y monotonía en inclusión.
- **Cuándo no usar / alternativa:** En problemas como multiplicación de cadenas de matrices o *Burst Balloons*, el término sumado depende de `k` ($a_l a_k a_r$), por lo que Knuth no aplica.
- **Contrato:** Matriz `C` 1-indexed de tamaño $(n+1) \times (n+1)$. Acota la búsqueda de `k` a `opt[l][r - 1] <= k <= min(r - 1, opt[l + 1][r])`.
- **Complejidad:** $\mathcal{O}(N^2)$ tiempo y memoria.

```
// 3. Knuth's Optimization para DP de Intervalos en O(N^2)
// Resuelve dp[l][r] = min_{l <= k < r} (dp[l][k] +
// dp[k+1][r]) + C(l, r)
ll solve_knuth(int n, const vector<vll>& C) {
  vector<vll> dp(n + 1, vll(n + 1, 0));
  vector<vi> opt(n + 1, vi(n + 1, 0));
  for (int i = 1; i <= n; ++i) opt[i][i] = i;

  for (int len = 2; len <= n; ++len) {
    for (int l = 1; l + len - 1 <= n; ++l) {
      int r = l + len - 1;
      dp[l][r] = INF;
      int kl = opt[l][r - 1], kr = min(r - 1,
        opt[l + 1][r]);
      for (int k = kl; k <= kr; ++k) {
        ll val = dp[l][k] + dp[k + 1][r] + C[l][r];
        if (val < dp[l][r]) { dp[l][r] = val; opt[l][r] = k;
          }
      }
    }
  }
  return dp[1][n];
}
```

- **Trampas y adaptaciones:** (1) Iterar siempre por longitud creciente `for (int len = 2; len <= n; ++len)` para que `opt[l][r - 1]` y `opt[l + 1][r]` ya estén calculados.

Bloque 9: Geometría Computacional 2D

9.1 Geometría 2D: `Pt`, `PtLL` (Entero), Segmentos, Polígonos (Área, Centroide, Punto en Polígono $\mathcal{O}(N)$ y Convexo $\mathcal{O}(\log N)$), `Convex Hull`, Rotating Callipers y Closest Pair

9.1.1 Punto 2D, Producto Punto, Producto Cruz y Orden Radial (`Pt`, `dot`, `cross`, `polar_cmp`)

- **Cuándo usar:** Base vectorial 2D para giros (`cross(a, b, c) > 0` es giro antihorario CCW), distancias y ordenamiento angular de vectores alrededor del origen (`polar_cmp`) en $[0, 2\pi)$ sin usar `atan2`.
- **Cuándo no usar / alternativa:** No usar `operator<` lexicográfico si buscas agrupar puntos con tolerancia `EPS` sin antes redondear, ya que la igualdad con `EPS` no es transitiva.
- **Contrato:** `operator<` y `operator==` son exactos para cumplir *strict weak ordering* en `std::sort` / `std::set`; para comparar con tolerancia usar `pt_eq_eps(a, b)`. `polar_cmp` separa el semiplano superior ($y > 0$ o $y = 0, x > 0$) del inferior y desempata por `cross(a, b) > 0`. Dependencias: `EPS` (§1.1).
- **Complejidad:** $\mathcal{O}(1)$ para productos vectoriales; $\mathcal{O}(N \log N)$ para ordenar N puntos con `polar_cmp`.

```
// 1. Estructura Base de Punto / Vector 2D (usar ld para
// general o ll para exacto)
struct Pt {
  ld x, y;
  Pt(ld x = 0, ld y = 0) : x(x), y(y) {}
  Pt operator+(const Pt& o) const { return {
    x + o.x, y + o.y}; }
  Pt operator-(const Pt& o) const { return {
    x - o.x, y - o.y}; }
  Pt operator*(ld k) const { return {x * k, y * k}; }
  Pt operator/(ld k) const { return {x / k, y / k}; }
  // Orden lexicografico exacto para cumplir strict weak
  // ordering en sort/set:
  bool operator<(const Pt& o) const {
    return x != o.x ? x < o.x : y < o.y;
  }
  bool operator==(const Pt& o) const {
    return x == o.x && y == o.y;
  }
  ld norm2() const { return x * x + y * y; }
  ld norm() const { return hypot(x, y); }
  // Rotacion 90 grados CCW
  Pt perp() const { return {-y, x}; }
  // Rotacion CCW por 'ang_rad'
  Pt rot(ld ang_rad) const {
    return {x * cos(ang_rad) - y * sin(ang_rad),
            x * sin(ang_rad) + y * cos(ang_rad)};
  }
};

int sgn(ld x) { return (x > EPS) - (x < -EPS); }
bool pt_eq_eps(const Pt& a, const Pt& b) {
  return abs(a.x - b.x) <= EPS && abs(a.y - b.y) <= EPS;
}
ld dot(const Pt& a, const Pt& b) {
  return a.x * b.x + a.y * b.y; }
ld cross(const Pt& a, const Pt& b) {
  return a.x * b.y - a.y * b.x; }
// Giro de a -> b -> c: > 0 izquierda (CCW), < 0 derecha
// (CW), 0 colineal
ld cross(const Pt& a, const Pt& b, const Pt& c) {
  return cross(b - a, c - a); }

// 2. Orden Radial (Polar Sort alrededor del origen (0, 0)
// en O(N log N))
int half_plane(const Pt& p) {
  return (p.y > 0 || (p.y == 0 && p.x > 0)) ? 0 : 1;
}
bool polar_cmp(const Pt& a, const Pt& b) {
  int ha = half_plane(a), hb = half_plane(b);
  return (ha != hb) ? (ha < hb) : (cross(a, b) > 0);
}
```

- **Trampas y adaptaciones:** (1) El vector nulo `(0, 0)` no tiene ángulo definido; filtrarlo antes de ordenar con `polar_cmp`.

9.1.2 Segmentos y Rectas: Pertenencia, Distancia e Intersección (`on_segment`, `dist_point_segment`, `segments_intersect`, `line_intersection`)

- **Cuándo usar:** Verificar si un punto cae sobre un segmento cerrado, medir la distancia de un punto a un segmento, comprobar si dos segmentos cerrados se tocan o cruzan, y hallar el punto de intersección de dos rectas no paralelas.
- **Cuándo no usar / alternativa:** Antes de llamar `line_intersection(a, b, c, d)`, verificar que las rectas no sean paralelas (`abs(cross(b - a, d - c)) > EPS`).
- **Contrato:** `segments_intersect` cubre tanto cruces propios como casos colineales donde un extremo toca al otro segmento (`on_segment`). Dependencias: `Pt`, `dot`, `cross`, `sgn` (§9.1.1).
- **Complejidad:** $\mathcal{O}(1)$ tiempo y memoria.

```
// 3. Segmentos y Rectas: Punto en segmento, Distancia e
// Interseccion
bool on_segment(const Pt& p, const Pt& a, const Pt& b) {
  return sgn(cross(a, b, p)) == 0 && sgn(dot(a - p,
    b - p)) <= 0;
}
ld dist_point_segment(const Pt& p, const Pt& a,
  const Pt& b) {
  if (pt_eq_eps(a, b)) return (p - a).norm();
  ld d2 = (b - a).norm2();
  ld t = clamp(dot(p - a, b - a) / d2, (ld)0.0, (ld)1.0);
  Pt proj = a + (b - a) * t;
  return (p - proj).norm();
}
bool segments_intersect(const Pt& a, const Pt& b,
  const Pt& c, const Pt& d) {
  int d1 = sgn(cross(a, b, c)), d2 = sgn(cross(a, b, d));
  int d3 = sgn(cross(c, d, a)), d4 = sgn(cross(c, d, b));
  if (d1 * d2 < 0 && d3 * d4 < 0) return true;
  return on_segment(c, a, b) || on_segment(d, a, b) ||
         on_segment(a, c, d) || on_segment(b, c, d);
```

```
}
// Interseccion de dos rectas infinitas (a->b) y (c->d)
// (asume no paralelas)
Pt line_intersection(const Pt& a, const Pt& b, const Pt& c,
  const Pt& d) {
  return a + (b - a) * (cross(c - a, d - c) / cross(b - a,
    d - c));
}
```

- **Trampas y adaptaciones:** (1) Para la distancia entre **dos segmentos** AB y CD: si `segments_intersect(a, b, c, d)` es `true` la distancia es `0`; si no, es el mínimo de las 4 distancias `dist_point_segment` de cada extremo al otro segmento.

9.1.3 Polígonos: Área, Centroide, Inclusión $\mathcal{O}(N)$ / Convexo $\mathcal{O}(\log N)$ y Kernel Entero (`polygon_area` , `polygon_centroid` , `point_in_polygon` , `point_in_convex_polygon` , `PtLL`)

- **Cuándo usar:** Calcular el área de un polígono simple (Shoelace), su centro de masa (`polygon_centroid`), probar si un punto está dentro/en el borde/fuera de un polígono general en $\mathcal{O}(N)$ o convexo en $\mathcal{O}(\log N)$, o calcular el doble del área en enteros exactos de 128 bits (`polygon_area2_int`).
- **Cuándo no usar / alternativa:** No usar `point_in_convex_polygon` si el polígono no es convexo o está en sentido horario (CW) o tiene vértices colineales duplicados en los bordes.
- **Contrato:** `point_in_polygon` y `point_in_convex_polygon` retornan `2` (en el **borde**), `1` (**estrictamente dentro**) o `0` (**fuera**). `polygon_area2_int` retorna $2 \cdot \text{Área} \in \mathbb{Z}$ sin usar punto flotante. Dependencias: `Pt` , `on_segment` (§9.1.1–§9.1.2).
- **Complejidad:** Área, centroide y `point_in_polygon` en $\mathcal{O}(N)$; `point_in_convex_polygon` en $\mathcal{O}(\log N)$.

```
// 4. Poligonos: Area (Shoelace), Centroide y Punto en
// Poligono O(N)
ld polygon_area(const vector<Pt>& p) {
  ld a = 0;
  for (int i = 0; i < sz(p); ++i) a += cross(p[i], p[(
    i + 1) % sz(p)]);
  // Sin dividir entre 2 es siempre entero en coords enteras
  return abs(a) / 2.0;
}
// Retorna: 2 si esta en el BORDE, 1 si esta ESTRICTAMENTE
// DENTRO, 0 si FUERA
int point_in_polygon(const Pt& q, const vector<Pt>& p) {
  int n = sz(p), wn = 0;
  for (int i = 0; i < n; ++i) {
    Pt a = p[i], b = p[(i + 1) % n];
    if (on_segment(q, a, b)) return 2;
    if (a.y <= q.y) {
      if (b.y > q.y && cross(a, b, q) > EPS) wn++;
    } else {
      if (b.y <= q.y && cross(a, b, q) < -EPS) wn--;
    }
  }
  return wn != 0 ? 1 : 0;
}

// Centroide de un poligono simple de area no nula en O(N):
Pt polygon_centroid(const vector<Pt>& p) {
  Pt c(0, 0);
  ld a2 = 0;
  for (int i = 0; i < sz(p); ++i) {
    ld cr = cross(p[i], p[(i + 1) % sz(p)]);
    c = c + (p[i] + p[(i + 1) % sz(p)]) * cr;
    a2 += cr;
  }
  return c / (3.0L * a2);
}

// Punto en Poligono Convexo CCW en O(log N) (sin puntos
// colineales redundantes):
// Retorna: 2 en BORDE, 1 ESTRICTAMENTE DENTRO, 0 FUERA.
int point_in_convex_polygon(const Pt& q,
  const vector<Pt>& p) {
  int n = sz(p);
  if (n < 3) return 0;
  if (cross(p[0], p[1], q) < -EPS || cross(p[0], p[n - 1],
    q) > EPS) return 0;
  if (on_segment(q, p[0], p[1]) || on_segment(q, p[0],
    p[n - 1])) return 2;
  int l = 1, r = n - 2, idx = 1;
  while (l <= r) {
    int mid = (l + r) / 2;
    if (cross(p[0], p[mid], q) >= -EPS) { idx = mid;
      l = mid + 1; }
    else r = mid - 1;
  }
  ld c = cross(p[idx], p[idx + 1], q);
  if (c < -EPS) return 0;
  if (on_segment(q, p[idx], p[idx + 1])) return 2;
  return 1;
}

// Kernel de Puntos Enteros con cross y doble area en i128
// (sin error de punto flotante):
struct PtLL {
  ll x, y;
  PtLL(ll x = 0, ll y = 0) : x(x), y(y) {}
  PtLL operator+(const PtLL& o) const { return {
    x + o.x, y + o.y}; }
  PtLL operator-(const PtLL& o) const { return {
    x - o.x, y - o.y}; }
  bool operator<(const PtLL& o) const {
    return x != o.x ? x < o.x : y < o.y; }
  bool operator==(const PtLL& o) const {
    return x == o.x && y == o.y; }
};
inline __int128 cross128(const PtLL& a, const PtLL& b) {
  return (__int128)a.x * b.y - (__int128)a.y * b.x;
}
inline __int128 cross128(const PtLL& a, const PtLL& b,
  const PtLL& c) {
  return cross128(b - a, c - a);
}
inline __int128 polygon_area2_int(const vector<PtLL>& p) {
  __int128 a2 = 0;
  for (int i = 0; i < sz(p); ++i) a2 += cross128(p[i], p[(
    i + 1) % sz(p)]);
  return a2 < 0 ? -a2 : a2;
}
```

- **Trampas y adaptaciones:** (1) Por el **Teorema de Pick**, en un polígono con vértices enteros, $2 \cdot \text{Área} = 2I + B - 2$, donde $B = \sum \gcd(|\Delta x_i|, |\Delta y_i|)$ son los puntos enteros en el borde e I los puntos estrictamente interiores.

9.1.4 Envolvente Convexa de Andrew (`convex_hull`)

- **Cuándo usar:** Obtener el menor polígono convexo (en sentido antihorario CCW) que contiene a un conjunto de N puntos en el plano.
- **Cuándo no usar / alternativa:** Si los puntos se insertan online y solo son gráficas de rectas $y = kx + m$, usar `LineContainer` (§8.4.1).
- **Contrato:** Elimina puntos duplicados y descarta puntos colineales intermedios en los lados (`cross(...) <= 0`). Retorna los vértices del casco en orden CCW sin repetir el primer vértice al final. Dependencias: `Pt` , `cross` (§9.1.1).
- **Complejidad:** $\mathcal{O}(N \log N)$ tiempo y $\mathcal{O}(N)$ memoria.

```
// 5. Convex Hull (Monotone Chain de Andrew en O(N log N) —
// orden CCW)
vector<Pt> convex_hull(vector<Pt> pts) {
  int n = sz(pts), k = 0;
  if (n <= 1) return pts;
  sort(all(pts));
  pts.erase(unique(all(pts), pt_eq_eps), pts.end());
  n = sz(pts);
  if (n <= 2) return pts;
  vector<Pt> h(2 * n);
  for (int i = 0; i < n; ++i) {
    while (k >= 2 && cross(h[k - 2], h[k - 1],
      pts[i]) <= EPS) k--;
    h[k++] = pts[i];
  }
  for (int i = n - 2, t = k + 1; i >= 0; --i) {
    while (k >= t && cross(h[k - 2], h[k - 1],
      pts[i]) <= EPS) k--;
    h[k++] = pts[i];
  }
  h.resize(k - 1);
  return h;
}
```

- **Trampas y adaptaciones:** (1) Si el problema exige **conservar** los puntos colineales en el borde del Convex Hull, cambiar `<` (`<= 0` pasa a `< 0`) en la condición del `while` y cuidar de no duplicar los extremos.

9.1.5 Diámetro de Polígono Convexo por Rotating Callipers (`rotating_callipers_max_dist2`)

- **Cuándo usar:** Encontrar la mayor distancia euclidiana al cuadrado entre dos puntos de una nube de N puntos (pasándole su `convex_hull` CCW) en $\mathcal{O}(N)$.
- **Cuándo no usar / alternativa:** Para la **menor** distancia entre dos puntos, el par más cercano no tiene por qué estar en el Convex Hull; usar `closest_pair_distance` (§9.1.6).
- **Contrato:** Recibe un polígono convexo CCW `h` (resultado de `convex_hull`). Avanza el puntero antipodal `j` mientras el área del triángulo con la arista `i -> i+1` crezca. Dependencias: `Pt` , `cross` (§9.1.1).
- **Complejidad:** $\mathcal{O}(N)$ sobre el casco convexo ($\mathcal{O}(N \log N)$ incluyendo `convex_hull`).

```
// 6. Rotating Callipers: Mayor distancia al cuadrado en un
// poligono convexo CCW
ld rotating_callipers_max_dist2(const vector<Pt>& h) {
  int n = sz(h);
  if (n <= 1) return 0;
  if (n == 2) return (h[0] - h[1]).norm2();
  ld max_d2 = 0;
  for (int i = 0, j = 1; i < n; ++i) {
    Pt edge = h[(i + 1) % n] - h[i];
    while (cross(edge, h[(j + 1) % n] - h[i]) > cross(edge,
      h[j] - h[i]))
      j = (j + 1) % n;
    max_d2 = max({max_d2, (h[i] - h[j]).norm2(),
                  (h[(i + 1) % n] - h[j]).norm2()});
  }
  return max_d2;
}
```

- **Trampas y adaptaciones:** (1) Siempre pasar a `rotating_callipers_max_dist2` el polígono **sin puntos colineales redundantes** generado por `convex_hull` .

9.1.6 Par de Puntos Más Cercanos por Sweep-Line (`closest_pair_distance`)

- **Cuándo usar:** Hallar la mínima distancia euclidiana entre dos puntos distintos (o `0` si hay puntos duplicados) de un conjunto de N puntos en $\mathcal{O}(N \log N)$.
- **Cuándo no usar / alternativa:** Si las distancias son Manhattan ($|x_1 - x_2| + |y_1 - y_2|$) y quieres el **máximo**, basta $\max(\max(x+y) - \min(x+y), \max(x-y) - \min(x-y))$ en $\mathcal{O}(N)$.
- **Contrato:** Si `sz(pts) < 2` retorna `INF` . Detecta puntos idénticos antes de iniciar el barrido y retorna `0` sin producir división por cero ni comparación inválida en `std::set` . Dependencias: `Pt` (§9.1.1).
- **Complejidad:** $\mathcal{O}(N \log N)$ tiempo y $\mathcal{O}(N)$ memoria.

```
// 7. Par de Puntos Mas Cercanos (Sweep-Line con std::set en
// O(N log N))
// Precondicion: sz(pts) >= 2 (retorna INF si hay menos de 2
// puntos).
ld closest_pair_distance(vector<Pt> pts) {
  if (sz(pts) < 2) return INF;
  sort(all(pts));
  for (int i = 1; i < sz(pts); ++i)
    if (pts[i] == pts[i - 1]) return 0;
  set<pair<ld, ld>> active; // Guarda {y, x}
  ld best = INF;
  int j = 0;
  for (int i = 0; i < sz(pts); ++i) {
    while (j < i && pts[i].x - pts[j].x >= best) {
      active.erase({pts[j].y, pts[j].x});
      j++;
    }
    auto it1 = active.lower_bound({pts[i].y - best, -INF});
    auto it2 = active.upper_bound({pts[i].y + best, INF});
    for (auto it = it1; it != it2; ++it) {
      ld dx = pts[i].x - it->second,
         dy = pts[i].y - it->first;
      best = min(best, hypot(dx, dy));
    }
    if (best == 0) return 0;
    active.insert({pts[i].y, pts[i].x});
  }
  return best;
}
```

- **Trampas y adaptaciones:** (1) El `std::set` activo durante el barrido ordena por `(y, x)` (no por `(x, y)`) para poder acotar el rango vertical $[y - d, y + d]$ con `lower_bound` .

9.2 Geometría Analítica Básica y Círculos: Proyección, Reflexión, Distancia Punto-Recta y `Circle`

9.2.1 Proyección, Reflexión y Distancia Punto-Recta (`proj_point_line` , `reflect_point_line` , `dist_point_line`)

- **Cuándo usar:** Proyectar ortogonalmente un punto P sobre una recta infinita AB, reflejar P respecto al espejo AB, o medir la distancia perpendicular de P a la recta infinita AB.
- **Cuándo no usar / alternativa:** Si el espejo es un **segmento acotado** AB y buscas distancia, usar `dist_point_segment` (§9.1.2).
- **Contrato:** Maneja el caso degenerado $A = B$ retornando A (o $|P - A|$ en distancia). Dependencias: `Pt` , `dot` , `cross` (§9.1.1).
- **Complejidad:** $\mathcal{O}(1)$ tiempo y memoria.

```cpp
// 1. Proyeccion, Reflexion y Distancia Punto-Recta infinita
// (a -> b)
Pt proj_point_line(const Pt& p, const Pt& a, const Pt& b) {
  if (pt_eq_eps(a, b)) return a;
  Pt v = b - a;
  return a + v * (dot(p - a, v) / v.norm2());
}
Pt reflect_point_line(const Pt& p, const Pt& a,
  const Pt& b) {
  Pt pr = proj_point_line(p, a, b);
  return pr * 2.0L - p;
}
ld dist_point_line(const Pt& p, const Pt& a, const Pt& b) {
  if (pt_eq_eps(a, b)) return (p - a).norm();
  return abs(cross(a, b, p)) / (b - a).norm();
}
```

- **Trampas y adaptaciones:** (1) El camino más corto desde S hasta T tocando una recta AB tiene longitud $|\text{reflect}(S, A, B) - T|$.

9.2.2 Geometría de Círculos: Intersecciones y Tangentes (`Circle`, `circle_line_intersection`, `circle_circle_intersection`, `circle_tangents_from_point`)

- **Cuándo usar:** Hallar los puntos de corte entre una recta y un círculo, entre dos círculos, o los puntos de tangencia trazados a un círculo desde un punto exterior P.
- **Cuándo no usar / alternativa:** Siempre distinguir el código `-1` de `circle_circle_intersection` (círculos idénticos con infinitos puntos de contacto) del código `0` (disjuntos).
- **Contrato:** Usa `sqrt(max((ld)0.0, h2))` para evitar `NaN` cuando la recta o el círculo son casi tangentes ($h^2 \approx -10^{-16}$). Dependencias: `proj_point_line` (§9.2.1), `Pt`, `pt_eq_eps` (§9.1.1).
- **Complejidad:** $\mathcal{O}(1)$ tiempo y memoria.

```cpp
// 2. Circulos: Interseccion Recta-Circulo, Circulo-Circulo
// y Tangentes
struct Circle { Pt c; ld r; };

vector<Pt> circle_line_intersection(const Circle& C,
                                    const Pt& a,
    const Pt& b) {
  if (pt_eq_eps(a, b)) return {};
  Pt pr = proj_point_line(C.c, a, b);
  ld d2 = (pr - C.c).norm2(), r2 = C.r * C.r;
  if (d2 > r2 + EPS) return {};
  ld h = sqrt(max((ld)0.0, r2 - d2));
  Pt dir = (b - a) / (b - a).norm();
  if (h <= EPS) return {pr};
  return {pr - dir * h, pr + dir * h};
}

// Retorna {estado, puntos}: -1 coincidentes, 0 disjuntos, 1
// tangente, 2 secantes
pair<int, vector<Pt>> circle_circle_intersection(
    const Circle& A, const Circle& B) {
  Pt v = B.c - A.c;
  ld d = v.norm();
  if (d <= EPS)
    return (abs(A.r - B.r) <= EPS)
             ? make_pair(-1, vector<Pt>{})
             : make_pair(0, vector<Pt>{});
  if (d > A.r + B.r + EPS ||
      d + min(A.r, B.r) < max(A.r, B.r) - EPS)
    return {0, {}};
  ld x = (d * d + A.r * A.r - B.r * B.r) / (2.0L * d);
  ld h2 = A.r * A.r - x * x;
  Pt base = A.c + v * (x / d);
  if (h2 <= EPS) return {1, {base}};
  ld h = sqrt(max((ld)0.0, h2));
  Pt nrm = v.perp() * (h / d);
  return {2, {base - nrm, base + nrm}};
}

// Puntos de contacto de las tangentes a C trazadas desde un
// punto exterior P
vector<Pt> circle_tangents_from_point(const Circle& C,
  const Pt& p) {
  Pt v = p - C.c;
  ld d2 = v.norm2(), r2 = C.r * C.r;
  // Punto estrictamente interior
  if (d2 < r2 - EPS) return {};
  if (abs(d2 - r2) <= EPS) return {p};
  ld x = r2 / d2, h = sqrt(max((ld)0.0,
    d2 - r2)) * C.r / d2;
  Pt base = C.c + v * x, nrm = v.perp() * h;
  return {base - nrm, base + nrm};
}
```

- **Trampas y adaptaciones:** (1) En problemas de cobertura angular sobre un círculo (ej. cuántos círculos cubren el borde de C_i), convertir los dos puntos de `circle_circle_intersection` en ángulos con `atan2` y hacer sweep-line (§3.7.1).

9.3 Intersección de Semiplanos (Halfplane Intersection)

- **Cuándo usar:** Construir el polígono convexo que resulta de intersectar N semiplanos dirigidos (región a la izquierda del vector $A \to B$), como el núcleo de visibilidad de un polígono (*Art Gallery Kernel*) o factibilidad de programación lineal en 2D.
- **Cuándo no usar / alternativa:** Si solo necesitas saber el valor óptimo lineal en 2D o más dimensiones sin los vértices del polígono, `Simplex` (§7.8) también lo resuelve.
- **Contrato:** Cada `Halfplane(a, b)` representa el semiplano a la **izquierda** del rayo $a \to b$ (`cross(b - a, q - a) >= 0`). Acota regiones abiertas con una caja `[-box, box]^2`. Si la intersección es vacía o degenerada (segmento/punto de área 0), retorna `{}`. Dependencias: `Pt`, `cross` (§9.1.1).
- **Complejidad:** $\mathcal{O}(N \log N)$ tiempo y $\mathcal{O}(N)$ memoria.

```cpp
// Semiplanos a la izquierda del rayo p -> p + v
struct Halfplane {
  Pt p, v;
  ld angle;
  Halfplane() {}
  Halfplane(Pt a, Pt b) : p(a), v(b - a), angle(atan2(v.y,
    v.x)) {}
  bool out(const Pt& q) const {
    return cross(v, q - p) < -EPS; }
  bool operator<(const Halfplane& o) const {
    if (abs(angle - o.angle) > EPS) return angle < o.angle;
    // Mas restrictivo primero si son paralelos
    return cross(v, o.p - p) < 0;
  }
};

Pt halfplane_intersect(const Halfplane& a,
  const Halfplane& b) {
  ld c = cross(a.v, b.v);
  return a.p + a.v * (cross(b.p - a.p, b.v) / c);
}

// Retorna poligono convexo CCW (vacio si la interseccion es
// vacia o degenerada).
// Si box > 0, acota regiones abiertas al cuadrado [-box,
// box]^2.
vector<Pt> halfplane_intersection(vector<Halfplane> h,
  ld box = 1e9) {
  if (box > 0) {
    h.push_back({{-box, -box}, {box, -box}});
    h.push_back({{box, -box}, {box, box}});
    h.push_back({{box, box}, {-box, box}});
    h.push_back({{-box, box}, {-box, -box}});
  }
  sort(all(h));
  deque<Halfplane> dq;
  for (const auto& cur : h) {
    if (!dq.empty() && abs(cur.angle - dq.back(
      ).angle) <= EPS) continue;
    while (sz(dq) >= 2 && cur.out(halfplane_intersect(dq[sz(
      dq) - 2], dq.back())))
      dq.pop_back();
    while (sz(dq) >= 2 && cur.out(halfplane_intersect(dq[0],
      dq[1])))
      dq.pop_front();
    if (!dq.empty() && abs(cross(cur.v, dq.back(
      ).v)) <= EPS) {
      if (dot(cur.v, dq.back().v) < 0 && cur.out(dq.back(
        ).p)) return {};
    }
    dq.push_back(cur);
  }
  while (sz(dq) >= 3 && dq[0].out(halfplane_intersect(dq[sz(
    dq) - 2], dq.back())))
    dq.pop_back();
  while (sz(dq) >= 3 && dq.back().out(halfplane_intersect(
    dq[0], dq[1])))
    dq.pop_front();
  if (sz(dq) < 3) return {};

  vector<Pt> poly;
  for (int i = 0; i < sz(dq); ++i) {
    const auto& a = dq[i];
    const auto& b = dq[(i + 1) % sz(dq)];
    if (abs(cross(a.v, b.v)) <= EPS) return {};
    poly.push_back(halfplane_intersect(a, b));
  }
  return convex_hull(poly);
}
```

- **Trampas y adaptaciones:** (1) Para el núcleo de un polígono dado en orden CCW, agregar un `Halfplane(p[i], p[(i + 1) % n])` por cada lado del polígono.

9.4 Suma de Minkowski de Polígonos Convexos

- **Cuándo usar:** Construir en $\mathcal{O}(N + M)$ el polígono convexo $P + Q = \{p + q : p \in P, q \in Q\}$, útil para colisión de polígonos convexos (P intersecta a $Q + v \Leftrightarrow v \in P + (-Q)$) o máximo de dos funciones convexas.
- **Cuándo no usar / alternativa:** Ambos polígonos de entrada deben estar en orden **antihorario (CCW)**.
- **Contrato:** Soporta casos degenerados donde P o Q tienen 1 o 2 puntos. Reordena ambos polígonos empezando por el vértice de menor (y, x) y fusiona sus vectores de aristas por ángulo. Dependencias: `Pt`, `cross` (§9.1.1).
- **Complejidad:** $\mathcal{O}(|P| + |Q|)$ tiempo y memoria.

```cpp
// Suma de Minkowski P + Q para dos conjuntos convexos CCW
// (admite punto o segmento)
vector<Pt> minkowski_sum(vector<Pt> P, vector<Pt> Q) {
  if (P.empty() || Q.empty()) return {};
  if (sz(P) == 1) {
    for (auto& q : Q) q = q + P[0];
    return Q;
  }
  if (sz(Q) == 1) {
    for (auto& p : P) p = p + Q[0];
    return P;
  }
  auto reorder = [](vector<Pt>& pts) {
    int pos = 0;
    for (int i = 1; i < sz(pts); ++i)
      if (pts[i].y < pts[pos].y || (
        pts[i].y == pts[pos].y && pts[i].x < pts[pos].x))
        pos = i;
    rotate(pts.begin(), pts.begin() + pos, pts.end());
  };
  reorder(P); reorder(Q);
  int n = sz(P), m = sz(Q);
  P.push_back(P[0]); P.push_back(P[1]);
  Q.push_back(Q[0]); Q.push_back(Q[1]);

  vector<Pt> res;
  int i = 0, j = 0;
  while (i < n || j < m) {
    res.push_back(P[i] + Q[j]);
    ld c = cross(P[i + 1] - P[i], Q[j + 1] - Q[j]);
    bool adv_i = (i < n) && (j == m || c >= -EPS);
    bool adv_j = (j < m) && (i == n || c <= EPS);
    if (adv_i) i++;
    if (adv_j) j++;
  }
  return res;
}
```

- **Trampas y adaptaciones:** (1) La distancia mínima entre dos polígonos convexos disjuntos P y Q es la distancia del origen $(0, 0)$ al polígono $P + (-Q)$.

Bloque 10: Búsqueda en Mitades (MITM), Parsing y Clique Máximo

10.1 Meet in the Middle (MITM): Subset Sum Exacto, Conteo con Multiplicidades y Máximo $\leq S$

10.1.1 Meet in the Middle: Conteo de Subconjuntos con Suma Exacta (`get_subset_sums`, `mitm_subset_sum_count`)

- **Cuándo usar:** Contar cuántos de los 2^N subconjuntos suman exactamente `target` cuando $N \in [25, 42]$ y los valores $a_i \leq 10^{12}$ no permiten DP por suma.
- **Cuándo no usar / alternativa:** Si $N \leq 20$, basta una sola mitad; si $\sum a_i \leq 10^6$, usar Knapsack DP (§8.1.1).
- **Contrato:** Usa `inplace_merge` para generar las $2^{N/2}$ sumas de cada mitad **ya ordenadas en** $\mathcal{O}(2^{N/2})$ sin pagar el factor $\log(2^{N/2})$ de `std::sort`. Cuenta multiplicidades con `equal_range`.
- **Complejidad:** $\mathcal{O}(2^{N/2} \cdot (N/2))$ tiempo y $\mathcal{O}(2^{N/2})$ memoria.

```cpp
// Genera las 2^k sumas de subconjuntos de un rango [l, r)
// en O(2^k)
```

```cpp
vll get_subset_sums(const vll& a, int l, int r) {
  vll sums = {0};
  for (int i = l; i < r; ++i) {
    int cur_sz = sz(sums);
    sums.resize(2 * cur_sz);
    for (int j = 0; j < cur_sz; ++j)
      sums[cur_sz + j] = sums[j] + a[i];
    inplace_merge(sums.begin(), sums.begin() + cur_sz,
      sums.end());
  }
  return sums; // Ya queda ordenado en O(2^(r-l))!
}

// 1. Cuenta cuantos subconjuntos (con multiplicidad) suman
// 'target'
ll mitm_subset_sum_count(const vll& a, ll target) {
  int n = sz(a), mid = n / 2;
  vll L = get_subset_sums(a, 0, mid);
  vll R = get_subset_sums(a, mid, n);
  ll ways = 0;
  for (ll x : L) {
    auto [it1, it2] = equal_range(all(R), target - x);
    ways += (it2 - it1);
  }
  return ways;
}
```

- **Trampas y adaptaciones:** (1) Los subconjuntos se distinguen por sus **índices** (si `a = {5, 5}` y `target = 5`, cuenta 2 subconjuntos).

10.1.2 Meet in the Middle: Mayor Suma de Subconjunto $\leq S$ (`mitm_max_subset_sum_le`)

- **Cuándo usar:** Encontrar la mayor suma de un subconjunto que no exceda `limit` ($N \leq 42, \text{limit} \leq 10^{18}$) combinando las dos mitades ordenadas con dos punteros en $\mathcal{O}(2^{N/2})$.
- **Cuándo no usar / alternativa:** Asume `limit >= 0` (pues el subconjunto vacío suma `0`).
- **Contrato:** Genera `L` y `R` ordenados con `get_subset_sums` y barre `x` creciente en `L` contra `j` decreciente en `R` en tiempo lineal sobre las listas. Dependencias: `get_subset_sums` (§10.1.1).
- **Complejidad:** $\mathcal{O}(2^{N/2})$ tiempo y memoria.

```cpp
// 2. Mayor suma de subconjunto <= limit (usando two
// pointers en O(2^(N/2)))
ll mitm_max_subset_sum_le(const vll& a, ll limit) {
  int n = sz(a), mid = n / 2;
  vll L = get_subset_sums(a, 0, mid);
  vll R = get_subset_sums(a, mid, n);
  ll best = -INF;
  int j = sz(R) - 1;
  for (ll x : L) {
    while (j >= 0 && x + R[j] > limit) --j;
    if (j >= 0) best = max(best, x + R[j]);
  }
  return best;
}
```

- **Trampas y adaptaciones:** (1) Gracias a `inplace_merge` + dos punteros, este algoritmo corre en $\mathcal{O}(2^{N/2})$ puro sin ningún factor logarítmico.

10.2 Evaluador de Expresiones Aritméticas (Shunting-Yard Parser)

- **Cuándo usar:** Evaluar en $\mathcal{O}(|S|)$ una cadena con números enteros, paréntesis `()`, operadores binarios `+ - * /` y signos unarios `+x` y `-x` respetando precedencia y asociatividad.
- **Cuándo no usar / alternativa:** Si el problema permite usar Python y la expresión no excede límites de seguridad ni tiene división `/` con semántica hacia cero para negativos, ver §12.15.
- **Contrato:** Usa el centinela `UNARY_NEG = '~'` (compatible con `-funsigned-char`) con prioridad `3` y asociatividad a derecha (`- - 5 == 5`).
- **Complejidad:** $\mathcal{O}(|S|)$ tiempo y memoria.

```cpp
struct ExpressionParser {
  static constexpr char UNARY_NEG = '~';
  int priority(char op) const {
    // Menos unario (-x), asociativo a derecha
    if (op == UNARY_NEG) return 3;
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return -1;
  }
  void apply_op(stack<ll>& st, char op) const {
    if (op == UNARY_NEG) {
      ll x = st.top(); st.pop();
      st.push(-x);
    } else {
      ll b = st.top(); st.pop();
      ll a = st.top(); st.pop();
      if (op == '+') st.push(a + b);
      if (op == '-') st.push(a - b);
      if (op == '*') st.push(a * b);
      if (op == '/') st.push(a / b);
    }
  }
  ll evaluate(const string& s) const {
    stack<ll> st;
    stack<char> op;
    bool may_be_unary = true;
    for (int i = 0; i < sz(s); ++i) {
      if (isspace((unsigned char)s[i])) continue;
      if (s[i] == '(') {
        op.push('('); may_be_unary = true;
      } else if (s[i] == ')') {
        while (op.top() != '(') {
          apply_op(st, op.top()); op.pop();
        }
        op.pop(); may_be_unary = false;
      } else if (
        s[i] == '+' || s[i] == '-' || s[i] == '*' || s[i]
          == '/') {
        char cur_op = s[i];
        if (may_be_unary && (
          cur_op == '+' || cur_op == '-')) {
          if (cur_op == '-') cur_op = UNARY_NEG;
          else continue;
        }
        while (!op.empty() && (
          (cur_op != UNARY_NEG && priority(op.top(
            )) >= priority(cur_op)) ||
          (cur_op == UNARY_NEG && priority(op.top(
            )) > priority(cur_op)))) {
          apply_op(st, op.top()); op.pop();
        }
        op.push(cur_op); may_be_unary = true;
      } else {
        ll val = 0;
        while (i < sz(s) && isdigit(s[i]))
          val = val * 10 + (s[i++] - '0');
        --i;
        st.push(val); may_be_unary = false;
      }
    }
    while (!op.empty()) {
      apply_op(st, op.top()); op.pop();
    }
    return st.top();
  }
};
```

- **Trampas y adaptaciones:** (1) Para evaluar expresiones modulares, reemplazar `a / b` en `apply_op` por `mul_mod(a, inv_fermat(b))` (§7.4.1).

10.3 Clique Máximo (Bron-Kerbosch Branch & Bound)

- **Cuándo usar:** Hallar el mayor subconjunto de vértices mutuamente adyacentes (**Maximum Clique**) o el mayor conjunto independiente (**Maximum Independent Set**, pasándole el grafo complemento) para $N \leq 60$.
- **Cuándo no usar / alternativa:** Si el grafo es bipartito, el Maximum Independent Set se halla en tiempo polinomial con König (`BipartiteMVC`, §6.6.3).
- **Contrato:** Nodos **0-indexed** (`0..n-1`) con $N \leq 64$ representados en máscaras `ull`. Usa pivoteo de Tomita (`u = __builtin_ctzll(P | X)`) y poda por cota superior (`sz(cur) + popcount(P) <= sz(best_clique)`).
- **Complejidad:** $\mathcal{O}(3^{N/3})$ peor caso teórico (extremadamente rápido con poda en grafos de competencia hasta $N \approx 55$).

```cpp
// Max Clique para N <= 64 usando mascaras ull
struct MaxClique {
  int n;
  vector<ull> adj;
  vi best_clique;

  MaxClique(int n) : n(n), adj(n, 0) {}

  void add_edge(int u, int v) {
    adj[u] |= (1ULL << v); adj[v] |= (1ULL << u);
  }
  void expand(ull R, ull P, ull X, vi& cur) {
    if (P == 0 && X == 0) {
      if (sz(cur) > sz(best_clique)) best_clique = cur;
      return;
    }
    if (sz(cur) + __builtin_popcountll(P) <= sz(
      best_clique)) return;
    int u = __builtin_ctzll(P | X);
    ull cand = P & ~adj[u];
    while (cand) {
      int v = __builtin_ctzll(cand);
      cand &= ~(1ULL << v);
      cur.push_back(v);
      expand(R | (1ULL << v), P & adj[v], X & adj[v], cur);
      cur.pop_back();
      P &= ~(1ULL << v);
      X |= (1ULL << v);
    }
  }
  vi solve() {
    best_clique.clear();
    ull P = (n == 64 ? ~0ULL : (1ULL << n) - 1);
    vi cur;
    expand(0, P, 0, cur);
    return best_clique;
  }
};
```

- **Trampas y adaptaciones:** (1) El **Maximum Independent Set** de G es exactamente el `MaxClique` del grafo complemento $\overline{G}$ (agregar arista (u, v) si y solo si $u \neq v$ y $(u, v) \notin E(G)$).

Bloque 11: Compendio Teórico, Teoremas, Fórmulas e Identidades

11.1 Condiciones de Optimización de Programación Dinámica (DP Speedups)

- **Tabla autoritativa de recurrencias y complejidades**: ver **§8.4**.
- **Desigualdad del Cuadrilátero**: $C(a,c) + C(b,d) \leq C(a,d) + C(b,c)$ para todo $a \leq b \leq c \leq d$. Es condición suficiente para que $opt[k][i] \leq opt[k][i+1]$ en Divide & Conquer DP. Si además $C(b,c) \leq C(a,d)$ (monotonicidad en inclusión), garantiza $opt[l][r-1] \leq opt[l][r] \leq opt[l+1][r]$ en Knuth DP.
- **CHT ($\mathcal{O}(N)$ vs $\mathcal{O}(N \log N)$)**: Lograr $\mathcal{O}(N)$ con dos punteros exige **simultáneamente** que las pendientes m_j se inserten en orden monótono y que las abscisas de consulta x_i sean monótonas. Si solo m_j es monótona pero x_i es arbitraria, se requiere búsqueda binaria sobre la envolvente en $\mathcal{O}(N \log N)$; si las pendientes tampoco son monótonas, usar `LineContainer` ($\mathcal{O}(N \log N)$).
- **WQS / Aliens Trick**: Para **minimizar** $f(k)$ (costo mínimo usando exactamente k transiciones/grupos), se requiere que $f(k)$ sea **convexa** ($f(k+1) - f(k) \geq f(k) - f(k-1)$); para **maximizar**, que $f(k)$ sea **cóncava** ($f(k+1) - f(k) \leq f(k) - f(k-1)$). Al penalizar cada grupo con λ, si hay tramos colineales en $f(k)$, desempatar siempre tomando la menor (o mayor) cantidad de grupos para reconstruir K.

11.2 Combinatoria (Expresada en Recurrencias, Sumatorias $\sum$ y Productorias $\prod$)

- **Coeficiente Binomial $\binom{n}{r}$ y Permutaciones $P(n,r)$**:
  - **Productoria directa $\mathcal{O}(r)$**: $\binom{n}{r} = \prod_{i=1}^{r} \frac{n-r+i}{i}$, $P(n,r) = \prod_{i=0}^{r-1}(n-i)$.
  - **Recurrencia multiplicativa (1 for-loop)**: $C_{n,0} = 1$, $C_{n,i} = C_{n,i-1} \cdot \frac{n-i+1}{i}$ $(i = 1..r)$.
  - **Recurrencia aditiva de Pascal (DP $\mathcal{O}(n^2)$ sin dividir, cualquier módulo)**: $C_{n,0} = C_{n,n} = 1$, $C_{n,r} = C_{n-1,r-1} + C_{n-1,r}$.
- **Coeficiente Multinomial ($\sum_{i=1}^{m} k_i = n$)**:
  - **Productoria acumulativa de binomiales**: $\binom{n}{k_1,k_2,\ldots,k_m} = \prod_{i=1}^{m} \binom{\sum_{j=1}^{i} k_j}{k_i} = \prod_{i=1}^{m} \binom{n-\sum_{j<i} k_j}{k_i}$.
- **Stars and Bars ($\sum_{i=1}^{k} x_i = n$)**:
  - **Sin cota superior**: Con $x_i \geq 0$: $\binom{n+k-1}{k-1} = \prod_{i=1}^{k-1} \frac{n+i}{i}$. Con $x_i \geq 1$: $\binom{n-1}{k-1} = \prod_{i=1}^{k-1} \frac{n-k+i}{i}$.
  - **Con cota superior $0 \leq x_i \leq c$ (Sumatoria)**: $\sum_{j=0}^{\lfloor n/(c+1) \rfloor} (-1)^j \binom{k}{j} \binom{n-j(c+1)+k-1}{k-1}$.
- **Identidades de Sumatorias con Binomiales**:

- **Binomio / Subconjuntos**: $\sum_{k=0}^{n}\binom{n}{k}a^{n-k}b^k=(a+b)^n$; $\sum_{k=0}^{n}\binom{n}{k}=2^n$; $\sum_{k=0}^{n}k\binom{n}{k}=n2^{n-1}$.
  - **Palo de Hockey (Hockey-Stick)**: $\sum_{i=r}^{n}\binom{i}{r}=\binom{n+1}{r+1}$.
  - **Vandermonde (Convolución)**: $\sum_{k=0}^{r}\binom{m}{k}\binom{n}{r-k}=\binom{m+n}{r}$.
- **Sumas Polinomiales y Geométricas ($S_n = S_{n-1} + f(n)$)**:
  - $\sum_{k=1}^{n}k=\frac{n(n+1)}{2}$; $\sum_{k=1}^{n}k^2=\frac{n(n+1)(2n+1)}{6}$; $\sum_{k=1}^{n}k^3=\left(\frac{n(n+1)}{2}\right)^2$.
  - $\sum_{k=1}^{n}k^4=\frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$.
  - $\sum_{k=0}^{n}x^k=\frac{x^{n+1}-1}{x-1}$ $(x\neq 1)$; $\sum_{k=1}^{n}kx^k=\frac{x-(n+1)x^{n+1}+nx^{n+2}}{(1-x)^2}$.
- **Números de Catalan (C_n)**:
  - **Recurrencia multiplicativa $\mathcal{O}(n)$**: $C_0=1$, $C_n=C_{n-1}\cdot\frac{2(2n-1)}{n+1}=\frac{1}{n+1}\prod_{i=1}^{n}\frac{n+i}{i}$.
  - **Recurrencia en sumatoria (DP $\mathcal{O}(n^2)$ sin dividir)**: $C_0=1$, $C_n=\sum_{i=0}^{n-1}C_i\cdot C_{n-1-i}$.
  - **Forma cerrada**: $C_n=\frac{1}{n+1}\binom{2n}{n}=\binom{2n}{n}-\binom{2n}{n+1}$ $(1,1,2,5,14,42,132,429,1430\ldots)$.
  - Aplicaciones: Paréntesis balanceados (n pares); triangulaciones de polígono ($n+2$ lados); árboles binarios (n nodos); caminos de Dyck de $(0,0)$ a $(2n,0)$.
- **Desarreglos (Derangements D_n — permutaciones sin puntos fijos)**:
  - **Recurrencia de 1 paso $\mathcal{O}(n)$**: $D_0=1$, $D_i=i\cdot D_{i-1}+(-1)^i$.
  - **Recurrencia aditiva de 2 pasos**: $D_0=1, D_1=0$, $D_i=(i-1)(D_{i-1}+D_{i-2})$.
  - **Sumatoria explícita**: $D_n=n!\sum_{k=0}^{n}\frac{(-1)^k}{k!}=\sum_{k=0}^{n}(-1)^k\binom{n}{k}(n-k)!$. Con k puntos fijos: $\binom{n}{k}D_{n-k}$.
- **Números de Stirling de 1ra Especie $\begin{bmatrix}n\\k\end{bmatrix}$ (permutaciones de n con k ciclos)**:
  - **Recurrencia DP $\mathcal{O}(nk)$**: $\begin{bmatrix}0\\0\end{bmatrix}=1$, $\begin{bmatrix}n\\k\end{bmatrix}=(n-1)\begin{bmatrix}n-1\\k\end{bmatrix}+\begin{bmatrix}n-1\\k-1\end{bmatrix}$.
  - **Productoria generadora**: $\sum_{k=0}^{n}\begin{bmatrix}n\\k\end{bmatrix}x^k=\prod_{i=0}^{n-1}(x+i)$.
- **Números de Stirling de 2da Especie $\begin{Bmatrix}n\\k\end{Bmatrix}$ (particionar n en k cajas no vacías)**:
  - **Recurrencia DP $\mathcal{O}(nk)$**: $\begin{Bmatrix}0\\0\end{Bmatrix}=1$, $\begin{Bmatrix}n\\k\end{Bmatrix}=k\begin{Bmatrix}n-1\\k\end{Bmatrix}+\begin{Bmatrix}n-1\\k-1\end{Bmatrix}$.
  - **Sumatoria directa $\mathcal{O}(k\log n)$**: $\begin{Bmatrix}n\\k\end{Bmatrix}=\frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$. (Sobreyectivas $n\to k$: multiplicar por $k!$).
- **Números de Bell (B_n — particiones totales de n elementos)**:
  - **Recurrencia en sumatoria**: $B_0=1$, $B_n=\sum_{k=0}^{n-1}\binom{n-1}{k}B_k=\sum_{k=0}^{n}\begin{Bmatrix}n\\k\end{Bmatrix}$.
- **Particiones de Enteros $p(n)$ (Recurrencia Pentagonal de Euler $\mathcal{O}(n\sqrt{n})$)**:
  - $p(0)=1$, $p(n)=\sum_{k\geq 1}(-1)^{k-1}\left[p\left(n-\frac{k(3k-1)}{2}\right)+p\left(n-\frac{k(3k+1)}{2}\right)\right]$.
- **Lema de Burnside y Collares (Sumatoria sobre simetrías / divisores)**:
  - $|X/G|=\frac{1}{|G|}\sum_{g\in G}|X^g|$; $\text{Collares}(n,k)=\frac{1}{n}\sum_{d|n}\phi(d)k^{n/d}$.

11.3 Teoría de Números y Aritmética Modular

- **Ecuaciones Diofánticas Lineales**:
  - $ax+by=c$ tiene solución entera si y solo si $g=\gcd(a,b)\mid c$.
  - Si (x_0,y_0) es una solución particular, todas las soluciones son: $x=x_0+k\cdot\frac{b}{g}$, $y=y_0-k\cdot\frac{a}{g}$ $(k\in\mathbb{Z})$.
- **Problema de las Monedas de Frobenius (Chicken McNuggets Theorem)**:
  - Para $\gcd(a,b)=1$, el mayor número entero que NO puede expresarse como $ax+by$ $(x,y\geq 0)$ es: $g(a,b)=a\cdot b-a-b$.
  - El número exacto de enteros positivos no representables es $\frac{(a-1)(b-1)}{2}$.
- **Teorema de Wilson**:
  - Un entero $p>1$ es primo si y solo si $(p-1)!\equiv -1\pmod p$.
- **Teorema de Euler y Reducción de Torres de Potencias**:
  - Si $\gcd(a,m)=1$, $a^{\phi(m)}\equiv 1\pmod m$.
  - En general para cualquier a: $a^b\equiv a^{b\pmod{\phi(m)}+\phi(m)}\pmod m$ si $b\geq\phi(m)$.
- **Fórmula de Inversión de Möbius**:
  - $g(n)=\sum_{d|n}f(d)\Leftrightarrow f(n)=\sum_{d|n}\mu(d)g\left(\frac{n}{d}\right)$.
  - Propiedad clave: $\sum_{d|n}\mu(d)=[n=1]$.
  - Identidad de coprimalidad: $[\gcd(i,j)=1]=\sum_{d|\gcd(i,j)}\mu(d)$.
- **Símbolo de Legendre y Criterio de Euler**:
  - $\left(\frac{a}{p}\right)\equiv a^{(p-1)/2}\pmod p\in\{1,-1,0\}$.
  - a es residuo cuadrático mod p primo impar si $\left(\frac{a}{p}\right)=1$.
  - Ley de Reciprocidad Cuadrática: $\left(\frac{p}{q}\right)\left(\frac{q}{p}\right)=(-1)^{\frac{p-1}{2}\frac{q-1}{2}}$.
- **Teorema de los Dos Cuadrados de Fermat**:
  - Un primo impar p se puede escribir como a^2+b^2 si y solo si $p\equiv 1\pmod 4$.
  - Un entero n es suma de dos cuadrados si y solo si todo factor primo $q\equiv 3\pmod 4$ aparece con exponente par en su factorización.
- **Ternas Pitagóricas Primitivas**:
  - Todo triplete $a^2+b^2=c^2$ con $\gcd(a,b,c)=1$ se genera mediante: $a=m^2-n^2$, $b=2mn$, $c=m^2+n^2$, con $m>n>0$, $\gcd(m,n)=1$, y m,n de distinta paridad.
- **Raíces Primitivas**:
  - g es raíz primitiva mod n si el orden multiplicativo de g es $\phi(n)$.
  - Existen raíces primitivas solo para $n\in\{1,2,4,p^k,2p^k\}$ con p primo impar.
  - Para verificar si g es raíz primitiva mod p: verificar que $g^{(p-1)/q}\not\equiv 1\pmod p$ para todos los factores primos q de $p-1$.
- **Período de Pisano (Fibonacci mod M)**:
  - El período $\pi(M)$ de Fibonacci módulo M siempre existe y es $\leq 6M$.
  - $\pi(10^k)=1.5\times 10^k$ para todo $k\geq 3$. $\pi(10^9)=1.5\times 10^9$.

11.4 Teoría de Grafos, Árboles y Redes

- **Teorema Matriz-Árbol de Kirchhoff (Matrix-Tree Theorem)**:
  - Sea $L=D-A$ la matriz Laplaciana del grafo ($D_{ii}=\deg(v_i)$, $A_{ij}=$ aristas entre i y j).
  - El número de árboles generadores (spanning trees) del grafo es igual al valor absoluto de cualquier cofactor de L (eliminar fila i y columna i y calcular $\det(L_{ii})$).
  - En grafos dirigidos: el número de arborescencias dirigidas enraizadas en r es el cofactor del Laplaciano dirigido $L^{\text{dir}}=D^{\text{in}}-A$ eliminando la fila y columna r.
- **Fórmula de Cayley y Bosques**:
  - Número de árboles etiquetados en n vértices: n^{n-2}.
  - Número de bosques etiquetados en n vértices con k componentes conexas donde los vértices $1,\ldots,k$ pertenecen a componentes distintas: $k\cdot n^{n-k-1}$.
- **Código de Prüfer**:
  - Biyección entre árboles etiquetados con n vértices y secuencias de longitud $n-2$ con elementos en $\{1,\ldots,n\}$.
  - Un vértice con grado d aparece exactamente $d-1$ veces en el código de Prüfer.
- **Teorema de König y Relaciones de Gallai**:
  - En grafos bipartitos: Matching Máximo = Min Vertex Cover. Max Independent Set = V − Matching Máximo.
  - Teorema de Gallai (para grafos generales sin vértices aislados): $\alpha(G)+\tau(G)=V$ (Indep Set + Vertex Cover = V). $\alpha'(G)+\tau'(G)=V$ (Matching Máximo + Edge Cover Mínimo = V).
- **Teorema de Dilworth (Anticadenas en Posets)**:
  - En cualquier conjunto parcialmente ordenado (DAG transitivo), el tamaño de la **anticadena más grande** es igual al número mínimo de cadenas disjuntas necesarias para cubrir todos los elementos.
  - Se resuelve modelando como Maximum Bipartite Matching.
- **Teorema de Tutte-Berge y Matrimonio de Hall**:
  - Condición de Hall para emparejamiento completo de X a Y en bipartito: $|N(S)|\geq|S|$ para todo subconjunto $S\subseteq X$.
  - Fórmula de Tutte-Berge: El tamaño del matching máximo en cualquier grafo es $\frac{1}{2}\min_{U\subseteq V}(|V|+|U|-\text{odd}(G-U))$.
- **Criterio Gráfico de Erdős-Gallai**:
  - Una secuencia de grados $d_1\geq d_2\geq\ldots\geq d_n$ corresponde a un grafo simple no dirigido si y solo si $\sum d_i$ es par y para todo $1\leq k\leq n$: $\sum_{i=1}^{k}d_i\leq k(k-1)+\sum_{i=k+1}^{n}\min(d_i,k)$.
- **Emparejamiento Estable (Gale-Shapley)**:
  - Si los proponentes son los hombres, el algoritmo genera el emparejamiento estable **óptimo para los hombres** y **pésimo para las mujeres** en $\mathcal{O}(N^2)$.
- **Flujo Máximo con Demandas (Cotas Inferiores $l(u,v)\leq f(u,v)\leq c(u,v)$)**:
  - Crear super-fuente S' y super-sumidero T'.
  - Para cada arista (u,v) con capacidad $[l,c]$, agregar arista en la red con capacidad $c-l$.
  - Para cada vértice u, definir balance $b(u)=\sum_v l(v,u)-\sum_w l(u,w)$.
  - Si $b(u)>0$, agregar arista (S',u) con capacidad $b(u)$.
  - Si $b(u)<0$, agregar arista (u,T') con capacidad $-b(u)$.
  - Agregar arista infinita (T,S) con capacidad ∞. Si el flujo máximo desde S' satura todas las aristas que salen de S', existe circulación factible.
- **Fórmula de Euler para Grafos Planares**:
  - $V-E+F=1+C$, donde C es el número de componentes conexas ($V-E+F=2$ si es conexo).
  - Corolarios para grafos simples conexos con $V\geq 3$: $E\leq 3V-6$ (si no tiene triángulos, $E\leq 2V-4$). Siempre existe al menos un vértice con grado ≤ 5.

11.5 Geometría y Distancias

- **Teorema de Pick**:
  - Para un polígono simple con vértices en coordenadas enteras (retícula): Área $=I+\frac{B}{2}-1$ donde I es el número de puntos enteros estrictamente interiores y B es el número de puntos enteros en el borde.
  - Un **segmento cerrado** entre (x_1,y_1) y (x_2,y_2) contiene exactamente $\gcd(|x_2-x_1|,|y_2-y_1|)+1$ puntos enteros; al sumar sobre las aristas cerrando el polígono (contando cada vértice una sola vez, arista semiabierta $[P_i,P_{i+1})$), la contribución de cada lado al total B del borde es $\gcd(|x_2-x_1|,|y_2-y_1|)$.
- **Fórmula de Herón y Radios de Triángulos**:
  - Semiperímetro: $s=\frac{a+b+c}{2}$.
  - Área: $A=\sqrt{s(s-a)(s-b)(s-c)}$.
  - Radio del incírculo (inradius): $r=\frac{A}{s}$.
  - Radio del circuncírculo (circumradius): $R=\frac{abc}{4A}$.
- **Distancia de Gran Círculo (Fórmula de Haversine)**:
  - Distancia entre dos puntos en una esfera de radio R con latitud/longitud (ϕ_1,λ_1) y (ϕ_2,λ_2) en radianes: $d=2R\arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right)+\cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)}\right)$.

11.6 Teoría de Juegos y XOR

- **Teorema de Sprague-Grundy**:
  - Todo juego **imparcial, finito/terminante (acíclico)** de información perfecta bajo **convención normal** (gana quien realiza la última jugada) es equivalente a un montón de Nim con tamaño igual a su valor de Grundy $G(u)=\text{mex}\{G(v):u\to v\}$.
  - Para la suma disjunta de juegos independientes: $G(S_1+S_2+\ldots)=G(S_1)\oplus G(S_2)\oplus\ldots$
  - El estado es **ganador** para el primer jugador si y solo si la suma XOR es $\neq 0$.
- **Juego de Nim**:
  - Posición ganadora $\Leftrightarrow x_1\oplus x_2\oplus\ldots\oplus x_n\neq 0$.
  - **Regla de Misère Nim** (pierde quien toma la última ficha): Se juega idéntico al Nim normal con suma XOR $\neq 0$, EXCEPTO cuando todos los montones tienen tamaño ≤ 1: en ese caso, la posición es ganadora si y solo si el número de montones de tamaño 1 es **par**.
- **Juego de Moore (Nim_k)**:
  - En cada turno, el jugador puede retirar fichas de entre 1 y k montones distintos.
  - Condición de victoria: Sumar cada columna binaria de los tamaños de los montones en base 10; la posición es perdedora si y solo si la suma de cada columna binaria es un múltiplo de $k+1$.

11.7 Reglas Lógicas de 2-SAT

Cláusula Deseada	Implicaciones en el Grafo de 2-SAT
p debe ser Verdadero	$\neg p\Longrightarrow p$
p debe ser Falso	$p\Longrightarrow\neg p$
Al menos uno ($p\vee q$)	$\neg p\Longrightarrow q$ y $\neg q\Longrightarrow p$
No ambos $\neg(p\wedge q)\equiv(\neg p\vee\neg q)$	$p\Longrightarrow\neg q$ y $q\Longrightarrow\neg p$
Equivalentes ($p=q$)	$p\Longrightarrow q$, $q\Longrightarrow p$, $\neg p\Longrightarrow\neg q$, $\neg q\Longrightarrow\neg p$
Distintos / XOR ($p\neq q$)	$p\Longrightarrow\neg q$, $\neg p\Longrightarrow q$, $q\Longrightarrow\neg p$, $\neg q\Longrightarrow p$
A lo sumo uno entre $x_1,\ldots,x_n$	Crear prefijos $p_i=p_{i-1}\vee x_i$: $x_i\Longrightarrow p_i$, $p_{i-1}\Longrightarrow p_i$, $p_{i-1}\Longrightarrow\neg x_i$.

- **Solubilidad**: El sistema 2-SAT es satisfacible si y solo si para ninguna variable x, x y $\neg x$ caen en la misma Componente Fuertemente Conexa (SCC). La asignación se obtiene eligiendo el valor cuya SCC aparece después en el orden topológico.

11.8 Tabla de Constantes, Primos y Módulos Frecuentes

Tipo	Constante / Primo	Propiedades Clave
Estándar	$10^9+7 = 1000000007$	Primo común en Codeforces/ICPC ($2 \times 10^9+14$ entra en signed 32-bit).
Estándar 2	$10^9+9 = 1000000009$	Ideal para segundo módulo en doble rolling hash con 10^9+7.
Primos Hash	$10^9+21, 10^9+33, 10^9+87$	Primos grandes para evitar hacks en tablas hash.
NTT Primo 1	$998244353 = 119 \cdot 2^{23}+1$	Raíz primitiva $g=3$. Soporta transformadas de tamaño hasta $2^{23} \approx 8.38 \times 10^6$.
NTT Primo 2	$1004535809 = 479 \cdot 2^{21}+1$	Raíz primitiva $g=3$. Soporta tamaños hasta $2^{21} \approx 2.09 \times 10^6$.
NTT Primo 3	$469762049 = 7 \cdot 2^{26}+1$	Raíz primitiva $g=3$. Soporta tamaños hasta $2^{26} \approx 6.71 \times 10^7$.
64-bit Prime	$10^{18}+9$ (1000000000000000009)	Primo de 64 bits para pruebas con Miller-Rabin.
64-bit Prime	$10^{18}+31$ (1000000000000000031)	Segundo primo de 64 bits.
Bases Miller-Rabin	`2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37`	Conjunto determinista para probar primalidad de cualquier $N < 2^{64}$.
Primos Gemelos	$(10^9+7, 10^9+9)$	Pareja de primos gemelos útiles para hashes combinados.

Bloque 12: Python 3 para ICPC — Trucos, BigInt, Bases y Librería Estándar

¿Cuándo usar Python en competencia?: - Cuando el problema requiere **BigInt** (miles de dígitos sin límite de 64 o 128 bits). - Cuando toque convertir o parsear **bases arbitrarias** o formatear binario/hexadecimal. - Cuando requieras **fechas, días transcurridos y calendarios** exactos (`datetime`). - Cuando se necesiten **fracciones exactas** (`Fraction`) o **100+ decimales** (`Decimal`). - Cuando parsear strings en C++ tome decenas de líneas y en Python salga con `re` o `eval` . - Cuando una DP con estados complejos (strings o tuplas) se resuelva en 5 líneas con `@cache` .

12.1 Template Base, Fast I/O, Límites de Recursión y Digits en BigInt

- **Cuándo usar:** Plantilla inicial en Python 3 para lectura masiva de tokens (`sys.stdin.read().split()`), recursión profunda (`setrecursionlimit`) y conversión de enteros de más de 4300 dígitos a texto (`set_int_max_str_digits`).
- **Cuándo no usar / alternativa:** En problemas interactivos donde debes leer línea por línea y hacer `flush` , no usar `sys.stdin.read()` (que espera hasta `EOF`); usar `sys.stdin.readline()` y `print(..., flush=True)` .
- **Contrato:** En Python 3.11+, imprimir un entero de más de 4300 dígitos lanza `ValueError` si no se llama antes a `sys.set_int_max_str_digits(10**6)` . Dependencias: librería estándar de Python 3.
- **Complejidad:** $\mathcal{O}(\text{tokens})$ para lectura completa de entrada.

```
import sys
from math import gcd, isqrt, comb, perm, factorial
from collections import defaultdict, Counter, deque
from fractions import Fraction
from decimal import Decimal, getcontext
from functools import cache
import bisect
import itertools
import re

# Aumentar limite de recursion para DFS profunda
sys.setrecursionlimit(10**6)

# OBLIGATORIO en Python 3.11+ para BigInt > 4300 digitos a
# string
sys.set_int_max_str_digits(10**6)

def solve():
    # Lectura ultra rapida de todos los tokens
    input_data = sys.stdin.read().split()
    if not input_data:
        return
    it = iter(input_data)
    # n = int(next(it))

if __name__ == '__main__':
    solve()
```

- **Trampas y adaptaciones:** (1) En PyPy3 / CPython, una recursión profunda con `@cache` o DFS que supere 10^5 niveles puede agotar la pila del sistema operativo aunque `setrecursionlimit` sea alto; preferir BFS/iterativo si $N > 10^5$.

12.2 Conversión de Bases Arbitrarias (Base $A \leftrightarrow B$) y Formateo

- **Cuándo usar:** Convertir números (incluso gigantes) entre cualquier par de bases $A, B \in [2, 62]$ o formatear en binario/octal/hexadecimal con relleno de ceros.
- **Contrato:** `int(s, base)` soporta nativamente bases $2 \leq \text{base} \leq 36$; `to_base(n, b)` soporta bases hasta 62 (`0-9A-Za-z`) y maneja `n = 0` y negativos.
- **Complejidad:** $\mathcal{O}(L^2)$ para conversión de cadenas de longitud L en BigInt.

```
# 1. De cualquier Base B a Entero Decimal: int(string, base)
num = int("1A3F", 16)       # Hexadecimal a int -> 6719
num = int("101101", 2)      # Binario a int -> 45
num = int("Z9", 36)         # Base 36 a int -> 1269

# 2. De Decimal a Binario, Octal, Hexadecimal nativo:
s_bin = bin(45)[2:]         # "101101"
s_oct = oct(45)[2:]         # "55"
s_hex = hex(45)[2:].upper() # "2D"

# Con f-strings y padding de ceros (ej. 8 bits):
f_bin = f"{45:08b}"         # "00101101"
f_hex = f"{45:04X}"         # "002D"

# 3. De Decimal a cualquier Base B (2 <= B <= 62):
CHARS = ("0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"
         "abcdefghijklmnopqrstuvwxyz")

def to_base(n: int, b: int) -> str:
    if n == 0:
        return "0"
    res = []
    neg = n < 0
    n = abs(n)
    while n > 0:
        res.append(CHARS[n % b])
        n //= b
    if neg:
        res.append('-')
    return "".join(reversed(res))

# Conversion directa Base A -> Base B:
def base_a_to_b(s: str, base_a: int, base_b: int) -> str:
    return to_base(int(s, base_a), base_b)
```

- **Trampas y adaptaciones:** (1) `bin(x)` devuelve el prefijo `'0b'` (y `'-0b'` si es negativo); usar `f'{x:b}'` evita tener que cortar `bin(x)[2:]` .

12.3 Aritmética Gigante (BigInt), Potencia Modular y Raíz Entera

- **Cuándo usar:** Operar con enteros de miles de dígitos sin overflow, calcular $b^e \pmod m$ en $\mathcal{O}(\log e)$ con `pow(b, e, mod)` , truncar divisiones hacia cero (`trunc_div`) y extraer $\lfloor \sqrt{N} \rfloor$ exacta con `math.isqrt(N)` .
- **Cuándo no usar / alternativa:** Nunca escribir `(b ** e) % mod` (que construye el número gigante b^e en memoria y da TLE/MLE); usar siempre el tercer argumento `pow(b, e, mod)` .
- **Contrato:** En Python, `-7 // 3 == -3` (redondea hacia $-\infty$); `trunc_div(a, b)` trunca hacia cero igual que C++ usando solo aritmética entera (sin `int(a / b)` , que falla para $> 10^{16}$ por precisión de `float`). En Python 3.8+, `pow(a, -1, m)` calcula directamente el inverso modular. Dependencias: `from math import isqrt` .
- **Complejidad:** `pow(b, e, m)` en $\mathcal{O}(\log e)$; `isqrt(N)` en $\mathcal{O}(M(\text{bits}))$.

```
# Potencia Modular BigInt en O(log e): pow(b, e, mod)
# Mucho mas veloz que (b ** e) % mod
ans = pow(3, 10**18, 10**9 + 7)

# Division entera con negativos (sin usar float, exacto para
# BigInt):
# En Python -7 // 3 da -3 (floor hacia -infinito).
# Para truncar hacia cero como en C++ sin perder precision
# en BigInt:
def trunc_div(a: int, b: int) -> int:
    return -(
        abs(
            a) // abs(
                b)) if (
                    a < 0) ^ (b < 0) else abs(a) // abs(b)

# Raiz Cuadrada Entera Exacta: isqrt(N)
# Retorna floor(sqrt(N)) sin perder precision de punto
# flotante
r = isqrt(10**50)  # 10^25 exacto (double fallaria aqui)
```

- **Trampas y adaptaciones:** (1) Nunca usar `int(math.sqrt(N))` para $N > 10^{15}$; usar siempre `math.isqrt(N)` .

12.4 Fracciones Exactas (`Fraction`) y Decimales de Alta Precisión (`Decimal`)

- **Cuándo usar:** Geometría, probabilidades o sistemas lineales pequeños donde el punto flotante de 64/80 bits da WA por precisión, o cálculo de raíces/divisiones con 100+ cifras decimales exactas.
- **Cuándo no usar / alternativa:** Si el denominador crece exponencialmente en miles de pasos, `Fraction` se vuelve lento por el cálculo de GCD en cada suma.
- **Contrato:** Construir siempre `Fraction(num, den)` o `Fraction('0.25')` (desde `str` , nunca desde `float` binario inexacto). En `Decimal` , ajustar `getcontext().prec = 100` antes de operar. Dependencias: `fractions` , `decimal` .
- **Complejidad:** $\mathcal{O}(\log(\text{den}))$ por operación racional.

```
# 1. Fracciones Exactas (simplifica automaticamente)
from fractions import Fraction

f1 = Fraction(3, 6)         # 1/2
f2 = Fraction("0.25")       # 1/4
res = f1 + f2               # 3/4
print(f"{res.numerator}/{res.denominator}")

# 2. Decimales con precision configurable (ej. 100 digitos)
from decimal import Decimal, getcontext
getcontext().prec = 100     # 100 digitos significativos

pi_approx = Decimal(22) / Decimal(7)
raiz_2 = Decimal(2).sqrt()
```

- **Trampas y adaptaciones:** (1) `Decimal(0.1)` hereda la imprecisión binaria de `float` ; pasar siempre strings `Decimal('0.1')` o enteros `Decimal(1) / Decimal(10)` .

12.5 Manejo de Fechas y Calendarios (`datetime`)

- **Cuándo usar:** Contar días exactos entre dos fechas, avanzar/retroceder K días o determinar el día de la semana respetando años bisiestos automáticamente.
- **Cuándo no usar / alternativa:** Si los años superan `9999` (`datetime.MAXYEAR = 9999`), reducir el año módulo `400` (el calendario gregoriano se repite exactamente cada 400 años = 146097 días).
- **Contrato:** `weekday()` retorna `0` para Lunes hasta `6` para Domingo. Dependencias: `import datetime` .
- **Complejidad:** $\mathcal{O}(1)$ tiempo y memoria.

```
import datetime

# 1. Crear fecha (Ano, Mes, Dia)
d1 = datetime.date(2024, 2, 28)
d2 = datetime.date(2026, 10, 25)

# 2. Diferencia en dias:
dias = (d2 - d1).days       # 970 dias exactos

# 3. Sumar dias a una fecha:
futuro = d1 + datetime.timedelta(days=100)

# 4. Dia de la semana: 0 = Lunes, 6 = Domingo
dia_sem = d1.weekday()

# 5. Formatear fecha:
texto = d1.strftime("%Y-%m-%d, %A")
```

- **Trampas y adaptaciones:** (1) El ciclo gregoriano de 400 años tiene exactamente $400 \times 365 + 97 = 146097$ días, que es divisible por 7 (por lo que los días de la semana también coinciden cada 400 años).

12.6 DP con Memoización Automática (`@cache` / `cache_clear`)

- **Cuándo usar:** Prototipar rápidamente DP top-down sobre estados dispersos (Digit DP, juegos, árboles) sin dimensionar arreglos multidimensionales a mano.
- **Cuándo no usar / alternativa:** Si el estado es denso y hay millones de transiciones simples, un bucle iterativo en C++ (Bloque 8) es $20\times$–$50\times$ más rápido.
- **Contrato:** Todos los argumentos de la función decorada con `@cache` deben ser *hashables* (`int` , `str` , `tuple` ; nunca `list`). Llamar `dp.cache_clear()` al final de cada caso de prueba en problemas multi-test. Dependencias: `from functools import cache` .
- **Complejidad:** Tiempo y memoria proporcionales al número de estados alcanzables.

```
from functools import cache

# DP de Mochila o Digit DP con memoizacion automatica:
@cache
def dp(i: int, rem_weight: int) -> int:
    if i == n or rem_weight == 0:
        return 0
    # Opcion 1: No tomar
    res = dp(i + 1, rem_weight)
    # Opcion 2: Tomar
    if weight[i] <= rem_weight:
        res = max(res, val[i] + dp(i + 1,
            rem_weight - weight[i]))
    return res

# Limpiar memoizacion entre casos de prueba:
dp.cache_clear()
```

- **Trampas y adaptaciones:** (1) Si olvidas `dp.cache_clear()` entre casos de prueba que leen arreglos globales `weight` / `val` , Python reutilizará resultados del caso anterior y dará WA.

12.7 Extracción de Números en Texto Sucio con Expresiones Regulares (`re`)

- **Cuándo usar:** Parsear entradas con formato sucio (coordenadas entre paréntesis, comas, signos, etiquetas de texto) extrayendo todos los enteros en una sola línea.
- **Cuándo no usar / alternativa:** Si la entrada solo contiene enteros separados por espacios, `map(int, sys.stdin.read().split())` es más rápido.
- **Contrato:** `re.findall(r'-?\d+', s)` extrae todos los enteros con o sin signo `-` . Dependencias: `import re` .
- **Complejidad:** $\mathcal{O}(|S|)$ tiempo y memoria.

```
import re

s = "pos: (x=15, y=-42), vel=[+100, -5], count=8"
# Extrae TODOS los enteros con o sin signo en 1 linea:
nums = [int(x) for x in re.findall(r'-?\d+', s)]
# nums = [15, -42, 100, -5, 8]
```

- **Trampas y adaptaciones:** (1) Si `'2026-11-01'` debe leerse como tres números positivos separados por guiones y no como `2026, -11, -1` , usar `r'\d+'` sin el `-?` .

12.8 Frecuencias y Álgebra de Multiconjuntos (`Counter`)

- **Cuándo usar:** Contar frecuencias de elementos o caracteres en $\mathcal{O}(N)$, hallar los K más frecuentes (`most_common(K)`) u operar multiconjuntos con suma `+` , resta `-` , intersección `&` (mínimo) y unión `|` (máximo).
- **Cuándo no usar / alternativa:** Si un adversario construye colisiones contra el hash de enteros en Codeforces/AtCoder con `Counter(int)` , envolver las llaves con `x ^ RANDOM_SEED` o convertirlas a `str` .
- **Contrato:** La resta `c1 - c2` elimina automáticamente las llaves cuya frecuencia quede ≤ 0; para conservar negativos usar `c1.subtract(c2)` . Dependencias: `from collections import Counter` .
- **Complejidad:** $\mathcal{O}(N)$ esperado.

```
from collections import Counter

c1 = Counter([1, 2, 2, 3])
c2 = Counter([2, 3, 3, 4])

# Operaciones algebraicas de multiconjuntos:
suma  = c1 + c2     # Suma conteos: {2: 3, 3: 3, 1: 1, 4: 1}
# Resta (mantiene solo positivos): {1: 1, 2: 1}
resta = c1 - c2
# Minimo conteo (interseccion): {2: 1, 3: 1}
inter = c1 & c2
# Maximo conteo (union): {1: 1, 2: 2, 3: 2, 4: 1}
union = c1 | c2

# Top K elementos mas frecuentes en O(N log K):
top2 = c1.most_common(2) # [(2, 2), (1, 1)]
```

- **Trampas y adaptaciones:** (1) `sum(c.values())` (o `c.total()` en Python 3.10+) devuelve el total de elementos contando multiplicidades.

12.9 Agrupación y Grafos Instantáneos (`defaultdict`)

- **Cuándo usar:** Agrupar elementos por clave (ej. anagramas por firma ordenada `''.join(sorted(w))`) o representar grafos dispersos con etiquetas arbitrarias (strings, tuplas) sin inicializar llaves previamente.
- **Cuándo no usar / alternativa:** Si los nodos son enteros `0..N-1` , una lista de listas `[[] for _ in range(N)]` es más rápida y consume menos memoria.
- **Contrato:** Al acceder a `d[k]` por primera vez, invoca la fábrica (`list` , `int` , `set`) y crea la entrada automáticamente. Dependencias: `from collections import defaultdict` .
- **Complejidad:** $\mathcal{O}(1)$ promedio por acceso/inserción.

```
from collections import defaultdict

# Grafo sin tamano previo:
adj = defaultdict(list)
adj[u].append(v)

# Agrupar anagramas:
grupos = defaultdict(list)
for w in words:
    grupos["".join(sorted(w))].append(w)
```

- **Trampas y adaptaciones:** (1) Iterar `for k in d:` mientras lees `d[otra_llave]` dentro del bucle lanza `RuntimeError: dictionary changed size during iteration` ; usar `d.get(otra_llave)` si no quieres crear llaves vacías.

12.10 Rotación y Transposición de Matrices en 1 Línea (`zip(*mat)`)

- **Cuándo usar:** Transponer una grilla $R \times C$ (convertir columnas en filas) o rotarla 90° en sentido horario o antihorario en una sola línea.
- **Contrato:** `zip(*mat)` empareja los elementos de igual columna de cada fila. `col[::-1]` invierte cada columna para rotar 90° en sentido horario.
- **Complejidad:** $\mathcal{O}(R \cdot C)$ tiempo y memoria.

```
mat = [
    [1, 2, 3],
    [4, 5, 6]
]

# 1. Transponer matriz (columnas a filas):
trans = [list(col) for col in zip(*mat)]

# 2. Rotar 90 grados sentido horario:
rot90 = [list(col[::-1]) for col in zip(*mat)]

# 3. Rotar 90 grados sentido antihorario:
rot90_ccw = [list(col) for col in zip(*mat)][::-1]
```

- **Trampas y adaptaciones:** (1) Si las filas son strings (`['abc', 'def']`), usar `[''.join(col) for col in zip(*mat)]` para obtener una lista de strings.

12.11 Álgebra de Conjuntos Nativos (`set`)

- **Cuándo usar:** Operaciones de intersección (`A & B`), unión (`A | B`), diferencia (`A - B`), diferencia simétrica (`A ^ B`) y prueba de subconjunto (`A <= B`).
- **Cuándo no usar / alternativa:** Si necesitas mantener los elementos **ordenados** con búsqueda binaria (`lower_bound`), el `set` de Python es una tabla hash desordenada; usar C++ `std::set` o `ordered_set` (§1.3).
- **Contrato:** `set()` crea un conjunto vacío (`{}` crea un diccionario vacío).
- **Complejidad:** $\mathcal{O}(|A| + |B|)$ para operaciones binarias entre conjuntos.

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}

inter = A & B       # Interseccion: {3, 4}
union = A | B       # Union: {1, 2, 3, 4, 5, 6}
dif   = A - B       # En A pero no en B: {1, 2}
simet = A ^ B       # Diferencia simetrica: {1, 2, 5, 6}
sub   = A <= B      # True si A es subconjunto de B
```

- **Trampas y adaptaciones:** (1) Un `set` no puede contener elementos mutables (`list` , `set`); usar `tuple` o `frozenset` si necesitas un conjunto de conjuntos.

12.12 Bits en Números Gigantes (`bit_count` , `bit_length`)

- **Cuándo usar:** Contar bits encendidos (`n.bit_count()` , equivalente a `popcount`) o el índice del bit más alto (`n.bit_length()` , cantidad de bits necesarios para representar `n`) en enteros de precisión arbitraria.
- **Cuándo no usar / alternativa:** En Python < 3.10 donde no exista `bit_count()` , usar `bin(n).count('1')` .
- **Contrato:** Para `n = 0` , `0.bit_count() == 0` y `0.bit_length() == 0` . Para $n > 0$, $\lfloor \log_2 n \rfloor + 1 = \text{n.bit_length}() - 1$. Dependencias: Python 3.10+.
- **Complejidad:** $\mathcal{O}(\text{palabras de } n)$.

```
n = 10**60

# Cantidad de bits encendidos (popcount) de un BigInt:
unos = n.bit_count()

# Cuantos bits ocupa el numero:
bits_necesarios = n.bit_length()
```

- **Trampas y adaptaciones:** (1) En Python los enteros con signo negativo tienen extensión infinita de bits en complemento a 2 (`~x == -x - 1`); aplicar siempre máscara `(x & ((1 << B) - 1))` si simulas registros de B bits.

12.13 Búsqueda Binaria con Proyección (`bisect` con `key`) y Ordenamiento Multi-Criterio

- **Cuándo usar:** Hacer `lower_bound` (`bisect_left`) o `upper_bound` (`bisect_right`) proyectando por un atributo con `key=` , u ordenar objetos por múltiples criterios ascendentes/descendentes.
- **Cuándo no usar / alternativa:** En Python < 3.10 el parámetro `key=` no existe en `bisect` ; en ese caso buscar sobre una lista de tuplas `(clave, ...)` .
- **Contrato:** El `sort` de Python (*Timsort*) es **estable** y $\mathcal{O}(N \log N)$ ($\mathcal{O}(N)$ en datos casi ordenados). Dependencias: `import bisect` .
- **Complejidad:** `bisect` en $\mathcal{O}(\log N)$; `sort` en $\mathcal{O}(N \log N)$.

```
import bisect

# 1. Búsqueda binaria con funcion key (Python 3.10+):
eventos = [(10, "A"), (25, "B"), (40, "C"), (90, "D")]
# lower_bound por el primer elemento (tiempo >= 30):
# idx = 2
idx = bisect.bisect_left(eventos, 30, key=lambda x: x[0])

# 2. Ordenamiento multi-criterio:
# 1) Puntos descendente (-x['pts'])
# 2) Penalidad ascendente (+x['pen'])
# 3) Nombre alfabetico (+x['nom'])
jugadores.sort(key=lambda x: (-x['pts'], x['pen'],
    x['nom']))
```

- **Trampas y adaptaciones:** (1) Para ordenar strings en orden **descendente** dentro de una tupla mixta sin implementar `cmp_to_key` , aprovechar la estabilidad de `sort` ordenando primero por el criterio secundario y luego por el primario.

12.14 Combinatoria y Generadores Nativos (`comb` , `perm` , `itertools`)

- **Cuándo usar:** Calcular $\binom{n}{k}$, $P(n,k)$ o $n!$ exactos en enteros gigantes, o iterar todas las permutaciones, combinaciones y productos cartesianos en pocas líneas para fuerza bruta o *stress testing*.
- **Cuándo no usar / alternativa:** Si necesitas $\binom{n}{k} \pmod{10^9+7}$ para $n = 10^6$, `math.comb(10**6, 5*10**5)` construye un número de 300 000 bits y da TLE; usar factoriales modulares (§7.4.2).
- **Contrato:** `comb(n, k)` retorna `0` si $k < 0$ o $k > n$. `permutations` , `combinations` y `product` devuelven iteradores perezosos en orden lexicográfico. Dependencias: `math` , `itertools` .
- **Complejidad:** $\mathcal{O}(k)$ operaciones BigInt para `comb(n, k)` .

```
from math import comb, perm, factorial
from itertools import permutations, combinations, product

# nCr y nPr gigantes sin overflow:
formas = comb(1000, 50)
p = perm(100, 3)
fact_100 = factorial(100)

# Generacion exhaustiva:
perms = list(permutations([1, 2, 3]))        # 3! formas
```

```python
combs = list(combinations([1, 2, 3, 4], 2))  # 4C2 formas
# 2^3 producto cartesiano
cart  = list(product([0, 1], repeat=3))
```

- **Trampas y adaptaciones:** (1) `itertools.combinations_with_replacement(iterable, r)` genera multiconjuntos de tamaño `r` con repetición permitida.

12.15 Evaluación de Expresiones Aritméticas (`eval`)

- **Cuándo usar:** Evaluar expresiones con paréntesis, `+`, `-`, `*`, `//`, `%`, `**` o bitmasks (`|`, `&`, `^`) directamente con el parser de Python.
- **Cuándo no usar / alternativa:** Si hay divisiones con operandos **negativos** y el problema exige truncamiento hacia cero estilo C++ (`-7 / 3 == -2`), `//` de Python dará `-3` (hacia $-\infty$); en ese caso usar `ExpressionParser` en C++ (§10.2).
- **Contrato:** La expresión regular `r'(?<!/)/(?!/)'` reemplaza únicamente las barras simples `/` por división entera `//` sin corromper los `//` que ya existían en la cadena. Dependencias: `import re` .
- **Complejidad:** $\mathcal{O}(|S|)$ para expresiones con enteros de tamaño fijo.

```python
# eval() procesa operadores (+, -, *, //, %, **)
expr = "((15 + 25) * 3) // 4"
resultado = eval(expr)      # 30

# Reemplazar solo '/' simples por '//' sin duplicar '//'
# existentes:
import re
expr_safe = re.sub(r"(?<!/)/(?!/)", "//", expr)
```

- **Trampas y adaptaciones:** (1) Nunca usar `.replace('/', '//')` porque convertiría un `'//'` ya existente en `'////'` (`SyntaxError`).